0x00-ToDo

- P1.plan
- P2.

0x00-算法分析设计 ☆

- 左神算法
- 剑指offer
- 力扣
  - 数组 ☆
  - 链表 ☆
  - 栈和队列☆
  - 树 ☆☆☆
    - 递归
    - 层序遍历
    - 前中后序遍历
    - BST
    - Trie
  - 哈希表 ☆
  - 字符串 ☆
  - 数组和矩阵 ☆
  - 图
    - 二分图
    - 拓扑排序
    - 并查集
  - 位运算 ☆
  - 双指针 ☆
  - 排序
    - 桶排序
    - 荷兰国旗问题
  - 二分 ☆
  - 分治
  - 搜索
    - BFS
    - DFS
    - 回溯
  - 贪心算法 ☆
  - 动态规划 ☆
    - 斐波那契数列
    - 矩阵路径和
    - 数组区间
    - 分隔数组
    - 最长递增子序列
    - 最长公共子序列
    - 01背包
    - 股票交易
    - 字符串编辑
  - 数学
- 问题迭代
  - 排序
  - 快排 ☆☆☆
  - 跳表
  - 红黑树
  - 布隆过滤器
  - 二叉树
  - 并查集
  - B树/B+树

- 图
- 单调栈
- Tire树
- 辗转相除法

面试算法

问题迭代

0x01-Java篇 ☆

基础 ☆☆☆

- 字符串 ☆☆☆
- OOP
- 关键字
- 异常
- IO
- 泛型
- 反射
- 拷贝
- 序列化
- Java8
- 注解
- 内部类
- 枚举类
- 问题迭代

集合类 ☆☆☆

- List
  - ArrayList ☆ ☆☆☆
  - Vector
  - LinkedList ☆☆☆
  - CopyOnWriteArrayList ☆
- Map
  - **HashMap** ☆ ☆☆☆
  - **ConcurrentHashMap** ☆☆☆☆
  - TreeMap
  - HashTable
  - LinkedHashMap
- Set
  - HashSet
  - TreeSet
  - LinkedHashSet

并发 ☆☆☆

- 并发基础
- 线程创建
- 基础线程机制
- 线程中断
- 互斥同步
- 线程协作/通信
- AQS工具类
- JUC工具类
- JMM内存
- 线程安全
- 锁相关
- 线程池原理
- ThreadLocal
- 问题迭代

JVM ☆☆☆

- JVM运行时数据区 ☆☆☆
- 垃圾回收算法 ☆☆☆
- 判断对象是否存活的条件
- 垃圾收集器 ☆☆☆

- 类加载器 ☆☆☆
- JVM 对象分配回收策略
- 类加载流程
- JVM调优
- 问题迭代

0x02-数据篇 ☆

- MySQL ☆☆☆☆
  - 表设计、三范式
  - 索引机制 ☆☆☆
  - 锁机制
  - 事务和隔离级别 ☆☆☆
  - SQL语句
  - 存储引擎
  - 日志模块
  - SQL优化、线上故障排查
  - 主从复制
  - 问题迭代
- Redis ☆☆☆
  - 0.介绍一下
  - 1.持久化 AOF/RDB ☆☆☆
  - 2.5种数据类型 场景 ☆☆☆
  - 3.缓存相关问题 ☆☆☆
  - 4.单线程模型 IO多路复用 ☆☆☆
  - 5.缓存过期淘汰策略
  - 6.高可用和高并发
  - 7.集群、主从复制、哨兵模式
  - 8.事务 ☆☆☆
  - 9.Redis特点
  - 10.发布订阅
  - 11.分布式锁
  - 11.面试常见题目

0x03-框架篇 ☆

- Servlet
- Spring ☆☆☆
  - AOP
  - IOC
  - 事务
  - Bean
  - 设计模式
  - 问题迭代
- Spring MVC
- Mybatis
- Hibernate
- Spring Boot
- 微服务
- Spring Cloud
  - 服务注册与发现
    - Eureka
    - Zookeeper
    - Consul
  - 负载均衡
    - Ribbon
    - OpenFeign
  - Hystrix 断路器
  - 路由网关
    - Zuul
    - GateWay
  - 分布式配置中心

Config
消息总线
SpringCloud Bus
消息驱动
SpringCloud Stream
分布式请求链路追踪
问题迭代
SpringCloud Alibaba
Nacos服务注册和配置中心
Sentinel实现熔断与限流
Seata处理分布式事务
0x04-科班篇 ☆
设计模式
面向对象六大原则
P1.单例模式 ☆☆☆
P2.装饰者模式
P3.代理模式 ☆☆☆
P4.工厂模式
P5.策略模式 ☆
P6.命名模式
P7.模板方法
P8.适配器模式
P9.责任链模式
P10.建造者模式
P11.观察者模式
问题迭代
计网
网络分层
数据链路层
网络层 ☆
传输层 ☆
应用层 ☆
问题迭代
思考
计组
操作系统
进程 ☆
线程 ☆
死锁 ☆
存储器
系统级IO
用户态内核态
中断
虚拟内存
内存管理 ☆
fork函数
Linux
1.linux查看文件内容的命令
2.linux常用命令
关机命令
用户注销
vim
linux主要目录结构
3.显示最近登录的5个用户名
4.生产环境服务器变慢，诊断思路和性能评估谈谈?
5.假如生产环境出现CPU占用过高，请谈谈你的分析思路和定位
6.查找特定文件
7.检索文件内容

        8.awk
        1、绝对路径用什么符号表示？当前目录、上层目录用什么表示？主目录用什么表示？切换目录用什么命令？
        2、怎么查看当前进程？怎么执行退出？怎么查看当前路径？
        3、怎么清屏？怎么退出当前命令？怎么执行睡眠？怎么查看当前用户id？查看指定帮助用什么命令？
        4、Ls命令执行什么功能？可以带哪些参数，有什么区别？
        5、查看文件有哪些命令
        6、列举几个常用的Linux命令
        7、你平时是怎么查看日志的？
        8、利用ps怎么显示所有的进程？怎么利用ps查看指定进程的信息？
        9、查看当前谁在使用该主机用什么命令？查找自己所在的终端信息用什么命令？
        10.使用什么命令查看磁盘使用空间？空闲空间呢？
        11.查看各类环境变量用什么命令
    文件操作
    问题迭代
软件工程
0x05-中间件
MQ
    1.为什么使用MQ
    2.MQ的优点和缺点
    3.四大MQ的有优缺点。
    4.可用行
    5.MQ重复消费问题
    6.消息数据丢失
    7.消息数据顺序消费
    8.消息积压
    9.设计一个架构
    10.应用场景
ZK
    Paxos
Netty
Dubbo
Tomcat
Nginx
ES
MongDB
思考
0x06-工具篇
Git/SVN
Maven
IDEA
0x07-分布式篇
D1.CAP BASE
D2.分布式锁 ☆☆☆
D3.分布式session
D4.分布式事务 ☆☆☆
D5.单点登录SSO、分布式会话
D6.分布式全局ID
D7.接口幂等性
D8.分布式限流
D9.正反代理
D10.一致性哈希·233-·66
D11.分布式算法`todo`
D12.问题迭代
0x08-容器篇
Docker
K8S

0x09-项目篇 ☆

- 1.白洋淀票务系统
- 2.宝利鑫光伏发电
- 3.雄安新区
- 4.轻量级RPC
- 5.问题迭代
- 6.HR问题
    - 1. 为什么选择我们公司
    - 2. 为什么做这个岗位
    - 3. 简历上的某个项目有什么困难，怎么解决的
    - 4. 对于实习的工作的成果的看法，思考
    - 5. 个人的优点缺点
    - 6. 职业规划是什么
    - 7. 你还有什么问题要问我吗

0X10-方法迭代

0x11-场景、系统设计

- 1.购物车
- 2.单点登录
- 3.秒杀系统
- 4.大数据问题
- 5.设计一个分布式锁 需要注意哪些事项。
- 6.限流问题
- 7.线上环境有遇到过OOM场景吗，说一下问题排查思路
- 8.线上CPU遇到过100%吗？说一下排查思路？
- 9.分布式事务的情景
- 10.SQL练练连
- 11.高并发抢票系统设计思路
- 12.附近人，打车
- 13.百度贴吧

0x12-白板编程

- 0.快排
- 1.归并
- 2.请手写一个死锁
- 3.两个程序依次输出 11/22/33 ？
- 4.单例
- 5.生产者消费者模型
    - 1.synchronized+notify+wait
    - 2.lock+Condition+awiait+singal
    - 3.volatile/BlockingQueue/AtomicInteger
- 6.个线程依序打印ABC
- 7.手写LRU算法

0x13-心得

- 简历相关
- **软件架构**
    - 重复代码
    - 框架设计
    - 项目依赖
    - 中台
    - **业务架构**
    - **技术架构**
        - **缓存视角**
    - 软件设计
- 工作
- 做项目一年的思考
    - 1.技术角度
    - 2.工作中和领导以及同事的合作，同部门以及跨部门协作
        - 1.与领导合作
        - 2.部门内部的合作

3.优势与缺陷。
4.自我反思
随笔
0x14-todo
0x15-书海遨游

0x00-ToDo

P1.plan

```
整理技能库 1.将知识树形化 2.大纲好复习 3.查漏补缺  4.offer
在生动的故事 没有细节也没用。
```

现在看书 一定要以搞明白一个知识点为主 不能整本书看 时间上赶不上，以面向面经复习 会看书。

任何一个技术都不是孤立存在的，一定要联系起来，多思考。形成一个整体。

方法迭代，【知识点概览->面试题自问自答】

P2.

0x00-算法分析设计 ☆

```
dp-数组-数学-字符串-树-哈希-dfs-二分搜索-贪心-bfs-双指针-栈-回溯-排序-图-位运算
队列-堆-链表-分治-递归-trie  【七分理解-三分记忆-三分背诵】
排列组合->DFS
最值->贪心/排序/dp
```

- 468. 验证IP地址
- 41. 缺失的第一个正数

左神算法

剑指offer

```
【数组】



【dp】


【分治】
33.二叉搜索树的后序遍历序列
```

```
    1.分治  根据数组最后数据判断为当前的根节点，然后分别求左子树和右子树。



【递归-dp】
10-1.斐波那契额数列   ☆☆
    1.递归   2.数组 3.dp
10-2.青蛙跳台阶【高频】  ☆☆
    1.递归 2，记忆化
10-3 变态跳台阶  ☆☆
    1.f(n) = 2^(n-1)

【回溯】
12.矩阵中的路径     ☆
    1.回溯
13.机器人的路径
    1.回溯   穷举所有路径  排除掉不符合条件的路径
34.二叉树中的合围某一值的路径  【面试之前必看】
    1.回溯算法
38.字符串的排列  【面试之前必看】
    1.回溯算法

【位运算】
15.二进制中的1的个数
    1.n&1 n>>>1 2.n&=(n-1)
64.求1+2+…+n
    1.&&判断

【字符串】
50.第一个只出现一次的字符  ★
    1.hash思想
58 - I. 翻转单词顺序
    1.字符串  先分隔  在逆序拼接
58 - II. 左旋转字符串
    1.先逆序前半部分  在逆序后半部分  最后逆序整个。
67.把字符串转换成整数
    1.注意符号  以及溢出

【堆】
40.最小的k个数
    1.堆  TOP(K)问题
41.数据流中的中位数
    1.大顶堆  和  小顶堆  结合使用

【排序】

【dfs】
54.二叉搜索树的第K大节点
    1.左中右为有序的递增序列  右中左就位递减序列  --k   + dfs就可以
```

力扣

数组 ☆

(排序/双指针/三指针/扫描方法(左->右、右->左))

add O(n) dele O(n) find O(1)

- 88.合并两个有序数组 [注意m是nums1的数据的长度 不是整个数组的长度] ☆☆☆
  - 1.合并后排序
  - 2.双指针/从前往后
  - **3.双指针/从后往前** 注意判断条件 p2 p1 p
- 75.颜色分类
- 16.16 部分排序

- [ ] **四数之和-454**

  a+b -(c+d)

- [ ]

链表 ☆

(虚拟头结点/双指针/快慢指针/翻转/中间节点)

- [ ] **找出两个链表的头结点-160**

  剑指offer原题 两个指针 先走自己的 走完在走对方 直到相遇

- [ ] **链表反转-206** ☆☆☆

  一个临时节点 pre 一个当前节点cur 一个下一个节点next 不断交换

- [ ] **归并两个有序的链表-21** ☆☆☆

  如果有一个链表为null，返回对方。否则谁小 递归谁

- [ ] **从有序链表中删除重复节点-83** ☆☆☆

  如果当前节点为null 或者 下一个节点为null 直接返回head，然后 判断下一个节点和当前节点是否相等 做逻辑处理

- [ ] **删除链表的倒数第N个节点-19** ☆☆☆

  快慢指针 + 先走k步 `slow.next = slow.next.next`

- [ ] **两两交换链表中的节点-24**

  存储临时节点 交换 连接旧链表 1.递归。 2.迭代法 3.

- [ ] **链表求和-445**

  两个栈 分别存储l1和l2的数据，pop 进位。 链表在while的时候进行构建。 **头插法**

- [ ] **回文链表-234** ☆☆☆

  借助栈 将链表全部入栈 依次弹出 和 head节点依次比较，如果不相同 返回false

- [ ] **分隔链表-725**

  遍历链表 sum % / 拿到应该放多少个 每次mod--

- [ ] **奇偶链表-328** ☆☆☆

  奇指针 偶指针 头指针 连接

- [ ] **链表有环- 141** ☆☆☆☆☆

  a.快慢指针 b.哈希表存储

- [ ] **K 个一组翻转链表** ☆☆☆

- [ ] **237.删除链表中的节点**

  1、要删除节点的值修改成后一个节点的值,删除后一个节点

- [ ] **23.合并K个排序链表**

- [ ] **25.K 个一组翻转链表** ☆☆☆
- [ ] **2.两数相加**
- [ ] **142. 环形链表 II**
- [ ] **237. 删除链表中的节点**
- [ ] **445. 两数相加 II**
- [ ] **6.从尾到头打印链表** 1.栈 2.递归 3.头插法
- [ ] **18.删除链表的节点** 1.迭代查找 2.快慢指针删除法 pre.next = fast.next
- [ ] **22.链表中倒数第K个节点** ☆☆ 1.快慢指针 快指针走k步
- [ ] **24.反转链表** ☆☆ 1.背下来
- [ ] **25.合并两个排序的链表** ☆☆ 1.前置pre + 遍历 l1 l2
- [ ] **35.复杂链表的复制** 1.hashMap存储Node 在依次遍历查找next 和 randomNode
- [ ] **52.两个链表的公共节点** ☆☆☆ 1.两个指针 谁先走完 走对方的 直到相遇。
- [ ] **合并两个链表-1699**

  保存a之前的值，以及b之后的链表，将链表插入到这个中间

栈和队列☆

- [ ] **栈实现队列-232** ☆☆☆

  in out in 进行push out进行pop 没有的话 从in中拿 注意不能in添加一部分数据 pop() 会有数据顺序问题
- [ ] **队列实现栈-225** ☆☆☆

  queue来解决 `while(size -- > 1) queue.add(queue.poll)`
- [ ] **最小栈-155** ☆☆☆

  一个数据栈 一个min minValue 记录每次push的最小值 pop时 注意min的为null
- [ ] **有效的括号-20** ☆☆☆

  将[({ 加入栈中，一对一对消除 如果栈为null 在遍历过程中 说明有剩余元素，直接返回false
- [ ] **每日温度-739** ☆☆☆

  单调栈
- [ ] **下一个元素更大元素-503**

  单调栈 + 循环 %
- [ ] **滑动窗口最大值-239** ☆☆☆
- [ ] **最大二叉树-654**
- [ ] **9.用两个栈实现队列** ☆☆☆ 1.in out 两个栈 删除的三种情况
- [ ] **30.包含min函数的栈** ☆☆☆ 1.data 和 min栈 保证minValue的值的变化
- [ ] **59.队列的最大值** 1.两个双端队列 一个存储数据 一个存储最大值。
- [ ] **59.2 滑动窗口最大值** 1.双端队列
- [ ] **502. IPO**
- [ ] **柱状图中最大的矩阵-84**

树 ☆☆☆

递归

递归

- [ ] **二叉树的最大深度-104** ☆☆☆

  return max(maxPath(r.left),maxPath(r.right))+1;

- [ ] **平衡二叉树-110** ☆☆☆

  在遍历树的过程中，分别计算左子节点和右子节点之差。如果超过1 说明不平衡

- [ ] **543.二叉树的直径**

  类似上面 只不过记录在遍历中一个最大值就可以。max

- [ ] **翻转二叉树-226**

  遍历的过程中 将左右子树的节点交换 第三方变量

- [ ] **617.合并二叉树** ☆☆☆

  递归过程中，合并r1.val+=r2.val; 因为是节点的赋值相加操作，所以不需要进行节点的转换。**【考点：根左右变形】**

- [ ] **判断路径和是否等于一个数-112** ☆☆☆

  递归调用，当节点左右子节点没有的时候，并且当前节点的值等于最后的sum ,因为需要递归求解左子树。【考点：存在左子树 或者 右子树上】

- [ ] **路径总和-437**

  递归统计，注意需要创建一个pathCount，头结点调用pathCount() 而子节点需要调用pathSum()

- [ ] **子树-572**

  树的子树也是同样，只不过需要加一个条件 因为不能保证从根节点开始，所以需要从左子树和右子树以及根节点开始遍历

- [ ] **树的对称-101** ☆☆☆

  在递归中只需要考虑那种情况可能出现不为子对称，a.如果都不为null 那么一定是遍历到节点了，b.如果有一个为null，为false 节点值不同 false 剩余情况继续递归 左子节点和右子节点 右子节点和左子节点 【判断哪些属于对称 哪些不属于对称】

- [ ] **二叉搜索树最小深度-111**

  考虑左子节点的最小深度和右子节点的最小深度，如果左右子节点有一个为null 那么返回l+r+1 否则返回最小者+1

- [ ] **左叶子之和-404**

  左叶子节点的条件 需要当前节点的左右子节点都为null 才可以，所以递归求解左右子树。

- [ ] **最长同值路径-687**

  最长路径无非就是三种情况 a.左子树中 b.右子树中 c.整个树中

- [ ] **二叉树中第二小的节点-671**

  我们来考虑一下，第二小的条件，只要出现一个不等于root.val的值，一定找到了次小者。如果左子节点为null，因为题目给出的条件是0个子节点，或者2个子节点。那么当前是不存在次小者的。否则话 遍历求出左子树和右子树最小者

层序遍历

层序遍历一般都是借助于队列来实现。类似于水波纹一样的效果。

- [ ] **二叉树的层平均值-637**

层序遍历一般都是借助于队列来进行先进先出，而我们只需要记录每一层的综合除以当前的个数就可以。

- [ ] **找树左下角的值-513**

  层序遍历无非就是队列，左下角的节点，我们可以在插入队列中元素顺序改变一下，只需要先插入右子节点，在插入左子节点。

前中后序遍历

> 树的遍历对于递归比较简单，对于非递归模式，要熟练使用栈来解决。判断好左右节点的压入栈的顺序。

- [ ] **非递归实现二叉树的前序遍历-144** ☆☆☆

  借助栈 先压入右节点 在压入左节点

- [ ] **非递归实现二叉树的后续遍历-145** ☆☆☆

  前序遍历是根左右，后续遍历是左右根 根右左 就是后序遍历的反转版本，我们只需要改变一下前序遍历反转一下就可以了

- [ ] **非递归实现二叉树的中序遍历-94** ☆☆☆

  中序遍历，我们需要先将二叉树中的最左节点添加到栈中，等最左节点为null,返回节点，弹出的一定是最左节点，中序遍历是左中右，所以根据将左节点的值添加到list中，而继续看当前这个最左节点是否有右节点，如果有将有节点的左节点添加到栈中。注意while条件`while(cur!=null || !stack.isEmpty())` 因为当节点不为null，或者栈中有元素的时候，都还需要执行。

- [ ] **590.N叉数的后序遍历**
- [ ] **589.N叉树的前序遍历**
- [ ] **429.N叉数的层序遍历**

BST

> 遇见BST问题，一定要将问题靠向根节点的所有左子树小于根节点的值，根节点的所有右子树大于根节点的值
> 解题套路 一般都是递归

- [ ] **修建二叉搜索树-669**

  考虑问题，先从最顶端思考问题。先考虑当前值节点是否大于最大值，就选择左子树 小于最小值就选择右子树，然后求出左子树 右子树。

- [ ] **二叉搜索树中第K小的元素-230**

  中序遍历，记录当前的第几个元素

- [ ] **把二叉搜索树转换为累加树-538**

  中序遍历改版，因为较大的一定在右边，所以先递归右节点，在遍历左节点，在递归过程中，计算较大值的累加和。

- [ ] **二叉查找树的最近公共最先-235** ☆☆☆

  判断当前节点在左子树还是右子树中

- [ ] **二叉树的最近公共祖先-236** ☆☆☆

  递归求解左右子节点 当`root==null || root==p || root == q`返回`root`

- [ ] **将有序数组转换为二叉搜索树-108**

  递归求解 二分 分别构造`root.left root.right`

- [ ] **有序链表转换二叉搜索树-109**

分治+快慢指针(找到中间节点)

- [ ] **两数之和-653**

  中序遍历+二分查找

- [ ] **二叉搜索树的最小绝对差-530**

  中序遍历+记录前一个节点+记录最小值

- [ ] **二叉搜索树中的众数-501**

  中序遍历+a.如果当前前值节点不为null ，并且当前前值节点和当前节点值相等 cur++ 否则 cur=1。如果当前cur大于max，则max修改成cur，并将当前集合中的数 删除，添加当前节点，否则 如果cur==max ，直接添加当前节点，保存当前节点为前置节点。

- [ ] **222.完全二叉树的节点个数**
- [ ] 101.对称二叉树 ☆☆☆
- [ ] **124.二叉树的最大路径和** ☆☆☆
- [ ] **114.二叉树展开为链表**
- [ ] **100.相同的树**
- [ ] **99.恢复二叉搜索树**
- [ ] **验证二叉搜索树-98**
- [ ] **从前序与中序遍历序列构造二叉树-105**
- [ ] **7.重建二叉树** ☆☆☆ 1.前序+中序 递归解决
- [ ] **26.树的子结构** 1.递归调用 recur
- [ ] **27.二叉树的镜像** ☆☆☆ 1.递归 2.栈结构
- [ ] **28.对称的二叉树** ☆☆ 1.递归 判断左右子节点
- [ ] **32-1 从上到下打印二叉树-左右顺序** ☆☆☆ 1,队列 和下面的类似
- [ ] **32-2 从上到下打印二叉树-层序遍历** ☆☆☆ 1.队列 2.递归
- [ ] **32-3 从上到下打印二叉树-左右+逆序** ☆☆☆ 1.队列 + flag 逆序标志
- [ ] **37.序列化二叉树** ☆☆☆ 1.序列化 二叉树-》字符串 2.反序列化 字符串-》二叉树
- [ ] **55-1 二叉树的深度** ☆☆☆ 1.dfs
- [ ] **55-2 平衡二叉树** ☆☆☆ 1.dfs flag标志
- [ ] **68-1 二叉搜索树的最近公共祖先** ☆☆☆ 1.根据BST的特点 DFS
- [ ] **68-2 二叉树的最近公共祖先** ☆☆☆ 1.dfs 递归左右子节点
- [ ] **二叉树的右视图-199** ☆☆☆

  队列 size

Trie

> 解决的问题点是什么呢？
>
> 查找一个字符串的是否存在或字符串前缀是否存在

- [ ] **实现一个Trie-208**

  Trie主要内部封装一个Trie数组，大小为26，将存储的字母-'a' 存入Trie数组中。类似一个Trie数是一个26个Tire的数组。添加 如果当前为null new Trie()，查找需要查找到最后，前缀的话 只要出现为null，就说明没有找到。否则找到了。

- [ ] **单词搜索-2 -212**

利用Trie来实现查找，加上回溯算法 不断查找 排除不符合条件的字符串。注意边界条件

哈希表 ☆

- [ ] **两数之和-1** ☆☆

  hashmap

- [ ] **存在重复元素-217**

  hashset

- [ ] **最长和谐序列-594**

  hashmap存储 num 和 num 对应出现的个数 当num+1 出现 计算当前num+num+1的总个数

- [ ] **最长连续序列-128**

  hashset 记录数组元素， 遍历 nums 当出现num-1 len++ num+1 len++ 同时判断当前集合中是否存在有这个元素 不存在直接continue

- [ ] **字母异位词分组-49**

字符串 ☆

- [ ] **有效的字母异位词-242**

  nums记录字符的个数 一个加 一个减 判断是否等于0

- [ ] **最长回文串-409**

  分为奇数和偶数字符，偶数直接进行计算，如果最终结果小于字符串长度，len++

- [ ] **同构字符串-205**

  数组 记录出现的个数

- [ ] **回文字符串-647**

  奇数和偶数之和，whle 判断是否相等

- [ ] **回文数-9**

  right 和 x 平分 最后判断注意特殊情况

- [ ] **计算二进制子串696**

  a b cnt 分别计算

- [ ] **字符串转换整数-8**
- [ ] **罗马数字转整数-13**
- [ ] **最长公共前缀-14**
- [ ] **回文对-336**
- [ ] **复原IP地址-93**
- [ ] **转换成小写字母**
- [ ] **最后一个单词的长度-58**
- [ ] **字符串中第一个唯一字符-387**
- [ ] **反转字符串-344**

数组和矩阵 ☆

- [ ] **删除排序数组中的重复项-26** ☆☆☆

双指针 i 指向开始的位置0 j=1 只要nums[i] != nums[j] 就添加到数组i位置。

- [ ] **移动零-283**

  cnt 非零移动到前面 后面补上0 b.过雪球解法

- [ ] **重塑矩阵-566**

  定义一个[r] [c] [index/n] [index%n] 确定行列号数据

- [ ] **最大连续1的个数-485**

  cur max 分别记录当前的值

- [ ] **搜索二维矩阵-2-240**

  行列 计算 剑指offer-4

- [ ] **有序矩阵的第K小的元素-378**

  二分查找 堆

- [ ] **错误的集合-645**

  通过交换元素 是元素在应有的位置上

- [ ] **寻找重复数-287**

  快慢指针

- [ ] **优美的队列-667**
- [ ] 数组的度-697
- [ ] **托普利茨矩阵-766**

  行 列 加1 解决

- [ ] **数组嵌套-565**

  for for 记录nums[j] = -1 交换 j = t; cnt max

- [ ] 分隔数组-769
- [ ] 3.数组中重复的数字
  1.hashSet 2.数组下标法(鸽巢机制)
- [ ] 4.二维数组中的查找
  1.暴力 2.线性查找 从右上角查找 小于列-- 大于行++
- [ ] 5.替换空格 1.stringbuffer
- [ ] 21.调整数组顺序使奇数位于偶数前面
  1.双指针 2.先分开 在合并
- [ ] 29.顺时针打印矩阵 左到右 上到下 右到左 下到上
- [ ] 39．数组中出现次数超过一半的数字 ☆☆☆ 投票法
- [ ] 62.圆圈中最后剩下的数字 ans = (ans+m)%i

图

二分图

- [ ] 判断是否为二分图-785

拓扑排序

- [ ] 课程安排的合法性-207
- [ ] 课程安排的顺序-210

并查集

- [ ] 冗余连接-684
- [ ] 208.实现Trie前缀树
- [ ] 547-朋友圈

位运算 ☆

- **汉明距离-461** ☆☆☆

  亦或 + 计算1的值

- **只出现一次的数字-136** ☆☆☆

  遍历一遍 亦或就可以

- **丢失的数字-268**

  x ^ i ^ nums[i] x ^ nums.length

- 数组中不重复的两个元素-260

- **颠倒二进制位-190**

  ans << 1 + n&1 n>>>0

- **不用额外变量交换两个整数**

  `a = a ^ b b = a ^ b a = a ^ b`

- **2的n次方-231** ☆☆☆

  `n > 0 && (n&(n-1)) == 0`

- **4的n次方-342**

  `n > 0 && ((n & (n-1)) == 0) && (n & 0xaaaaaaaa) == 0`

- **交替位二进制数-693**

  `int ans = n ^ (n >> 1); return (ans & (ans + 1)) == 0;`

- **两整数之和-371**

  `low = a ^ b; carry = a & b; a = low; if (carry == 0) break; b = carry << 1;`

- **比特位计数-338**

  `nums[i] = nums[i & (i-1)] + 1;` 取前一个元素的1 110 取10的基础上加1 就可以了。

- **位1的个数-191** ☆☆☆

-

双指针 ☆

双指针主要用于遍历数组，两个指针指向不同的元素，从而协同完成任务。

- [ ] **有序数组的Tow Sum-167** ☆

  一个指向最小一个指向最大 如果sum>target j-- sum<target i++ 否则就找到

- [ ] **平方数之和-633** ☆☆☆

  i*i + j * j = 判断当前是否等于 j需要开方一下，为什么呢 因为当等于c的时候一半以上一定超出范围

- [ ] **反转字符串中的元音字符-345** ☆☆☆

  如果不是元音字符 i ++ j-- 否则 两者交换 i ++ j-- 操作

- [ ] **回文字符串-680** ☆☆☆

  策略 删除第一个或者最后一个 如果都不相等 则不能成为回文字符串

- [ ] **归并两个有序数组-88**

  i ,j , k 从后往前移动

- [ ] **判断链表是否有环-41**

  双指针

- [ ] **最长子序列-524** ☆☆☆

排序

- Kth Element-215

桶排序

- 出现频率最多的K个元素-347
- 按照字符出现次数对字符串排序-451

荷兰国旗问题

- [ ] **按颜色进行排序-75**

  one two three

二分 ☆

二分查找的前提 1.目标函数单调性（单调递增或者递减） 2.存在上下界（bounded） 3.能够通过索引访问（index accessible）

- [ ] **求开方-69** ☆☆☆

  l <= r sqrt = x/m; r=m-1 l=m+1;

- [ ] **寻找比目标字母大的最小字母-744**

  类比找重复数字 l<=r l=m+1 r=m-1

- [ ] **有序数组的Single Element -540**

  l<r 保证m是偶数 否则m-- 如果nums[i]==nums[i+1] l=m+2 否则 r =m; 返回l

- [ ] **第一个错误的版本-278**

  通过判断错误版本的位置 如果当前有说明在 l m之间 r=m 否则的话 就在m r 之间 l=m+1

- [ ] **旋转数组的最小数字-153** ☆☆☆

  通过在区间判断 如果nums[m] > nums[r] 那么一定再后边 l = m+1 否则的话就在前边 r = m

- [ ] **查找区间-34**

  通过查找target 和target+1 位置 来查找target区间

- [ ] **搜索旋转排序数组-33** ☆☆☆

  通过判断是在前一个 还是以一个

- [ ] **有效的完全平方数-367**

  二分

- [ ] **搜索二维矩阵-74** ☆☆☆

  类比一维数组，控制好边界条件， 行=mid/clo 列 mid%clo

- [ ] **寻找峰值-162**

二分

- [ ] **有序矩阵中第K小的元素-378**

一个升序数组，找目标数。（递归和非递归的二分） `todo`

- [ ] **有效的完全平方数-367**

  二分即可， long 类型 mid x int case过不去

- [ ] **11.旋转数组的最小数字** 1.二分
- [ ] **53 - I. 在排序数组中查找数字 I** 1 .二分
- [ ] **53 - II. 0～n-1中缺失的数字** 1 .二分
- [ ] **57.和为S的两个数字** 1.二分+双指针

分治

- [ ] 给表达式加括号-241
- [ ] 不同的二叉搜索树-95

搜索

BFS

- [ ] 组成整数的最小平方数数量-279
- [ ] 最短单词路径-127

DFS

- 查找最大的连通面积-695
- [ ] **岛屿数目-200** ☆

  染色 四连通

- [ ] 好友关系的连通分量数目-547
- [ ] **被围绕的区域-130** ☆

  将边界的O设置为# 然后 最后将#设置成O，剩余的O就是X中的O

  forfor forfor

- 能到达的太平洋和大西洋的区域-417

回溯

- 电话号码的字母组合-17
- Ip地址划分-93
- **单词搜索-79**
- 输出二叉树中所有从根到叶子节点的路径 -257
- **排列-46** ☆
- 含有相同元素求排列-47
- 组合-77
- 组合求和-39
- 含有相同元素的组合求和-40
- 1-9数字的组合求和 -216
- **子集-78** ☆☆☆
- 含有相同元素的求子集-90
- 分隔字符串使得每个部分都是回文树-131

- 数独-37
- **N皇后-51** ☆☆☆
- **组合综合-39**
- **至少有K个重复字符的最长子串-395**

贪心算法 ☆

贪心算法是一种在每一步选择中都采取在当前状态下最好或最优的选择，从而希望全局是一个最优的解。

- [ ] **分配饼干-455**

  贪心策略：sort后，只要保证用最小的去满足最小的孩子胃口，就可以出现局部最优解，->全局最优解。

- [ ] **无重叠区间-435**

  贪心策略：sort 先计算出当前多少重叠的，总数减去不重叠的 就是不重叠的空间

- [ ] **用最少的箭头射爆气球-452**

  贪心策略：sort 只要当前气球的范围在上一个气球的范围就直接continue 否则的话 cnt++ 进行交换

- [ ] **根据身高和序号重组队列-406**

  按照升高身高升序，序号降序排列

- [ ] **种植花朵-605**

  只有当前后都是0 才可以种花

- [ ] **判断是否为子序列-392**

  直接借助IndexOf() 查看当前元素是否存在过

- [ ] **子数组最大的和-53** ☆☆☆

  直接max记录最大值 sum 记录当前最大值

- [ ] **跳跃游戏1-55**

  从后往前推 nums[i]+i > pos 就可以移动

- [ ] **跳跃游戏2-45**

  从前往后推 尽力找到最大的步数

- [ ] **柠檬水找零-860**

  问题细化 只需要考虑 five ten 之间的关系

动态规划 ☆

(贪心算法与动态规划的不同在于他对每个子问题的解决方案都能做出选择，不能回退，动态规划则会保

- 三步dp 【状态 选择】
  - 1.寻找重复子问题
  - 2.dp方程 状态定义 `一维dp` `二维dp` `常量dp`
  - 3.状态转移 选择或不选
- 递归->记忆化递归->dp 二维dp->一维dp->常量dp 【常规套路】
- **无后效性**
  - 状态下决策的收益，只与状态和决策相关，与到达该状态的方式无关。
- 分治是自顶向下解决问题，dp是自底向上解决问题

树形dp 存以前的运算结构。并根据以前的结果对当前进行选择，有回退功能)

斐波那契数列

- [ ] **爬楼梯**

  1.递归 2.记忆化递归 3.dp 4.常量dp

- [ ] **强盗抢劫-1-198**

  状态定义： dp[i] i天抢劫到的钱数

  状态转移：抢劫的策略是 dp[i] = Math.max(dp[i-2]+nums[i],dp[i-1]);

  优化

  - 二维dp->一维dp -> 常量dp

- [ ] **强盗抢劫-2-213**

  只需要考虑是抢劫第一个 还是抢劫最后一个。通过求出两者最大值。

  递归调用抢劫1 max((0,n-2) (2,-n-1))

- [ ] **强盗抢劫-3-337**

  只需要考虑爷孙 与 子 节点的最大和即可。

```
int [] l = recur(root.left);//左节点的打劫情况
int [] r = recur(root.right);//右节点的打劫情况
res[0] = Math.max(l[0],l[1])+Math.max(r[0],r[1]);
res[1] = l[0]+r[0]+root.val;//只打劫当前节点
```

优化【递归->记忆化->树形dp(一维)】

矩阵路径和

- [ ] **矩阵的最小路径和-64**

  从上 或者 从左 dp[j] = min(dp[j],dp[j-1]); dp[j]+=gird-i-j;

- [ ] **不同路径-62**

  状态定义：dp[i] [j] i j 位置的步数

  状态转移：填充1 dp[j] = dp[j] + dp[j-1];

  优化【二维dp 一维dp】

数组区间

- [ ] **数组区间和-303**

  初始化计算sum[i] = sum[i-1]+nums[i-1] 返回sum[j+1]-sum[i]

- [ ] **数组中等差递增子区间的个数-413**

  当A[i]-A[i-1] == A[i-1]-A[i-2]时 dp[i] = dp[i-1]+1; 最后计算dp之和 cnt

- [ ] **乘积最大子数组-152** ☆

  如果min max res 出现负数 min <-> max 交换

分隔数组

- [ ] **整数拆分-343**

  状态定义： dp[i] 当前的次数

  状态数组： dp[i] = max(dp[i],max(j*dp[i-j],j * (i-j))); //max(i,f(n-i),i*(n-i))

- [ ] **完全平方数-279**

状态定义：dp[i] 几个数字

dp[i] = min(dp[i],dp[i-j*j]+1);

- [ ] **解码方法-91**

  不同策略进行dp dp[i]+=dp[i-1] dp[i]+=dp[i-2]

最长递增子序列

- [ ] **最长递增子序列-300** ☆☆☆

  状态定义：dp[i] i位置 最长上升子序列的长度

  状态转移：dp[i] = max(dp[i],dp[j]+1); 记录max

- [ ] **一组整数对能够构成的最长链-646**

  状态定义：dp[i] i位置最长长度

  状态转移：当满足pairs[j] [1] < pairs [i] [0] dp[i] = max(dp[i],dp[j]+1);

- [ ] **最长摆动子序列-376**

  状态定义 up down 分别计算依次递推+1

最长公共子序列

- [ ] **最长公共子序列-1143**

  状态定义 dp[i] [j]

  状态转移方程 a.相等 dp[i] [j] = dp[i-1] [j-1] b.不等 dp[i] [j] = max(dp[i-1] [j],dp[i] [j-1]);

01背包

- [ ] **划分数组为和相等的两部分-416**

  状态定义：dp[i] 当前位置是否可以分割

  状态转移：dp[i] = dp[i] || dp[i-num]

- [ ] **目标和-494**

  dp[i] = dp[i] + dp[i-num];

- [ ] **01字符构成最多的字符串-474**

  dp[i] [j] = max(dp[i] [j], dp[i-zero] [j-ones] +1);

- [ ] **找零钱-322**

  dp[i] = Math.min(dp[i],dp[i-coins[j]]+1);

- [ ] **找零钱的硬币数组合-518**
- [ ] **字符串按单词列表分隔-139**
- [ ] **组合综合-377**

股票交易

- [ ] **需要冷却期的股票交易-309**

  状态定义 dp[i] [j] a. j=0 不持股 b.j=1 持股 c.j=2 冻结期

  转移方程 a.持股状态 b.不持股状态 c.冻结期(从不持股状态转移过来)

  优化 【二维dp -> 常量dp】

- [ ] **需要交易费用的股票交易-714**

  状态定义 dp[i] dp[0] dp[1] 当前是否有股票

优化 【一维dp -> 常量 dp】

- [ ] **只能进行两次的股票交易-123**

  状态定义 dp[n] [5] 定义五种状态 a.初始化 b.第一次买入 c.第一次买入 d.第二次买入 e.第二次卖出

  分别后一个状态依托于前一个状态 dp[i] [0] -> dp[i] [1] -> dp[i] [2] -> dp[i] [3] -> dp[i] [4]

  优化 【二维dp -> 常量dp】

- [ ] **只能进行k次的股票交易-188**

  记录每次交易的次数

- [ ] **只能进行1次的股票交易-121** ☆☆☆

  状态定义 dp[i] [j] i天的钱数+ j 是否持有股票 0->卖了

  转移方程

  dp[i] [0] = max(dp [i-1] [0], dp[i-1] [1]+ prices[i]) 当前没有 a.上一天也没有 b.上一天有 现在卖了

  dp[i] [1] = max(dp [i-1] [1], dp[i-1] [0] -prices[i]) 当前有 a.上一天有 b.上一天没有 现在买了一股

  优化 【二维dp -> 常量dp -> 记录最大差值 】

- [ ] **可以交易多次的股票交易-122**

  和进行1次交易一样

  优化 【常量dp -> 每次只要后一个大于前一个就交易 】

字符串编辑

- [ ] **删除两个字符串的字符使他们相等-583**

  类比成最长公共子序列

- [ ] **编辑距离-72**

  状态定义 dp[i] [j]

  状态转移 a.相等 dp[i] [j] = dp[i-1] [j-1] b.不等 min**(dp[i-1] [j],dp [i-1] [j-1], dp[i] [j-1])** +1

- [ ] **复制黏贴字符-650**

- [ ] 42.连续子数组的最大和 ☆☆☆ 1.dp 49.丑树 1.dp

数学

- [ ] **多数元素-169**

  投票法 `v += (p == num) ? 1 : -1;`

- [ ] **除自身以外数组的乘积-238**

  先计算右边 在计算左边 `res=k k*=nums[i] for res[i]*=k k*=nums[i]`

问题迭代

排序

不受初始元素影响的排序：选择，堆，归并。

类别	排序方法	时间复杂度			空间复杂度	稳定性
		平均复杂度	最好情况	最坏情况	辅助存储	
插入排序	直接插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	希尔排序	$O(n^{1.3})$	$O(n)$	$O(n^2)$	$O(1)$	不稳定
选择排序	直接选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	堆排序	$O(nlog_2n)$	$O(nlog_2n)$	$O(nlog_2n)$	$O(1)$	不稳定
交换排序	冒泡排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	快速排序	$O(nlog_2n)$	$O(nlog_2n)$	$O(n^2)$	$O(n)$或$O(log_2n)$	不稳定
归并排序		$O(nlog_2n)$	$O(nlog_2n)$	$O(nlog_2n)$	$O(n)$	稳定
基数排序		O(nd))	O(nd))	O(nd))	O(r+n)	稳定
n为数据规模，基数排序中r为基数，表示r进制，d为位数						

快排 ☆☆☆

- code看白板编程
- （时间复杂度，最好和平均都是 O(nlgn)，最差是 O(n*n),当数据几乎有序时是最差的，这是退为冒泡，空间复杂度 O(nlgn) **快速排序在无序时效率最高，有序时效率低。**
- 堆排序、基数排序、归并排序、选择排序的排序次数和初始化状态无关，即最好情况和最坏情况一样
- **快排改进：**
  - 1）和插入排序组合，由于快速排序在处理小规模数据时表现不好，因此在数据规模小到一定程度时，改用插入排序，具体小到何种程度，一般文章说 5-10.（SCISTL硅谷标准模块采用 10）
  - 2）中轴元素（左边比他小，右边比他大）可以取最左边、最右边、中间这三个位置的元素中的中间值。
  - 3）分成三堆，一方面避免相同元素，另一方面降低了子问题规模（一堆小于一堆等于一堆大于）。

跳表

- Skip list(跳表）是一种可以代替平衡树的数据结构，默认是按照Key值升序的。Skip list让已排序的数据分布在多层链表中，以0-1随机数决定一个数据的向上攀升与否，通过“空间来换取时间”的一个算法，**在每个节点中增加了向前的指针**，在插入、删除、查找时可以忽略一些不可能涉及到的结点，从而提高了效率。
- 实现
  - `ConcurrentSkipListMap`
- 思想 空间换时间
- 插入 查找 删除 时间复杂度都是O(logN)
- **构造过程**
  - 1、给定一个有序的链表。 2、选择连表中最大和最小的元素，然后从其他元素中按照一定算法随即选出一些元素，将这些元素组成有序链表。这个新的链表称为一层，原链表称为其下一层。 3、为刚选出的每个元素添加一个指针域，这个指针指向下一层中值同自己相等的元素。Top指针指向该层首元素 4、重复2、3步，直到不再能选择出除最大最小元素以外的元素。

- **为什么要随机节点出现的次数**
  - 新插入一个节点之后，就会打乱上下相邻两层链表上节点个数严格的2:1的对应关系。如果要维持这种对应关系，就必须把新插入的节点后面的所有节点（也包括新插入的节点）重新进行调整，这会让时间复杂度重新蜕化成O(n)。删除数据也有同样的问题。
  - skiplist为了避免这一问题，它不要求上下相邻两层链表之间的节点个数有严格的对应关系，而是为每个节点随机出一个层数(level)。比如，一个节点随机出的层数是3，那么就把它链入到第1层到第3层这三层链表中。

红黑树

- 红黑树是一种二叉查找树【BST树的强化：有自平衡特性】
  - 性质1：每个节点要么是黑色，要么是红色。
  - 性质2：根节点是黑色。
  - 性质3：每个叶子节点（NIL）是黑色。
  - 性质4：每个红色结点的两个子结点一定都是黑色。
  - **性质5：任意一结点到每个叶子结点的路径都包含数量相同的黑结点。**

布隆过滤器

- **功能**
  - **能够判断元素一定不存在**
- **基本操作**
  - (1)添加元素

    布隆过滤器的原理是，**当一个元素被加入集合时，通过K个hash函数将这个元素映射成一个位数组中的K个点，把它们置为1。**检索时，我们只要看看这些点是不是都是1就（大约）知道集合中有没有它了：如果这些点有任何一个0，则被检元素一定不在；如果都是1，则被检元素很可能在。这就是布隆过滤器的基本思想。

    <因为不同的key，可能经过K个hash函数之后产生的多个位下标是相同的[或者多个key构成了一个新的映射集合]，**导致误判**>
  - (2)判断元素是否存在

    当我们要判断一个元素是否在布隆过滤器中时，我们把这个值传入k个hash函数中获得映射的k个点。这一次我们确认下是否所有的点都被置为1了，如果有某一位没有置为1则这个元素**肯定不在集合中**。如果都在那这个元素就**有可能在集合中**。
- **工程实践**
  - Redis缓存
  - 垃圾邮件 评论

二叉树

- **AVL树特点可以说一下嘛？**
  - 任意一个结点的key，比它的所有左孩子key大，比它的所有右孩子key小；【同时具备二叉查找树的性质】
  - 任意结点的孩子结点之间高度差距最大为1；【为什么平衡】

  平衡二叉树【AVL树】是BST的一种优化。
- **完全二叉树和满二叉树的区别**
  - 比如一个树的高度是h，那么从1层到h-1层开始，每层节点个数都达到最大值，最后一层节点从左到右进行连接，就是完全二叉树。而满二叉树的节点必须满足，最后一层不能有空节点存在。

- 如何求解完全二叉树的深度？
  - 因为完全二叉树最底层从左到右是连续的，所以左边走到底就能获得最大高度【深度】。

并查集

B树/B+树

- B树和B+树的时间复杂度如下：

算法	平均	最差
空间	O(*n*)	O(*n*)
搜索	O(log *n*)	O(log *n*)
插入	O(log *n*)	O(log *n*)
删除	O(log *n*)	O(log *n*)

写在前面，好像不同的教材对b树，b-树的定义不一样。我就不纠结这个到底是叫b-树还是b-树了。

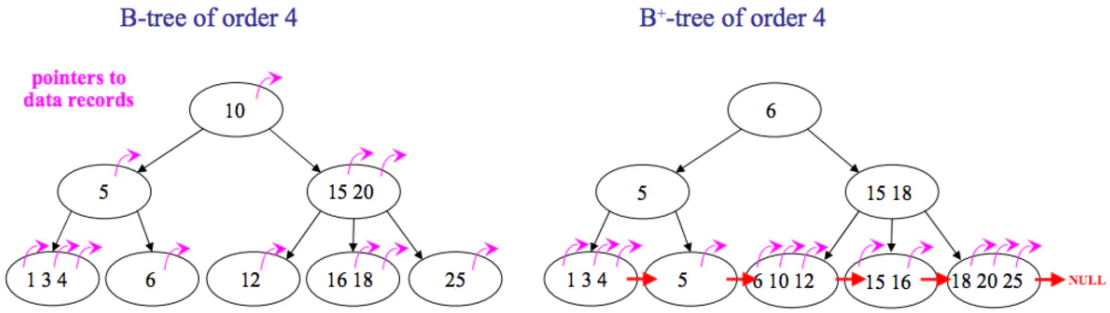


image.png

如图所示，区别有以下两点：

1. B+树中只有叶子节点会带有指向记录的指针，而B树则所有节点都带有，在内部节点出现的索引项不会再出现在叶子节点中。
2. B+树中所有叶子节点都是通过指针连接在一起，而B树不会。

B+树的优点：

1. **非叶子节点不会带上指向记录的指针，这样，一个块中可以容纳更多的索引项，**一是可以降低树的高度。二是一个内部节点可以定位更多的叶子节点。
2. 叶子节点之间通过指针来连接，范围扫描将十分简单，而对于B树来说，则需要在叶子节点和内部节点不停的往返移动。**具体的来讲，如何想扫描一次所有数据，对于b+树来说，可以从因为他们的叶子结点是连在一起的，所以可以横向的遍历过去。而对于b-树来说，就这能中序遍历了。**

B树的优点： 对于在内部节点的数据，可直接得到，不必根据叶子节点来定位。

B树长什么样子？

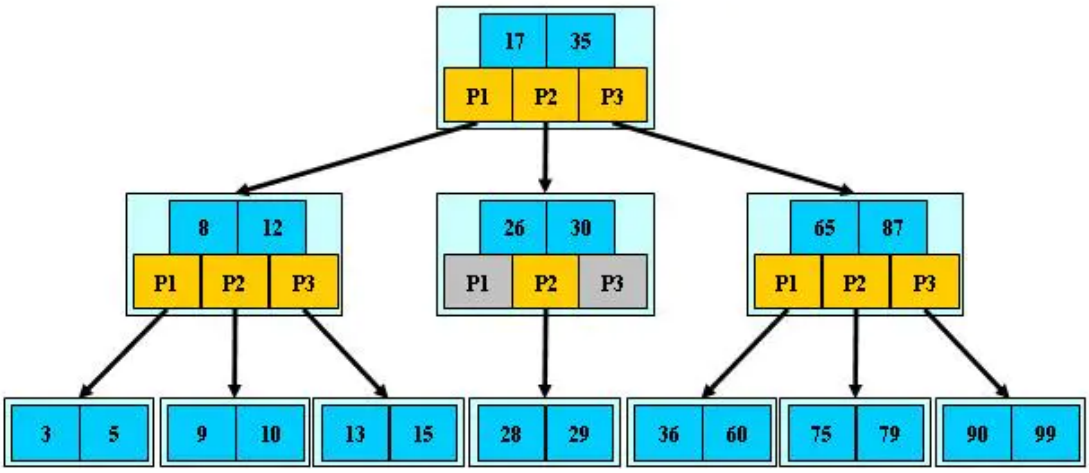


image.png

红黑树和B树应用场景有何不同？

为什么要设计红黑树？

先说一下红黑树，红黑树有一个比较复杂的规则，红的结点balala怎么样，黑的结点balalal怎么样。大一大二学这些的时候，傻呵呵的想背课文一样背下来，当也不知道为什么要设计成这样。换一句话说，为什么平衡树和红黑树的区别是什么？为什么有了平衡树还要设计出来红黑树？

红黑树的规则： 1）每个结点要么是红的，要么是黑的。 2）根结点是黑的。 3）每个叶结点（叶结点即指树尾端NIL指针或NULL结点）是黑的。 4）如果一个结点是红的，那么它的俩个儿子都是黑的。 5）对于任一结点而言，其到叶结点树尾端NIL指针的每一条路径都包含相同数目的黑结点。

现在想想，我的理解是**平衡树（AVL）**更平衡，结构上更加直观，时间效能针对读取而言更高，**但是维护起来比较麻烦！！！**（插入和删除之后，都需要rebalance）。但是，红黑树通过它规则的设定，确保了插入和删除的最坏的时间复杂度是O(log N) 。

设计红黑树的目的，**就是解决平衡树的维护起来比较麻烦的问题，红黑树，读取略逊于AVL，维护强于AVL，每次插入和删除的平均旋转次数应该是远小于平衡树。**

小结一下：

能用平衡树的地方，就可以用红黑树。用红黑树之后，读取略逊于AVL，维护强于AVL。

红黑树 和 b+树的用途有什么区别？

1. 红黑树多用在内部排序，即全放在内存中的，STL的map和set的内部实现就是红黑树。
2. B+树多用于外存上时，B+也被成为一个磁盘友好的数据结构。

为什么b+磁盘友好？

1. 磁盘读写代价更低 树的非叶子结点里面没有数据，这样索引比较小，可以放在一个blcok（或者尽可能少的blcok）里面。避免了树形结构不断的向下查找，然后磁盘不停的寻道，读数据。这样的设计，可以降低io的次数。
2. 查询效率更加稳定 非终结点并不是最终指向文件内容的结点，而只是叶子结点中关键字的索引。所以任何关键字的查找必须走一条从根结点到叶子结点的路。所有关键字查询的路径长度相同，导致每一个数据的查询效率相当。
3. 遍历所有的数据更方便 B+树只要遍历叶子节点就可以实现整棵树的遍历，而其他的树形结构要中序遍历才可以访问所有的数据。

图

- 拓扑排序

- 最小生成树

单调栈

单调栈——单调栈的维护是 O(n)的时间复杂度。所有元素只会进入栈一次，并且出栈后再也不会入栈。

性质：1）单调栈里元素有单调性；

2）元素加入栈前，会在栈顶端把破坏栈单调性的元素都删除；

3）使用单调栈可以找到元素向左遍历第一个比它小的元素，也就可以找到元素向左边遍历第一个比它大的元素。

Tire树

百度时，一输入“北京”，搜索框下面就会出现“北京爱情故事”，“北京公交”，“北京语言大学”等，实现这类技术用的数据结构是什么？答案：trie 树，又称为单词查找树，字典树，是一种树形结构，是一种哈希树的变种，是一种用于快速检索的多叉树结构。其核心思想是：空间换时间。利用字符串的公共前缀来降低查询时间的开销以达到提高效率的目的。Trie 树的性质：1）根节点不包含字符，除了根节点以外的每个节点包含一个字符 2）从根节点到某一个节点，路径上经过的字符连接起来，为该节点对应的字符串 3）每个节点的all 子节点包含的字符串不同。

辗转相除法

辗转相除法：用来求两个自然数的最大公约数（已知 a，b，c 为正数，若 a 除以 b 余数是 c，则[a,b]=[b,c]，其中[a,b]表示 a 和 b 的最大公约数）代码见 p14 反 时间复杂度 O(nlogn) a*b*=*最大公约数*最小公倍数

```
private static int gongyueshu (int a, int b) {
    int c = a % b;
    while (c != 0) {
        a = b;  b = c; c = a % b;
    }
    return b;
}
```

面试算法

- 给定一个字符串，请将字符串里的字符按照出现的频率降序排列 lc 451
- 快排的原理解释下，时间复杂度是多少，快排是稳定的吗，类似的稳定排序算法有哪些
- **二分算法，二分算法的具体使用场景？**
    - 基本思路就是通过前后指针进行查找中间元素target是否是目标元素。前提是数组，且元素必须是有序的。虽然跳表从某种程度上也是基于二分的思路，但是需要更多的空间存储next指针域。
    - 使用场景
    - 缺点 有序 数组。解决方案 上升维度，二叉树。

问题迭代

- **重新理解数据结构与算法**
    - 数据结构与算法

程序=数据结构+算法。

数据：在 java 的世界里面，数据，分为两种，基本数据类型，是由 java 编程语言内置的，对象数据类型， 可以又程序员自由定制。

结构：数据存储的结构

算法离不开数据，而数据离不开存储结构 算法必须基于数据，而数据是基于存储结构而存在的

算法必定会基于特定的数据结构来开展 而不同的数据结构，解决同一个问题的算法，必然不同

数据结构为算法提供了支撑，而好的算法，可能需要一种全新的数据结构

数组的缺陷如下：

1，大小固定；

2，删除一个元素之后，会形成一个空缺，让数组中的元素失去了彼此挨在一起的特性；

链表可以解决数组的问题

第一，链表的空间是无限的，第二，链表删除一个元素，永远不会出现空缺，他们依然紧紧地联系在一起

为了解决数据检索问题。出现了二叉树。

检索领域的王者 hash

根据哈希和树的这些特性，在数据库索引领域，也分为 **hash 索引**和b-tree 索引

hash 索引的查询效率是最高的，但是 hash 索引不支持范围查询

b-tree 索引的查询效率稍微慢一点，但是它支持的范围查询非常强大

具象数据类型

变量，数组，链表，二叉树，哈希表 他们有完全清晰的形态和结构

抽象的数据结构

在 java 领域，抽象数据结构说的是 List，Set，Map，Queue，Stack 这几个东西

0x01-Java篇 ☆

基础 ☆☆☆

字符串 ☆☆☆

- StringBuffer --> 线程安全 （使用 synchronized 关键字实现）
- StringBuilder --> 非线程安全
- 底层实现均为**可修改数组**（char, JDK 9 以后是byte数组）
- 第5讲 | String、StringBuffer、StringBuilder有什么区别？
- **可变性** String 不可变 StringBuffer 和StringBuilder是可变的。
    - **不可变的好处**
        - 1.因为是不可变的 所以才可以实现字符串常量池，不同的对象都指向了池中的一个对象。
        - 2.安全性
        - 3.适合做key
- **性能** String的每次修改都相当于创建了一个新的对象，性能比较低。而StringBuffer使用syn来保证线程安全性，性能优于String 但是不如StringBuilder。

- 直接原因：不能，被final修饰。
- **Intern()** 先去常量池中查找有 直接返回。没有直接在常量池中创建并返回。
- 注意点：Java 字符串在初始化的时候，已经在常量池中有了。
- String new的过程 2个对象
  - 常量池里
  - 堆
- **常量池主要用于存放两大类常量：**
  - 1）字面量、符号引用。字面量相当于 java 语言层面常量的概念，符号引用包括类和接口的全限定名，字段名称和描述名称，方法名称和描述符。
- **运行时常量池**有动态性，java 语言并不要常量一定只有在编译时产生，也就是并非预置入 class 文件中常量池的内容才能放入常量池，运行期间有新的常量也可放入池中，比如String 的 intern 方法。优：对象共享，节省内存空间，节省运行时间。
- VS
  - 从使用者的角度来看，String是不可变字符串，后两者是可变字符串。具体的String不可变通过final修饰。以及每次都会创建一个新的对象。buffer是线程安全的可变字符串，builder是非线程安全的可变字符串。一般来说，在非并发场景下，建议使用builder。buffer的线程安全内部采用了同步锁进行，在性能上有一定的逊色。当然了 builder在5之后才有的，buffer5之前就存在了。

根本原因：

（1）为了安全考虑，比如下面的例子，在将String参数传入函数之后不会发生改变，对比StringBuffer

（2）为了效率考虑：在大量使用字符串的时候，**指向的是常量池中同一个常量**，节省内存空间，提高效率

1.==既可以对基本数据类型比较 也可以对引用数据类型比较 而equals只能进行对象之间的比较

2.== 对于对象比较 是判断对象的地址是否相同 而对于基本数据类型比较的是值的大小

3. equals默认比较的是对象的地址 我们可以重写 自定义比较规则。

小知识点

equals和hashcode

1.对象的equals相等 则hashcode一定相同

2.hashcode相同 equals不一定相同

```
首先==与equals是有明显区别的。
==强调栈中的比较，可以理解为地址比较
equals强调对象的内容比较
String s=“hello”；会在栈中生成hello字符串，并存入字符串常量池中。
String t=“hello” ；创建时，会在字符串常量池中寻找，当找到需要的hello时，不进行字符串的创建，引用已有的。 所以，s==t返回true，s.equals(t)也是true。
char c[]={'h','e','l','l','o'}; c==s这个是不存在的，==两边类型不同
t.equals(c)这个语句在anObject instanceof String这步判断不会通过，也就是cha[] 压根不能与String相比较，类型不是相同的。返回false
```

OOP

- OOP的理解
  - **面向对象是一种思想，可以将复杂问题简单化**，主要体现就是**封装继承多态**。

- 多态 (**编译时期和运行时期** 前提是必须存在继承或者实现这样 程序在运行时期可以根据创建对象的类型 动态指定运行那个方法 使用场景的话 框架中，我们通过自定义一个抽象类，接口，自己实现一个子类，动态调用我们的自定义方法)
  - 1.**父类引用指向子类对象**
  - 2.**接口引用指向实现类对象**
- 封装
  - **将事物封装成一个类，减少耦合，隐藏细节**。保留特定的接口与外界联系。内部结构发生变化不会影响外部接口的调用方。
  - 具体体现：将属性私有化setter getter
- 继承**【本质就是一个鸡蛋模型】** private 只是不能访问，并不是并不继承。
  - 从一个已知类中派生出一个新的类，新类可以拥有已知类的行为和属性。覆盖和重写。
- 普通的类方法是可以和类名同名的，和构造方法唯一的区分就是，构造方法没有返回值。

- **重载和重写的区别**
  - 重写 发生在父子类中，方法名 参数等一致。 **运行期**
  - 重载 在一个类中 存在多个同名的方法 参数类型 顺序/个数不同 **编译期**
    - 小知识点 **不同的返回值类型不能决定重载了**，因为编译器并不能通过返回值类型的决定调用哪一个方法，存在歧义性。 **与方法的返回值类型与访问权限无关**
- **接口与抽象类的区别** ☆
  - 抽象类中可以没有抽象方法，但有抽象方法一定是抽象类
  - 抽象类是单继承 接口是多实现
  - 抽象类是公共属性 方法的抽取。而接口是定义一套规范。
    - **应用场景**
      - **当我们只定义一些行为方法的规范 而不需要具体的上层的抽象实现。推荐使用接口**
      - **除了定义一下基础的规范，还有一定的公共基础类的抽取，那么推荐使用抽象类**
  - **知识点拓展** 8之后 可以在接口中定义默认方法，当出现多个接口都定义了默认方法。我们的一般做法有两种，一是重写接口中的默认方法，二是在实现类中指定要使用哪个接口中的默认方法。

    为什么会出现默认方法？

    在面向抽象编程，由于之前定义的接口 需要修改，需要修改所有实现类的方法，为了给已经存在的接口增加新的方法并且不影响已有的实现，引入了默认方法。

- **equals 和 == 理解**
  - == 对于基本数据类型来说，比较的是数值是否相等，对于引用类型来说 比较的是对象的地址值。
  - equals 对于没有重写该方法的类 默认使用的object类的equals 比较的是地址，重写的话 一般都是比较内容是否相等。
- hashcode持此方法是为了提高哈希表（例如`java.util.Hashtable`提供的哈希表）的性能。

  对象的地址转换成的唯一数字

1.1 equals的作用

用来判断其他的对象是否和该对象相等。

```
public boolean equals(Object obj) {
    return (this == obj);
}
```

Override Equals方法时的规则：

- **自反性**：对于任何非空引用值 x，x.equals(x) 都应返回 true。
- **对称性**：对于任何非空引用值 x 和 y，当且仅当 y.equals(x) 返回 true 时，x.equals(y) 才应返回 true。
- **传递性**：对于任何非空引用值 x、y 和 z，如果 x.equals(y) 返回 true， 并且 y.equals(z) 返回 true，那么 x.equals(z) 应返回 true。
- **一致性**：对于任何非空引用值 x 和 y，多次调用 x.equals(y) 始终返回 true 或始终返回 false， 前提是对象上 equals 比较中所用的信息没有被修改。
- **非空性**：对于任何非空引用值 x，x.equals(null) 都应返回 false。

1.2 重写equals为什么要重写hashcode

java的集合有两类，一类是List，还有一类是Set。前者有序可重复，后者无序不重复。当我们在set中插入的时候怎么判断是否已经存在该

元素呢，可以通过equals方法。但是如果元素太多，用这样的方法就会比较满。

于是有人发明了哈希算法来提高集合中查找元素的效率。 这种方式将集合分成若干个存储区域，每个对象可以计算出一个哈希码，可以将

哈希码分组，每组分别对应某个存储区域，根据一个对象的哈希码就可以确定该对象应该存储的那个区域。

hashCode方法可以这样理解：它返回的就是根据对象的内存地址换算出的一个值。这样一来，当集合要添加新的元素时，先调用这个元素

的hashCode方法，就一下子能定位到它应该放置的物理位置上。如果这个位置上没有元素，它就可以直接存储在这个位置上，不用再进行

任何比较了；如果这个位置上已经有元素了，就调用它的equals方法与新元素进行比较，相同的话就不存了，不相同就散列其它的地址。这

样一来实际调用equals方法的次数就大大降低了，几乎只需要一两次。

为了保证使用Map接口时，“相同”对象的hashCode也是相同的

HashMap对象是根据其Key的hashCode来获取对应的Value。

在重写父类的equals方法时，也重写hashcode方法，**使相等的两个对象获取的HashCode也相等**，这样当此对象做Map类中的Key时，两个equals为true的对象其获取的value都是同一个，比较符合实际。

1.3 equals和hashCode的关系

一个好的hashCode的方法的目标：**为不相等的对象产生不相等的散列码**，同样的，相等的对象必须拥有相等的散列码。

如果两个对象equals，那么它们的hashCode必然相等，但是hashCode相等，equals不一定相等。

- **java为什么不支持多继承？**
  - 从技术实现的角度来看，降低编程的复杂度。
- 2.String.intern() 和常量池关系
- **3.Object相关方法**
  - hashcode
    - hashcode方法存在主要是提高判断对象是否相等的效率，当对象equals相等时，hashcode一定相等，hashcode相等 对象不一定相等。
  - equals
    - 自反性

        - 对称性
        - 传递性
        - 一致性
    - notify()和notifyAll()
        - notify方法随机唤醒一个 notifyAll()唤醒所有在等待的线程
- **Java中父类和子类初始化顺序（小红书）**
    - 优先级排序
        - 1. 父类中静态成员变量 **和** 静态代码块
        - 2. 子类中静态成员变量 **和** 静态代码块
        - 3. 父类中普通成员变量 **和** 代码块，父类的构造函数
        - 4. 子类中普通成员变量 **和** 代码块，子类的构造函数
        - (其中 “和” 字两端按照代码先后顺序执行)
        -
    - java中父类和子类初始化顺序
        - 被动引用：1）通过子类引用父类的静态字段，不会导致子类的初始化，即父类静态代码块执行，子类静态代码块不执行。2）通过数组定义来引用类，不会触发此类的初始化，

          `eg，SuperClass [] ss=new SuperClass[10]`

          3）常量在编译阶段会存入调用的类的常量池，本质上没有直接引用到定义常量的类，因此不会触发定义常量的类的初始化。
    - 1.成员变量：编译和运行都参考左边。

      2.成员函数（非静态）：编译看左边，运行看右边

      3.静态函数：编译和运行都看左边。
- **基础数据类型**
    - ①无论如何，Integer与new Integer不会相等。不会经历拆箱过程， ②两个都是非new出来的Integer，如果数在-128到127之间，则是true,否则为false java在编译Integer i2 = 128的时候,被翻译成-> Integer i2 = Integer.valueOf(128);而valueOf()函数会对-128到127之间的数进行缓存 ③两个都是new出来的,都为false ④int和integer(无论new否)比，都为true，因为会把Integer自动拆箱为int再去比
    - 包装类和基础类区别
        - 第一点的话，包装类提供了缓存功能，在-128/127之间有一个缓存区域，但是对于基础类来说，是没有的。第二点，包装类提供了更多封装的方法，可以基于这些方法做一些相关的操作。但是需要去权衡，本身类就占用的空间比较大，Int 空间更小。 第三点，初始化类型不同 一个是null 一个是0。
    - 1.装箱就是自动将基本数据类型转换为包装器类型（int-->Integer）；调用方法：Integer的 **valueOf(int) 方法**

      ```
      public static Integer valueOf(int i) {
          if (i >= IntegerCache.low && i <= IntegerCache.high)
              return IntegerCache.cache[i + (-IntegerCache.low)];
          return new Integer(i);
      }
      Integer对象内部维护了一个IntegerCache对象，当使用valueof对象时，先去这个缓存里看有没有，有的话直接从缓存中拿 没有的话 直接创建一个新的。
      ```

      拆箱就是自动将包装器类型转换为基本数据类型（Integer-->int）。调用方法：Integer的intValue方法

```
public int intValue() {
    return value;
}
```
直接返回value值

在Java SE5之前 需要手动 5以后有了自动装箱的特性。

```
static {
    // high value may be configured by property
    int h = 127;
    String integerCacheHighPropValue =

 sun.misc.VM.getSavedProperty("java.lang.Integer.IntegerCache.high");
    if (integerCacheHighPropValue != null) {
        try {
            int i = parseInt(integerCacheHighPropValue);
            i = Math.max(i, 127);
            // Maximum array size is Integer.MAX_VALUE
            h = Math.min(i, Integer.MAX_VALUE - (-low) -1);
        } catch( NumberFormatException nfe) {
            // If the property cannot be parsed into an int, ignore it.
        }
    }
    high = h;
    cache = new Integer[(high - low) + 1];
    int j = low;
    for(int k = 0; k < cache.length; k++)
        cache[k] = new Integer(j++);
    // range [-128, 127] must be interned (JLS7 5.1.7)
    assert IntegerCache.high >= 127;
}
```
直接看源码 可以看到Integer将 -128 - 127之间的值缓存起来了。所以这样的结果。

1.面向过程：

一种较早的编程思想，顾名思义就是该思想是**站着过程的角度思考问题**，强调的就是功能行为，**功能的执行过程**，即先后顺序，而每一个功能我们都使用函数（类似于方法）把这些步骤一步一步实现。使用的时候依次调用函数就可以了。

2.面向对象：

一种**基于面向过程的新编程思想**，顾名思义就是该思想是站在对象的角度思考问题，我们把多个功能合理放到不同对象里，强调的是具备某些功能的对象。

具备某种功能的实体，称为对象。面向对象最小的程序单元是：类。面向对象更加符合常规的思维方式，稳定性好，可重用性强，易于开发大型软件产品，有良好的可维护性。 在软件工程上，面向对象可以使**工程更加模块化，实现更低的耦合和更高的内聚。**

类是对象的抽象 对象是类的具体。类是对象的模板，对象是类的实例。

封装:通常我们在创建一个类的时候，会添加上方法或者属性的权限修饰符。但是public private方法的含义不得而至。private就是一个类私有的属性或者函数。举一个栗子 一台电脑 对于用户来说提供了一些公共的USB接口，用户使用USB接口可以很方便使用，但是电脑内部的构造是十分复杂的。而usb相当于是一个触发接口，插入U盘后 os会做一系列动作来完成，而这些复杂的事情就是电脑自己封装起来的。私有的，对外暴露给用户的才是公开的方法。 这就是封装的思想。

多态:通常定义一个父类用子类来继承父类，Person p = new Studet();这是属于父类引用指向子类的对象，但是interface的引用 从另一个层面体现出多态特定，定义一个Interface 定义规则，实现类实现接口中的抽象方法。

这也是多态的体现 接口引用指向实现类对象。也是面向接口编程最基本的思想。在框架设计中 我们通过都会定义一系列接口(接口中定义抽象方法，框架的调用基于接口中抽象方法的调用 具体实现类执行) 方法重载实现的是编译时的多态性，而方法重写实现的是运行时的多态性。

抽象是将一类对象的共同特征总结出来构造类的过程，包括**数据抽象**和**行为抽象。**

关键字

- ***static***
  - 在没有创建任何对象的前提下，仅仅通过类本身来调用 static 方法。这实际上正是 static 方法的主要用途
  - A Guide to the Static Keyword in Java
  - **内存角度来看** 静态成员变量存放在常量池中。加载时就放入常量池中，而非静态成员存放在堆中，局部变量存放在栈中。
  - **运行时期来看** 代码被编译成`.class`文件时，由static修饰的代码块 成员变量 方法被放入内存的方法区中，
- **super&this**
  - **基础概念** this是访问本类实例属性和方法 super是子类访问父类的属性或方法
  - **查找范围** this先查找本类，没有的话在查父类 super直接访问父类。
  - **使用** this单独使用时，代表本类 super在子类覆盖父类方法时，访问父类同名方法。
- ***final***
  - 被final修饰的类不能被继承
  - 被final修饰的方法不能被重写
  - 被final修饰的变量不能被修改
  - **变量被final修饰后不能再指向其他对象，但可以重写**
- ***finally***
  - try-catch-finally 中包含 return 的情况分析（字节跳动）
    - "finally块中的内容会先于try中的return语句执行，如果finall语句块中也有return语句的话，那么直接从finally中返回了"
  - “不要在 finally 代码块中使用 return 语句”（《码出高效》 5.2）
- **finalize**
  - Object中方法，子类可以覆盖父类 实现资源清理和垃圾回收之前调用此方法。
- **break**
  - 如何跳出当前的多重嵌套循环 break标记
- **switch**
  - 在Java7之前，switch只能支持 byte、short、char、int或者其对应的封装类以及Enum类型。在Java7中，也支持了String类型 String byte short int char Enum 类型

异常

- 1.**Throwable的Exception 和Error**
  - error是java程序运行时系统错误或者资源耗尽。
    - `stackOverFlowError`
    - `outOfMemoryError`
  - exception
    - **(编译时异常)受检异常** 需要在代码中显示处理，否则编译错误。
      - `IOException` `SqlException`
    - **(运行时异常)非受检异常** 运行时异常 继承自RuntimeException

        - `FileNotFoundException` `ClassNotFoundException` `IndexOutOfBoundsException` `ArrayStoreException`

    ps：异常作为java体系结构中一个重要的组成部分，对于程序中出现的错误可以部分规避，但是对于系统级别的错误是无能为力的。整个体系结构为Throwable 下的error 和 exception。而error是程序运行过程中出现的系统故障或者资源用尽。而exception是程序中可能抛出的异常，同来来说分为两部分。一类是非受检异常 一类受检异常。受检异常通常需要我们做出一定的显示代码的处理 比如try catch or thorws 而非受检异常是程序运行期间 抛出的比如空指针等。

- **2.try catch finally**
    - catch中异常的类型需要逐渐扩大，不能前面大于后面的异常类型。
    - 若 try，finally 都有 return，则忽略 try 的 return 语句。
    - **异常处理的基本原则**
        - 尽量不要捕获通用异常，捕获特定异常。有利于发现问题。
        - 不要忽略异常
        - 本模块不知道如何处理的时候，抛给上层模块进行业务处理。
        - 捕获异常后应该记录log日志。方便排查。
        - 不要使用trycatch捕获整块代码。
- **3.throw和throws的区别**
    - throw语句用在方法体内，表示抛出异常由方法体内语句处理，一定是抛出了异常。
    - thrwos语句在方法声明的后面，该方法的调用者要对异常进行处理，可能发生了异常，也可能没有发生异常。
    - 位置不同

      1.throws用在函数上，后面跟的是异常类，可以跟多个；而 throw用在函数内，后面跟的是异常对象。

      功能不同：

      2.throws用来声明异常，让调用者只知道该功能可能出现的问题，可以给出预先的处理方式；throw 抛出具体的问题对象，执行到throw，功能就已经结束了，跳转到调用者，并将具体的问题对象抛给调用者。也就是说throw 语句独立存在时，下面不要定义其他语句，因为执行不到。

      3.throws表示出现异常的一种可能性，并不一定会发生这些异常；throw则是抛出了异常，执行throw则一定抛出了某种异常对象。

      4.两者都是消极处理异常的方式，只是抛出或者可能抛出异常，但是不会由函数去处理异常，真正的处理异常由函数的上层调用处理。
- **4.NoClassDefFoundError 和 ClassNoFoundException 有什么区别？**
    - 1.一个是error级别 一个是exception级别
    - 2.classNodeFoundException是由class.forName方法动态加载，没有加载到，就抛出ClassNotFonudException异常。
    - 3.NoClassDefFoundError是在加载类的时候订阅不到，**通俗一点来说就是编译的时候可以找到，但是运行的时候找不到类。**
- **5.为什么说try catch 耗费资源呢？？**
    - **从虚拟机层面来说，构造异常实例需要生成该异常的栈轨迹，需要逐一访问当前线程的栈帧，并记录调试信息，包括栈帧所指向方法的名字，方法所在的类名，方法名，以及在代码中的第几行触发该异常信息等，这就是耗费资源的原因。**
- **6.为什么finally总是最后执行？？**
    - **编译器的作用，在编译代码时，会拷贝finally代码的内容，分别放在try-catch代码块中正常执行流程和异常执行流程的路径出口中，这样不管那个流程 都会执行finlly块。**
    - 1、不管有木有出现异常，finally块中代码都会执行；

```
public static void main(String[] args) {
    try {
        throw new RuntimeException();
    }catch (Exception e){
        e.printStackTrace();
    }finally {
        System.out.println("1");
    }
}
java.lang.RuntimeException
    at com.ncst.exception.Exception2.main(Exception2.java:12)
1
```

2、当try和catch中有return时finaly仍然会执行；

```
try {
    return;
}catch (Exception e){
    e.printStackTrace();
}finally {
    System.out.println("1");
}
1
```

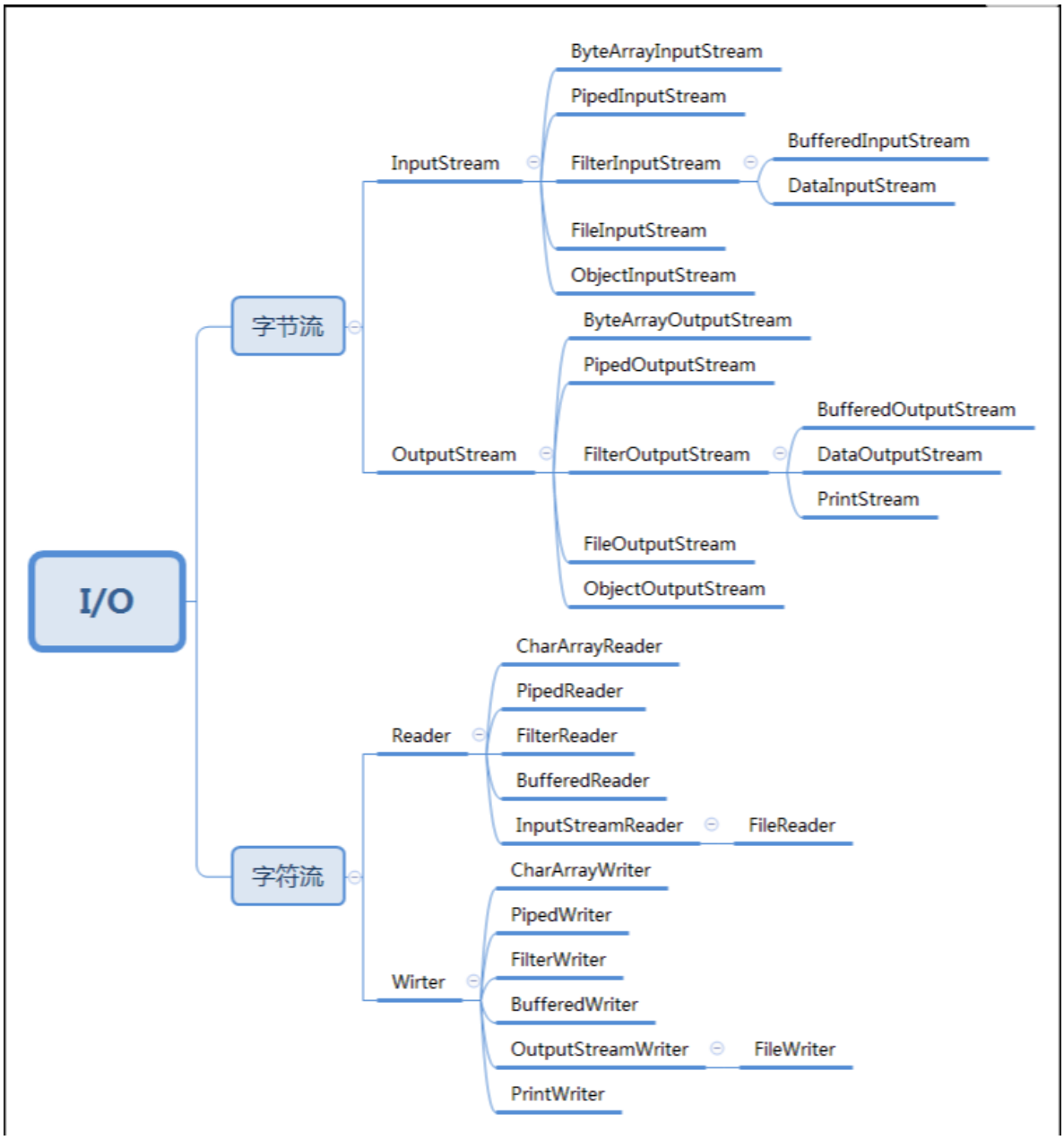
3、finally是在return后面的表达式运算后执行的（此时并没有返回运算后的值，而是先把要返回的值保存起来，管finally中的代码怎么样，返回的值都不会改变，任然是之前保存的值），所以函数返回值是在finally执行前确定的； 4、finally中最好不要包含return，否则程序会提前退出，返回值不是try或catch中保存的返回值。

```
private static int test() {
    try {
        return 1;
    }catch (Exception e){
        e.printStackTrace();
        return  2;
    }finally {
        System.out.println("1");
        return 3;
    }
}
```

- **重写理解异常机制？**
  - 很多人对于异常的理解就是程序中出现的bug,其实并不是，**只不过是异常是一种程序中的保护机制**。为了让程序更加健壮而做出的一种设计。举一个例子，我们程序中当出现一个null对象调用方法时，会抛出NullPointerException，从而程序出现**中断**，打印在控制台上，程序终止。生成异常报告。
  - a.要不自己内部解决 解决不了 b.抛给上级调用者
- **Exception和Error的区别？**

  Error 类一般指与虚拟机相关的问题，比如系统崩溃，虚拟机错误，内存空间不足，对于这种错误导致的应用程序中断，仅靠程序本身无法恢复和预防，遇到这样的错误，建议让程序终止。Exception 表示程序可以处理的异常，遇到这类异常，应该尽可能处理异常，使程序恢复运行，而不应该随意终止异常。

IO

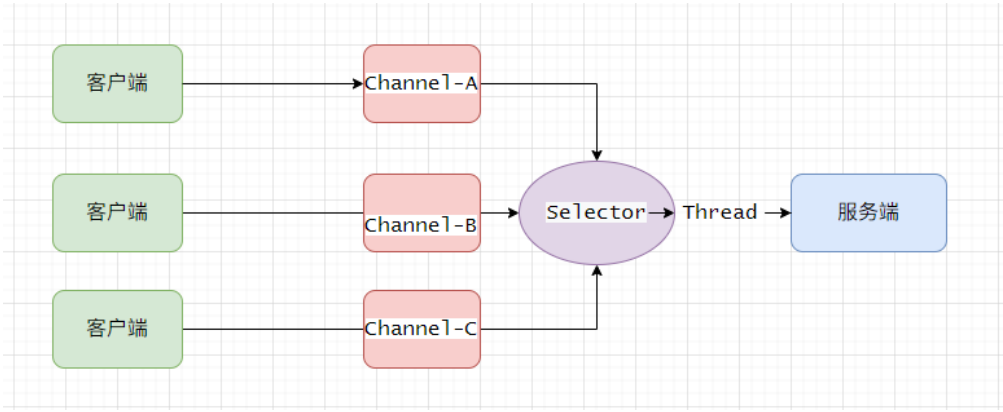


- 同步 / 异步 --> **关注的是消息通信机制**（区别最大在于异步的话调用者不需要等待处理结果）
  - **数据就绪后需要自己去读是同步，数据就绪直接读好在回调给程序是异步。**
- 阻塞 / 非阻塞 --> 关注的是程序在等待调用结果（消息，返回值）时的状态
  - **阻塞是挂起当前线程，调用线程结果才会返回，非阻塞 不会立即返回结果，不会阻塞当前线程**
- **BIO**
  - **阻塞同步**，客户端发送消息后，需要服务端为每个用户的请求创建一个单独的线程来执行。以流式处理数据，面向流的 io 一次处理一个字节，一个输入流产生一个字节，一个输出流消费一个字节。
  - 适用场景：连接数比较小，对于服务器要求比较高。
- **NIO**
  - **同步非阻塞**，NIO改变了之前BIO之前为每个请求创建一个单独的线程的机制，而是采用了选择器进行轮询执行客户的请求，提高了服务端的复用能力。 NIO以块处理。面向块的 io，每一个操作都在一步中产生或消费一个数据块。
  - 适用场景：连接数多 时间短，聊天服务器，弹幕系统
  - NIO中的buffer和inputstream select知识点
  - NIO实现原理

- **AIO**
  - 异步非阻塞
- **五种常见网络IO模型**
  - **BIO**(阻塞IO)
    - 分类 字节 InputStream OutputStream、字符Reader、Writer
    - 同步并阻塞 一个请求 一个线程
    - 应用场景
      - 连接数目比较少 且固定的架构 简单易理解

    BIO BS流程的理解

    首先 服务端启动一个serversocket监听一个固定的端口，接收来自客户端的请求，客户端发送3次握手过程，建立连接。服务端接收客户端的连接请求，(**注意每个客户端的请求 服务端都要使用一个新的线程进行处理客户端请求，服务端需要阻塞等待客户端的请求 以及 客户端的数据的发送。比较耗费IO和网络资源。**)，当服务端没有数据时，客户端会阻塞read，等到服务端的数据写入。

    - BIO存在的问题
      - 1.每个客户端请求都要创建一个独立的线程 与对应的客户端进行数据read 业务处理 数据write
      - 2.并发量比较大时，需要创建大量线程来处理，系统资源占用比较大。
      - 3.建立连接之后，如果没有数据读取，则线程阻塞在read操作上，造成线程阻塞。
  - **NIO(非阻塞 IO)**
    - 

    - 同步非阻塞
    - 三大核心部分
      - **Channel 通道**
        - 可以把 channel 看作 io 流，但它和 io 流相比还有一些不同：1）channel 是双向的，既可以读又可以写，io 流是单向的；2）channel 可以进行异步读写；3）对 channel 的读写必须经过 Buffer 对象。
        - **分类**
          - Filechannel：从文件读取数据的。
          - Datagramchannel：读写 udp 的网络协议数据。
          - Socketchannel：读写 tcp 网络协议数据。
          - Serversocketchannel：可以监听 tcp 连接。
      - **Buffer(缓冲区)**
        - Buffer 的三个属性：1）capacity：buffer 有一个固定大小，也就是 capacity，你只能往里面写 capacity 个 byte、long、char 等类型；2）

position：当你写数据时，position 表示当前位置，即下一个可以开始写的位置。初始时 position 为 0，当写入一个数据时，position 会向前移动到下一个可插入数据的 buffer 中，position 最大为 capacity-1；当读数据时，也就是从某个特定位置开始读，应当将 position 从写模式切换到读模式，position 被置为 0。3）limit：在写模式下，limit 表示你最多能写多少数据，此时 limit=capacity；在读模式中，limit 表示你最多能读到多少数据。所以当切换到读模式时，limit 被设置为写模式下position。

        - **Selector(选择器)**
            - 网络编程 nio（异步 io）异步 io 是一种没有阻塞的读写数据的方法。selector 是一个对象，可以注册到很多 channel，监听各个 channel 上发生的事情，并且能够根据事件情况决定 channel 读写，这样通过一个线程管理多个 channel，就可以处理大量网络连接。有了selector，可以用一个线程处理所有 channel。线程之间的切换对操作系统来说，代价是很高的，并且每个线程也会占用一定的系统资源，所以对系统来说，线程越少越好（但若 CPU 有多个内核，不使用多任务是在浪费 CPU 能力）。
        - **NIO是面向缓冲区的**
        - 从文件读取数据分三步：1）从 FileInputStream 中获取 channel；2）创建 Buffer；3）从channel 中读数据到 Buffer。通过 nio 进行文件复制的代码：
        - 应用场景
            - 连接数目多 连接比较短 聊天服务器 弹幕系统 服务器间通信
    - BIO和NIO的区别
        - **BIO以流的方式处理数据**，而**NIO以块的方式处理数据**，块IO的效率比流IO高很多。
        - BIO是阻塞的，NIO则是非阻塞的
        - **BIO基于字节流和字符流进行操作**，**而NIO基于Channel通道和Buffer缓冲区进行操作**，数据总是从通道读取到缓冲区中的，或者从缓冲区写入到通道中的。
    - **AIO(异步IO)**
        - 异步非阻塞
        - 应用场景
            - 数据
- [怎样理解阻塞非阻塞与同步异步的区别？]
- [BIO,NIO,AIO 总结]（基本概念 + 实例 *TODO*）
- [Java NIO浅析]（思想 + 实例 *TODO*）
- [第11讲 | Java提供了哪些IO方式？ NIO如何实现多路复用？]（思想 + 实例 *TODO*）
- [Lesson: Basic I/O]（官方 IO Docs *TODO*）
- **重新理解IO流**
    - IO流对象的理解，大多数人的理解都是管道或者是文件，一种资源，这种理解，我认为是不正确的。IO流对象是二进制数据资源的抽象。并且是单向传输，也就是说同时只能允许一端进行操作。输入流就是二进制数据的source抽象。而数据的来源是不确定的，可能是网络或者本地 内存中等。刚开始学习IO流，会感觉为什么既然有File对象，那么FileInputStream对象获取数据呢，因为Java设计者，使用统一的Inputstream来描述数据源，通过FileInputStream对象API操作对象。输出流和出入流是相反的。输入流的来源是不确定的，而输出流的载体也是不确定的。对于OutputStream来说，具体什么载体我们统统不关系，他就是一个高度抽象化的抽象输出资源对象。比如我们可以通过数组进行下标的操作，但是通过ByteArrayInputStream封装数据后，可以提供一套更高层的抽象。就像Object抽象了所有的Java对象一样。

泛型

- **本质**：类型参数化，解决了不确定对象的类型问题 即**参数化类型**，泛型擦除：Java 编译器生成的字节码文件不包含有泛型信息，泛型信息将在编译时被擦除，这个过程称为泛型擦除。其主要过程为
  - 1）将所有泛型参数用其最左边界（最顶级的父类型）类型替换；
  - 2）移除 all 的类型参数。
- **优点**
  - 安全：不需要担心程序中出现类型转化的问题
  - 避免强制类型转换
  - 可读性比较高
- 1.泛型擦除
- 泛型赋值问题
- 泛型参数问题
- **泛型理解？**
- ### 1. 泛型为什么不安全

  泛型能够保证容器里存放元素的类型，但是通过运行时的反射机制能够绕过**泛型的编译验证**。

```
    public static void main(String[] args) throws Exception {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        //list.add("a"); // 这样直接添加肯定是不允许的
•
        //下面通过java的反射，绕过泛型来给添加字符串
        Method add = list.getClass().getMethod("add", Object.class);
        add.invoke(list,"a");
•
        System.out.println(list); //[1, a] 输出没有没问题
        System.out.println(list.get(1)); //a
    }
```

  ### 2. 泛型擦除

  Java中的**泛型基本上都是在编译器这个层次来实现的。在生成的Java字节码中是不包含泛型中的类型信息的。使用泛型的时候加上的类型参数，会在编译器在编译的时候去掉。**这个过程就称为**类型擦除。** 如在代码中定义的List和List等类型，在编译后都会编程List。JVM看到的只是List，而由泛型附加的类型信息对JVM来说是不可见的。Java编译器会在编译时尽可能的发现可能出错的地方，但是仍然无法避免在运行时刻出现类型转换异常的情况。类型擦除也是Java的泛型实现方法与C++模版机制实现方式之间的重要区别。

反射

- **什么是反射以及反射有什么具体应用**
  - Java反射机制是在**运行状态中**，对于任意一个类，都能够知道这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意一个方法和属性。这种 **动态获取的信息以及动态调用对象的方法的功能** 称为Java语言的反射机制
  - 应用
    - 框架设计的灵魂 --> 如：Spring 通过 XML 配置模式装载 Bean 的过程
    - 使用JDBC连接数据库时使用 Class.forName() 通过反射加载数据库的驱动程序
    - 动态加载类名，例如Spring动态加载Bean。
  - 什么是反射机制？反射机制的应用场景有哪些？（基本介绍）

- Java基础之—反射（非常重要）（反射实例 *TODO*）
- **怎么实现的？**
  - 调用`newInstance()`适用于无参情况下？
  - 调用构造方法 适用于无参和有参的情况下？
- **你知道动态代理和反射的关系吗 说一下？**
  - JDK提供的原生动态代理基于反射实现的。

拷贝

- **浅拷贝**
  - 条件 实现`Cloneable接口 并重写clone()`
  - **存在的问题**
    - 只拷贝对象的值对象类型，而引用类型不会拷贝。
      - 解决方案->**深拷贝**
- **深拷贝**
  - 复制整个对象信息，包含值和引用类型
  - 实现方式
    - 序列化实现克隆
    - 引用类型都实现克隆
- **拷贝的好处？**
  - 使用方便
  - 性能高
  - 隔离性
- **浅拷贝 & 深拷贝（字节跳动）**
  - 浅拷贝 --> 对基本数据类型进行值传递，对引用数据类型进行引用传递般的拷贝，**没有真实的创建一个新的对象**
  - 深拷贝 --> 对基本数据类型进行值传递，对引用数据类型，**创建一个新的对象**，并复制其内容
  - 细说 Java 的深拷贝和浅拷贝
  - 8.6: Pass by Value vs. Pass by Reference - Processing Tutorial

序列化

- 序列化 --> 把对象转换为**字节序列**的过程称为对象的序列化
- 反序列化 --> 把字节序列恢复为对象的过程称为对象的反序列化
- 注意点：**Java在序列化时不会实例化static变量和transient修饰的变量，因为static代表类的成员，transient代表对象的临时数据，被声明这两种类型的数据成员不能被序列化。**
- Java 序列化
- Java对象的序列化（Serialization）和反序列化详解（实例）
- 二叉树的序列化与反序列化
- 作用 持久化对象
- **serialVersionUID的作用是什么？**
  - 通过判断该UID 是否相同，是否可以修改成功本类数据以及相关属性
- **可序列化接口的用途是什么？**
  - 只是标识一下这个类可以被序列化。
- **序列化的方式?**

    - 原生方式
    - JSON格式->fastjson
    - Hession方式
- **比如我们通过创建对象存储在堆空间中，那么这个数据一定是二进制的。那么为什么还需要进行反序列化操作呢？**

  首先，我们要明确一点，那就是序列化的目的是什么：是为了将文件从本地特有的二进制，序列化成公共二进制。如果只是单纯的传输堆空间的二进制，接收方是无法进行查看，因为序列化遵循的不是一套二进制。

Java8

- lambda 表达式，以及使用 lambda 表达式的场景
- Stream

注解

- **作用**
    - **代替繁杂的配置文件 简化开发**
- **java中有哪些元注解？**
    - **@Target 说明注解所修饰的对象范围**
    - **@Retention 保留策略 注解被保留的时间长短**
        - **source 源文件保留**
        - **class class文件中有效**
        - **runtime 运行时 保留**
    - **@Documented 标记注解**
    - **@Inherited 继承标记注解**
- **应用场景**
    - 注解代替配置文件 SpringBoot提供了大量的注解来代替注解配置
- 自定义注解 注解处理器(反射和线程机制)`todo`
- ```
  public @interface MyAnn {
      String value();
      int value1();
  }
  @MyAnn(value = "xx",value1 = 100)
  public class MyClass2 {

  }
  ```

内部类

- **使用场景**
    - 可以作为多继承的一种实现方式
    - 方便将存在一定逻辑关系的类组织在一起，对外界隐藏。
    - 内部类可以方便继承或实现接口 是多继承的一种拓展
- **成员内部类**
    - **成员内部类可直接访问内部类 需要建立(外部类.this.xxx)**
    - 外部类访问成员内部类，必须先建立内部类对象。
- **静态成员内部类**
    - 注意静态成员内部类不能访问非静态外部类对象
  ```

- **局部内部类**
  - 成员内部类与局部内部类的区别？
    - 成员内部类可以访问任意修饰符，局部内部类不能访问任何修饰符
    - **局部内部类是声明在外部类的方法或其他作用域范围内的**，内部类是直接声明在外部类之中的，与方法和属性平级
- **匿名内部类**
- **权限问题**
  - 在Java中，可以将一个类定义在另一个类里面或者一个方法里边，这样的类称为内部类，广泛意义上的内部类一般包括四种：成员内部类，局部内部类，匿名内部类，静态内部类 。

    1.成员内部类

    （1）该类像是外部类的一个成员，可以无条件的访问外部类的所有成员属性和成员方法（包括private成员和静态成员）；

    （2）成员内部类拥有与外部类同名的成员变量时，会发生隐藏现象，即默认情况下访问的是成员内部类中的成员。如果要访问外部类中的成员，需要以下形式访问：【外部类.this.成员变量 或 外部类.this.成员方法】；

    （3）在外部类中如果要访问成员内部类的成员，必须先创建一个成员内部类的对象，再通过指向这个对象的引用来访问；

    （4）成员内部类是依附外部类而存在的，也就是说，如果要创建成员内部类的对象，前提是必须存在一个外部类的对象；

    （5）内部类可以拥有private访问权限、protected访问权限、public访问权限及包访问权限。如果成员内部类用private修饰，则只能在外部类的内部访问；如果用public修饰，则任何地方都能访问；如果用protected修饰，则只能在同一个包下或者继承外部类的情况下访问；如果是默认访问权限，则只能在同一个包下访问。外部类只能被public和包访问两种权限修饰。

    2.局部内部类

    （1）局部内部类是定义在一个方法或者一个作用域里面的类，它和成员内部类的区别在于局部内部类的访问仅限于方法内或者该作用域内；

    （2）局部内部类就像是方法里面的一个局部变量一样，是不能有public、protected、private以及static修饰符的。

    3.匿名内部类

    （1）一般使用匿名内部类的方法来编写事件监听代码；

    （2）匿名内部类是不能有访问修饰符和static修饰符的；

    （3）匿名内部类是唯一一种没有构造器的类；

    （4）匿名内部类用于继承其他类或是实现接口，并不需要增加额外的方法，只是对继承方法的实现或是重写。

    4.内部静态类

    （1）静态内部类是不需要依赖于外部类的，这点和类的静态成员属性有点类似；

    （2）不能使用外部类的非static成员变量或者方法。

枚举类

- 枚举类安全 不能被继承 编译后成为public static final类
- **应用场景**
  - 作为一个高级的常量类
  - switch判断

- **原理**
  - 通过javap分析`.class`发现都转换成了`public static final`字段

问题迭代

- **一个类的执行流程**
  - 1.不论是idea还是手动编译 都是将`.java`文件转换成`.class`文件 其中主要`.java->语法分析->词法分析->语义分析->.class`
  - 2.类加载器加载`.class`文件，其中涉及到类加载器的双亲委派模型。
  - 3.将class文件 进行**解析执行**成机器语言。当然hotspot也自带jit(即时编译器) **编译执行**在程序运行时间 动态编译 将热点编译成机器语言。
    - 这里存在一个问题那就是热点代码的判定。
      - **基于采样的热点判断**(定周期的查看每个线程栈顶的代码，经常出现在栈顶的代码就是热点代码，优点是简单，缺点比较不精确)
      - 基于计数的热点判定(给每个方法或者代码块设定一个计数，当达到一定的阈值就触发JIT编译)
        - **计数器**
          - **方法调用计数器**(存在热度衰减)
          - **回边计数器**(主要统计方法中循环体的执行次数)
  - **既然你说到了JIT即时编译器 可以说说你对它的理解吗 对象栈上分配？？**
    - JIT全称为Just in time compile 即时编译器，把经常运行的代码 作为热点代码编译成本地平台相关的机器码，并进行各种层次的优化，**除了具有缓存功能之外**，还支持底代码优化，包括**逃逸分析**、锁消除、锁膨胀、方法内联、空值检查消除、类型检查消除以及公共子表达式消除等。
    - **逃逸分析可以说说你的理解吗 ？？**
      - 先来说说什么是逃逸分析，就是分析对象的动态作用域，比如我们在方法中创建一个对象 很有可能通过参数进行函数调用的传参，被外部方法所引用，就为**逃逸分析。**
      - JIT的优化
        - **同步省略： 锁消除 当JIT编译器判断不会出现并发问题，会同步将syn去掉**
        - **标量替换**
          - **标量： 一个无法在被分解的数据的数据，比如基本数据类型**
          - **聚合量： 一个可以在被分解的数据 ，一个对象就是一个聚合量 可以在被分解为一个聚合量和标量**
      - **如果一个对象经过逃逸分析，发现外部不会引用，就会将对象分解为多个成员变量来替换，这个过程就是标量替换，好处在于不需要将对象直接分配在堆中，而是可以在栈上直接分配**
      - **逃逸分析的缺点?**
        - 技术不成熟，分析阶段比较耗时，如果一个对象不是逃逸的，那么得不尝试。
- **对跨平台的理解**
  - 由于java语言有跨平台的特性，是基于jvm来实现的，屏蔽了底层的硬件细节。究其原因在于对于不同的操作系统提供了不同的jvm，而jvm是依赖于每个系统。但是它所运行的`.class`字节码是通用的，对于在window上编写的`.java`文件编译后的`.class`文件，在linux中也可以运行。所以这才是跨平台的应用。
- **包装类和基本数据类型**

- 好处
  - 1.功能丰富 2.可以定义泛型参数 3.可以序列化 4.可以缓存一定范围内的数据
- 对于包装类来说 在-127-128之间 会被缓存 否则创建一个新的对象(在堆上创建对象)

```
public static Integer valueOf(int i) {
  if (i >= IntegerCache.low && i <= IntegerCache.high)
    return IntegerCache.cache[i + (-IntegerCache.low)];
  return new Integer(i);
}
```

- 应用场景
  - pojo类属性使用包装类
  - rpc方法返回值和参数必须使用包装类
  - 所有局部变量使用基本数据类型
- 基本数据类型一定存储在栈中吗

不一定。需要看基本数据类型定义的区域，**如果是方法内部 局部变量 则存储在java栈中**，随着栈的消失而消失。**如果是全局变量则存储在java堆中**，不会随着栈的消失而消失。

- **值传递 & 引用传递**
  - Java中方法参数传递方式是按**值传递**
  - 如果参数是基本类型，传递的是基本类型的字面量值的拷贝
  - 如果参数是引用类型，传递的是该参量所引用的对象在堆中地址值的拷贝
  - 什么是值传递和引用传递？
- **可以说一下你对高内聚和低耦合的理解吗？**
- **谈谈你对JavaBean的理解？**
  - 目的是为了实现程序中代码的向后兼容性，JavaBean不是一门技术，而是一种规范，定义Bean的一种方式。符合四个条件就可以。**1.所有私有属性为private 2.提供默认的构造方法 3.提供setter getter 4.实现序列化接口**
- **重新理解protected?**
  - 在学习Servlet中，直接去继承httpServlet 重写里面的方法，但是大多数人，并没有意识到为什么要这么做呢，并且他是protect保护的，也就是不同包的子类和同一个包下可以访问到这个方法。其实，原因比较简单，我们知道框架一般都是内置了一些基础方法，来让我们使用，但是，为了实现拓展性，我们可以实现protected的方法，框架在动态运行过程中，发现这个方法被覆盖了，直接调用用户的自定义方法。
  - 但是，凡事不是绝对的，也就是说，框架也可以将某个类抽象成抽象类，一些基础实现直接定义成absstract就可以了，用户自定义实现的时候，必须强制性实现。本身就是设计模式的不同体现。模板设计模式。灵活性更好。

集合类 ☆☆☆

List

ArrayList ☆ ☆☆☆

- **底层** 数组 **默认大小为10** 只有在add的时候 才会初始化容量，建议在初始化的时候，直接定义好大小
- **扩容** 1.5倍 `int newCapacity = oldCapacity + (oldCapacity >> 1);`
- **删除** 将当前元素之后的元素整体移动一个位置 然后置为null，gc回收。
- 非同步 同步可以使用Collections.syn

- **快速失败** 如果通过非迭代器的方式修改数据，会抛出concurrentmodifay异常。主要应用于检查错误。必须修改操作。modcount 记录改变的次数
- **RamdomAccess的作用** 仅用于表示可以随机访问
- **与Vectory的区别** （syn修饰过的）
    - **ArrayList和LinkedList的区别**
        - ArrayList底层使用了动态数组实现，实质上就是一个动态数组
        - LinkedList底层使用了双向链表实现，可以当堆栈、队列、双端队列使用
        - ArrayList在随机读取元素快 LikedList在修改删除元素节点快
        - ArrayList会预留一部分空间，空间不足会自动扩容。
        - LinkedList的开销在于节点的指针上以及记录的信息上。
- **多线程环境下怎么办？**
    - 推荐使用读写List **CopyOnWriteArrayList**

Vector

- **原理**： 初始化容量10 扩容1倍 synchronized修饰 **扩容2倍**

LinkedList ☆☆☆

- **结构：双向链表结构**
- 添加元素
    - addFirst 主要逻辑
- source: 1.首部添加尾部添加。正常添加。删除。

CopyOnWriteArrayList ☆

- **读写分离** 写操作在一个复制的数组上进行。需要lock锁。
- **如何保证线程安全的？**
    - **写时复制策略** 简单一点 如果只是读 多个线程都会取到到一份相同的数组。如果修改，会拷贝一份新的数据到新数组中，修改完新数组将引用直接改变成新的数组。旧数组引用直接废弃。
    - **可以说一下add()的过程吗？**
        - 首先获取一个本对象的lock锁，**上锁** 整个类全局只有一把锁
        - 获取数组，拷贝一份新的到新数组中
        - 修改完毕，将新数组的引用替换成旧数组的引用
        - **释放锁**
    - **可以说一下get()的过程？**
        - 直接获取。
    - **了解有哪些缺点吗？**
        - **数据的不一致性**，`CopyOnWriteArrayList`只能保证数据`最终一致性`，不能保证数据`实时一致性`
        - **内存占用** 由于写操作会复制一份相同的容量，所以占用内存时多一倍，可能造成频繁的GC。可以考虑使用其他并发安全容器。
- 适用场景
    - **读多写少**
    - 缺陷
        - 内存占用
        - 数据不一致 在某一时间段内，读操作不能保证读取到数据的实时性。

    - 不适合对数据一致性要求比较高的场景
- **谈谈你对数组和链表的理解 【高频】**
    - **首先从数据结构角度看**，数组是通过索引下标实现查看数据的，所以查询时间复杂度O(N) 但是删除和添加 如果不是在末尾进行操作的话，那么平均时间复杂度为O(n)。但是对于链表来说的话，因为链表的结构是一个节点 内部包含next域和val域，通过指针来操作，查询时间复杂度为O(n) 需要next指针不断下探。而删除和添加只需要移动前后节点的next域，所以时间复杂度为O(1)。**上面是从数据结构上来看，但是对于java这种高级语言来说，对数据和链表进行了封装，也既是抽象数据类型**。对应的首先分别是ArrayList和LinkdList，所以总结一下，数组适合于查询多修改操作少的场景，而链表恰恰相反适合于查询少修改操作多的场景
    - 上面是最基础的理解，没什么好说的。对于get ，Arraylist是O(1) 但是对于链表的get(n) 是O(n/2) 因此推荐使用迭代器进行get()，
- **谈谈你对数组和集合的理解？**
    - 可以从三个维度来看，数组存储的数据类型是单一的，而集合可以存储任意类型。
    - 数组长度固定 集合可以动态扩容
    - 数组效率比较高 但是集合的功能更加完善对于使用者来说api完善。

Map

HashMap ☆ ☆☆☆

- **底层数据结构**
    - **1.7 数组+链表 头插法(链表成环问题**`todo`**)** `Entry`节点
    - **1.8 数组+链表+红黑树** 8/64 6 尾插法 `Node`节点 **初始化大小**16，扩容2倍
        - **为什么要使用红黑树**
            - 解决hash碰撞
            - **为什么链表到红黑树的转换值为8**
                - **泊松分布概率函数** 在8的之后 命中率机会接近于0, - **resize扩容和树化冲突** - **红黑树个数<=6 转换成链表** 7是防止互转过于频繁。
- **hashmap的初始容量 加载因子 扩容增量是什么?**
    - **hashmap的初始容量是16，加载因子为0.75 扩容是元容量的1倍** 如果hashmap的容量是16一次扩容后是32，hashmap扩容元素个数为16*0.75=12之后开始扩容。
        - 值较高会减少空间开销和查找成本。
- **为什么说HashMap是不安全的?**
    - 同时put**碰撞**导致数据丢失
        - 当两个线程都添加数据 索引都在一个位置，添加的时候会丢失一个线程添加的数据。
    - 同时put**扩容**导致数据丢失
    - **死循环造成的CPU100%** 推荐coolshell 陈大
- **如何保证安全**
    - Collections.syn()
        - 了解原理吗
            - 使用了syn同步代码块，对于并发量高性能不高
    - ConcurrentHashMap
- **如何确定元素存放在哪个位置呢**
    - 1.计算出hash值 确定桶的下标
    - **为什么要将key的哈希码右移16位？**

        - int是32位，**为了更好的计算出hash是分散的**，所以将h无符号右移16位。然后在与h异或。就可以保证高16位与低16位都能参与到hash的计算。**尽最大努力减少哈希冲突**
- **为什么hashmap容量必须是2的次幂 这是为什么?**
    - **因为当容量是2的次幂时，h&(length-1)运算才等价于对length取模，也就是h%length & 比%效率更好**
    - **尽量均匀分布减少哈希冲突，提高hashmap查询效率**，
    - **那么hashmap如何保证容量是2的次幂？**
        - **1.默认初始化为16**
        - **2.对于目标自定义的容量大小 函数tableSizeFor()返回一个最接近2的幂次的最小值** 比如输入20 会返回32.
- **说一下你对put过程的理解 ？**
    - 1.数组为空 创建一个新的数组
    - 2.对**key计算hash 为空** 【如果table[0] 有key为null，进行value的覆盖，否则的话直接插入到tables[0]中】 计算在哪一个桶下标 直接插入。
    - 3.不为null 如果存在直接覆盖、
    - 4.判断是否是树节点 添加到红黑树中
    - 5.否则 就是链表 遍历链表【超过阈值 **树化**】key存在 直接覆盖
    - 6.是否超过容量 超过就**进行扩容resize**

  总体流程：调用put()存储键值对，根据键的hashcode计算在哪一个bucket下标中，创建一个entry节点，添加到对应的桶下标中，如果出现**hash冲突**，则添加到对应的链表和红黑树节点中。
- **说一下你对get过程的理解**
    - 获取存储的值的hashcode，找到对应的bucket下标，然后根据equals在链表和红黑树中找到对应的值
- **扩容机制** `todo`
    - **jdk7**
        - **resize()**
            - **默认两倍扩容**
            - **先判断是否达到Integer的最大值，如果是则不能扩容**
            - **创建一个新的Entry数组**
            - **将数据转移到新的Entry数组中**
                - **transfer()**
                    - **暂存旧的数据引用**
                    - **获取新数组长度**
                    - **遍历旧数组**
                    - **重新计算每个元素在新数组中的下标 rehash()**
                    - **使用头插法**
                    - **头插法 resize()容易死循环？**
    - **jdk8**
        - **resize**
            - **超过最大值不扩容**
            - **扩大问原来容量的两倍**
            - **通过高位运算e.hash & oldCap 确定元素是否需要移动，元素要么在原位置，要么在原位置移动2的次幂的位置。**

  hashmap默认的负载因子为0.75 当容量超过16*0.75=12时 会触发一次扩容，大小为之前容量的两倍 32，需要将数据进行**rehash** 计算一下在扩容之后数组的下标位置，多线程环境下 可能出现死循环。

- 死循环原因分析：
  - Hashmap 的 resize 在多线程的情况下可能产生条件竞争。因为如果两个线程都发现 hashmap 需要进行 resize 了，他们会同时试着调整大小。在调整大小的过程中，存储在链表中的元素的次序会反过来。因为移动到新的位置时，hashmap 并不会将元素放在链表的尾部，而是放在头部在，这是为了避免尾部遍历。（否则，针对 key 的 hashcode 相同的 entry 每次添加还要定位到尾节点）。如果条件竞争发生了，可能出现环形列表。之后，当我们调用get(key)操作时就可能发生死循环。

不安全有两点：1.死循环 2.数据丢失 3.迭代时不允许修改内容 4.只读的并发是安全的

- **Hash冲突的解决办法** ☆
  - 拉链法(hash采用)
  - 开放地址法
  - rehash
  - 建立公共溢出区
- **HashMap remove()删除策略**
  - **1.定位桶下标**
  - **2.如果第一个节点与删除节点的值相等 则将node指向该节点**
  - **3.如果是TreeNode节点 调用红黑树的查找逻辑定位待删除节点**
  - **4.否则是链表节点 通过链表指针.next查找到 并删除**
  - **5.删除节点 并修复链表或红黑树结构**
- **那些类适合做hashmap的键？**
  - String和Integer是适合的，因为都重写了equals和hashcode。可以有效提高hashcode性能，如果修改键值对，放入和获取不一样 是找不到原来对象的。
- **HashMap和HashTable的区别有哪些？**
  - hashmap是线程不安全的，hashtable是线程安全的 加syn修饰。
  - hashmap的key可以添加null值，hashtable的key和value都不可以添加null值
  - **初始容量 扩容方式**
    - hashtable的默认初始容量是11 扩容是2n+1
    - hashmap的默认初始容量是16 扩容是2*n
  - **可以举出HashMap不安全的例子吗？**
    - **hashmap多线程环境下扩容导致死循环的问题**
    - hashtable线程安全由于内部在put和remove等方法上使用了syn关键字，所以对于多线程修改数据来说，同一时刻只能有一个线程可以修改 保证了线程安全。
  - **快速失败机制 fast-fail**
    - **快速失败**是java集合的一种**错误检查机制**，修改或删除一个或多个映射的操作，仅更改实例中key对应的value并不是结构修改
    - **底层实现**
      - 遍历集合的时候 会有一个字段modCount 记录集合修改的次数，还有一个expectedModCount值，如果两者不同直接抛出并发修改异常。`ConcurrentModficationException()`

ConcurrentHashMap ☆☆☆☆

- **原理 : 1.7 数组+链表 (segment分段锁+reentrantlock 并发竞争** 自旋 阻塞 HashEntry) **同时支持16个并发写**
- **原理 : 1.8 数组+链表+红黑树 (cas+syn** cas自旋 sync保证 **Node**)
- **1.7 VS 1.8结构改变**
  - **数据结构** 并发度提高由原来的16提升到多个Node的数量。

    - **Hash碰撞** 链表->红黑树
    - **保证并发安全** 7采用分段锁 ReentrantLock 8 CAS+Syn
    - **查询复杂度** 链表:O(N) 红黑树:O(logN)
        - **可以说一下为什么要在链表增加到8转为红黑树呢?**
            - 红黑树占用空间比较大。
            - **泊松分布概率函数**
- **说一下put的流程**
    - 判断key value不为空
    - 计算hash值
    - 根据对应位置节点的类型 来赋值 或者helpTransfer,或者增长链表，或者给红黑树增加节点
    - 检查满足阈值 红黑树化
    - 返回oldVal
    - **1.8详细流程**
        - 自旋重试插入元素
            - 链表为空 初始化
            - 否则 判断索引位为空 通过CAS设置在索引位置
            - 判断当前节点是否是移动节点(桶正在扩容)
            - 否则 产生哈希冲突
                - 获取syn锁 **双重检查是否是原头结点**
                    - 判断当前桶的哈希值是否>0
                        - 遍历链表
                            - 判断key是否存在 是否允许覆盖 覆盖 break
                            - 新节点 **尾插法到链表中**
                    - 判断是否是红黑树结构
                        - 添加到红黑树中
            - 判断是否超过树化阈值 链表->红黑树
        - **是否需要扩容**
        - **既然你说到了扩容操作，可以谈谈你的理解吗 ？**
            - 由于操作的table都是同一个，**需要等扩容完毕才可以进行读写操作才能进行**。所以扩容的效率就成为整个并发的瓶颈点。扩容时，需要锁的保护。旧版的segment锁，锁粒度大。
                - 判断新table是否为空
                    - 创建一个两倍大小的实例 扩容失败异常机制
                - `TODO`
- **说一下get的流程**
    - 计算hash值,定位table索引，是首节点就返回。
    - 找到对应的位置，根据情况进行
    - 直接取值
    - 红黑树里找值
    - 遍历链表取值
    - 返回找到的结果
- **concurrenthashmap不安全吗？**

    - ConcurrentHashMap只是保证get put是安全的，不能保证之外数据的操作的数据安全性。这是一个误区。
        - 推荐方式
            - concurrenthashmap .**replace()**
    - **实际生产案例**
- **ConcurrentHashMap的key可以为null吗？**
    - 不可以

TreeMap

- 底层:**红黑树 会按照键值对排序**
- 键为字符串 按照字典默认排序，如果自定义类型 按照compareTo方法比较值。
    - 实现**comparable**【实现了可以进行排序】和**comparator**【比较器】的比较
        - 两者区别
            - Comparable实现比较简单，需要重新定义规则 需修改源码
            - Comparator不需要修改源码，在TreeMap的构造器中闯入一个指定规则的比较器就可以。

HashTable

- synchronized 锁住整个类 **不支持null键和值**
- Queue

  PriorityQueue 优先队列 堆实现。

LinkedHashMap

- **原理** LinkedHashMap可以记录下元素的插入顺序和访问顺序
- LinkedHashMap的内部Entry继承与HashMap.Node，这两个类都是实现Map.Entry<K,V>
- before,after 双向链表
- accessOrder 可以设定T/F 是否使用LRU算法。
-

Set

HashSet

- **原理** hashMap 用值做key,v为固定obj,**PRESENT**. **非安全的** 允许null值
- **保证元素的唯一性:判断元素的hashcode是否相同，如果相同 还会判断元素的equals方法 是否为true**
- 为什么采用Object作为value对象。
    - new Object()占据堆内存，一个空的object对象占用8Byte。，null只不会占用堆内存。
- **能简单说一下hashSet的put过程吗？其实就是hashmap的put过程**
    - 首先先比较一下hash值，如果不相等，直接添加到table中
    - **哈希码相同** 比较equals 不相等直接添加，否则不添加。
    - **为什么要使用使用hashcode进行对比呢？**
        - 可以有效提升键的缓慢搜索
    - **什么是哈希码？**
        - 哈希码是相对唯一，一种代表对象的int值，HashMap就是利用哈希码快速查找。

- **哈希码如何计算？**
  - 如果基本数据类型直接参与计算，引用类型 先计算该哈希值在参与计算。
- **只通过哈希码判断两个对象合适吗？**
  - 不合适，不同的对象很有可能哈希值相同

TreeSet

- **原理** 红黑树
- **怎么去重的？**
  - 通过comparable或者comparator

LinkedHashSet

- **原理** 类似hashSet使用hashMap LinkedHashSet使用LinkedHashMap
  - **如何保证数据数据的有序和唯一性？**
  - 底层由链表和哈希表组成，链表保证元素的有序既存储和取出一致，哈希表保证元素的唯一性。
-

遍历

- **Iterator和Listiterator的区别？**
  - Iterator可以遍历set和list，listiterator只能遍历list.
  - literator只能向前遍历，listiterator可以前后都遍历
  - listiterator只是实现了前者 并添加功能

数组和集合的转换

- 数组->集合 `Arrays.asList(xxx)`
- 集合->数组 `xxx.toArray()`
- 字符串->数组 `xxx.toCharArray()`
- 数组->字符串

Collections和Collection的区别？

- Collection是一个集合接口类，子类有Set和List。Collections是一个工具类 包含一些排序 二分 拷贝集合等

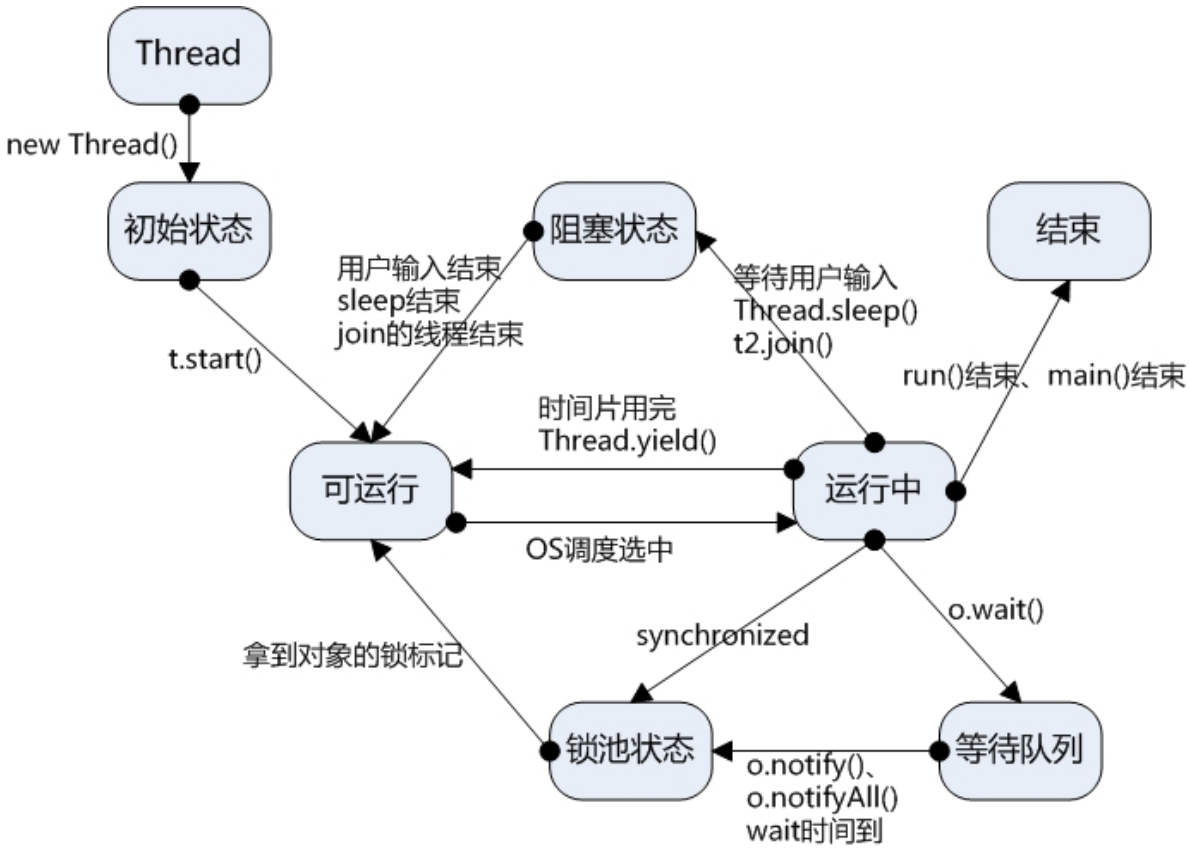
集合的好处

1.简化编程效率，基本上实现了常见数据结构与算法。

2.同步集合 保证安全类

并发 ☆☆☆

并发基础



- **线程状态** 6种状态
  - 新建 `new`
  - 可运行 `runnable`
  - 阻塞
    - 阻塞`blocked` syn 调用wait 进入同步代码块中或者同步方法中，等待获取一把排他锁。
  - 等待
    - 无限期等待 `wait` wait join
      - 调用了`Object.wait()`
      - `Thread.join()`
      - `LockSupport.park()`
    - 有限期阻塞
      - `Object.wait.` 带参的
      - `Thread.join()` 带参的
      - `LockSupport.parkNanos`
      - `LockSupport.parkUnit()`
  - `Blocked和wait的区别?` ☆
    - `blocked` 处于本状态的线程等待获取监视器锁以期望**进入(还没有进入)**同步代码块/方法中，比如A线程获取到同步代码块的锁，在执行，线程B尝试获取同步锁，获取不到处于`Blocked`状态。无法进入到同步块中。
    - 线程可以通过`wait` `join` `LockSupport.park` 进入`wait`状态，进入wait状态的线程等待唤醒`notify或notifyall`才有机会获取CPU的时间片来继续执行。
    - 区别
      - 与wating状态相关联是**等待队列**，与blocked状态相关联的是**同步队列**，一个线程由等待队列迁移到同步队列，线程状态将会由wait到blocked，可以说，blocked状态是处于wating状态到运行状态必由之路。wait>blocking->running

- 停止`stop`

线程状态大体上分为5类 1.新建 2.可运行 3.阻塞 4.定时阻塞 5.等待 6.停止 首先进行分析，新建一个线程其实就是创建好了，但是还没有调用start方法，可运行状态分为两种状态 要么当前线程在cpu上运行 要么 当前线程准备就绪 获取cpu时间片， 阻塞 当进入一个同步块中 别的线程获取不到锁，就需要等待获取锁进行，这是就会进行阻塞 然后阻塞中，无限期阻塞 调用无参的wait方法，等待另一个线程调用notify 调用join 等其他线程执行完毕，定时阻塞 调用了有参的sleep wait join 最后一个终止状态

- **线程活性故障有哪些？**
  - 由于资源的稀缺性或者程序自身的问题**导致线程一直处于非Runnable状态**，并且其处理的任务一直无法完成的现象被称为是线程活性故障，常见的活性故障有**死锁，锁死，活锁与线程饥饿**
  - **线程死锁** ☆
    - 多个线程永久等待对方而被永久暂停
    - 必备条件
      - **资源互斥** 一个资源只能被一个线程使用
      - **请求与保持条件** 一个线程因阻塞时对以获取的资源不释放
      - **不剥夺条件** 线程获取的资源 在未使用之前，不能强行剥夺
      - **循环等待条件** 若干线程之间形成一种头尾相接的循环等待资源关系
    - **针对于死锁，你有什么解决方案？**
      - **粗粒度的锁**
      - **锁排序法** 按照获取锁的顺序 来依次获取锁
    - **预防手段？**
      - 尽量使用 tryLock(long timeout, TimeUnit unit) 的方法 (ReentrantLock、ReentrantReadWriteLock)，设置超时时间，超时可以退出防止死锁；
      - **尽量使用 Java. util. concurrent 并发类代替自己手写锁；**
      - **尽量降低锁的使用粒度，尽量不要几个功能用同一把锁；**
      - **尽量减少同步的代码块。**
  - **线程锁死**
  - **活锁**
  - **锁饥饿**
- **线程常用方法**
  - **Sleep 和 Wait 的理解？**
    - Sleep 是Thread的静态方法，当前线程进入阻塞状态 睡眠n毫秒，当时间到了会解除阻塞状态，进入可运行状态，等待CPU的执行。 **不释放锁**
    - Wait是Object的方法， **必须和syn一起使用**，当调用notify notifyAll被调用后，会被解除阻塞**释放锁** 需要try 不然会有异常InterruptedExcpption
    - **对比**
      - 存在类的不同：sleep() 来自 Thread，wait() 来自 Object。
      - 释放锁：**sleep() 不释放锁**；wait() 释放锁。
      - 用法不同：sleep() 时间到会自动恢复；wait() 可以使用 notify()/notifyAll() 直接唤醒。
  - **Join方法**
    - 当前线程调用，其他线程全部停止，等当前线程执行完毕，接着执行。
  - **Yield方法**
    - 使得当前线程放弃分配的时间片，不阻塞，可运行状态，随时会获取时间片。
  - **线程唤醒**

使用 notify()/notifyAll() 方法唤醒线程。

    - notify() 方法随机唤醒对象的等待池中的一个线程；
    - notifyAll() 唤醒对象的等待池中的所有线程。

- 线程安全的理解
    - 首先来说，线程安全的前提是必须多个线程进行对数据的操作。我们需要分开来看，**一个是并发读**。因为读不会对数据造成修改，所以，并发读是没有问题的。**另一点是并发写**。当多个线程并发修改同一个数据的话，同一个数据可能出现数据不一致的问题。
    - 解决办法的话，进行加锁操作。一个是加 `syn`
    - 如果当多个线程访问同一个可变的状态变量时，没有使用合适的同步，那么程序就会出问题。有三种方法可以修复这个问题：**1）不在线程之间共享该变量。2）变量改为不可变。3）使用同步。**
    - **什么是线程安全**
        - a class is thread safe when it continues to behave correctly when accessed from multiple threads
        - 指某个函数、函数库在多线程环境中被调用时，能够正确地处理多个线程之间的共享变量，使程序功能正确完成
        - 可见性 / 原子性 / 有序性
        - 线程安全 Wiki
    - **描述Java下的并发编程（阿里）**
        - Java中实现并发编程的手段 --> 多线程
        - 线程的生命周期（新建 / 就绪 / 运行 / 阻塞 / 死亡）
        - 创建线程的方法
            - Runnable 接口
            - 继承 Thread
            - 通过 Callable 和 Future 创建线程
        - Java 多线程编程

线程创建

- 继承 Thread 类，重写 run 方法
- 实现 Runnable 接口，实现 run 方法
- 实现 Callable 接口，实现 call 方法
    - 优点：Callable 的调用是可以有返回值的，它弥补了之前调用线程没有返回值的情况
- 通过线程池技术实现
- **start和run区别？**
    - Start()和 run()——1）start 方法启动线程，真正实现多线程运行，通过调用 Thread 类的 start 方法来启动一个线程，这时此线程是处于就绪状态，并没有运行，若 cpu 调度该线程，则该线程就执行 run 方法；2）run 方法当做普通方法的方式调用，程序要顺序执行，要等 run 方法执行完毕，才可以执行下面的代码，程序中只有主线程这一个线程（除了 gc 线程）。

基础线程机制

- 1.Executor
- 2.Daemon
- 3.sleep
- 4.yield

线程中断

- interruptedException
  - Thread.sleep() 方法会抛出一个 InterruptedException 异常，当线程被 sleep() 休眠时，如果被中断，这会就抛出这个异常。
  - 注意：Thread.sleep() 方法由于中断而抛出的异常，是会清除中断标记的。
- 终止线程的方式有哪些？
  - **使用退出标志位终止线程**
  - **stop 但是不推荐使用**
    - 调用 stop() 方法会立刻停止 run() 方法中剩余的全部工作，包括在 catch 或 finally 语句中的，并抛出ThreadDeath异常(通常情况下此异常不需要显示的捕获)，因此可能会导致一些清理性的工作的得不到完成，如文件，数据库等的关闭。
    - 调用 stop() 方法会立即释放该线程所持有的所有的锁，导致数据得不到同步，出现数据不一致的问题。
    - 直接进行中断线程，会出现数据不一致问题。
    - 举一个栗子，比如线程A设置一个成员student的name和age，而另一个线程直接中断线程A，获取到的数据是不完整的。
  - **interrupted方法中断线程**
    - 需要注意一点就是**调用`interrupted`方法只是做一个标记，并不会马上进行中断线程**
    - 相关方法
      - `Thread.isInterrupted()` 判断是否被中断
      - `Thread.interrupted()` 判断是否被中断，并清除当前中断状态
  - **重写理解interrupted()**

    我们知道，通过调用interrupted()方法可以实现中断一个线程，并设置一个标志，其实大多数人，并没有真正理解到点上，浮于表面。首先，他的作用有两个，第一点，主要是对于一个阻塞状态的线程进行中断，比如说wait / sleep join。线程处于一个阻塞状态，等待获取锁资源，或者限定等待。但是由于外部条件改变了，需要将线程从先唤醒执行后面的逻辑，调用interrupted()来实现对线程的唤醒操作，打印出一个异常，从而执行后续的业务。一个比较好的例子，多个人都去上厕所，但是，只有一个坑位。剩余的人都在外面等。你等了一个小时，但是你女朋友不耐烦了。一个电话，从你原来的wait 状态中唤醒。去执行其他的业务逻辑了。

    第二点 就是interruped会设置线程一个中断位为true，调用isterruped()方法的作用呢，作用就是，我检查中断过，并会设置为false。 我们来想一下，为什么会有这样的需求呢，比如，可能针对某种业务来说，我可能会中断多次，但是，我如何标志你是那次中断呢，所以，当第一次调用interrupted() 方法会设置为true,但是调用istreeupetd会设置为false。一次就是我查看了，中断过一次了。

    第二个方法呢，Thread.currentThread.isInterrupted() 这个方法呢，调用之后 只是显示一下，之前中断过。被打扰过。 **在抛出interruptedException异常之前，会清除中断标志位** 所以在之前打印就位true 而之后就是false

    应用场景

    - 有一些相关的业务处理，比如线程运行过程中，需要停下来，处理一些相关的业务。就可以设置
    - Interrupt设置为true，为了多次中断 可以配合使用interrupted()，但是线程通信使用这个有点鸡肋，
    - 可以使用线程共享内存进行通信。

    总结

    interrupt重要用途就是打断sleep wait join 三个状态的休眠，并抛出异常，

互斥同步

- **说一下你对同步互斥的理解？**

  同步：多个线程并发访问共享数据时，保证共享数据在同一个时刻，只被一个（or 一些，使用信号量）线程使用，**而互斥是实现同步的一种手段**，临界区，互斥量，信号量都是主要的互斥实现方式。互斥是因，同步是果；互斥是方法，同步是目的。

- **同步锁synchronied** `升级过程todo`
  - 根本点:**synchronized 锁住的是括号里的对象，而不是代码**
  - 同步代码块 **()里的对象** this对象是不安全的。不是同一个对象。
    - 原理 使用 monitorenter 和 moniterexit 指令实现，monitorenter指令插入到同步代码块的开始位置，moniterexit 指令插入到同步代码块的结束位置，jvm 需要保证每一个 monitorenter 都有一个 moniterexit 与之对应。任何对象都有一个 monitor 与之相关联，当且一个 monitor 被持有之后，他将处于锁定状态。线程执行到 monitor 指令前，

      将会尝试获取对象所对应的 monitor 所有权，即尝试获取对象的锁
  - 静态代码块 **.class对象(作用于整个类)**
  - 同步方法 **this对象**
    - 原理：同步方法。依靠的是方法修饰符上的 ACC_SYNCHRONIZED 实现。Synchronized 方法则会被翻译为普通的方法调用和返回指令，比如 invokevirtual 指令，在 jvm 字节码层面并没有任何特别的指令来实现synchronized 修饰的方法，而是在 class 文件的方法表中将该方法的 access_flags 字段中的synchronized 标志位置为 1，表示该方法为 synchronized 方法，且使用调用该方法的对象 or该方法所属的 class 在 jvm 内部对象表示作为锁对象。
  - **锁升级过程** ☆（为什么锁升级 内核态<->用户态 ） `TODO`
    - 状态标记 **无锁->偏向锁(有延时)->轻量级锁(自旋锁)->重量级锁**
    - 偏向锁 自旋锁 都是用户态完成的，重量级锁是需要向内核申请
    - 无锁
    - **为什么需要偏向锁？**
      - 多数syn 在很多情况下，只有一个线程在运行
    - **偏向锁加锁流程？**
    - **偏向锁的效率一定比自旋锁高吗？**
      - 不一定，在明确知道会有多线程竞争的情况下，偏向锁肯定会涉及锁撤销，这时候直接使用自旋锁，JVM启动过程，会有很多线程竞争，默认启动不开，过一段时间开启。
    - **为什么有自旋锁还需要重量级锁？**
      - 自旋是消耗CPU资源的，锁时间太长，或者自旋锁线程多，CPU会大量消耗，重量级锁有等待队列，所以拿不到锁的进入等待队列，不需要消耗CPU资源。
    - 偏向锁 当前线程的id
    - 轻量级锁 (出现竞争的前提) CAS锁 如何保证一个线程可以获取成功 **如果没有成功自旋 自旋到一定次数会升级为一个Syn重量级锁**
    - 重量级锁
    - 在锁对象的对象头里面有一个 threadid 字段，在第一次访问的时候 threadid 为空，JVM（Java 虚拟机）让其持有偏向锁，并将 threadid 设置为其线程 id，再次进入的时候会先判断 threadid 是否尤其线程 id 一致，如果一致则可以直接使用，如果不一致，则升级偏向锁为轻量级锁，通过自旋循环一定次数来获取锁，不会阻塞，执行一定次数

之后就会升级为重量级锁，进入阻塞，整个过程就是锁升级的过程。

- **描述 synchronized 的底层实现**
  - 进入时，执行monitorenter 计时器+1，释放锁monitorexit时，计数器-1
  - 当一个线程判断到计数器为0时，则当前锁空闲，可以占用，反之，当前线程进入到等待状态。
  - 可以看出 JVM（Java 虚拟机）是采用 monitorenter 和 monitorexit 两个指令来实现同步的，monitorenter 指令相当于加锁，monitorexit 相当于释放锁。而 monitorenter 和 monitorexit 就是基于 Monitor 实现的。但是这块有一个细节，那就是通过反编译之后，可以发现有两个monitorexit这是为什么呢？主要在于一旦程序异常退出，就要执行将锁资源的释放，否则，就会出现死锁线程。
- **syn对一致性的保证**
  - **通过写线程冲刷处理器缓存，和读线程刷新处理器缓存保证可见性**
    - 获取锁之后，需要刷新处理器缓存，使得前面写线程所做的更新可以同步到本线程
    - 释放锁需要冲刷处理器缓存
- **syn对原子性的保证**
  - 互斥性
- **可重入**
  - 当一个线程试图获取一个它已经获取的锁时，这个获取动作就自动成功（自己可以再次获取自己的内部锁）
- Java线程同步：synchronized锁住的是代码还是对象

- **显示锁ReentrantLock**
  - new ReentrantLock() **默认创建的为非公平锁**，如果要创建公平锁可以使用 new ReentrantLock(true)。
  - 底层：内存屏障 包括释放屏障 存储屏障 加载屏障 和 获取屏障
  - 缓存一致性协议
  - **ReentrantLock有哪些优势？**
    - ReentrantLock 具备**非阻塞方式获取锁的特性**，使用 tryLock() 方法。ReentrantLock 可以中断获得的锁，使用 lockInterruptibly() 方法当获取锁之后，如果所在的线程被中断，则会抛出异常并释放当前获得的锁。ReentrantLock 可以在指定时间范围内获取锁，使用 tryLock(long timeout,TimeUnit unit) 方法。
- **轻量级锁Volatile**
  - 具体看JMM小结
- **对比**
  - **synchronized 和 ReentrantLock 比较**
  - 第15讲 | synchronized和ReentrantLock有什么区别呢？
    - lock是**显示锁**，提供了一些内部锁不具备的特性，但并不是内部锁的替代品，**显示锁支持公平锁和非公平锁的调度方式，默认采用非公平调度。**
    - syn 内部锁简单，但是不灵活，显示锁支持在一个方法内申请锁，并且在另一个方法释放锁，显示锁定义了一个trylock方法 尝试去获取锁，成功返回 失败并不会导致其执行的线程被暂停而是直接返回false 可以避免死锁。 **不可中断。 无法知道获取到锁** 才导致lock产生 1.6锁优化 几乎和lock性能持平
  - **synchronzied和volatile比较**
    - synchronized 既能保证可见性，又能保证原子性，而 volatile 只能保证可见性，无法保证原子性。比如，i++ 如果使用 synchronized 修饰是线程安全的，而 volatile 会有线程安全的问题。
  - > **1.原始构成**

> synchronized是关键字属于jvm层面。
>
> monitorenter(底层是通过monitor对象来完成的，其实wait/notify等方法也依赖于monitor对象只有在同步块或方法中才能调用wait/notify等方法)
>
> monitorexit
>
> Lock是具体的类(java.util.concurrent.locks.Lock)是api层面的锁
>
> **2.使用方法**
>
> synchronized 不需要用户去手动释放锁，当synchronized代码执行完后系统会自动让线程释放对锁的占用。
>
> ReentrantLock则需要用户去手动释放锁若没有主动释放锁，就有可能导致出现死锁现象。
>
> 需要lock()和unlock()方法配合try{}finally语句块来完成。
>
> **3.等待是否可中断**
>
> synchronized不可中断，除非抛出异常或正常运行完成。
>
> ReentrantLock可中断。 1.设置超时方法 tryLock(long timeout,TimeUnit unit)
>
> 2.LockInterruptibly()放代码块中，调用interrupt()方法可中断
>
> **4.加锁是否公平**
>
> synchronized非公平锁
>
> ReentrantLock两者都可以，默认非公平锁，构造方法可以传入boolean值 true为公平锁，false为非公平锁。
>
> **5.锁绑定多个条件Condition**
>
> synchronized没有
>
> ReentrantLock用来实现分组唤醒需要唤醒的线程们，可以精确唤醒，而不是想synchonized要么随机唤醒一个线程，要么唤醒全部线程。

- **锁的机制时什么？**
  - 锁有一个专门的名字：对象监视器。当多个线程同时请求某个锁时，则锁会设置几种状态来区分请求的线程；
    - 1）connection list：所以所有请求锁的线程将被首先放置到该竞争队列中；
    - 2）entry lsit：那些有资格成为候选人的线程被移到 entry lsit；
    - 3）wait set：那些调用wait 方法被阻塞的线程放置到 wait set 中。

线程协作/通信

- **join**
  - **join() 方法有什么用**
    - Thread.join() 把指定的线程加入到当前线程，**可以将两个交替执行的线程合并为顺序执行的线程**。比如在线程B中调用了线程A的join()方法，直到线程A执行完毕后，才会继续执行线程B。其实底层是调用了wait方法，当线程A执行完毕，A会唤醒B 然后从阻塞状态到可运行状态
    - Java多线程中join方法的理解
    - 简谈Java的join()方法
- **wait/notify/notifyAll**
  - notify/notifyAll
  - 注意点 a.必须在同步块中使用，不然会抛出异常 非法的监视锁
  - b.必须要先等待后唤醒，线程才能被唤醒

- **await/singal/singalAll**
- **LockSupport**
  - **是什么可以说一下吗？**
    - locksupport是一个创建锁和用于同步类相关的阻塞和解除阻塞的原语操作
  - unpark 解除阻塞 park 阻塞

AQS工具类

- 描述 AQS（AbstractQueuedSynchronizer）的作用（原题为 ReentrantLock里提供了一个很好的工具，你知道这个工具是什么吗？）
  - https://tech.meituan.com/2019/12/05/aqs-theory-and-apply.html
  - 是什么？
    - 是用来**构建锁**或者其它**同步器组件**的**重量级基础框架**及整个JUC体系的基石， 通过**内置的FIFO队列**来完成资源获取线程的排队工作，并通过一个int类变量表示持有锁的状态
  - 作用
    - 该类提供了一种框架，用于实现依赖FIFO等待队列的阻塞锁和同步器
  - 同步器和锁
    - 锁
      - 面向的是使用者 进一步封装了锁的具体过程，程序员只需要API相关的调用即可
    - 同步器
      - 面向锁的实现者 提出统一规范并简化实现锁的难度，屏蔽了同步状态管理，阻塞线程排队和通知，唤醒机制等。
  - 重要属性
    - **state-同步状态字段**
      - 代表加锁的状态 初始值为0
        - 0代表没有占用 1代表占用了 大于1代表重入锁
      - volatile 修饰保证线程间的可见性。
      - getState 同步状态当前值
      - setState 设置同步状态的值
      - *compareAndSetState* - 如果当前状态值等于期望值，则以原子方式将同步状态设置为给定的更新值。此操作具有 volatile 读和写的内存语义。
    - **CLH队列Node节点**
      - 尾部入队 头部出队
      - **Node节点**
        - `SHARED` 线程以共享模式等待锁
        - `EXCLUSIVE` 线程以独占模式等待锁
        - `waitStatus` 当前节点在队列中的状态
          - `volatile int waitStatus` Node的**等待状态waitState成员变量**
          - 状态
            - 0 Node初始化的默认值
            - `CANCELLED=1` 被取消申请锁的请求
            - `CANCELLED=-2` 节点在等待队列中，节点线程等待唤醒
            - `PROPAGATE=-3` 在SHARED下使用
            - `SIGNAL=-1` 线程准备好 等待释放资源

    - **支持模式**
        - 此类支持**独占模式（默认）和共享模式**:
            - 当以独占模式acquire时，其他线程尝试的acquired无法成功。独占模式有时只用 Sync Queue，但若涉及 Condition，则还有 Condition Queue。
            - 由多线程acquire的共享模式可能（但不一定）成功。共享模式时只用 Sync Queue。
- **CountDownLatch 门栓**
    - 让一些线程阻塞直到另一些线程完成一系列操作后才被唤醒。CountDownLatch主要有两个办法，当一个或多个线程调用await方法时，调用线程会被阻塞。其他线程调用countDown方法会将计数器减一(调用countDown方法的线程不会阻塞)当计数器的值变为零时，因调用await方法被阻塞的线程会被唤醒，继续执行。
    - 如何实现计数的？
        - **AQS**
    - **使用场景**
        - 人满发车
    - **典型用法**
        - 一个线程等待多个线程执行完毕，在继续自己的工作
        - 多个线程等待某一个线程的信号，同时开始执行
- **CyclicBarrier 循环栅栏**
    - cyclicBarrier 的字面意思**可循环(Cyclic)使用的屏障(Barrier)**,要做的事情是，让一组线程到达一个屏障(也可以叫做同步点)时被阻塞，直到最后一个线程到达屏障时，屏障才会开门，所有被屏障拦截的线程才会继续干活，线程进入屏障通过CyclicBarrier的await方法。
    - **现在需要测试系统的高并发性能，如何模拟高并发？**

      使用 cyclicbarrier，cyclicbarrier 初始化时规定一个数目，then 计算调用了 cyclicbarrier.await()

      进入等待的线程数，当线程数达到这个数目时，所有进入等待状态的线程将被唤醒并继续。

      Cyclic 就像他的名字一样，可看成是一个屏障，所有线程必须到达后才可以一起通过这个屏障。
    - CountDownLatch、CyclicBarrier作用是：若干个线程并发，直到最后一个线程完成才会往下执行，否则先到达的

      等待。又称之为线程屏障。可以设置等待超时时间。如果wait(0)，则永远等待下去。对于多个线程协同合作组合结果的场 景使用绝佳！比如：需要统计一个excel表格。里面有很多个sheet。每个sheet页都有一个结果。但是只有最后一个sheet页统计出来才能完整的统计出结果。先完成的就等待后面的完成才能继续。理解很简单吧。
- **Semaphore 信号量**
    - 信号量主要用于两个目的，一个是用于多个共享资源互斥使用，另一个用于并发线程数的控制。
- **CountDownLatch、CyclicBarrier区别**
    - CountDownLatch计数器只能使用一次。而CyclicBarrier可以使用reset()重置。
- **你可以大概说一下AQS的流程吗？**
    - 大概的整体流程是这样的，首先，我们创建三个线程A,B，C。线程A先拿到锁，执行任务。而B、C一直那不多锁，不能执行任务。被park。等A指向完毕之后，unpark(B);
    - `lock.lock()` 线程A通过CAS 设置上锁。而等线程B去获取锁的时候，CAS获取不到锁。于是进入 `acquire(1)` 进入 `nonfairTryAcquire` 再次尝试获取锁，获取不到、直接返回false。进入 `addwaiter()` 将当前节点添加到队列中 `enq(node)` ，因为 `t==null` ，所以先创

建一个哨兵结点。然后第二次自旋，将当前节点Node(ThreadB)，添加到队列中。调用`acquireQueued()`，拿到当前节点的前置节点。第三次获取，获取不到。进入`park()`等待。等待线程释放锁，`unpark()`操作。而在此时，线程A执行任务完毕，进行`lock.unlock()`操作。执行`release(1)`,通过head节点将下一个节点进行`unpark()`操作。而因为线程B被park()了，所以下一次就可以获取到锁，将队列中的哨兵结点进行修改。

JUC工具类

- 1.FutureTask
- 2.并发队列
  - 阻塞队列
    - BlockingQueue
    - **LinkedBlockingQueue是一个线程安全的阻塞队列，实现了先进先出等特性。**
  - 非阻塞队列
    - ConcurrentLinkedQueue 高效非阻塞并发队列【线程安全的LinkedList】
- 3.ForkJoin

JMM内存

- Java内存模型和线程基本操作
- 每个线程都有一个工作内存，线程只可以修改自己工作内存中的数据，然后再同步回主内存，主内存由多个内存共享。

  下面 8 个操作都是原子的，不可再分的：

  1）lock：作用于主内存的变量，它把一个变量标识为一个线程独占的状态。

  2）unlock：作用于主内存的变量，他把一个处于锁定状态的变量释放出来，释放后的变量才可以被其他线程锁定。

  3）read：作用于主内存变量，他把一个变量的值从主内存传输到线程的工作内存，以便随后的 load 操作使用。

  4）load：作用于工作内存的变量，他把 read 操作从主内存中得到的变量值放入工作内存的变量副本中。

  5）use：作用于工作内存的变量，他把工作内存中一个变量的值传递给执行引擎，每当虚拟机遇到一个需要使用到变量的值得字节码指令时将会执行这个操作。

  6）assign：作用于工作内存的变量，他把一个从执行引擎接收到的值付给工作内存的变量，每当虚拟机遇到一个给变量赋值的字节码指令时执行这个操作。

  7）store：作用于工作内存的变量，他把工作内存中一个变量的值传送到主内存中，以便随后 write 使用。

  8）write：作用于主内存的变量，他把 store 操作从工作内存中得到的变量的值放入主内存的变量中。

原子性 可见性 有序性 是多线程环境编程的线程安全的体现、

- **可以谈谈你对JMM的理解吗？**
  - 在我看来，JMM是对于现有计算机内存模型的抽象，原因在于处理器和内存之间速度差异导致 性能的下降，所以引入高速缓存、寄存器。将内存之间的数据进行同步到两者中。而对于处理器来说 不同的线程操作不同的寄存器数据 是不可见的，因此，需要必然会存在数据之间

不一致的问题，而JVM 对这种模式进行了抽象化 也既JMM，抽象为三个组件部分 **线程->工作内存->主内存**

- **原子性**
  - **可以说一下什么是原子性吗？**
    - 对共享变量访问操作，不管是查看还是修改都不允许出现一个不一样的状态，不可分割的。比如一个函数的功能是实现i++ i-- 那么不能出现一个中间状态，也就是i++ 执行完毕之后的数值 被其他线程访问到，那么就会出现并发线程安全问题，中间结果的不可见性 对于其他线程来说是透明的。
  - **有什么实现方式吗？**
    - **锁的排他性**，同一时刻，只有一个线程可以访问共享变量。
    - **CAS机制**
    - **语言规范**,除了long 和 double 其他的变量 写操作是原子性的
    - **Volatile** 可以保证修饰的变量写操作的原子性
  - **原子性有哪些注意的点吗？**
    - **原子性针对的是共享变量，局部变量不会存在共享问题**，也既不会存在并发问题。不会存在安全问题主要是 数据不会被多个线程共享访问修改，局部变量存储在栈中，一个方法从栈帧的入栈和出栈操作 只有一个线程可以访问到，所以是安全的？
    - **单线程环境下讨论原子性 没有意义**
    - **volatile只保证变量写操作的原子性，读写符合操作是不保证的。**
- **可见性**
  - **可以说一下什么是可见性的理解吗？**
    - 一个线程对于共享变量的更新，在后续操作中访问该变量是否可见。至于为什么会出现数据的不一致在于，处理器和内存之间 由于速度之间的量级差异，导致了性能问题，而在计算机中解决问题无外乎加入一个中间层，而形成了 处理器->寄存器、告诉缓存->内存 三层架构。而处理器每次都会优先从寄存器中读取已有的数据，而寄存器对于每个处理器来说并不是共享的。所以形成了 寄存器和内存数据的不一致性问题。所以也就出现了可见性问题、
  - **单处理器中，为什么会出现可见性的问题呢？**
    - 单处理器中，由于是并发多个线程执行，每个线程会保存当前自己独有的数据，其余线程是无法看到更新的数据，所以也存在共享数据不可见问题
  - **针对于可见性问题，你有什么解决方案吗？**
    - 当对数据读取的时候，**强制刷新处理器缓存**。将每个处理器更新的操作同步到当前通道中
    - 当对数据写入的时候，**冲刷处理器缓存**
- **有序性**
  - **说一下对有序性的理解？**
    - 一个处理器上线程运行的内存访问操作在另外一个处理器上运行的线程来看是否有序的问题？
  - **为什么要进行重排序呢？**
    - 提高程序的执行性能，编译器在任务不影响程序正确的性的前提下，会对源代码进行一定的调整，代码顺序，说白了是对内存读写操作的一种优化。指令重排序不影响单个线程，但是影响线程并发执行的正确性
  - **可以举一个栗子谈一下有序性吗？**
    - Instance instance = new Instance();
      - 在堆内存分配对象空间 通过指针碰撞或者空闲列表方式
      - 在堆内存初始化对象
      - 将堆地址指向instance

        - 2 3步可能会发生重排序，导致引用变量指向了一个不为null 但是也不完整的对象。**在多线程的单例模式中，必须通过volatile来禁止指令重排序**
    - **总结**
        - 原子操作是一组要么完全发生，要么没有发生，其余线程不会看到中间过程的存在，注意 **原子操作+原子操作 不一定是原子操作**
        - 可见性是一个线程对共享变量的更新对于另外一个线程是否可见的问题
        - 有序性是按照什么顺序执行的问题
        - **原子性+可见性->有序性**
- **先行发生原则** `todo`
- volatile 总线风暴
    - **介绍 volatile 关键字**
        - 保证变量的可见性（指示 JVM，这个变量是不稳定的，每次使用它都到主存中进行读取） & 防止指令重排序
        - 只能用于变量
        - 对一个 volatile 变量的写操作， Happens-Before 后续对这个 volatile 变量的读操作
        - **原理**
            - 观察加入 volatile 关键字和没有加入 volatile 时所产生的汇编代码，发现加入 volatile 时，会多出一个 lock 前缀指令，lock 前缀指令相当于内存屏障，内存屏障提供 三个功能：1）它确保指令重排序时不会把其后面的指令排到内存屏障之前，也不会把前面的放在内存屏障之后，即在执行到内存屏障这句指令时，在他前面的操作已经全部完成。2）它会强制将对缓存的修改操作立即写入主存。3）若是写操作，它会导致其他 cpu 中对应的缓存行无效。
        - 2.volatile关键字
    - **volatile变量的开销**
        - 不会导致线程上下文切换，但是其读取变量的成本较高，因为其每次都需要从高速缓存或者主内存读取，无法直接从寄存器中读取变量
    - **什么场景下使用volatile?**
        - v**适合多个线程共享一个状态变量，锁适合多个线程共享一组状态变量，可以将多个线程共享的一组状态变量合并成一个对象**。从而代替锁。
    - **Volatile变量在多个线程下一定安全吗**
        - 不一定 Volatile只保证数据的可见性，有序性，不保证原子性。

线程安全

- 线程安全的几种方案
    - 互斥同步，阻塞同步 synchronized lock 悲观并发策略
    - 非阻塞同步，乐观并发策略，先执行，产生冲突，补偿机制，CPU的CAS，Unsafe类
    - 无同步方案，方法无状态，不依赖堆上数据和公共系统资源
    - ThreadLocal
- **CAS**
    - **了解CAS的底层吗？**
        - UnSafe 直接通过内存偏移地址就可以访问
            - except value
            - old value
            - new value
    - **存在哪些问题？**

        - 1.循环时间长开销大
        - 2.只能保证一个共享变量的原子操作 多个变量的原子操作需要同步处理
        - 3.ABA问题
            - **你了解CAS的ABA问题吗？**
                - **ABA的解决方案呢？**
                    - **ABA(时间锁/版本号)**
                        - `AtomicStampedReference`
- **AQS(同步器和lock锁的基础)**
    - **核心组件**
        - State
        - FIFO队列
        - 协作工具类实现的获取、释放

锁相关

- **公平锁和非公平锁**
    - 前提概要 **JVM对资源的调度分为公平调度和非公平调度**
    - （先申请先得到）多个线程在等待同一个锁时，必须按照申请锁的时间顺序来依次获取，而非公平锁则不保证这一点。Synchronized 不是公平锁，reetrantlock 默认下也是非公平的，但是在构造函数中，可以设置为公平的
    - 公平调度
        - 按照申请的先后顺序授予资源的独占权
        - 体现:公平锁 lock
    - 非公平调度
        - 资源的持有线程在释放锁的时候，通知某一个线程B接着去执行，但是B从内核态到用户态需要一定的上下文切换时间，而这个间隙 C线程可以优先获取锁执行，当C执行的占用资源时间不长，释放锁后，B刚到达获取锁，那么就不会印象程序的并发执行。
        - 体现:非公平锁 syn lock
- 偏向锁
- **可重入锁和不可重入锁**
    - 可重入锁
        - 同一个线程外层函数获得锁之后，内层递归函数仍然能获取该锁的代码。**前提锁对象必须是同一个对象** ，从另外一个角度可重入锁，可以避免死锁，你想我之前获取了锁，在执行的时候，递归调用同一个同步方法，被阻塞了。显然是不合理的。
        - **可以详细说一下syn的可重入的原因吗？**
            - **锁对象内部维持了锁计数器和一个持有该锁线程的指针。**
        - `synchronized`和`ReentrantLock` 都是可重入锁.syn可重入的原因，在于monter记录了当前进入锁的次数，而lock依赖于aqs的state的次数。
    - 不可重入锁
        - CAS
- 共享锁和拍他锁
- **自旋锁和阻塞锁**
    - 自旋锁
        - 尝试获取锁的线程不会立即阻塞，而是采用循环的方式去尝试获取锁 好处是减少线程上下文切换的消耗，缺点是循环会消耗CPU

        - **互斥同步对性能最大的影响是阻塞的实现**，挂起线程和恢复线程的需要转入内核态中完成。若物理机器上有一个以上的处理器，能让 2 个 or2 个以上的线程同时并行执行，我们就可以让后面请求锁的那个线程“稍等一下”，但不放弃处理器的执行时间，看看持有锁的线程是否很快就释放锁，为了让线程等待，我们只需要让他执行一个忙循环（自旋），这项技术就是所谓的自旋锁。
        - **自适应自旋锁的自旋时间不再固定**，而是由前一次在同一个锁上的自旋时间及锁的拥有者的状态决定。
- 可中断锁
    - syn不可中断
- 轻量级锁：传统的锁是重量级锁，他是用系统的互斥量来实现。轻量级锁的本意是在没有多线程竞争的前提下，减少传统的重量级锁使用操作系统互斥量产生的性能消耗。
- 偏向锁：目的是消除数据在无竞争情况下的同步原语。如果说轻量级锁是在无竞争的情况下使用 CAS 操作去消除同步使用的互斥量，那么偏向锁就是在无竞争的情况下把整个同步消除掉。
- **锁消除：** 锁消除是指虚拟机的编译器在运行时，对一些代码上要求同步，但是被检测到不可能存在共享数据竞争的锁进行消除。
- 重量级锁
- 读写锁/独占锁 共享锁
    - 写锁
        - 指该锁一次只能被一个线程所持有，对ReentrantLock和Synchronized而言都是独占锁。
    - 读锁
        - 指该锁可以被多个线程所持有 对`ReentrantReadWriteLock`其读锁是共享锁
- 悲观锁
    - 悲观锁：就是很悲观，每次去拿数据的时候都认为别人会修改，所以每次在拿数据的时候都会上锁。这样别人想拿这个数据就会 block 直到它拿到锁。传统的关系型数据库就用到了很多这种机制，比如行锁，写锁等，都是在操作之前上锁。
    - 使用场景
- 乐观锁
    - 乐观锁：就是很乐观，每次去拿数据的时候都认为别人不会修改，所以不会上锁，但是在更新的时候会判断一下在此期间别人有没有去更新这个数据。适用于多读，比如 write_condition.两种锁各有优缺点，不能认为一种比一种好。
    - 使用场景
        - 读多写少的情况下，降低冲突。减少锁的开销，加大系统吞吐量。
    - **乐观锁与悲观锁的概念，常见实现（阿里）**
        - 乐观锁适用于**多读**的应用类型，这样可以提高吞吐量 / 悲观锁适合于**多写**
        - 乐观锁常见实现 --> 版本号机制 / CAS 算法
            - CAS 算法概念 / 缺点
                - 自旋 --> 循环尝试
                - ABA 问题
        - [何谓悲观锁与乐观锁]
        - [Compare-and-swap Wiki]
- 自旋锁 锁粗化 锁消除 自适应自旋锁、锁消除的依据是逃逸分析的数据支持。锁粗化：将多个连续加锁解锁操作链接起来扩展成一个范围更大的锁

线程池原理

- **Java中如何使用线程池，以及线程池有哪些重要的参数（阿里）**
  - 生产者 - 消费者 模式的一种实现
  - 线程池的好处
    - **降低资源消耗 / 提高响应速度 / 提高线程的可管理性**
  - 使用方法
    - ThreadPoolExecutor类
      - 线程池中的每一个线程被封装成一个 Worker 对象，ThreadPool 维护的其实就是一组 Worker 对象
      - 继承关系模型
        - `Executor`->`ExecutorServicer`->`AbstractExecutoServicer`->`ThreadPoolExecutor`
      - 一些主要参数
        - corePoolSize --> 核心线程数
        - maximumPoolSize --> 最大能创建的线程数（可以理解为当任务量突然过大时的一种补救措施）
        - workQueue --> 工作队列，为 BlockingQueue，用于存储等待执行的任务
  - 《码出高效》 7.4.2（线程池源码详解，关键方法逐行解释）
  - 22 | Executor与线程池：如何创建正确的线程池？（基本介绍和主要参数）
  - 深入理解 Java 线程池：ThreadPoolExecutor（ThreadPoolExecutor 源码和关键类分析）
  - Java并发编程：线程池的使用（ThreadPoolExecutor 源码 和 基本用法）
  - 第21讲 | Java并发类库提供的线程池有哪几种？ 分别有什么特点？（介绍 Executor 框架和从设计思想角度介绍ThreadPoolExecutor类）
  - **线程池的创建？**
    - `newFixedThreadPool` **执行长期的任务，性能好很多**
      - 定长线程池，可控制线程最大并发数，超出的线程会在队列中等待
      - core = max Queue:->LinkedBlockingQueue
    - `newCachedThreadPool` **执行很多短期异步的小程序或者负载较轻的服务器**
      - 可缓存线程池，线程池长度超过处理需要，可灵活回收空间线程，若无可回收，则新建线程
    - core = 0, max = MAX Queue:->SynchronouseQueue 有新任务就创建一个新的线程
    - `newSingleThreadExecutor` **一个任务一个任务执行的场景**
      - 单线程化的线程池，只会使用唯一的工作线程来执行任务，按照顺序执行
      - core = max = 1, Queue->LinkedBlockingQueue
- **参数详解**
  - `corePoolSize` 线程池中的常驻核心线程数 < core:新建 >core : 缓冲队列
  - `MaxnumPoolSize` 线程池能容纳执行的最大线程数 必须大于1
  - `keepAliveTime` 多余的空闲线程的存储时间 默认大于corePoolSize 才会生效 curNum >corePoolSize 一定时间销毁
  - `UNIT` KeepAliveTime的单位
  - `workQueue` 任务队列 被提交但位未被执行的任务
    - `LinkedBlokingQueue`
    - `ArrayBlokingQueue`
    - `SynchronizedQueue`

- `threadFactory` 表示生产线程池中工作线程的线程工厂，用于创建一般用默认的即可
- `handler` 拒绝策略
  - 队列已满，工作线程>=max
  - `AbortPlicy` 抛异常
  - `CallerRunsPolicy` 回退到调用者
  - `DiscardOldestPolicy` 抛弃最老的
  - `DiscardPolicy` 直接丢弃任务
- **创建线程的流程可以说一下吗？**
  - 首先创建一个线程，刚开始小于core 直接创建，当线程数大于core的线程 放入工作队列中，当工作队列中也满了之后，那么就创建新的线程，当大于max，不在创建，执行拒绝策略。抛出异常、回退到调用者、丢弃最老的、直接丢弃。其中一个就是线程数在core<num<max 当空闲到keepliveTime时间会回收这些空闲线程。回缩到corePoolSize。其实上面说的这是ArrayBlockingQueue队列的工作机制。
  - 第二种 SynchornousQueue 核心数满了，新来一个任务，创建一个新的线程处理，达到最大，采取拒绝策略，CacheThreadPool就是使用这个工作队列，配置的最大线程数是Integer的最大值，
  - 第三种是配置LinkedBlockingQueue ,这个queue是没有大小的，可以无限排，最大线程数就没有意义，相当于是一个固定大小的核心线程池，FiexThreadPool就是这种形式。
  - 重点抛出线程创建流程中有三种Queue的选择，而最常见的是Array的形式。
  - **核心在于阻塞队列**
- 线程池的状态了解吗【微聚未来】
- **实际工作中你是如何创建线程池的？**
  - 其实 一般来说不推荐直接使用jdk 提供现有的创建机制，而需要按照我们机器的性能 以及业务的处理性能，一般来说分为两种一种**CPU密集型**，需要进行大量的计算处理，需要尽量减少线程数量 **CPU+1个线程的线程池** 另一种**IO密集型**，需要进行IO操作，会被阻塞，为了补浪费CPU的计算性能，应该多配置线程数，

- **阻塞队列**
  - 基本操作
    - 当阻塞队列是空时，从队列中获取元素的操作将会被阻塞。
    - 当阻塞队列时满的时，往队列里添加元素的操作将会被阻塞。
  - **为什么要使用阻塞队列呢？**
    - 好处是我们不需要关心什么时候需要阻塞线程，什么时候需要唤醒线程，因为这一切BolckingQueue都给你一手包办了，在concurrent包发布以前，在多线程环境下，我们每个程序员都必须去自己控制这些细节，尤其还要兼顾效率和线程安全，而这会给我们的程序带来不小的复杂度。

实现类	特点
ArrayBlockingQueue	**由数组结构组成的有界阻塞队列**
LinkedBlockingQueue	**由链表结构组成的有界(但大小默认值有Integer.MAX_VALUE)阻塞队列**
PriorityBlockingQueue	支持优先级排序的无界阻塞队列
DelayQueue	使用优先级队列实现的延迟无界阻塞队列
SynchronousQueue	**不存储元素的阻塞队列，也既单个元素的队列**
LinkedTransferQueue	由链表结构组成的无界阻塞队列
LinkedBlockingDeque	由链表结构组成的双向阻塞队列

        - **SynchronousQueue**
            - SynchronousQueue没有容量。与其他的BlockingQueue不同，SynchronousQueue是一个不存储元素的BlockingQueue 每一个put操作都要等到一个take操作。
- **了解java中的队列吗？说一下你的理解？**
    - 从宏观上有一个Queue接口，而代码中都是通过继承 实现，下层到底层。总体来说 双端队列 阻塞队列 非阻塞队列
- **线程池的关闭 了解吗？**
    - 一种是`shutdown`不会立即终止线程池，等所有队列执行完毕在会终止。
    - `shutdomnNow()`执行该方法，线程池立即变成stop，并试图停止所有正在执行的线程
- **开放题：自己设计一个线程池需要考虑哪些问题点？**
- **线程池如何执行任务的？**

  本质上就是通过while(true) 来判断当前任务队列中是否存在任务，线程池线程竞争队列中的任务。

ThreadLocal

- "它通常用于同一个线程内，跨类、跨方法传递数据"
- ThreadLocal 不继承 Thread，也不实现 Runable 接口，ThreadLocal 类为每一个线程都维护了自己独有的变量拷贝。每个线程都拥有自己独立的变量，其作用在于数据独立。ThreadLocal 采用 hash 表的方式来为每个线程提供一个变量的副本。 key为this value
- **内存泄漏问题**
    - 什么是内存泄漏？
        - 某个对象不在有用，但是占用的内存却不能被回收
    - Thread->ThreadLocalMap->Entry(key 为null) ->Value
    - **如何解决内存泄漏？**
        - 调用remove() 删除Entry对象
- Thread ThreadLocal ThreadLocalMap三者关系
    - 一个Thread对象中都持有一个ThreadLocalMap成员
    - 一个ThreadLocalMap中包含多个ThreadLocal（用到多个ThreadLocal）
- **应用场景**
    - 每次HTTP请求都对应一个线程，线程之间互相隔离，这就是ThreadLocal的典型应用场景
- **重新理解ThreadLocal？**

首先来说的话，为了实现跨方法的数据存储，设置了ThreadLocalmap,可以理解为一个thread类都包含有一个map用来存储线程范围内对象或者变量值。

如果使用普通的hashmap

如果是普通的hashmap的话，由于key value都是强引用，set本地线程变量的目的是不是在这个方法在进入到内层方法之后，还能使用这些变量值，一定要明白一点，设置变量是为了给谁使用。是外层方法设置好，在内层使用。

函数的调用 函数的调用本质就是一个递归栈的过程，只不过启动函数main方法是一切方法的源头，从根本上来 刚开始我们设定一个值，在函数内层调用的时候来使用这个变量。

使用的三种情况 通常来说，第一种 本层给本层使用 这个没有必要 第二种 外层给内层使用 第三中 内层给外层使用

外层设置给内层用： 我们来假象如果key 和 value都是强引用，由于外层设置好了值，在内层中使用，但是由于内层中递归调用栈帧出栈操作，使用之后就不会在使用了。那么这个key 和value就是多余，当不及时回收的话，那么整个map中存储的数据就会越来越多。如果设置key为软引用呢，当发现只有自己一个人持有堆上对象时，说明其他程序是不会存在这个对象的强引用。本地线程的key就应该被回收了。 value为什么不设置为软引用呢。key是用来导航的，这个value可能来自本线程之外，如果key设置为软引用，很可能多个线程共享，其他线程获取的时候获取不到。 如果说，把value设置成若引用会导致这样一个问题，

试想这样一个场景，内层设置一个key 变量，弱引用的，由于内层会栈帧出栈，导致方法退出，那么强引用的变量会被设置为Null ，弱引用，gc的时候 会把map中的value清除掉。外层方法拿不到这个value值。因此，threadlocalmap不可能设置value为弱引用，必须使用强引用。但是为什么key可以设置成弱引用呢？

我们在内层方法中，设置了key 为弱引用，value为强引用。是不是面临和上面一样的问题。key会被回收了。外层方法获取key值时是不是先拿key，也就是说外层方法必须已经知道key，但是这是自欺欺人。如果在内层方法中new的，外层方法不可能获取该key的任何信息。这样的线程遍历，毫无意义。因为外层方法必须通过key来获取变量，所以，jdk设置为了弱引用。**内层方法设置线程给外层方法使用的例子** 所以不要内层给外层传。除非这个key是已知的。不会出现内存泄漏问题。

比如我们会外层方法中put (new A(),new B()) 外层方法都没有办法拿到这个引用，更何况内层方法。 **显然我们设定遍历一定是在外层方法中发挥作用，内层方法中也发挥作用。这样的变量才有意义。** 因此，外层传递给内层，一定是百分百持有该变量的。

再次强调value是强引用的原因，有可能存在内层往外层传递的情况 如果是弱引用 内层中new出来的对象，是没有办法使用的。

Java并发学习资源

- Java并发编程学习路线（学习思路篇）
- Java Concurrency in Practice（`TODO`）
- Java并发编程实战 （极客时间课程）
- 《码出高效》 7（几个并发工具类源码阅读）

问题迭代

- **为什么线程不安全呢？**
    - 当多个线程共享同一个变量，不进行同步读写操作，就会出现数据不一致问题。一般来说可以通过锁 原子变量，volatile。 基本上以下三种方式，可以修复。
        - 不在线程之间共享该状态变量
        - 将状态变量修改成不可变的
        - 在进行访问这个变量的时候，进行同步处理
- **可以说一下对多线程 高并发的理解吗？**

- 首先来说，对于一些用户量级比较少的系统应用来说，程序员大多不需要直接面对高并发 多线程问题，因为在数据库层面 中间件 框架中 数据库自身的特性 隔离性，事务特性就保证了并发问题，相关的锁机制等。但是，我们难免会遇到自己去设计一个相关的并发场景的问题。所以，就需要掌握相关的并发知识用以解决一定的业务问题。 目的 学**习多线程不是为了使用多线程，而是理解一些中间件框架的工作原理和方式。**
- 相关业务设计 ->**场景设计** 首先 对于千万级别的并发，我们需要提前设置一个map 进行 计算好 随机钱数和 份数，当在1分钟内，用户并发访问时，将用户抢到的红包，记录在一张临时的红包表中，等红包活动结束之后，启动一个后台线程进行账务业务的处理，转账对应的红包到用户中。重点在于 提前计算好 以及 活动之后的操作。
- **如何你面对的是一个千万级别的并发系统，你有什么设计方案 解决并发问题吗？**
  - **基本盘:根据实际的情况，精准定位问题的所在，不会盲目去做方案**
  - 横向拓展 (增加服务器数量)
    - **不能够解决的话，找到瓶颈点在于哪里?**
      - **数据库层面**
        - 读瓶颈
          - 读写分离
        - 写瓶颈
          - 分库分表
        - 表结构、索引等
        - **释放增加缓存**
        - 数据库表结构设计，索引，sql语句层面
      - **文件IO层面**
        - IO密集型 硬盘 增加机器 扩容
      - **网络带宽层面**
        - 网络带宽 构建CND集群

- **针对死锁问题你如何排查的？**
  - 通过jconslone工具
- **了解线程的调度策略吗？**
  - 线程体中调用了 yield() 方法，让出了对 CPU 的占用权；
  - 线程体中调用了 sleep() 方法，使线程进入睡眠状态；
  - 线程由于 I/O 操作而受阻塞；
  - 另一个更高优先级的线程出现；
  - 在支持时间片的系统中，该线程的时间片用完。
  - yield -> 让出 sleep->睡眠 IO->阻塞
- **了解线程的状态吗？**
  - 总体来说，线程的生命周期和人的生命周期相关。刚开始新创建(人出生)NEW 状态，还没有被调用，当一个线程调用start() 线程进入可运行状态，但是可运行状态包含两种 一个是线程在等待获取CPU时间片，或者线程已经在运行中，当线程进入一个syn 代码块中，但是获取不到锁的情况 进入阻塞状态，当再次拿到锁的时候，就再次进入可运行状态，但是当调用 wait 方法，等待唤醒，需要对应的线程调用notify/notifyAll 唤醒，当线程调用sleep(T), 就进入定时等待状态，当倒计时结束，恢复状态，最后一种就是线程终止，结束了，可能是正常结束，也可能是中途出现系统级别的异常，退出。
- **了解Thread类的优先级吗？**

    - 最小优先级是1 最大优先级是10 默认创建的线程是5 查看源码默认情况下，继承父线程的优先级。通俗一点说就是，创建线程的线程优先级是6 那么，创建的子线程就是6 **但线程优先级属性其实高度依赖于所处操作系统。Java 线程的优先级被映射到宿主机平台的优先级上时，优先级个数也许更多，也许更少。 所以一般也很少使用，但设置最低优先级是很方便的** - **守护线程有了解吗 及其作用？** - 为其他线程服务
        - **线程中断了解吗？**
            - 没有可以强制线程终止的方法，但 interrupt 方法可以用来请求终止线程。**没有任何语言方面的需求要求一个被中断的线程应该终止。中断一个线程不过是引起它的注意。被中断的线程可以决定如何响应中断。某些线程是如此重要以至于应该处理完异常 后， 继续执行，而不理会中断。但是，更普遍的情况是，线程将简单地将中断作为一个终止 的请求。**
- **实际项目中哪里用到了多线程？**
- **为什么要使用多线程 优势有哪些，用了就可以提高效率？**
- **线程安全问题是怎么产生的，如何解决？**
- **说一下你对多线程的理解？**
    - 首先来说，多线程的出现是随着进程级别资源的共享，为了压榨CPU时间片，所以，才使用多线程进行处理任务。因此，多线程的使用从根本上提高的执行效率。但是，引入了多线程也带来了一定的问题，比如数据安全，死锁问题。操作系统层面采用了一定的方案，但是对于JVM平台之上的程序员来说，JDK也提供了相关的互斥资源的访问，来保证数据的安全性。比如说常见的JMM 内存模型结构 volatile 保证主内存和工作内存，数据的可见性，有序性。相关的锁机制 syn lock锁保证在同一时间内，只会有一个线程访问同一个变量，但是绝对的互斥一定可以保证数据的安全性，但是对于一些只读的数据，我们可以采用读写分离的思想，copyonweiteList 等 将读和写 进行分离，保证读写之间的非互斥性。所以，总结一下，我们应该先明白多线程的背景，然后了解带来了问题，以及如何解决。三个点去理解。才能更好的解决问题。
- **死锁的理解？**

  死锁，从根本上来理解，就是一定是两个线程进行操作共享数据。具体的应用场景就是，比如，我们在操作数据库的时候，A事务和B事务都对C/D列进行了操作。只不过A先操作了C，然后B操作了D，当A操作D的时候，发现，我获取不到这个资源，B同样操作C，发现也获取不到这个资源，就僵死了。两个事务都执行不下去。发生了死锁，解决办法的话，一般，我们可以在开始就限定当前事务或者线程操作的资源，过程中别的线程不能操作，也就是锁的粒度进行粗化，另外一点就是保证获取锁的顺序性。A先执行完毕，B才可以获取锁。如此而已。

JVM ☆☆☆

JVM运行时数据区 ☆☆☆

> 前提概要 GC作用的区域是堆和方法区

- **堆** [[年轻代]eden s1 s2 复制算法 8:1:1] **15**-> [老年代] minGC **年轻代与老年代的比例是1:2** 【解决的问题点在于数据存储的问题，数据怎么放，放在哪?】
    - 一定15岁就进入老年代[不一定]
    - 大对象直接进入老年代
    - 含义：Java 堆（Java Heap）是 **JVM 中内存最大的一块，是被所有线程共享的**，在虚拟机启动时候创建，**Java 堆唯一的目的就是存放对象实例，几乎所有的对象实例都在这里分配内存**，随着JIT编译器的发展和逃逸分析技术的逐渐成熟，栈上分配、标量替换优化的技术将会导致一些微妙的变化，所有的对象都分配在堆上渐渐变得不那么绝对了。

如果在堆中没有内存完成实例分配，并且堆不可以再扩展时，将会抛出OutOfMemoryError。 Java 虚拟机规范规定，Java 堆可以处在物理上不连续的内存空间中，只要逻辑上连续即可，就像我们的磁盘空间一样。在实现上也可以是固定大小的，也可以是可扩展的，不过当前主流的虚拟机都是可扩展的，通过 -Xmx 和 -Xms 控制。

- 存放信息：**堆区存放的是程序员创建的对象**
- **堆内存上的对象回收也叫垃圾回收，那么垃圾回收什么时候开始呢？【超高频】**
  - 垃圾回收主要是完成**清理对象、整理内存**的工作。上面说到GC经常发生的区域是堆区，堆区还可以细分为新生代、老年代。新生代还分为一个Eden区和两个Survivor区。垃圾回收分为发生在**年轻代区域的回收(Minor GC)**和**老年代区域回收(Full GC)**
    - **Minor GC(年轻代GC)**
      - 对象优先在Eden区进行分配，当Eden区没有足够的空间，虚拟机发生一次MinorGC，因为Java大对象大都是朝生夕灭。所以Minor GC非常频繁，而且速度也很快。
    - **Full GC(老年代GC)**
      - Full GC发生在老年代的GC，当老年代没有足够的空间既发生Full GC,**发生Full GC一般都会有一次Minor**
      - **可以说一下Full GC 触发条件吗？**
        - 调用System.gc()
        - 老年代空间不足
        - 空间分配担保失败
        - JDK1.7及以前的永久代空间不足
        - Concurrent Mode Failure
  - **动态对象年龄判定**
    - **如果Survivor空间中相同年龄所有对象的大小总和大于Survivor空间的一半**，比如一共六个对象，三个对象是3岁，其中两个对象是2岁，相同年龄的3个3岁总和为9，而Survivor总和的一半是13,大于一半，那么**年龄大于等于该对象年龄的对象既可晋升到老年代。**
  - **空间分配担保**
    - 发生Minor GC时，虚拟机会检测之前每次晋升到老年代的平均大小是否大于老年代的剩余空间大小，如果大于，则进行依次Full GC(老年代GC)，如果小于，则查看HandlePromotionFailure设置是否允许担保失败，如果允许担保失败，会进行一次Minor GC, 不允许，则改为一次Full GC。

- **方法区**
  - 方法区（Methed Area）**用于存储已被虚拟机加载的类信息、常量、静态变量、即时编译后的代码等数据。** 永久代的垃圾回收主要回收废弃常量和无用的类。

    很多人把方法区称作“永久代”（Permanent Generation），本质上两者并不等价，只是 HotSpot 虚拟机垃圾回收器团队把 GC 分代收集扩展到了方法区，或者说是用来**永久代来实现方法区而已**，这样能省去专门为方法区编写内存管理的代码，但是在 JDK 8 也移除了“永久代”，使用 Native Memory 来实现方法区。 1.8为元空间 **方法区是一个概念，永久代和元空间只是实现而已。**

    当方法无法满足内存分配需求时会抛出 `OutOfMemoryError` 异常。
  - 1.8 前后变化，永久代被元空间替代
    - jdk8中出现了元空间替代永久代，元空间和永久带类似，都是对JVM规范中方法区的而实现，区别在于元空间并不在虚拟机中，**而是在本地内存**，默认情况下元空间的大小收到本地内存的限制。可以设置。

Jdk1.6及之前： 有永久代, 常量池1.6在方法区(永久代上) Jdk1.7： 有永久代，但已经逐步“去永久代”，字符串常量池，静态变量溢出在堆中 Jdk1.8及之后： 无永久代，常量池1.8在元空间

        - **为什么要使用元空间代替永久代？**
            - 字符串在永久代中，容易出现性能问题和内存溢出的问题。类和方法的信息等比较难确定大小，因此对于永久代的大小指定比较困难，太小容易出现永久代溢出。使用元空间则使用了本地内存。
    - **存放数据**：存放全局变量和静态变量+类信息(构造方法/接口定义)+运行时常量池
- **方法栈** 【栈不存在垃圾回收问题 解决的是程序运行的问题，如何执行，如何处理数据】
    - 含义：Java 虚拟机栈（Java Virtual Machine Stacks）描述的是 Java 方法执行的内存模型，每个方法在执行的同时都会创建一个**栈帧（Stack Frame）用于存储局部变量表(编译器确定下来的)、操作数栈、动态链接、方法出口、堆内存中某个对象的引用变量等信息，每个方法从调用直至执行完成的过程，都对应着一个线帧在虚拟机栈中入栈到出栈的过程。**
        - 如果线程请求的栈深度大于虚拟机所允许的栈深度就会抛出 `StackOverflowError` 异常。
        - 如果虚拟机是可以动态扩展的，如果扩展时无法申请到足够的内存就会抛出 `OutOfMemoryError`异常。
        - **总结 a.超出规定的 `StackOverFlowError` b.动态拓展 `OOM`**
    - 与堆的区别(非逃逸对象的栈上分配)
    - **存放数据**：**栈区存放函数的参数值，局部变量的值等** 八种数据类型的变量+对象的引用变量+实例方法
    - **【了解栈帧中存储的是什么信息吗？】**
        - 本地变量：输入参数和输出参数以及方法区内的变量
        - 栈操作：记录出栈、入栈的操作
        - 栈帧数据：包括类文件，方法等
    - **说一下静态变量和局部变量的区别？**
        - 静态变量初始化的阶段分为两步 一个在准备阶段 会被赋予0值， 另一个在初始化阶段赋予程序员在代码中的初始值，
        - 而局部变量，没有准备阶段这一步骤，所以需要方法被调用的之后就必须赋予一个初始化的值，这也就是为什么我们局部变量需要初始化的原因了。
    - **i++和++i 放在字节码中分析一下？**
    - **基于栈式架构的虚拟机地址紧凑，但是对于一个数的操作要进行出栈和入栈 是否性能上有降低，有什么优化吗？**
        - Hop Sopt JVM的设计者们提出了**栈顶缓存(Tos Top-Of-Stack Cashing)技术**，将栈顶元素全部缓存到物理cpu的寄存器中，以此降低对内存的读、写次数，执行执行引擎的执行效率。
    - 注意点： **方法调用时，会创建栈帧在栈中，调用完是程序自动出栈释放，而不是gc释放**
- **本地栈**
    - 含义：本地方法栈（Native Method Stack）与虚拟机栈的作用是一样的，只不过虚拟机栈是服务 Java 方法的，而本地方法栈是为虚拟机调用 Native 方法服务的。

      在 Java 虚拟机规范中对于本地方法栈没有特殊的要求，虚拟机可以自由的实现它，因此在 Sun HotSpot 虚拟机直接把本地方法栈和虚拟机栈合二为一了。
    - **重新理解native方法**

        - 前言：我们知道可以通过C语言进行编写程序，经过编译之后，就可以执行。但是对于特定的函数调用，C语言也只直接调用现成的系统级别的函数。这些函数会被编译之后成功ABI。而Java中native方法则是调用系统函数，在程序员看来只是一个API。但不是操作系统提供的API。我们应该容忍这些，相当于在系统级上进行的应用层的编程。但是，我们又可以从另一个角度看，比如，我们可以直接通过处理器级别的指令直接写程序，不需要编译也可以执行。但是缺点就是效率低下。对比汇编就是如此。但是每个语言所解决的问题域是不同的。不能一视同仁。
        - **native的形态是什么样子？**

          我们知道通过编写`.java`文件 经过编译之后变成了`.class`而这个二进制文件就是被JVM所解析执行的，说白了class文件就是JVM的源代码。只不过为了实现跨平台，抽取了一层的JVM。那么native方法是怎么执行呢，native方法由于只是一种声明 编译之后还是原来的样子，但是当运行时，会调用系统库函数来执行对应的逻辑。

- **程序计数器**
    - 含义：程序计数器（Program Counter Register）是一块较小的内存空间，它可以看作是当前线程所执行的字节码的行号指示器。在虚拟机的概念模型里，字节码解析器的工作是通过改变这个计数器的值来选取下一条需要执行的字节码指令，分支、循环、跳转、异常处理、线程恢复等基础功能都需要依赖这个计数器来完成。**JVM中的PC寄存器是对物理PC寄存器的一种抽象模拟**

      由于 JVM 的多线程是**通过线程轮流切换并分配处理器执行时间的方式**来实现的，也就是任何时刻，一个处理器（或者说一个内核）都只会执行一条线程中的指令。因此为了线程切换后能恢复到正确的执行位置，每个线程都有独立的程序计数器。

      如果线程正在执行 Java 中的方法，程序计数器记录的就是正在执行虚拟机字节码指令的地址，如果是 Native 方法，这个计数器就为空（undefined），**因此该内存区域是唯一一个在 Java 虚拟机规范中没有规定 OutOfMemoryError 的区域。**

      为什么说如果执行native方法时，pc就为空呢，首先如果一个方法被修饰为native，是属于操作系统的代码，而java方法的执行需要JVM进行执行，程序的执行就是靠pc进行指引，在JVM中只管Java程序的执行，线程的陷入都需要维护这个PC计数器，所以PC就执行当前正在执行的地址。但是native方法，当加载到JVM中时，发现一个方法是native方法，是一个标记，就会调用系统函数来执行，而具体的程序执行逻辑，并不属于JVM体系结构内的事情，自然维护JVM就是空。而native方法的执行就要靠系统的PC进行维护。**也就是说Java方法工作在JVM中，而native方法工作在操作系统层面中。**

      `xx.java`文件 其实就是JVM进程执行的一个资源，JVM并没有做全部的事情，JVM通过解析`x.java`程序来执行对应的逻辑。而另一部分的执行 调用操作的基础函数代码库来执行，而操作系统就是一个基础的代码池。程序员只需要进行API级别的调用，忽略其细节。
    - **唯一不会OOM的区域**
- **full GC 代码那些影响？**
- **full GC触发条件、Minor GC**
- **OOM 介绍 ？**

  OOM是一个比较大的话题，我们需要对于不同运行时方法区 分开看，比如常见的就是栈溢出，堆区中 超过堆内存限制，或者堆GC 空间很少，Direct buffer 直接内存分配超过限制 以及系统限制每个进程创建的线程数。
    - **StackOverflowError**
        - **区域：栈**
        - **原因：**递归调用 导致超出在栈空间中所限制大小
        - **解决方案：** 规范递归调用的终结条件
    - **OutOfMemoryError:Java heap space**

        - **区域：堆**
        - **原因：**对象大小超出堆内存大小
        - **解决方案：** 扩大内存，减少不要对象的内存占用
    - **OutOfMemeoryError:GC overhead limit exceeded**
        - **区域：堆**
        - **原因：** GC回收内存比较少，很快又占满了。
        - **解决方案：** ？？
    - **OutOfMemeoryError:Direct buffer memory**
        - **区域：物理内存**
        - **原因:** NIO 在操作物理内存分配的内存 超过了物理内存
    - **OutOfMemeoryError:unable to create new native thread**
        - **区域: 系统限制**
        - 原因 每个系统限定一定数量的线程数
    - **OutOfMemeoryError:Metaspace**
- **对象的栈上分配 ？JIT编译器**
- **如何解决互联网应用中gc停顿问题，如何解决碎片问题？**

垃圾回收算法 ☆☆☆

- **复制算法**
    - 含义：复制算法是将**内存分为大小相同的两块，当这一块使用完了，就把当前存活的对象复制到另一块，**
    - **然后一次性清空当前区块**。不会有内存碎片。空间上有浪费。
    - 缺点：空间占用 **空间利用率** 此算法的缺点是只能利用一半的内存空间。
- **标记清除**
    - 含义：此算法执行分两阶段，**第一阶段从引用根节点开始标记所有被引用的对象**，第二阶段**遍历整个堆，把未标记的对象清除。此算法需要暂停整个应用，同时，会产生内存碎片**。
    - 先标记-后清除 **内存碎片 标记-清除效率不高**
- **标记整理**
    - 含义：此算法结合了“标记-清除”和“复制”两个算法的优点。分为两个阶段，第一阶段**从根节点开始标记所有被引用对象**，第二阶段**遍历整个堆，把清除未标记对象并且把存活对象“压缩”到堆的其中一块**，按顺序排放。此算法避免了“标记-清除”的碎片问题，同时也避免了“复制”算法的空间问题。
    - 先标志-后整理 不会有**内存碎片**
- **分代算法** (主流)
    - 主要在堆中 **新生代->[复制算法] 老年代->[标记整理、标记清除] 比例：1:2**
    - 一个对象的一生：我是一个普通的Java对象，我出生在Eden区，在Eden区我还看到和我长的很像的小兄弟，我们在Eden区中玩了挺长时间。有一天Eden区中的人实在是太多了，我就被迫去了Survivor区的“From”区，自从去了Survivor区，我就开始漂了，有时候在Survivor的“From”区，有时候在Survivor的“To”区，居无定所。直到我18岁的时候，爸爸说我成人了，该去社会上闯闯了。于是我就去了年老代那边，年老代里，人很多，并且年龄都挺大的，我在这里也认识了很多人。在年老代里，我生活了20年(每次GC加一岁)，然后被回收。
    - 新生代 **Eden、From Survivor、To Survivor 比例 8:1:1**
        - **能说一下 新生代垃圾回收的过程 ？**
            - Eden区+From Survivor 存活的对象 拷贝到To Survivor区
            - 清空Eden区 和 From Survivor区
            - From Survivor区 和To Survivor分区交换(from->to to->from)
        - **为什么新生代要分两个Survivor分区呢？**

        - 空间利用率和程序运行的效率都是最优的。
        - 如果只有一个s区，每次垃圾回收之后，存活的对象都会进入到老年代，老年代会满了之后 会触发`Full GC`,比较耗时，所以合理调整比例，可以优化调优
        - 不需要新生代频繁GC
        - 利用率比较高 年龄到15升级到老年代。
        - **Survivor的存在意义，就是减少被送到老年代的对象，进而减少Full GC的发生，Survivor的预筛选保证，只有经历16次Minor GC还能在新生代中存活的对象，才会被送到老年代。**
        - **设置两个Surivovr区的最大的好处就是解决了碎片化、**

          为什么一个Survivor 不行呢

          假设 我们只有一个Survivor区，新创建的对象存储在Eden区中，当Eden区满的时候，触发一个Major GC，将存活的对象 转存到Survivor区中。反复执行，当Eden区再次满了，此时进行 Major GC,因为Eden区中 和 Survivor中都或有存活的对象，而直接将Eden区的对象转存到Survivor中。S区中会有碎片。

          而如果是两个s区的话，当Eden区满 触发Major GC，因为S1区 GC会有碎片，所以 直接将存活的对象 存储到S2，就可以防止内存碎片化。如果反复执行。s1 和 s2 来回复制。如果对象复制达到16次 会被送到老年代中。
    - GC停顿 stw 安全点
    - **什么时候发生Yong GC？针对年轻代**
        - 当`Eden`区满了之后，就会触发Young GC。
    - **什么时候会发生Full GC?**
        - 分配担保过程中，当不允许担保失败时，或者老年代中最大可用连续空间小于历次晋升到老年代中对象平均空间，触发`Full GC`

判断对象是否存活的条件

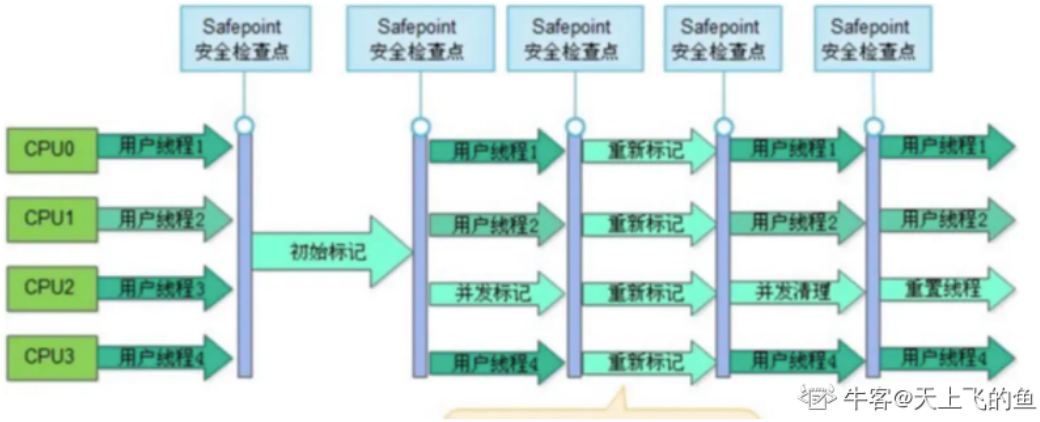
- **引用计数器算法** (强软弱虚)
    - 含义：引用计算器判断对象是否存活的算法是这样的，**给每一个对象设置一个引用计数器**，每当有一个地方引用这个对象的时候，计数器就加 1，与之相反，每当引用失效的时候就减 1。
    - **GC中循环引用问题**
        - 比如A中引用了B B中引用了A
- **可达性分析算法** 【HotSpot使用的】
    - 含义：在主流的语言的主流实现中，比如 Java、C#，甚至是古老的 Lisp 都是使用的可达性分析算法来判断对象是否存活的。这个算法的核心思路就是**通过一些列的“GC Roots”对象作为起始点，从这些对象开始往下搜索，搜索所经过的路径称之为“引用链”。当一个对象到 GC Roots 没有任何引用链相连的时候，证明此对象是可以被回收的。**
    - **那些对象可以当做gcRoot？栈内存中的引用变量是否都可以作为root?**
        - **Java虚拟机栈中的引用对象**
            - `Person p = new Person();`
        - 本地方法栈中JNI(既本地方法Native)引用的对象
        - **方法区中类静态引用对象**
            - `public static final Person p = new Person()`
        - **方法区中常量引用对象**
            - `public final Person p = new Person()`

        - 被启动类(bootstrap加载器)加载的类和创建的对象
    - **引出强软弱虚四种引用 两次标记过程(finalize方法)？**
        - 不管是引用计数法还是可达性分析算法都与对象的引用有关，对象的引用决定对象的生死。
            - **强引用**
                - **即使该对象以后永远都不会被用到JVM也不会回收，因此强引用是造成java内存泄漏主要原因**
                - `Person p = new Person()`; 只要对象引用存在 不会被回收。
            - **软引用**
                - `当内存不足时，才会回收软引用对象` `softReference`JVM确保在抛出OOM异常之前 清理软引用对象。
            - **弱引用**
                - 下次GC时会被回收 `WeakReference`
                - **可以谈谈弱引用和虚引用的使用场景吗?**
                    - 比如一个从本地磁盘上读取较多的图片信息系统，第一点 如果直接读取IO 和 网络比较耗时，如果一次全部加载内存会成为一个瓶颈。
                    - 比较好的一个设计思路 将图片路径作为key,value存储图片对应关联的对象 使用弱引用，每次GC，都会回收一部分对象数据。
            - **虚引用**
                - 无法获取一个虚引用的对象，目的在于 当这个对象被垃圾回收器回收时。可以收到系统的一个通知。`PhantomReference`
                - **目的：跟踪对象被垃圾回收的状态**
                - 结合finalize()
                    - 任何一个对象只会被系统执行一次`finalize()`
                    - finalize 一个对象死亡至少要经过两次标记。
                        - 第一次 发现没有对象可达的引用链。**第一次标记** 进行一次筛选。如果没有重写或者已经被调用过，则没有必要执行 否则执行
                        - 如果有必要执行 会被放入一个Queue队列 finalize方法是对象逃脱死亡命运的最后一次机会，只要在第二次finalize 中有引用链 重新连接 对象不会被GC

垃圾收集器 ☆☆☆

主要解决的问题消除或减少工作线程因内存回收而导致停顿的努力一直在进行着，

- **新生代回收器：**
    - **Serial**
        - **单线程收集器，需要stw Client模式下新生代收集器**
        - 优点：**简单高效**，没有线程交互的开销，专心做垃圾收集可以获得最高的单线程收集效率。
    - **ParNew**
        - Serial的多线程版本
    - **Parallel Scavenge**
        - 复制算法，目标达到一个可控制的吞吐量。 **自适应调节策略**
- **老年代回收器：**
    - **Serial Old**

        - 单线程收集器，标记整理 CMS 的后备预案 serial的老年代版本
    - **Parallel Old**
        - Parallel Scavenge的老年代版本 **追求吞吐量**
    - **CMS** ☆☆☆
- **CMS**
    -


    - **以获取最短回收停顿时间为目标**，基于**清除算法**，老年代收集器，过程分为四个步骤：
        - **初始标记**：标记 GC Roots 能直接关联的对象，速度很快。**STW**
        - **并发标记**：从 GC Roots 的直接关联对象开始遍历整个对象图，耗时较长但不需要停顿用户线程。
        - **重新标记**：修正并发标记期间因用户程序运作而导致标记产生变动的记录。**STW**
        - **并发清除**：清理标记阶段判断的已死亡对象，不需要移动存活对象，该阶段也可与用户线程并发。
    - **优点**
        - 降低垃圾回收的停顿时间为目的，很显然具有并发收集，停顿时间低的优点。
    - **缺点：**
        - ① 对处理器资源敏感，并发阶段虽然不会导致用户线程暂停，但会降低吞吐量。
        - ② 无法处理浮动垃圾，有可能出现并发失败而导致 Full GC。
        - ③ 基于清除算法，**会产生空间碎片**。大对象分配困难。
    - 相关问题
        - **如果在重新标记之前刚好发生了一次MinorGC,会不会导致重新标记阶段stw时间太长？**
            - **不会的，**在并发标记阶段其实还包括了一次并发的**预清理节阶段**，虚拟机会主动**等待年轻代发生垃圾回收**，这样可以将重新标记对象引用关系的步骤放在**并发标记阶段**，有效降低重新标记阶段stw时间。
        - **CMS垃圾回收器在那些阶段是没用用户线程参与的？**
            - 初始标记 和 重新标记
- **G1**
    - G1 GC 仍然存在着年代的概念，但是其内存结构并不是简单的条带式划分，而是类似棋盘的一个个 `Region`。`Region` 之间是**复制算法**，但整体上实际可看作是标记 - 整理（Mark-Compact）算法，可以**有效地避免内存碎片**，尤其是当 Java 堆非常大的时候，G1 的优势更加明显。**根据回收时间来优先回收价值最大的region**
    - 特点
        - 并行与并发
        - 分代收集
        - 空间整和
        - 可预测的停顿

    - 过程
        - 初始标记
        - 并发标记
        - 最终标记
        - 筛选回收
            - 首先对各个Region的回收价值饿成本进行计算，根据用户期望的GC停顿时间来指定回收计划。
- **ZGC** `todo`

  JDK11 中加入的具有实验性质的低延迟垃圾收集器，目标是尽可能在不影响吞吐量的前提下，实现在任意堆内存大小都可以把停顿时间限制在 10ms 以内的低延迟。

  基于 Region 内存布局，不设分代，使用了读屏障、染色指针和内存多重映射等技术实现可并发的标记整理。ZGC 的 Region 具有动态性，是动态创建和销毁的，并且容量大小也是动态变化的。

类加载器 ☆☆☆

- **类加载器**
    - 通过一个类的全限定类名来获取此类的二进制字节流，实现这个动作的代码就是类加载器。
    - **比较两个类是否相同，只要这两个类是同一个类加载器加载的前提下才有意义，**否

      则即使这两个类来源于同一个 class 文件，被同一个虚拟机加载，只要加载他们的加载器不同，他们就是不同的类
- **类加载器分类**
    - **启动类加载器** Bootstrap ClassLoader
        - 启动类加载器负责加载存放在 `jdk\jre\lib` 下，或被 `-Xbootclasspath` 参数指定的路径中的类，启动类加载器无法被 Java 程序直接引用。
    - **拓展类加载器** Extension ClassLoader
        - 拓展类加载器负责加载 `jdk\jre\lib\ext` 目录中，或者由 `java.ext.dirs` 系统变量指定的路径中的所有类库
    - **应用类加载器** Application ClassLoader
        - 应用类加载器负责加载到用户类路径(classpath)所指定的类，开发者可以直接使用该类加载器。若应用程序中没有定义过自己的类加载器，一般情况下，这个就是程序中默认的类加载器。
    - 自定义的类加载器
    - **双亲委派模型**

      若一个类加载器收到了类加载请求，它首先不会自己去尝试加载这个类，而是把所这个请求委派给父类加载器去完成，每一层的加载器都是如此，因此 all 加载请求最终都应该传送到顶级的启动类加载器。只有当父类加载器反馈自己无法加载时（他的搜索范围中没有找到所需的类）时，子加载器才会尝试自己去加载

      好处 双亲委派模型好处：eg，object 类。它存放在 rt.jar 中，无论哪个类加载器要加载这个类，最终都是委派给处于模型顶端的启动类加载器加载，因此 object 类在程序的各种加载环境中都是同一个类。
- **类加载器的职责**
    - **全盘负责**
        - 当一个类加载器负责加载某个class时，该class所依赖的和引用的class也将由该类加载器负责载入。除非显示使用另外一个类加载器来载入。

    - **父类委托**
        - 类加载机制会让父类加载器加载该类，只有父类加载器无法加载该类时才尝试从自己的类路径中加载该类。父类委托机制是为了**防止内存中出现多份同样的字节码**，保证java程序安全稳定运行
    - **缓存机制**
        - 缓冲机制将会保证所有加载过的Class都会被缓存，当程序中需要使用某个class时，先从缓存区寻找该class,只有缓存区不存在，系统才会读取该类的二进制数据，并将其转换成class对象，存入缓存区，这就是为什么修改了Class，必须重新启动JVM，程序的修改才会生效。
- **双亲委托机制**(如何破坏，loadClass 方法)
- 类加载机制，以一种什么形式存在于内存中，如何找到这个`.class`文件，什么时候需要类加载 `todo`
- 为什么需要这么多类加载器 ？ `todo`
- **结合类加载机制 说一下tomcat加载机制？**
- **重写理解类加载器？**
    - 首先一点，对于任意的`.class`文件，类加载器的作用就是将这个类似源程序的文件加载到JVM，JVM进行一系列的解析 验证 初始化，生成一个Class对象。但是对于类加载器来说，本身也是一个类，那么这个类是如何记载，所以最顶层并不是Java语言编写的，而是C++进行写的类加载器，也被称为Bootstrap类加载器，加载一些最基础的类，`rt.jar`，而除了基础的类，我们还有两个类加载器`Ext Classloader`和`App ClassLoader`分别加载加载对应的`ext.jar`和classpath路径下的类。
    - 好了，来梳理一下这个流程，比如我们自定义了一个包`com.lang.Object`，main方法启动之后，APP classLoad进行加载，首先他会查看之前是否加载过这个类，如果没有加载过，先问父类，父类也就是EXT有没有加载过，如果也没有，那么就会到系统类加载器，系统类加载因为会加载rt.jar 会加载一个Object，直接往下通知说，你不要加载了。我已经加载了过了。那么就可以保证一些非法的类被加载。
    - **后面的自定义Classloader的场景** 上面就是基本的知识点，但是对于tomcat下webapp下，我们可以部署多个项目，而多个项目中的自定义类是相互隔离的，这个是如何做的呢，其实这就是属于在一个java虚拟机进程中划分成不同的区，需要我们自定义classLoad的子类，这样就可以设定不同的classloader的区域。每个区域都有一个自定义的classloader类进行管理。简称为一个Web Context。而对于tomcat下lib目录下的jar 本身也是一个classpath。但是对于每个项目来说也是一个classpath，之前说的classpath是整个Java进程的，而这个classpath是针对特定的应用来说。我们应该理解这些。而不是记忆化记忆。
- **重新理解双亲委派？**
    - 首先说一下，默认存在三个类加载器bootstarp extstarp appstarp。没什么可以说的，启动一个main方法，当前这个类就是用户类，而非用户类就是系统的相关基础包，也就是bootstarp 和 extstarp相关的基础类。所以在main方法启动之前，会提前加载好一堆基础类。main线程先去加载主类，主类在classpath下，把主类加载完毕，就完成对类变量的初始化，初始化结束之后。main线程开始调用main方法，进入第一个方法。加载主类,app自己先不加载，而调用父类 ext 也不加载，调用boot ，只有Boot加载不了才会通知子类加载，所以，所有加载的一定是App。一旦加载成功，我们就可以调用主类的getClassLoader方法，查看被哪个类加载器加载的。**Classpath是什么样的存在的呢？**
    - 在启动JVM的时候，使用classpath参数指定的目录就是classpath。也就是APP加载的目录。
    - 好了，在main方法中，`A a = new A()` 那么这个类加载器是谁加载的呢？APP吗。其实不一定，这一行使用的类加载器是外围类的类加载器，外围类是主类，被APP加载。

- 如果在main所在的类中方法 `B b = new B()` 是那一个类加载器。在调用的时候，通过A的对象来调用，那么，这个方法内部的B类，就是被A使用的类加载器加载的。所在外围类的加载器。`Thread.currentThread.setContextClassLoader` 设置线程上下文的类加载器。假如在main方法过程中，我们定义了 `C c = new C()` 因为主类加载器是APP，但是如果这个C类是Ext下的拓展下的子类，那么就会被Ext拓展器加载。调用c.getClassLoadr 一定是Ext。如果这个C类中有一个方法 `D d = new D()` D类应该被哪个加载器加载，D类应该被Ext加载。如果这个D类在classpath下呢，是不是当前是不存在加载classpath的类加载器呢？ 所以 重点来了，最终加载一定会出现 class Not found 我们需要设置一个线程上线文类加载器，如果是一个第三方的类呢，**JDBC中的类被Boot加载，最后却需要到classpath下找驱动类，说白了，一些系统类需要反过来调用用户类** 根据双亲委派模型，这是打破了这种模型。解决方法呢？ 就是在线程上下文中，把App类加载器绑定。然后线程运行到需要D类的这一行的时候，通过getContextClassLoader把App取出来。让后使用App.classLoader方法去加载，这就是classLoader的引用。**主类的ClassLoader一定是App，如果需要App，一定通过主类去获取。主要就是避免进入一个类中，内部使用的加载器是外部类的。我可以强制让内部类代码使用我绑定的类加载器**，
- 如果一行代码是 `A a = new A() class.forName()` 使用的外围类类加载器。`如果使用的是XXXClassLoader.findClass()` 完全指定了哪个类加载器。`Thread.currentThread.getContextClassLoader` 把它取出来，绕过默认使用外围类的限制。
- tomcat是一个JVM进程，tomcat里也有boot ext app 前两者使用java目录下的lib 而 app使用tomcat下的lib目录。tomcat管理了很多的webapp,每一个webapp都是独立隔离的。
- 我们知道，tomcat 中可以并存多个 webapp，并且 tomcat 本身也是 java 程序，那么，tomcat 自身有依赖 的 java 类库，webapp 有自己独立的依赖 java 类库，webapp 有共享的 java 类库等。

  在 tomcat 的目录体系中，common 目录中的 java 类库可以被 tomcat 自身和所有的 webapp 使用；server 目录中的 java 类库只能 被 tomcat 自己使用而不能被 webapp 使用；shared 目录中的 java 类库可以被所有的 webapp 共享，但是不能被 tomcat 自身使用；

  WEB-INF 中的 lib 和 classes 的 java 类库只能独立的 webapp 使用而不对外暴露。

  那么，tomcat 为了实现这样的类库隔离性质，绝对不可能只是设置一个 classpath，如果只有唯一的 classpath，将 tomcat 包括 所有的 webapp 所依赖的第三方类库都丢在 classpath 中，那么就做不到这样的隔离性质了。
- common shared server目录，每个webapp下的lib目录。有三个webapp。现在启动tomcat就有三个ServletContext对象，使用了一个map 维护了这三个对象 key是String value分别是ServletContext对象。如何初始化呢？假如由于开发时间不同，使用的servlet版本是不同的，2.2 2.5 3.0，那么每个项目下都有对应的webapp-lib目录，每个项目启动的时候需要从自己对应的lib下加载目录。需要加载整个项目时，创建三个自定义的类加载器，X1 X2 X3 x1.findClass("com.ncst.XXX.class") 创建了三个ServletContext.class，然后用反射,newInstance() 三个实例。ssm框架中最外围的对象是不是ServletContext对象。Servlet是Spring mvc的入口，spring mvc中的controller,包括Spring 容器，也是在ServletContext中初始化的。是一个顶级对象。**classpath就是web-inf下的lib目录 以及classes目录了。** tomcat初始化三个ServletContext的时候，使用了三个自定义类加载器。相当于三个自定义的类加载器的私有空间下加载类。
- 针对于tomcat是否符合双亲委派模型？我们来分析。由于自定义的类加载器是继承App的，而我们的weblib下是没有基础类，并没有出现问题，所以一定是符合双亲委派模型。APP是每个自定义私有类加载器的parent,APP如果又被tomcat使用。是不是存在一问题，那就是tomcat和webapp 都是APP的类加载器子类，如果tomcat和webapp下都使用了jdbc，那么当加载的时候，webapp查找使用app进行加载，最终导致app没有加载到jdbc的包。最终报

错。因此，可以总结出tomcat弃用了classpath。comm是tomcat和webapp公用的，server是tomcat私有的，shared是被webapp共享。

- **为什么自定义类加载器？**

  因为灵活，默认app,ext,boot可以满足大多数情况的java应用程序，但是有一些特殊的情况需要我们更加细致的类加载。

- **从动态代理角度理解类加载器？**

  - 动态代理中，使用jdk实现，需要我们传递一个类加载器，为什么要类加载器呢，其实从根本上`.class`加载到内存中，就是一堆二进制流，而如何将这些文件加载到JVM中，就需要我们设定对应的类加载器。通过自定义类加载器，我们重写`findClass()`通过调用`super.defineClass()`;生成对应的Class对象，而动态代理的字节码是由程序中动态生成的。我们来分析一下，为什么需要类加载器呢，实际上，只是使用这个`defineClass()方法`方法来生成Class对象。并不是使用`findClass`或者`loadClass()`方法。字节码都是程序提前生成好的。**ClassLoader抽象类的defineClass已经帮我们实现了这个生成的Class的解析过程**，所有自定义的ClassLoader包括AppClassLoader都继承它。**动态代理生成的类名是程序生成的，然后把生成的字节流传递进去** 调用defineClass方法，返回Class实例。
  - 那么在程序中，我们需要定义类加载，而这个类加载器一般都是App,动态代理中，生成一个代理类，首先就是程序生成二进制数据，二进制字节码，是程序动态生成的。而需要一个类加载器去执行`defineClass()`而我们传递的类加载器就是执行这个`classloader`的`defineClass()`
  - **动态代理中自定义类加载器还是app呢？**

    App加载的类是全局存在的，所有被App加载的，在永久区永久存在。这就是全局对象。方法区对象的回收，有三个条件。1、该类的ClassLoader被回收 2、Class的所有实例化对象都回收 3、不在有任何对象可以通过类名访问这个Class。App是不会被回收的，方法区叫永久代就是这个原因。

    自定义类加载器，一定的时候，可以将自定义的累加器不可达，直接回收加载的对象。

JVM 对象分配回收策略

- **内存分配策略**
  - **对象优先在Eden分配**
  - 大对象直接进老年代
    - 避免在Eden区及两个S区之间来回复制，产生大量的内存操作
  - 长期存活对象进老年代
  - **动态年龄判定**
    - 为了更好的适应不同程序的内存状况。
    - S区中相同年龄的对象总和大于S区的一半以上，>=这个年龄的值可以直接到old区 无需等待15岁
  - **空间分配担保**
    - 具体见**遇见offer-JVM** 脑图
    - 在发生 minor gc 前，虚拟机会检测老年代最大可用的连续空间是否>新生代 all 对象总空

      间，若这个条件成立，那么 minor gc 可以确保是安全的。若不成立，则虚拟机会查看

      HandlePromotionFailure 设置值是否允许担保失败。若允许，那么会继续检测老年代最大可

用的连续空间是否>历次晋升到老年代对象的平均大小。若大于，则将尝试进行一次minor

gc，尽管这次 minor gc 是有风险的。若小于或 HandlePromotionFailure 设置不允许冒险，则

这时要改为进行一次 full gc。

- Minor gc前
  - 老年代最大可用连续空间大于新生代all对象总空间吗？ minor gc安全【极端情况下 所有新生代对象都直接到老年代中】
- 不安全
  - `HandlerPromotionFailure` 是否允许担保失败
    - 不允许 直接 Full GC 允许进行下一步
      - 老年代最大可用连续空间大于》历次晋升带老年代中对象的大小
        - 大于
          - 进行Minor GC
        - 小于 进行Full GC

- **对象回收？**
  - 回收与分配是相对的，在4大垃圾回收算法中，主要是那些标记为可以清除的对象，就可以被回收。

类加载流程

- **new的过程？**
- 总结一下

  1.遇到new先去检查指令参数看常量能否定位到一个类的符号引用。看是否解析 加载 初始化过 没有进行加载

  2.内存分配 两种 指针碰撞 空闲列表

  3.内存分配的安全性问题 1.乐观锁 2.TLAB

  4.初始化赋值。对象的init方法

  在Java中我们创建对象都会用new进行创建，下面我来接收一下new之后对象创建及内存分配的具体的过程

  一：虚拟机遇到一条new指令后，先去检查这条指令参数是否能在常量池中定位到一个类的符号引用，并检查这个符号引用代表的类是否已被加载、解析和初始化过，如果没有，那必须先执行相应的**类加载**过程。

  二：类加载检查通过后，接下来虚拟机为新生对象**分配内存**，因为对象所需内存的大小在类加载后是完全确定的(用引用，堆中存放实例),所以只需分配一个固定大小的内存即可。分配内存的方法用两种(Java虚拟机中)

  ①**指针碰撞**：假设Java堆中内存是绝对规整的，所有用过的内存都存放在一边，空闲的内存存在在另一边，中间放着一个以指针为分界点的指示器，分配内存就是把指针往空闲的那侧移动对象需要的内存即可。但是如果Java内存堆不是完整的，就没有办法进行简单的指针碰撞，

  ②**空闲列表**：空闲列表就是虚拟机先对内存分块，**并用一个表记录每个内存块是否使用的情况**，为对象分配内存时只需更新列表上的记录即可，选择哪一种分配方式取决于对中的内存使用情况是否完整。

  但是在为对象分配内存的同时，需要考虑其它线程是否也在这一时刻需要分配，因此需要考虑到线程的安全问题，通过以下两种方法进行解决：

①同步处理[**乐观锁**]：一种是对分配内存的空间动作进行同步处理--实际上虚拟机采用CAS配上失败重试的方式保证更新操作的原子性

②本地线程分配缓存TLAB:另一种是把内存分配的动作按照线程在不同的空间之中进行，及每个线程先预先分配一定的缓存。

TLAB的全称是**Thread Local Allocation Buffer**，即**线程本地分配缓存区**，这是一个线程专用的内存分配区域。:)

三：内存分配完成后，虚拟机将分配到的内存空间都初始化为零值(不包括对象头)，如果采用的是TLAB的方法分配，则这一步骤在分配之前完成。这一步操作保证了对象的实例字段在Java代码中可以不付初始值就直接使用，程序访问的值为false。这个解决了以前我在上Java课与老师争论在执行构造方法是否会先初始化，明显我对了，但是当时没有拿出有说服力的证据。

四：在上面的步骤完成后，从虚拟机的视角，一个新的对象已经产生了，但是从Java程序的视角来看，对象创建方法才刚刚开始，还要执行对象的init方法（构造方法），一个对象才完成创建

JVM加载一个类的整套流程说一下？

- 类从加载到虚拟机内存中开始，到卸载出内存为止，它的整个生命周期包括：加载-验证-准备-解析-初始化-使用-卸载，其中验证-准备-解析称为链接。
- **加载**
  - 将**类的`.class`文件中的二进制数据读入到内存中，将其放在运行时数据区的方法区内，然后在堆区创建一个java.lang.String对象，用来封装类在方法区内的数据结构。**
  - 在加载阶段，虚拟机需要完成下面 3 件事：

    1）通过一个类的全限定名获取定义此类的二进制字节流；`Class.forName()`

    2）将这个字节流所表示的静态存储结构转化为方法区运行时数据结构 内存结构

    3）在内存中生成一个代表这个类的 class 对象，作为方法区的各种数据的访问入口。
- **链接**
  - **验证**
    - 确保被加载的类的正确性，包括文件格式验证，元数据验证，字节码验证以及符号引用验证。
    - 验证的目的是为了确保 clsss 文件的字节流中包含的信息符合当前虚拟机的要求，且不会危害虚拟机自身的安全。验证阶段大致会完成下面 4 个阶段的检验动作：
      - 1）文件格式验证
      - 2）元数据验证
      - 3）字节码验证
      - 4）符号引用验证
      - {字节码验证将对类的方法进行校验分析，保证被校验的方法在运行时不会做出危害虚拟机的事，一个类方法体的字节码没有通过字节码验证，那一定有问题，但若一个方法通过了验证，也不能说明它一定安全}。
  - **准备**
    - 准备阶段为类的静态变量分配内存，并将其初始化为默认值，假设一个类变量的定义`public static int val = 3`，那么变量val在准备阶段过后的初始值不是3而是0。
      - 准备阶段是正式为类变量分配内存并设置变量的初始化值得阶段，这些变量所使用的内存都将在方法区中进行分配。（不是实例变量，且是初始值，若 public static int a=123;准备阶段后 a 的值为 0，而不是 123，要在初始化之后才变为 123，但若被 final 修饰，public static final int a=123;在准备阶段后就变为了 123）
  - **解析**
    - 将类中**符号引用转换为直接引用**，符号引用以一组符号来描述所引用的目标，符号可以是任何形式的字面量，只要使用时能够无歧义的定位到目标即可。

- **初始化**
  - 为类的静态变量赋予正确的初始值，JVM负责对类进行初始化，主要对类变量进行初始化。
  - **什么时候进行初始化操作？**
    - 在遇到下列情况时，若没有初始化，则需要先触发其初始化（加载-验证-准备自然需要在此之前）：
      - 1）1.使用 new 关键字实例化对象 2.读取或设置一个类的静态字段 3.调用一个类的静态方法。
      - 2）使用 java.lang.reflect 包的方法对类进行反射调用时，若类没有进行初始化，则需要触发其初始化
      - 3）当初始化一个类时，若发现其父类还没有进行初始化，则要先触发其父类的初始化。
      - 4）当虚拟机启动时，用户需要制定一个要执行的主类（有 main 方法的那个类），虚拟机会先初始化这个类。

（加载，验证，准备，解析，初始化）

```
    -  加载流程(加载[加载时候生成方法区对应的类数据结构，以及生成对应的class对象 [数组没有对应的类 如何生成]]，验证，准备[分配类变量空间，复制后延后到初始化]，解析，初始化[结合Java新建对象各部分执行顺序分析])
```

JVM调优

> 调优对于大多数人来说，就是进行参数配置调优，这个理解是很险隘的。应该以一个更加宏观的视角去看待。不同的维度。比如，根据gc的频繁与否，我们可以尽量将一些朝生夕死的对象在年轻代灭了，而不是在老年代。
>
> 我来举一个栗子，比如我们遇到一个高并发场景下的订单系统，创建一个订单需要0.2MB的对象。1S中处理200个订单，那么1S中就创建200MB的对象。
>
> 我们来看一下jvm内存划分，一共是3GB内存，eden区800MB s0 100MB s1 100MB，old区域2GB 我先普及一下基础知识点，对象默认创建的eden区域，对于大对象直接到old区域。在s区中只要相同年龄的对象大于s区的一半大小直接到old空间。或者经过固定年龄到old区域中。 好了，接着我们的分析。eden区域800MB，也就是大约需要8S eden区域就会满了。我们假定800MB对象100MB中不是垃圾，另外都是垃圾。那么经过一次YCG之后，就会剩余100MB对象，但是我们的S区域只有100MB，并且更糟糕的是由于这100MB对象由于年龄都是相同的，并且占有s区一半以上空间，会直接到old区中。而Old区域2GB，大概需要2.5min 就会被填满。进行依次Old YC oldGC是比较消耗时间的，需要进行标记清除。
>
> 好了，对于用户来说，2.5min就会发生一次停顿一次。不可忍受。
>
> 好了，有了具体的问题，我们来分析一下，应对的策略。一般我们都会加钱 扩容内存容量。这没问题。扩大新生代的空间，没问题吧，但是过大的新生代空间是进行YGC是比较耗时的。而我们采用第二种策略。
>
> 在原有基础上，修改一下新生代和老年代的空间的大小即可。
>
> 将新生代和老年代空间划分成 eden 1.4GB s0 300MB s1 300MB 老年代1GB，怎么样。由于之前100MB的对象不断进入到老年代中发生Old GC，这样我们可以将100MB对象拦截在S区中。避免发生Old GC。
>
> 上面只是一个维度，我们也可以从多个角度看 比如栈帧的空间是固定的1MB，可以减少栈帧的大小。或者并行的垃圾收集器，增大垃圾回收器。

- **相关工具**

- **jps：虚拟机进程状况工具**

  列出正在运行的虚拟机进程，使用 Windows 的任务管理器或 UNIX 的 ps 命令也可以查询，但如果同时启动多个进程，必须依赖 jps。`jps -l` 显示类名

- **jstat：虚拟机统计信息监视工具**

  监视虚拟机各种运行状态信息，显示本地或远程虚拟机进程中的类加载、内存、垃圾收集、即时编译器等运行时数据。

- **jinfo：Java 配置信息工具**

  查看虚拟机各项参数，使用 `jps -v` 可查看虚拟机启动时显式指定的参数，但如果想知道未显式指定的参数只能使用 `jinfo -flag`。

- **jmap：Java 内存映像工具**

  用于生成堆转储快照，还可以查询 finalize 执行队列、Java 堆和方法区的详细信息，如空间使用率，当前使用的是哪种收集器等。

- **jhat：虚拟机堆转储快照分析工具**

  JDK 提供 jhat 与 jmap 搭配使用分析堆转储快照。jhat 内置微型的 HTTP/Web 服务器，堆转储快照的分析结果后可以在浏览器查看。

- **jstack：Java 堆栈跟踪工具**

  用于生成虚拟机当前时刻的线程快照，定位线程出现长时间停顿的原因，如线程间死锁、死循环、请求外部资源导致的长时间挂起等。

- **jconsole 查看运行类信息**

  死锁

- **JVM调优参数配置**
  - **JVM堆内存的分配**
    - `-Xmx:512` 等价`-XX:MaxHeapSize`设置最大堆内存为 512 M；**【默认是物理内存的1/4】**
    - `-Xms:215` 等价`-XX:InitialHeapSize` 初始堆内存为 215 M；**【默认初始化为物理内存1/64】**
    - `-Xmn2G`设置年轻代大小为2G
    - `-XX:SurvivorRatio` 设置新生代 Eden、Form Survivor、To Survivor 占比。**【默认8:1:1】**
      - survivor 大了，会浪费空间，空间利用率低。**若 survivor 太小，会使一些大对象在 minor gc**

        时直接从 eden 区到 old 区，让 old 区的 gc 频繁 【JVM调优策略】
    - `-XX:NewRatio` 设置分代垃圾回收器新生代和老生代内存占比；**【默认1:2】**
    - `-Xmx10240m -Xms10240m -Xmn5120m -XXSurvivorRatio=3`
    - ```
      -Xmx10240m: 代表最大堆
       -Xms10240m: 代表最小堆
       -Xmn5120m: 代表新生代
       -XXSurvivorRatio=3: 代表Eden:Survivor = 3    根据Generation-Collection算法(目前大部分JVM采用的算法)，一般根据对象的生存周期将堆内存分为若干不同的区域，一般情况将新生代分为Eden ，两块Survivor;     计算Survivor大小， Eden:Survivor = 3，总大小为5120,3x+x+x=5120  x=1024
      新生代大部分要回收，采用Copying算法，快！
      老年代 大部分不需要回收，采用Mark-Compact算法
      ```
      ```

        - **堆内存上对象的分配与回收**
            - `-XX:MaxNewSize` 设置最大年轻区内存；
            - `-XX:MaxTenuringThreshold=5` 设置新生代对象经过 5 次 GC 晋升到老年代；**【默认值15】**
            - `-XX:PretrnureSizeThreshold` 设置大对象的值，超过这个值的大对象直接进入老生代；
            - `-XX:HandlePromotionFailure` 是否允许担保失败
    - **JVM方法区的分配**
        - `-XX:PermSize`设置非堆内存初始化值，**【默认是物理内存的1/64】**
        - `-XX:MaxPermSize`设置最大非堆内存(方法区)内存大小 **【默认是物理内存的1/4】**
    - `-XX:UserTLAB` 是否启用TLAB空间分配。
    - **方法区(元空间)参数设置**
        - `-XX:MetaspaceSize` 指定元空间大小
        - `-XX:MaxMetaspaceSize` 指定元空间最大空间
    - **基本信息修改**
        - `-XX:+PrintGCDetails` 打印GC收集信息
        - `-XX:+UseSerialGC` 是否使用串行垃圾收集器
    - **JVM默认值**
        - `-XX:+PrintFlagsInitial` 查看初始默认值
        - `-XX:+PrintFlagsFinal` 查看JVM默认配置值
    - **Java栈**
        - `-Xss` 等价于`-XX:ThreadStackSize`设置单个线程的大小
    - **直接物理内存参数**
        - `-XX:MaxDirectMemorySize` 设置最大物理内存
    - **X参数**
        - `-Xint` 解释执行
        - `-Xcomp` 第一次使用就编译成本地代码
        - `-Xmixed` 混合模式，JVM自己来决定是否编译成本地代码
    - **查看JVM运行时参数**
        - `-XX:+PrintFlagsInitial`
        - `-XX:+PrintFlagsFinal`
- **内存调优 故障排查**
    - **如何排查一个线上的服务异常？**
        - 首先查看当前进程的JVM启动参数，查看内存设置是否存在明显问题
        - 查看GC日志，查看GC频率和时间是否明显异常
        - 查看当前进程的状态信息top -Hp pid,包括线程个数等信息
        - jstack pid查看当前的线程状态，是否存在死锁相关信息。
        - jstat -gcutil pid查看当前进程的gc状态
        - jmap -heap pid查看当前进程的堆信息，包括使用的垃圾收集器等信息
        - 用jvsiual工具打开dump二进制文件，分析是什么对象导致了内存泄漏，定位到代码处，进行code review.
- **常见的OOM原因有哪些**
    - 数据库资源没有关闭
    - 加载特别大的图片
    - 递归次数过多，并一直没有释放资源。
- 调用jdk的Jconsole查看内存、线程、GC、对象内存占用等情况。依情况而定位具体如何去优化，比如一个对

象异常的占用内存。就要去程序里找找这个对象是否有问题了。

问题迭代

- **一个`.java`在JVM如何执行 详细说说你的理解？**
  - 首先程序在执行之前先要把 Java 代码（.java）转换成字节码（.class），JVM 通过类加载器（ClassLoader）把字节码加载到内存中，但字节码文件是 JVM 的一套**指令集规范**，并不能直接交给底层操作系统去执行，因此需要特定的**命令解析器执行引擎**（Execution Engine）将字节码翻译成底层机器码，再交由 CPU 去执行，CPU 执行的过程中需要**调用本地库接口**（Native Interface）来完成整个程序的运行。
  - `.java` -> `.class` -> 类加载器 -> 内存 -> 命令解析器执行引擎 -> 底层机器码 -> `cpu` -> 本地库接口
  - **那么可以在详细说一下对象创建过程中的内存分配吗？**
    - **创建对象的过程**
      - ① 当 JVM 遇到字节码 new 指令时，首先检查能否在常量池中定位到一个类的符号引用，并检查该类是否已被加载。

        ② 在类加载检查通过后虚拟机将为新生对象分配内存。

        ③ 内存分配完成后虚拟机将成员变量设为零值，保证对象的实例字段可以不赋初值就使用。

        ④ 设置对象头，包括哈希码、GC 信息、锁信息、对象所属类的类元信息等。

        ⑤ 执行 init 方法，初始化成员变量，执行实例化代码块，调用类的构造方法，并把堆内对象的首地址赋值给引用变量。
    - **对象的内存分配有两种方式 在②中**
      - **指针碰撞方式**
        - Java的堆中内存是绝对规整，用过的内存在一边，没用过的在另一边，中间有一个指示指针，所有的对象分配就是将指针往一边移动对象等大小的空间。
      - **空闲列表方式**
        - 内存不规整，已使用的和未使用的内存相互交替，维护一个列表用来记录那块内存是可用的，在分配的时候找到一个足够大的空间分配对象实例，并维护更新列表上的记录。
        - 小知识点:堆内存规整与否与使用的垃圾回收器**是否拥有压缩整理功能来决定的**
    - **既然你说了内存分配方式，那么在并发环境下安全性如何保证 ？**
      - 对分配内存空间的动作进行**同步处理**，通过**CAS+失败重试**的方式保证更新指针操作的原子性。
      - 把分配内存的动作按照线程划分在不同的空间之中，既给每个线程一定的堆内存空间，称为**本地线程分配缓存(TLAB)**,当TLAB用完之后，才进行同步锁定，虚拟机是否使用TLAB可以通过参数设定。`-XX:+/-UserTLAB`
      - 内存分配过程`todo`
    - **大概聊了对象内存分配，可以说说对象的内存布局吗？** `TODO`
      - **对象头**
        - 标记字段（在 32 位和 64 位 jvm 中分别为 32bits 和 64bits）+类型指针
      - **实例数据**
      - **对齐填充**
      - ps：若对象是一个 java 数组，则对象头中还必须有一块用于记录数组长度的数据。因为虚

拟机可以通过普通 java 对象的元数据信息确定 java 对象的大小，但是从数组的元数据中无

法确定数组的大小。

对象大小必须是 8 字节的整数倍。

- **谈一下对内存溢出和内存泄漏的理解？**
  - **内存泄漏**：一个不再被程序使用的对象或变量 一直占据在内存中就是内存泄漏。 这些对象一般具有两个特点。1.对象是可达的，在有向图中。2.对象是无用的，不会被gc回收。
  - **内存溢出**：程序申请内存，没有足够的内存，报错OutOfMemory。
    - **内存位置**
      - **堆溢出**
      - **栈溢出**
      - **运行时常量池溢出**
      - **方法区溢出**
  - **实际场景**
    - **长生命周期对象引用短生命周期**。比如缓存系统，当缓存一个对象到缓存系统时，长时间不访问该对象，那么由于缓存对象被缓存引用长期持有引用，所以不会释放对象。
    - **连接相关资源** jdbc session 及时释放
    - **内部类对象持有外部类对象的引用**。当我们通过外部类对象获取一个内部类对象的引用，如果内部类对象不释放这个引用，那么外部类永远无法释放，因为内部类对象持有外部类对象的引用。造成内存泄漏。
    - **改变哈希值**，当对象存到hashset中时，如果修改了这个对象计算hash值的字段，那么我们无法再次通过对象引用找到存储在hashset中引用，也就造成了内存泄漏。
  - 内存泄漏会造成内存溢出(创建对象时，由于空间不够)
- **对象被访问的时候是怎么被找到的？**
  - 前提概要，当我们创建一个对象，会在栈内存中持有该对象的变量，指向堆内存中的某个具体实例。
  - **句柄访问方式**
    - 引用中存储的就是对象的句柄地址，句柄中包含了对象实例数据与类型数据各自的具体地址信息。
    - 好处
      - 稳定 对象被移动 只会改变句柄中的实例数据指针，reference本身不需要修改。
  - **直接指针访问方式**
    - 通过指针直接定位，开销小，HotSpot直接使用。
    - 好处
      - 节省一次指针定位的时间开销。
- **说说内存回收的过程?**
- **内存快照抓取和MAT分析DUMP文件了解吗？**
- **什么是OOM,什么是StackOverflowError 有哪些分析方法？**
  - **Java中什么时候会发生OOM（华为）**
  - Java内存溢出(OOM)异常完全指南
- **谈一下你对JVM的理解 以及java8之后的改变？**
- **描述JVM中常见的垃圾回收器，如CMS，以及JVM调优思路（美团）** `TODO`
  - 一文了解JVM全部垃圾回收器，从Serial到ZGC
  - 第27讲 | Java常见的垃圾收集器有哪些？
  - 《码出高效》 P134~P138

- **JVM垃圾回收的时候如何确定垃圾，是否知道什么是GCRoots**
  - **什么是垃圾？**
    - 不在使用的对象所占的内存空间就是垃圾。
  - **要进行垃圾回收，如何判断一个对象是否是可以被回收的？**
    - 引用计数法->循环引用问题
    - 可达性分析
      - **什么是GCRoots 了解哪些对象可以用作GCRoots吗**
        - XXXX
      - **其中主要还涉及到强 软 弱 虚引用？**
- **G垃圾回收算法和垃圾收集器的关系?分别是什么请你谈谈？**
  - GC算法(复制/标清/标整/分代）是内存回收的方法论，垃圾收集器就是算法落地实现
  - 因为目前为止还没有完美的收集器出现，更加没有万能的收集器，只是针对具体应用最合适的收集器，进行分代收集
  - 分类
    - 串行垃圾回收器(Serial)
      - 它为单线程环境设计并且只使用一个线程进行垃圾回收，会暂停所有的用户线程。所以不适合服务器环境
    - 并行垃圾回收器(Parallel)
      - 多个垃圾回收线程并行工作，此时用户线程是暂停的，适用于科学计算/大数据处理等弱交互场景
    - 并发垃圾回收器(CMS)
      - 用户线程和垃圾收集线程同时执行（不一定是并行，可能交替执行），不需要停顿用户线程 互联网公司多用它，适用于对响应时间有要求的场景
    - G1垃圾回收器
      - G1垃圾回收器将堆内存分割成不同的区域然后并发的对其进行垃圾回收
- **生产环境服务器变慢，诊断思路和性能评估谈谈?**
- **假如生产环境出现CPU占用过高，请谈谈你的分析思路和定位**
- **对于JDK自带的JVM监控和性能分析工具用过哪些？一般你是怎么用的？**
- **JVM和JMM区别**
  - JMM是规范 而JVM必须实现。
- **类加载和Class.forName的区别？**
  - 通过 classloader 加载类实际上就是加载的时候并不对该类进行解析，因此也不会初始化，而 class 类的 forName 方法则相反，使用 forName 方法加载的时候会将 class 进行解析与初始化。
  - 首先我们得了解反射，那么我们就知道反射是运行状态中对任意一个知道名字的类进行属性和方法的获取。区别如下：

    class.forName()前者除了将类的.class文件加载到jvm中之外，还会对类进行解释，执行类中的static块。

    而classLoader只干一件事情，就是将.class文件加载到jvm中，不会执行static中的内容,只有在newInstance才会去执行static块。
- **JVM 如何知道一个对象要分配多大的内存**

  当类被加载如内存后，jvm 就知道。JVM 可以通过普通 java 对象的元数据信息确定 java对象的大小。

类需要同时满足下面 3 个条件才是“无用的类”：

  - **1.该类的 all 实例都已经被回收**
  - **2.加载改类的 classloader 已经被回收**
  - **3.该对象的 java.lang.class 对象没有在任何地方被引用，无法再任何地方通过反射访问该类的方法。**
- 垃圾收集器是自动运行的，一般情况下，无须显示的请求垃圾收集器，调用 System 类

  的 gc 方法可以运行垃圾收集器，但这样并不能保证立即回收指定对象。

  1）垃圾收集器并不是一个独立的平台，他具有平台依赖

  2）一段程序可以建议垃圾回收执行，但是不能强迫他执行

  3）当一个对象的 all 引用都被置为 null，这个对象就可以变为能被垃圾回收

  调用 System.gc()，这是一个不确定的方法。Java 中并不保证每次调用该方法就一定能够启动垃圾收集，他不过是会向 JVM 发送这样一个申请，到底是否真正执行垃圾收集，一切都是未知数。

- **重学结合操作系统与JVM？**
  - 首先从从操作系统层面来说，分为用户态和内核态。具体来说的话，内核态可以使用CPU的所有指令集，对所有硬件拥有全部的执行能力。用户态，只有有限的指令。一旦涉及到系统调用，必须从用户态到内核态的切换。系统内部的调用对于程序员来说是透明的，而用户态到内核态切换，需要等到内核态执行完毕，才可以执行下去。一个好的例子就是，你需要使用电脑，但是电脑的拥有这不是你，而是别人，你需要别人把电脑打开。你才可以使用。但是很多文件不是不能使用的。**CPU指令集是机器码的语法，机器码是CPU指令集的集合，一个操作系统的可执行应用程序，包含了用户态的CPU机器码+系统调用**
  - 举一个栗子，一个程序，可能包含50个CPU操作的机器码 20条系统调用。每个系统调用都会陷入到操作系统中，操作系统拥有完全的CPU操作权限。对各种硬件的驱动程序进行访问。
  - 而操作系统提供给我们内核调用就是ABI，每一个内核调用就会调用ABI接口。应用程序二进制接口。比如对应上面的程序，可能执行到22条指令的时候，我们需要进行磁盘的IO操作，但是我们应用程序的用户态代码没有这个权限，只能通过内核态的ABI接口进行处理。所以就会陷入到内核态进行处理。等内核态处理完毕时，我们才会进行处理。
  - **ABI VS API** 其实两者不是相同的东西,API叫做应用编程接口，高级语言编程时，书写源代码的时候，使用的编程接口。高级语言最后一定会被编译。生成一个本地机器码内部，很有可能就有ABI和API的对应。而ABI是针对二进制机器码而言的。针对的点是不一样的，一个是应用层级别，一个是CPU指令集级别。
  - 但是仅仅由内核提供的库是远远不够的，很多时候有很多第三方的动态链接库来实现，而动态链接库又去调用内核库。
  - **JVM** jvm内部其实也是包含了众多的核心类库就是jdk基础类库，虚拟机启动的时候，需要加载这些文件才可以执行，很多人说 字节码是跨平台，其实这是不对的，linux下的file类和window下的file 你能说一样吗，根本不一样，都是对应平台下硬件的IO封装，跨平台说的是用户类的跨平台。通俗一点说，就是基于jdk提供的这一套，编写的类 无论是jdk 还是spring hibernate都是基于第三方类库，而核心类是不一样的，不同的平台不同的实现，但是编程接口是一样的，只不过jdk进行了封装。最终用户类都需要和核心类进行连接。**JDK系统的api核心类 提供了java虚拟机运行的内核基础** 虚拟机不仅仅为java服务，字节码也不仅仅为java服务。 **java api类比成虚拟机内核，三方jar包 都是动态链接库 自己写的类就是应用程序**
  - **JVM工作原理** 首先需要明确一点就是JVM是操作系统之上的应用层软件，只不过专门为执行解析`.class`文件，首先启动的时候，先进行初始化操作 加载`rt.jar ext/xx.jar` 一些核心类库，接着执行`classpath`下的用户自定义类`Main.java` 运行字节码 1.解释执行 2.动态编译
  - **解析字节码 一边解析字节码 一边和操作系统内核打交道 将解析好的字节码翻译成本地机器码 然后交给操作系统去执行** 字节码会进行链接 ，比如源码中写了一个file类，编译之后会得到java api的file核心类的字节码进行链接。而file类的字节码包含了本地native的调用，这些java解析字节码的时候可以获知。java解析执行字节码 生成本地机器码，机器码中又有操作系统的库调用，系统调用。

0x02-数据篇 ☆

MySQL ☆☆☆☆

表设计、三范式

- **重新理解三范式？**
  - **一个列不可在分割，一个字段不可以分成多个列**
    - 举一个例子，比如一个用户表，每个用户有一个兴趣字段，但是，每个用户不止一个兴趣，通常有些人会将兴趣字段进行添加成多个，用多个兴趣1，兴趣2，这样去表示。其实这是违背了第一范式，一点就是，当增加字段时。拓展性和维护性不易。我们应该将这种字段提取出形成一个兴趣表，用兴趣id进行查找。但是需要进行权衡，根据具体的业务。
  - **一个表中所有的列都应该依赖主键**
    - 一个学生表中主键是id，学生有对应的姓名 性别 成绩，但是如果出现一个列是班级人数，那么这个班级人数和stuid值不具备依赖关系的，如果仅仅为了业务上数据展示，其实不可取，而这个班级人数和班级id是耦合的。需要权衡。
  - **列和列之间不应该出现依赖传递**
    - 一个表有商品数量和商品价格，但是如果还有一个商品总价。而这个商品总价是依赖于00商品数量和商品价格的。**第三范式的问题就是数据不一致。**
  - **主键属性之间无传递依赖**
  - 表结构的设计从一定程度上决定了程序编写的BUG数。

索引机制 ☆☆☆

MySQL官方对索引的定义为：**索引（Index）是帮助MySQL高效获取数据的数据结构。InnoDB存储引擎的索引模型底层实现数据结构为B+树，所有数据都存储在B+树中.** 可以得到索引的本质：**索引是数据结构。**

- 优势

1.类似大学图书馆建书目索引，提高数据检索的效率，降低数据库的IO成本优势s 2.通过索引列对数据进行排序，降低数据排序的成本，降低了CPU的消耗

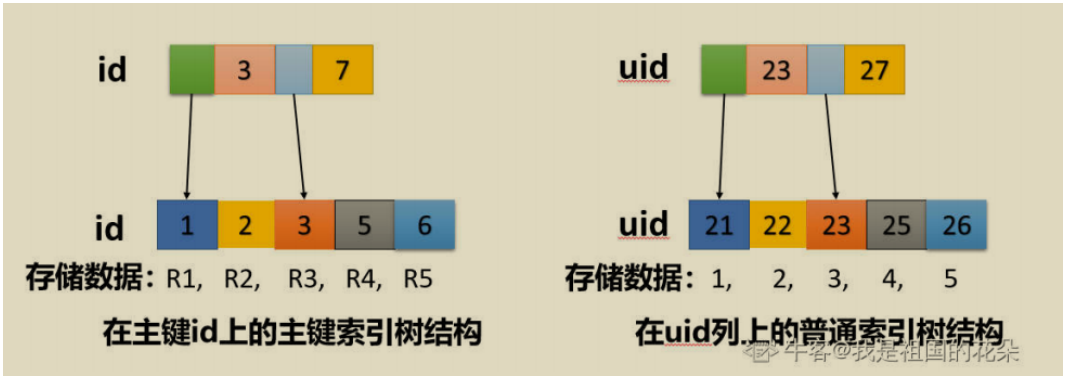
- 劣势

虽然索引大大提高了查询速度，同时却会降低更新表的速度，如对表进行INSERT、UPDATE和DELETE。 因为更新表时，MySQL不仅要保存数据，还要保存一下索引文件每次更新添加了索引列的字段，都会调整因为更新所带来的键值变化后的索引信息 一实际上索引也是一张表，该表保存了主键与索引字段，并指向实体表的记录，所以索引列也是要占用空间的

可以说一下那些列适合建立索引，那些列不适合建立索引吗？ ☆☆☆☆

- 那种情况下需要创建索引
  - **主键自动建立唯一索引** 聚簇索引
  - **频繁作为查询条件的字段应该创建索引**
  - **查询中与其他表关联的字段，外键关系建立索引。**
  - 单键/组合索引的选择问题，组合索引性价比更高
  - 查询中排序的字段，排序字段若通过索引访问将大大提高排序速度。
  - 查询中统计或分组字段
- 那种情况下不要创建索引

    - **表记录太少**
    - **经常修改的表或字段**
    - where 条件用不到的字段
    - 过滤性不好的不适合建索引
    - 底层结构 二叉查找树（ALV）-》B树-》B+树 -》Hash索引等区别
- **索引是什么，有哪些常见索引，以及为什么MySQL使用B+树做索引 而不是B树** ☆☆☆
    - 索引 --> 一种数据结构 **以上都是一种结构 索引是存储引擎层实现的。**
    - **为什么要使用 B+ 树，而不是 B 树、Hash、红黑树或二叉树？**
        - 二叉树(不保证平衡)->二叉平衡树(AVL)->Hash树->红黑树->B树->B+树
        - B 树：**不管叶子节点还是非叶子节点，都会保存数据**，这样导致在非叶子节点中能保存的指针数量变少（有些资料也称为扇出），指针少的情况下要保存大量数据，只能增加树的高度，导致 IO 操作变多，查询性能变低。
        - Hash：虽然可以快速定位，但是**没有顺序**，**IO 复杂度高，并且不支持范围查询**。
            - 适用场景
                - 等值查询 1.无法排序 2.hash冲突 3.只能等值查找
        - 二叉树：树的高度不均匀，**不能自平衡**，查找效率跟数据有关（树的高度），并且 IO 代价高。
            - 存储的数据比较少，并且当数据量多的时候，高度比较高。
        - 红黑树：树的高度随着数据量增加而增加，IO 代价高。
        - **常见的索引模型**
            - 哈希表 数组 树
    - B+ 树做索引优势
        - AVL, 红黑树等二叉树，查找过程中要进行许多次的磁盘读取操作，非常耗时（逻辑结构上相近的节点在物理结构上可能会差很远）
        - B树
            - **由于B+树的内部节点只存放键，不存放值，因此，一次读取，可以在内存页中获取更多的键，有利于更快地缩小查找范围 而B树叶子节点和非叶子节点都存储值**
            - B+树天然具备排序功能 --> B+树所有的叶子节点数据构成了一个**有序链表**，在查询大小区间的数据时候更方便，数据紧密性很高，缓存的命中率也会比B树高
            - B+树查询效率更稳定 --> B+所有关键字数据地址都存在叶子节点上，所以每次查找的次数都相同，所以查询速度要比B树更稳定
    - **倒排索引了解吗 说一下**
        - 在辅助表中存储了单词与单词自身在一个或多个文档中所在位置之间的映射
            - inverted file index {单词，单词所在文档的id}
            - full inverted index {单词，{单词所在文档的id，在具体文档的位置}}
- **聚集索引（Clustered Index）和非聚集索引的区别？** ☆☆☆
    - 聚集索引 (**主键索引**)--> **其索引树的叶子节点中存的是整行数据(叶子节点存储真实的数据行)，**指数据库表行中数据的物理顺序与键值的逻辑（索引）顺序相同（正文内容本身就是一种按照一定规则排列的目录）**MySQL聚簇索引的B+树子节点是相连接的**
        - 每个表只能有一个聚集索引，因为目录只能按照一种方法进行排序
    - 非聚集索引(**普通索引**)-->叶子节点内容是主键的值，在InnoDB里，非主键索引也被称为二级索引
        - 非聚集索引与聚集索引区别
            - 叶子节点并非数据节点
            - 叶子节点为每一个真正的数据行存储一个键-指针 对

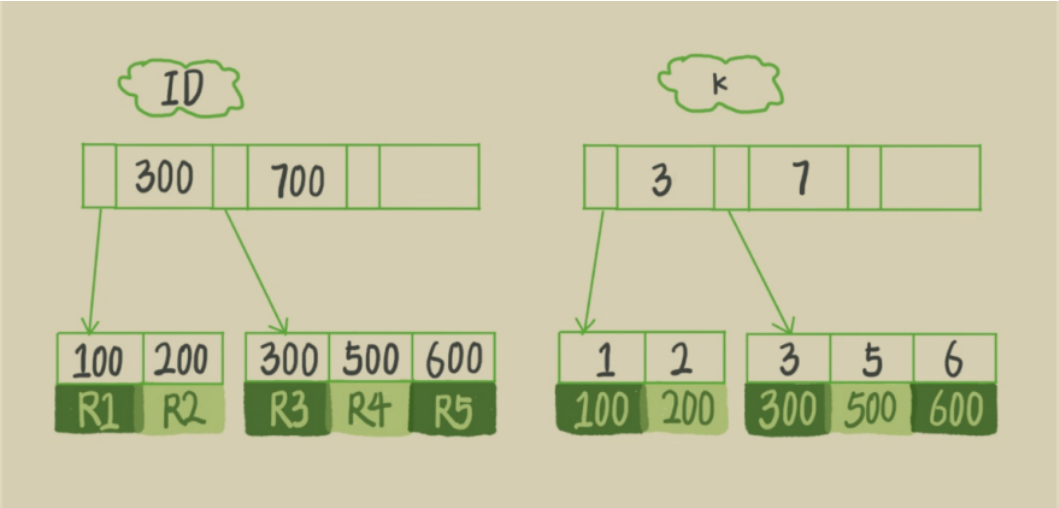
        - 叶子节点中还存储了一个指针偏移量，根据页指针及指针偏移可以定位到具体的数据行
    - 

    - 速度对比
        - **主键索引树的叶子节点存储的是完整的记录(R1)，而普通索引树上存储的是其对应的主键的值。(还需要根据主键值去查询一次。)**
    - **聚簇索引和非聚簇索引 加锁问题**
    - **回表问题了解吗？** ☆☆☆
        - `select * from user where id = 1` 只需要根据主键索引树查找一次就可以。
        - `select * from user where uid = 1` **需要根据普通索引查询对应的主键的值 再去主键索引树中查找一次。这个过程为回表**
    - **是不是所有的非聚集索引都需要回表操作？** ☆☆☆
        - 如果一个索引已经包含了我们需要查找的信息，那么就不需要回表操作，也被称为**覆盖索引**，合理的建立覆盖索引，也是SQL优化的常用手段
        - 比如我们建立一张只有age,name的两个表字段的索引表。当我们通过`select age,name from t where id = 1`;就可以通过覆盖索引，避免回表操作。
        - **回表中的索引下推了解吗？说一下你的理解？** ☆☆☆
            - 在索引遍历过程中，对索引中包含的其余字段先做判断，直接过滤掉不满足条件的记录，减少回表次数，提升查询效率。
        - 聚簇索引->非聚簇索引->回表问题->覆盖索引->索引下推【根本点在于解决减少查询次数 磁盘IO】
- **索引类别**
    - 单值索引
        - 即一个索引只包含单个列，一个表可以有多个单列索引
    - 唯一索引
        - 索引列的值必须唯一，但允许有空值
    - **唯一索引和普通索引查找的区别？** ☆☆☆
        - **查询过程**
            - 对于`select * from T where k= 5`
            - 对于普通索引来说，查找到第一个满足提交的值，会依次向后查找，直到有一个不满足k=5的值。
            - 对于唯一索引来说，因为唯一保证了唯一性，所以直接返回。
            - 对比
                - 因为InnoDB是按照数据也进行读写的，而每个数据页大小为16KB，每次读取都会将一个数据也读取到内存中，因为数据都是相连的，所以只需要一个指针和一次计算。对于极端情况下，概率也是极小的。
        - **更新过程**

        - change buffer，当需要更新数据时，如果数据页在内存中，直接进行更新。不在内存中，就将数据写入到change buffer中，当数据页某一次读取到内存时，将change buffer中的操作更新到数据页中。**change buffer是一个可持久化的文件，数据在内存和磁盘都会保留一份**，将change buffer中的操作应用到数据页，得到最新结果的过程称为merge。
        - 对于数据页在内存中，会直接更新。但是对于数据页不在内存中，对于唯一索引，会将数据页加载到内存中，进行判断是否冲突，更新值。但是普通索引来说，直接将更新操作写入到change buffer中就可以了。**将数据页读入到内存中，会涉及到磁盘IO操作，changer buffer可以减少随机磁盘访问，提升系统性能。**
    - **change buffer的适用场景**
        - **对于写多读少的业务系统**来说，比较适合将数据写入到change buffer中。
    - 但是对于写完立即读取的数据，大量的读取会将change buffer需要修改的数据页读取到内存中，所以不适合这种场合。**change buffer+普通索引 可以提升性能**
- **联合索引/复合索引**
    - 即一个索包含多个列
        - 为什么要使用联合索引

          1.减少开销 2.覆盖索引 3.效率高
    - **主键索引、非主键索引**
        - 主键索引等于聚簇索引 聚集索引
        - 非主键索引等于非聚簇索引 非聚集索引
    - **覆盖索引**
        - 在普通索引上的查询已经可以直接获取到结果，不需要回表操作，这样的索引就是覆盖索引
    - 优点是显著提高查询效率，最常见的MySql性能优化手段。
        - 辅助索引不包含整行记录的所有信息，故其大小要远小于聚集索引。可以大量减少IO操作。
    - **自适应Hash索引**
        - InnoDB 存储引擎会监控表上各索引页的查询。如果观察到建立索引可以带来速 度提升，则建立哈希索引，称之为自适应哈希索引（Adaptive Hash Index，AHI）。
    - **全文索引**
        - 在MyISAM中引擎上，通过关键字查找
    - **空间索引**
        - 空间索引是对空间数据类型的字段建立的索引，MySQL中的空间数据类型有四 种，GEOMETRY、POINT、LINESTRING、POLYGON。 在创建空间索引时，使用SPATIAL关键字。 要求，引擎为MyISAM，创建空间索引的列，必须将其声明为NOT NULL。
    - 如何查看一个表结构是否有索引
    - `explain select * from tableName`
- **索引最左前缀原则** ☆☆☆
- 最左匹配原则也叫最左前缀原则，是 MySQL 中的一个重要原则，指的是索引以最左边为起点任何连续的索引都能匹配上，当遇到范围查询（>、<、between、like）就会停止匹配。 生效原则来看以下示例，比如表中有一个联合索引字段 index(a,b,c)：
    - where a=1 只使用了索引 a；
        - where a=1 and b=2 只使用了索引 a,b；
        - where a=1 and b=2 and c=3 使用a,b,c；
        - where b=1 or where c=1 不使用索引；
        - where a=1 and c=3 只使用了索引 a；

        - where a=3 and b like 'xx%' and c=3 只使用了索引 a,b。
    - 最左前缀匹配原则，非常重要的原则，mysql会一直向右匹配直到遇到范围查询(>、<、between、like)就停止匹配，**比如a = 1 and b = 2 and c > 3 and d = 4 如果建立(a,b,c,d)顺序的索引，d是用不到索引的**，如果建立(a,b,d,c)的索引则都可以用到，a,b,d的顺序可以任意调整。
    - =和in可以乱序，比如**a = 1 and b = 2 and c = 3 建立(a,b,c)索引可以任意顺序**，mysql的查询优化器会帮你优化成索引可以识别的形式
- **索引失效了解吗？** ☆☆☆
    - 1）条件中有 or，即使其中有条件带索引也不会使用（要想使用or 又想让索引生效，只能将 or 条件中每个列加上索引）
    - 2）like 查询，以%开头 `like '%xx'` 失效
    - 3）若列类型为字符串，则一定要在条件中将数据用引号引起来，否则不使用索引
    - 4）**若 mysql 估计使用全表扫描要比索引快，则不使用索引**
    - 5）对索引进行运算导致索引列失效
    - 6）使用内部函数导致索引失效，这样应当创建基于函数的索引
    - 7）b-树，is null 不会用，is not null 会用。
- **InnoDB的索引模型？**
    - **InnoDB是根据主键顺序以索引的形式存放的**，这种存储方式的表称为**索引组织表**。比如我们创建一个表，索引类型分为主键索引和非主键索引。**主键索引的叶子节点存的是整行数据，InnoDB里，主键索引也被称为聚簇索引，非主键索引的叶子节点内容是主键的值，在innoDB里，非主键索引也被称为二级索引**
    - **主键索引和普通索引有什么区别？**
        - `select * from T where ID = 500` 直接根据主键索引结构查找到整个数据 **主键查询方式**
        - `select * from T where k = 5;` 先搜索K树，查找到id=500，根据ID搜索树进行在查找一次，这个过程就是回表 **普通索引查找方式**
        - 区别：普通索引查找方式比主键查询方式多查找一次，应该尽量使用主键查询方式



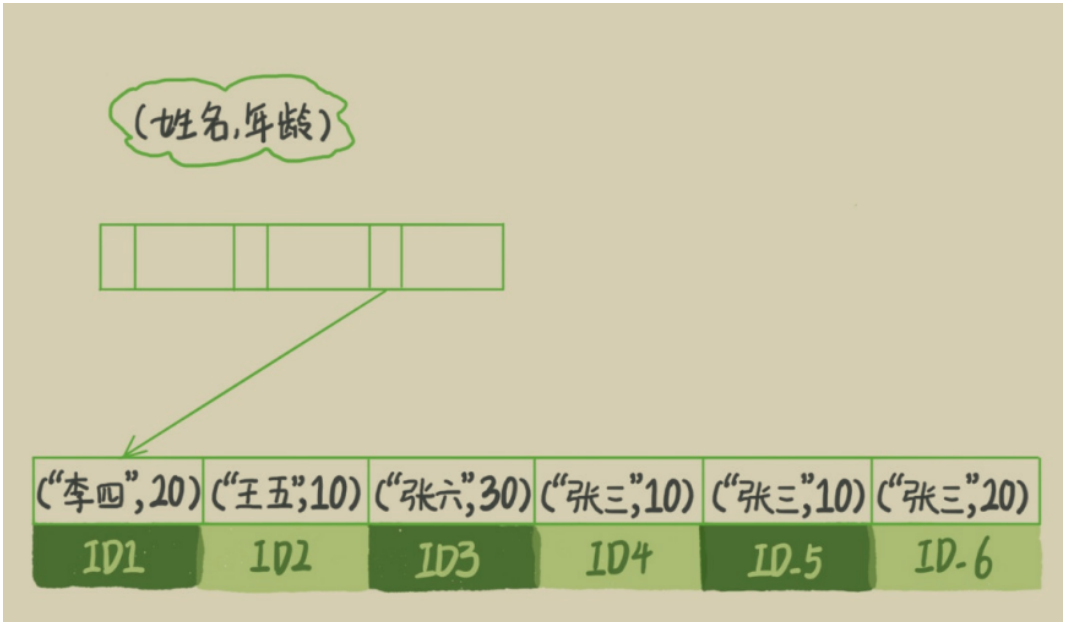
- **索引维护**
    - 当我们插入一条id=700，直接插到R5后面就可以，但是如果插入的数据是400，就需要移动出一个空闲的位置，而如果这个数据页时满的，那么就需要将当前数据页的部分数据进行移动到另一个数据页，而这个过程就是**分页**，分页是比较影响性能的。当删除数据时，就会有数据的合并，**合并页**，一个分页的逆过程。
- **自增主键的相关问题？**

- 一般我们的建表语句中都采用的是自增模式，对于有序数据来说，是比较合理的，因为会按照id自增逐渐添加到树中。但是具体到业务中，这是无法预测的。如果只是保证id的唯一性，那么用身份证号这种数据，对于非主键索引来说，每个叶子节点存储的都是主键的值。叶子节点存储空间是比较大的。显然，**主键索引越小，普通索引的叶子节点越小，普通索引占用的空间也越小。**
- **业务字段做主键的场景？**
  - 只有一个索引，该索引时一个唯一索引。可以避免搜索两颗树。

- **深入浅出索引-下？**
  - `select * from t where between k=3 and k = 5;`需要查找几次索引树。好了，我们来分析一下，由于k这一列是由索引树的，会先查找K索引树，找到k=3的值 所对应的Id值，在根据主键Id去Id索引树中找到对应的数据行，而k=5也一样，而这个过程就是回表操作。查找了3次搜索K索引树，2次回表操作。是否能进一步优化，减少会表操作呢？
  - **覆盖索引**

    `select id from t where between k=3 and k=5` 因为查询结构中只需要返回id，那么我们在K索引树上就可以找到对应的Id值，无序回表操作，而这就是覆盖索引。**由于覆盖索引，能减少回表的次数，因为调优中合理使用覆盖索引时提升系统性能的一个关键点**

    在实际工作中，遇到不同的需求进行权衡是否进行设定索引，如果某个字段是唯一的，并且大量相关数据根据这个唯一字段查找，那么可以根据这个字段索引树，减少回表操作。合理的设定索引需要进行权衡利弊。
  - **最左前缀原则**

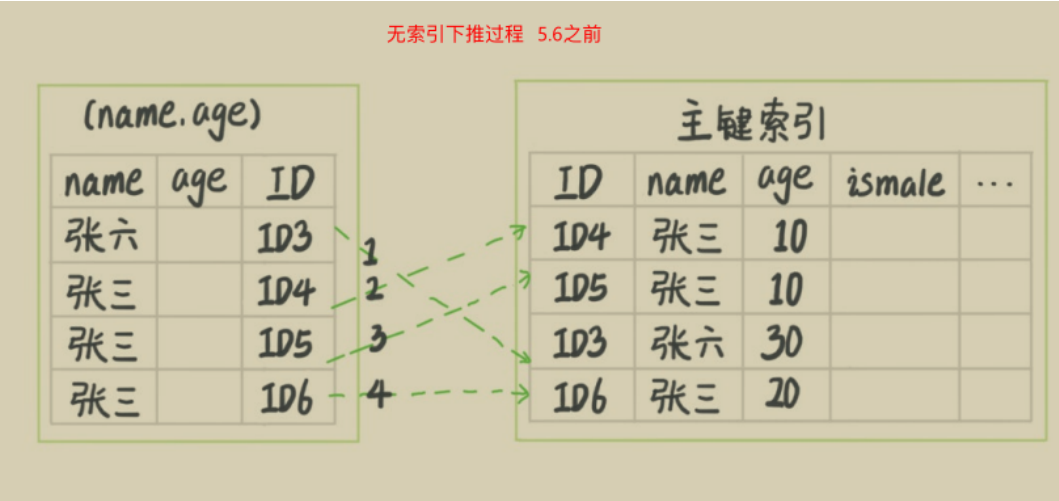
    

    合理使用最左前缀原则。比如查询张三 可以查找到ID4 继续向后遍历，就可以了。只要满足最左前缀，就可以利用来加速索引，最左前缀可以是联合索引的最左N个字段，或字符串索引的最左M个字符。

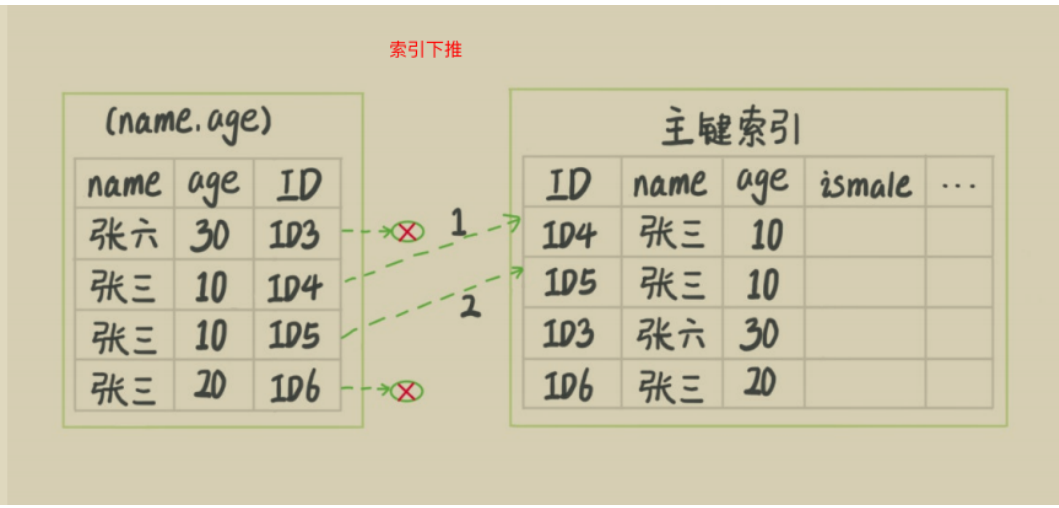
    如何安排索引内的字段顺序？

    如何可以通过调整，减少一个组合索引中一个，就将删除索引提到前面。同时也要考虑空间的因素。
  - **索引下推**

    `select * from tuser where name like "张%" and age =10 and ismale = 1` 根据最左匹配原则，可以根据联合索引`(name,age)` 索引树找到 `张%` 开头的数据，但是对于`age` 和 `ismale` 需要查找到主键，在进行回表操作。当然，这是在mysql 5.6 之前，在5.6之后，有一个索引下推的概念，在遍历的过程中，对索引包含的字段先做判断，直接过滤掉不满足的记录，减少回表次数。



可以分析一波，对于无索引下推，只会先去联合索引中，找到对应的张开头的列，然后根据主键ID去主键索引树中查找对应的行记录。但是符合条件的数据只有2条。



而相比之下，索引下推，会预先判断不符合条件的数据，在联合索引中查找了age，将不符合条件的列直接过滤，减少了回表的次数。

- **MySQL选错索引**
  - `force index(a)` 使用优化器强制使用a索引。
  - `show table x`
  - `analyze table x` 重新统计索引信息
  - `like"%aaa%"` 不会使用索引， `like" aaa%` 可以使用索引 `like "%aaa"` 也不会使用索引
  - **优化器的逻辑**
    - 选择索引是优化器的工作。扫描越少的行数，访问磁盘的次数越少，消耗CPU资源越少。
    - 区分度：一个索引上不同的值越多，所以的区分度就越大。一个索引不同的值的个数，就是基数。基数越大，索引的区分度越好。
    - **区分度是如何统计的？** 在InnoDB上会选择N个数据页，通过统计每个数据页的不同值，计算平均值，乘以数据页的大小。
    - 通过非主键索引会有回表操作，而对于数据量大的查询，优化器会认为直接通过主键减少回表操作，是一种更好的方式。
  - **索引选择异常和处理**
    - 第一种方式 采用`force index` 强制选择一个索引
    - 第二种 考虑修改语句
    - 第三种 有些场景下，新建一个更合适的索引 或删除误用的索引

- 1.什么是索引 其实按照生活中的 我们看书都是看目录，这个目录就是一个索引，用以确认具体的页码。索引只是一个简要的信息，而具体的信息，在索引指定的位置中。

2.索引的结构 如果 我们查询name = 'xx' 通过where name = 'xx' 可以在name 上建立一个索引，此时，直接在索引中先查找对应的值的位置。 比如 xx 表中第234行 通过索引确定数据的位置，直接去对应的位置查找对应的值，就可以了。 这里 我们要讨论一下 为什么索引很快呢，在于索引的逻辑结构是一个B树。 如果将国家类比成一个根节点，那么各个省份就是根节点下的子节点，省份下面又有市 一次类推 县 乡 村如果我们要查询某一个地级市。 查询只需要2次就可以，根节点到省 省到市，而这种查询速度是很快的，虽然同比hash来说 相形见绌 但是对于上百万的数据，这个数据的时间复杂符 还是很客观的。类比solr 等 都是这样的思想。所以 我们学一个技术 **要掌握这个技术背后的思想**，而不是说用用就可以了。

锁机制

- **为什么要使用锁机制呢？**
    - 支持对共享资源进行并发的访问，**提供数据的完整性和一致性**。
- 锁的基本概念和大类别(悲观并发控制、乐观并发控制、MVCC)
- **MVCC多版本并发控制以及undo log（回滚日志）** `todo`
- **悲观锁和乐观锁？**
    - 悲观锁
    - 乐观锁
        - CAS->ABA->时间戳、版本号
    - 悲观锁大多数情况下，依靠数据库的锁机制。乐观锁大多基于数据版本，记录机制实现。数据版本。为数据增加一个版本标识，比如增加一个 version 字段。读数据时，将版本号一块儿读出，之后更新时，版本号加 1，将提交数据的版本号与数据库表对应记录的当前版本号进行对比。若提交的大于数据库里面的，则可以更新，否则认为是过期数据。将乐观锁策略在存储过程中实现，对外只开放基于此存储过程的数据更新途径，而不是将数据表直接对外公开。
    - **重新理解悲观锁和乐观锁？**
        - 1.悲观锁 之前for update 可以看到 虽然 for update解决了并发上丢失更新的问题。 举个例子:当一个用户在取钱操作的时候，中途有事 暂停取钱 一直处于暂有排他锁，其他与他有商业合作的人 在这个期间 没有办法给他转账 也就是写操作事务，for update是一种悲观主义 ，总是认为 自己在写操作的时候 别人总是会进行破坏操作，因此这就是悲观锁。具有排他性

          2.乐观锁 本质上是提供了一个版本号的机制，用来控制，比如 当前数据库的版本好为10 当你进行写操作 获取到当前版本号10 然后操作 balance =balance +100; 但是这个时候 你突然接收到一个电话，没有及时提交。在这个期间，你女朋友 也进行了写操作 获取了当前数据库的版本号 进行了操作， balance = balance - 1000 , 然后提交了。数据库检测到提交的版本号与当前数据库版本号是一致的 所以就提交成功 当你电话打完之后 你去继续提交这个事务，就会出现提交失败，因为当前数据库的版本号为11 而你提交的版本号是10 说明在你操作期间 别人进行了操作，因此操作失败。这就是基于多版本号的乐观锁的机制。 3.数据库乐观锁的设计 update set money = money - 100 ,version = version + 1 where version = version; 数据库加版本号，然后代码里面就通过版本号进行处理，每次更新不是都返回受影响的行数吗，如果受影响的行数为0， 重新获取最新的数据，再次进行操作。 业务设计: 如果我要你查询余额，对余额大于10000的账户，增加100个积分 - 余额表 积分表 操作 update jifenTable set jifen = jifen + 100 where jid = 001 and exists(select 1 from account where aid = 001 and oldVersion = version)

4.乐观锁 1.代码复杂 设计上有挑战 2.对于客户端的版本号不一致的冲突提示？ 3.存在操作失败的风险。。当用户提交一个事务后 很有可能由于当前版本号与数据库中版本号不一致而导致事务操作失败。 悲观锁 1.而对于悲观锁，只需要一个select for update，简单直观，不容易出错

在并发中，如果使用乐观锁，将会死一大片，这对于我们的系统来说就是一个噩梦。 5.乐观锁和悲观锁使用场景 悲观锁: 如果account账户表，只有一个写业务，只有取款，没有其他的并发操作，那么使用悲观锁是必须的。 我们除了取款（存在丢失更新），还有转账，收款等大并发的业务 我们就不能锁定了 一定要放开，让其他大并发业务能够正常进行 因为哪些业务都是一个update，不存在丢失更新 所以这个并发大 ?? 怎末去权衡。

> 我看了网上很多的博客说 并发小用悲观 并发大用乐观，这是人云亦云的。
>
> 假如，只有取款业务（并发很大，存在丢失更新） 你能用了乐观锁？ 一秒钟来了1000个取款，难道你要999个都提示失败吗
> 如果使用乐观锁 并发很大，乐观锁导致的后果越来越严重。
>
> 网络上说的并发 是说的其他对于该数据的并发写 而不是仅仅是这个丢失更新的写 就像我说的，你不仅仅只有取款，还有转账，收款
> 说的是影响了这个非丢失更新的大并发。
>
> 所以使用悲观锁和乐观锁需要进行具体业务具体的权衡。

- **读锁？**
  - 单存的读 不会存在读阻塞
  - 读之后并修改 `select from update` 行级锁，当不走索引，是表级锁。
    - 对于查询的数据如果只是显示，那么我们不需要进行`for update`，如果查询的数据要做相关的业务，我们需要使用，因为这块在并发量下，会出现更新丢失的问题。
    - 1.早期数据库锁机制 早期的数据库是共享锁，如果select一行，会对该行添加上一个共享锁。共享锁可以共享读取，但是当一个写操作的时候 这个写操作必须等待共享锁操作完毕才可以执行，也就是在共享锁占用期间，其他的写操作是阻塞的。 因此，存在一个问题 当读并发很高的时候，写锁很难获取到写的权限。对于高并发的数据库 是一个瓶颈。？？？？ 2.MVCC 进一步发展 提供了一个MVCC的机制，称为多版本并发控制。 每一个修改都在副本中完成，一旦事务提交，其他事务就可以看到已提交的数据。这样很大程度上提升了数据库系统的并发度。 就可以并发读写，但是在这种情况下。对于排他锁来说就有问题。 3.排他锁问题 当同时有两个排他锁进行事务操作的时候，第2个排他锁就会被阻塞，只有第一个执行完毕，才可以执行第二个。也就是说 他们不能同时 进行修改，否则就会出现丢失更新。即第一个事务提交的数据，被第二个事务所覆盖。第一个事务的数据被丢失。 总结:现在的数据库是可以并发读操作，在写的时候 也可以读。但是在写和写还是会存在排他影响。 4.select for update讨论？ select for update 不是简单的读取数据，而是在查询数据的时候，其他的写事务 不能操作改行数据。 我们要保证在该事务的整个 过程中，其他的并发事务不能操作本行数据。select for update 具有排他性。select 不会阻塞 select 但是 select for update 会阻塞 select for update 举一个例子：加入 你和你女朋友同时给你的账户存钱 一人存1000 ，这个操作是同时发生的 一个前 一个后 必然会出现 后一个覆盖前一个 事务。最后的银行卡余额不是2000 而是1000 。 但是 使用 select for update 你先for update 这个时候，你女朋友是可以看到这个0元余额。但是她不可以对这个账户进行操作 因为for update具有排他性，当你提交完毕事务后，你女朋友才可以操作，这样就可以保证在一个事务写的时候，别人可以看，但是不妨碍 你的写操作，无伤大雅。但是 她是不可以操作的。因为for update具有排他性。
- **独占锁和拍他锁？**

    - 独占锁 `select from update`
    - 共享锁
    - **意向锁：将锁定的对象分为多个层次**
- **行级锁什么时候会锁定整张表？**
    - InnoDB行级锁是通过锁记录实现的，**如果更新的列没有建立索引是会锁定整张表的。**
- MVCC的基本概念，时间戳版本号，快照版本与当前版本，使用场景等
- **mysql如何解决死锁?**
    - 主要是多个事务使用行级锁对某行数据加锁造成的。
    - 解决方案
        - 业务逻辑上的死锁解决方案
            - **指定锁的获取顺序**
            - **大事务拆分成各个小事务**
            - **在同一个事务中，一次锁定尽量多的资源，减少死锁概率**
            - **给表建立合适的索引以及降低事务的隔离级别等**
        - 数据库的设置来解决死锁
            - **通过 innodb_lock_wait_timeout 来设置超时时间，一直等待直到超时；**
            - **发起死锁检测，发现死锁之后，主动回滚死锁中的某一个事务，让其他事务继续执行。**
        - **总结** 业务上，首先需要保证加锁的顺序，不能A需要a,b两个锁，而B也需要a,b两个锁，A按照a,b去加锁而B按照b,a加锁，大概率情况下会发生死锁的情况。减少锁的粒度。或者如果当前需要a,b锁，一次性锁定当前事务所需要的锁。这是第一点，第二点的话 从数据库本身来说，我们需要保证一旦发生死锁，在一定的时间内要释放锁，也就是设定超时时间。或者发生死锁后，可以回滚其中一个事务到初始状态。
- **全局锁了解吗？缺点说一下？**
    - 全局锁就是对整个数据库实例加锁，它的典型使用场景就是做**全量逻辑备份**，这个时候整个库会处于完全的只读状态。
    - 缺点：使用全局锁会使整个系统不能执行更新操作，所有的更新业务会出于等待状态；如果你是在从库进行备份，则会导致主从同步严重延迟。
    - **全局锁和表锁？**
        - 全局锁

          因为在给全局锁会导致只能只读数据，如果存在一个主从节点。

          对于主节点来说，在这个期间是不能进行处理执行更新操作，整个业务系统处于暂停状态。

          对于从节点来说，主从复制之间会存在一定的延时，会有一个数据不一致的窗口期。

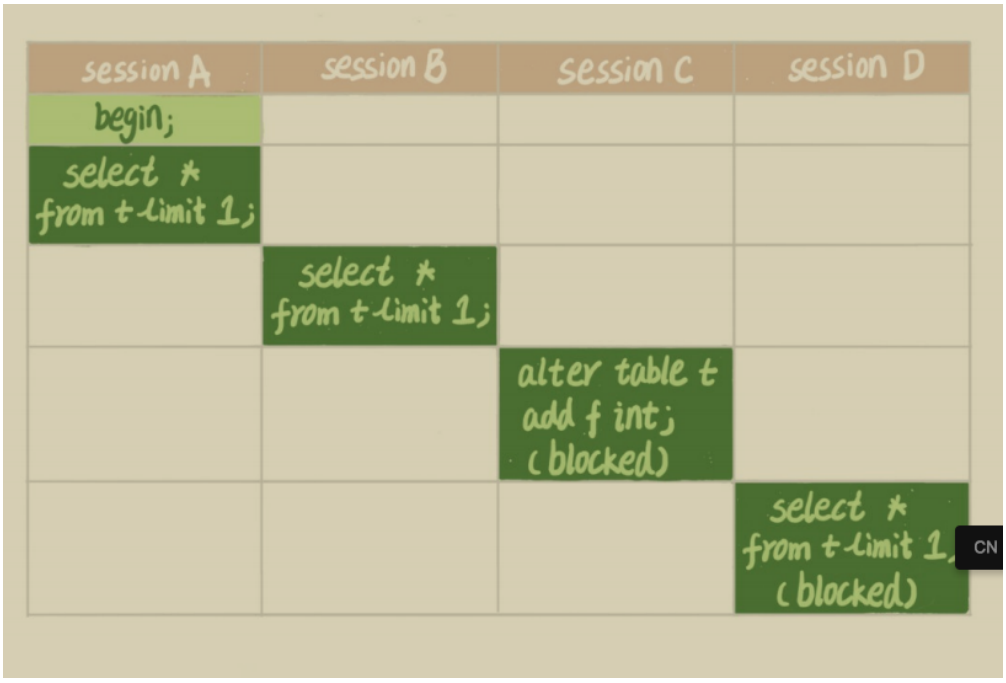
          如果不加全局锁的话，我们来假设一个实际场景，对于一个餐厅管理系统来说，用户付钱，然后给我们对应的用餐。这是属于一个事务中，如果在期间进行备份了一个中间状态。有两种情况，一种是先付钱，只备份了付钱状态，原来20元，备份了当前的状态，付款后成了10元，但是没有备份用餐情况，最终的结果是由于之前备份了20元的日志，导致当前钱数20元，用餐了。与之相反就是钱数少了，但是没有用餐情况。这种情况在业务上是不允许存在的。**不加锁的话，系统备份得到的库不是一个逻辑时间点，这个视图是不一致视图。**
        - **表锁**

          MySQL 表级别的锁 有两种，一种是表锁，一种是元数据锁(meta data lock MDL)

          表锁：`lock tables t write` 加写锁，其他线程写操作就是阻塞的。也就是读读不互斥，读写互斥。写写互斥。因为表锁的粒度比较大，支持的并发度就降低了。所以支持行锁的InnoDB存储引擎来说，比较推荐使用行锁进行操作。

MDL 当进行遍历数据的时候，另一个用户在添加数据时，就会出现数据不一致，对于前者来说。因此,**当对一个表做增删查改时，加DML读锁。而对表结构进行修改时，会加DML写锁。**

好了，我们来分析一下，因为读读不会操作阻塞，所以，多个线程进行真删改查数据时不会出现阻塞。但是对表结构的修改是写锁，会操成写锁阻塞。当有两个线程同时对表结构修改时，一个线程必须等另一个线程操作完毕，才可以修改表结构。



好了，我们来分析一个案例，当Session A开启一个事务进行操作，由于DML会自动加另一个读锁，而Session B也会获取DML读锁。都可以读取到数据，但是对于Session C由于修改修改表结构，需要获取DML 写锁，而读写时互斥的。会被阻塞，而如果之后session D 进行读取数据获取DML读锁，就会被DML写锁阻塞。导致系统崩溃。

如何解决上面的实际场景：1.kill 长事务操作 2.如果当前库时热点库，设定DML写锁获取锁的时间，超时返回。

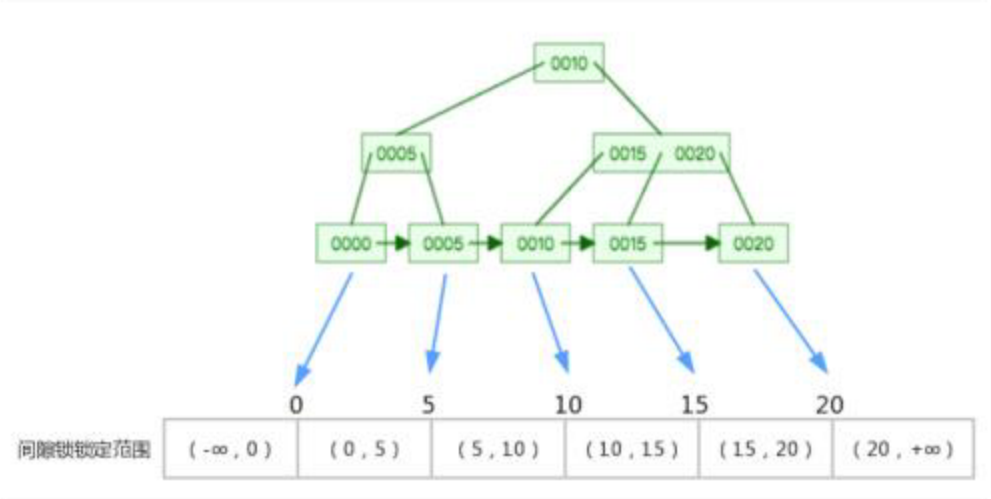
- **InnoDB 存储引擎有几种锁算法？** ☆☆☆
  - Record Lock — 单个行记录上的锁；
  - **了解具体实现吗？**
    - **只有通过索引条件检索数据，InnoDB 才使用行级锁，否则 InnoDB 将使用表锁**。使用 for update 来实现行锁，具体脚本如下：

      ```
      select * from t where id=1 for update
      ```

      其中 id 字段必须有索引。
  - Gap Lock — **间隙锁**，锁定一个范围，不包括记录本身；**开区间**。
    - 当我们用范围条件而不是相等条件检索数据，并请求共享或排他锁时，InnoDB会给符合条件的已有数据记录的索引项加锁；**对于键值在条件范围内但并不存在的记录，叫做“间隙（GAP)”**， InnoDB也会对这个“间隙”加锁，这种锁机制就是所谓的间隙锁（Next-Key锁）。
    - 间隙锁解决了可重复读下幻读问题
  - Next-Key Lock — 锁定一个范围，包括记录本身。
  - 三者之间的关系
    - 将表的自增id看做是一个区间，gap lock是一个区间，而record lock是行锁。

所以 **gap + record lock = next-key lock。前开后闭区间**。行锁就是锁定一行记录，但是当我们进行插入数据的时候，是更新记录之间的间隙，所以为了解决幻读(添加后的数据 被别的用户看到)，只锁定一个区间，如果是锁定整个区间的话，成本太大，于是InnoDB引入了间隙锁。

间隙锁，顾名思义就是锁定两个值之间的空隙。当插入5条记录后，就产生了6个间隙锁。



- **针对锁优化有什么建议？**
  - 尽可能让所有数据检索都通过索引来完成，避免无索引行锁升级为表锁。
  - 合理设计索引，尽量缩小锁的范围
  - 尽可能较少检索条件，避免间隙锁
  - 尽量控制事务大小，减少锁定资源量和时间长度
  - 尽可能低级别事务隔离
- **当我们使用for update的时候，会对一个不存在的数据进行加锁吗？**
  - 结论 并不会进行加锁，而是加了一个间隙锁。`gap lock`,比如，我们的数据库中表有两个字段`id=4`和`id=7`当执行事务A`select * from t where id = 3 for update`会加锁一个间隙锁，而锁定的区间就是(0,5),是一个开区间，当执行事务`select * from t where id = 3 for update`时会被阻塞吗？并不会被阻塞，结论就是对于一个不存在行加`for update`时，会加一个间隙锁，`gap lock`只保护你的写操作，并不会保护你的读操作。即使锁定了相同的区间，也是这样。 但是如果事务A进行`insert 3`操作，会等待事务B的锁释放，而如果事务B进行`insert 3`会被事务A的锁阻塞。这也是常见的一种死锁方式。
- **两阶段锁，死锁和死锁检测？**
  - 行锁是由引擎层各个引擎自己实现的，MyIASM引擎不支持行锁，InnoDB支持行锁。对于不支持行锁的执行引擎来说，并发粒度比较低。
  - **两阶段锁**



上面是两个不同的事务，事务A开启一个事务，而在这个期间事务B也操作同一行数据，就会出现阻塞。结论可想而知，因为行锁的存在，所以会导致阻塞现象出现。那么事务B什么时候，才可以执行呢，也就是必须等到事务A执行完毕commit后才可以执行。**在InnoDB事务中，行锁是在需要的时候才加上的，但并不是需要了就立刻释放，需要等事务执行完毕才释放，这就是两阶段锁。一个开始一个结束。**

有了上面的基础，我们假设一个实际业务场景，电影院在线售票系统，A用户需要在B电影院进行购票，而具体的操作就是先支付，扣减用户A的账户余额，增加B余额。售票成功。如果为了系统的可维护性，可以在这个事务中添加一条回顾事务日志。但是如果在这个期间，用户C也需要购票，那么，我们如何降低并发度，也就是用户C也需要对B的余额进行操作。为了更好的提高并发度，降低B账号阻塞其他用户，将B的操作顺序改成 A log B，可以很好的运行。

但是 如果有一天商家进行秒杀活动，大量用户操作B的账户。系统会出现死锁问题。也就是系统执行的事务不多，但是CPU接近于100%。

死锁和死锁检测



事务A和事务B同时去获取对方的锁，就会出现死锁现象。解决方式1、超时时间 2、手动解除一个事务占有的资源。

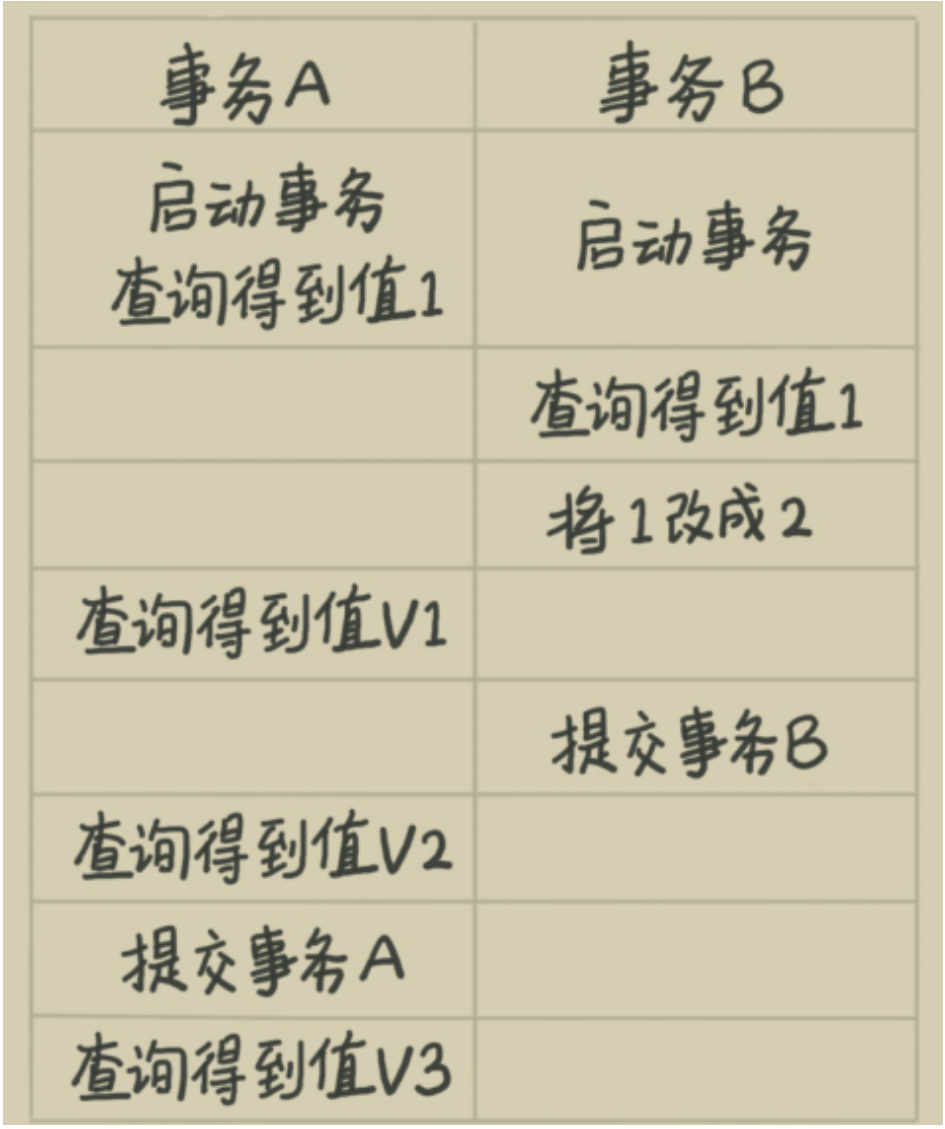
事务和隔离级别 ☆☆☆

事务的本质：事务会把数据库从一种一致状态转化为另一种一致性状态。

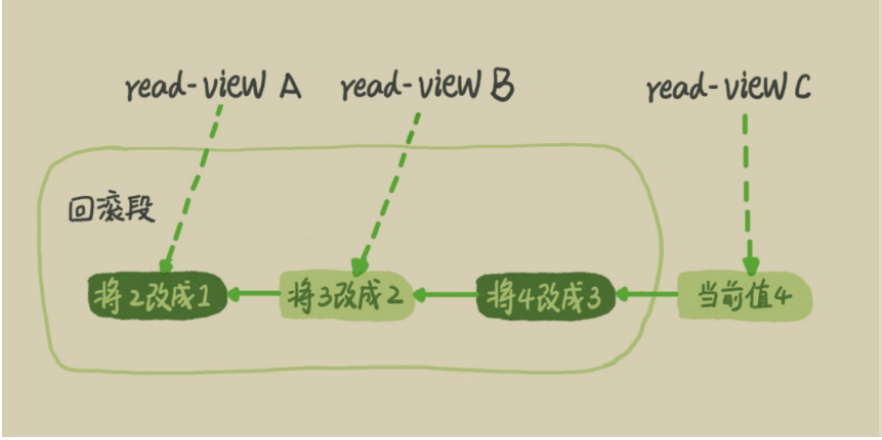
- **范式可以说说你的理解吗？**
  - 第一范式：所有字段都是不可分解的原子值
  - 第二范式：确保数据库中每一列和主键相关，而不能只与主键的某一部分相关
  - 第三范式：每一列都和主键直接相关，而不能间接相关
- **事务ACID的理解说一下？** ☆☆☆
  - **A(Atomicity)原子性**
    - 原子性通常来说，我们将操作一个联合的过程 当做一个原子，要么全部成功，要么全部失败，不会出现提交到一半 另一半成功，一半失败。原子性 包含了不可在分隔的最小粒度单元。
  - **C(Consistency)一致性**
    - 一致性是从一个开始的状态到另一个结束的状态。 原子性基于一致性的。原子性说的是前后状态的保持，但是，我们知道计算机指令执行是有一个之间差的，也就是第一秒提交的事务，在第三秒执行完毕，但是如果一个用户在第二秒进行访问，得到的结果是不是会出现不一致现象。所以一致性保证了只会有一个前状态和一个后状态，绝对不会出现一个中间态。
    - **分布式中解决一致性问题的思想？**
      - 上面说的是在一个单体应用中，操作的是同一个数据库。但是在分布式事务中，我们通常一个用户的操作会涉及到多个数据库的操作。网络延时等问题，需要如何才能保证数据之间的一致性呢，通常来说，ACID的C是一种强一致性，而分布式中是采用了最终一致性，也就是在用户操作完毕之后，在一定的时间内不会立即进行同步，而是在一段间隔内，数据最终到达最终一致性。而开源产品中，zk就是一个分布式治理的框架，在数据一致性的最终一致性上是的产品。
  - **I(Ioslation) 隔离性**
    - 事务隔离级别

	脏读	不可重复读	幻读
1. 读未提交	P	P	P
2. 读已提交	S	P	P
3.重复读	S	S	P
4.序列化	S	S	S

  - `select @@tx_isolation;`
    - 每个读写事务的对象对其他事务的对象的操作是不可见的， 通过事务隔离级别可以保证数据对于另一个事务的影响。通常使用锁来实现，是一种细粒度的策略。可以提高事务之间的并发度、
    - **事务隔离级别以及造成的问题解决了那些问题 可以说一下吗？**
      - **Serializable（序列化）** --> 可避免脏读，不可重复读，幻读
        - 对于同一行记录，读数据会加读锁，写数据会加写锁。通过加锁降低并发度来实现事务数据操作的唯一性。
        - 幻读 --> 在事务执行过程中，当两个完全相同的查询语句执行得到不同的结果集。这种现象称为幻读，**幻读:查询到添加的数据 脏读:查询到修改的数据**
      - **Repeatable read（可重复读）** --> 可避免脏读，不可重复读，但可能出现幻读

    - 不可重复度 --> 在一次事务中，当一行数据获取两遍得到不同的结果表示发生了“不可重复读”
- **Read committed（已提交读）** --> 可避免脏读，但是可能会造成不可重复读
    - 脏读 --> 当一个事务允许读取另外一个事务修改但未提交的数据时，就可能发生脏读
- **Read uncommitted（未提交读）** --> 级别最低，什么都避免不了
    - 丢失更新 --> 先提交的事务，会被后提交的事务覆盖
    - 比如有两个事务A从1增加到10，但是B可以看到这个中间的过程，因为数据的是未提交读，可以看到A事务没有提交的数据。
- **MySQL 默认使用 repetable read 的事务隔离级别。**
- 

- 我们以上面这个图为准，来说一下在四种隔离级别情况下，V的值分别是多少。
- **读未提交：** V1=2 V2=2 V3=2 对于读未提交，可以读到事务B的中间状态。
- **读已提交** V1=1 V2=2 V3=2 事务A只能读取到事务B提交之后的结果
- **可重复读** V1= 1 V2=1 V3=2 对于事务A来说 在一个事务中读取到的数据必定是一致的。
- **串行化** V1=1 V2=1 V3=2 串行行本质上就是加锁的操作，只有当事务A先执行完毕，事务B才会继续执行。
- **那么是如何实现的呢？**

        - 以视图的逻辑结构为准，**可重复读是在事务开启的时候创建的，在整个期间都在用这个视图。读已提交是SQL语句开始的创建的，读未提交是没有视图概念的。只返回最新的数据，而串行化，直接加锁的方式来避免并行访问**
        - 

        - 对于一条记录，将字段修改成2、3、4 会存储对应的回滚日志。用以区分不同视图下的数据。**同一条记录存在多个版本，这就是多版本并发控制MVCC**，比如我们想拿到1的值，就必须从后往前依次回滚到最终的状态。

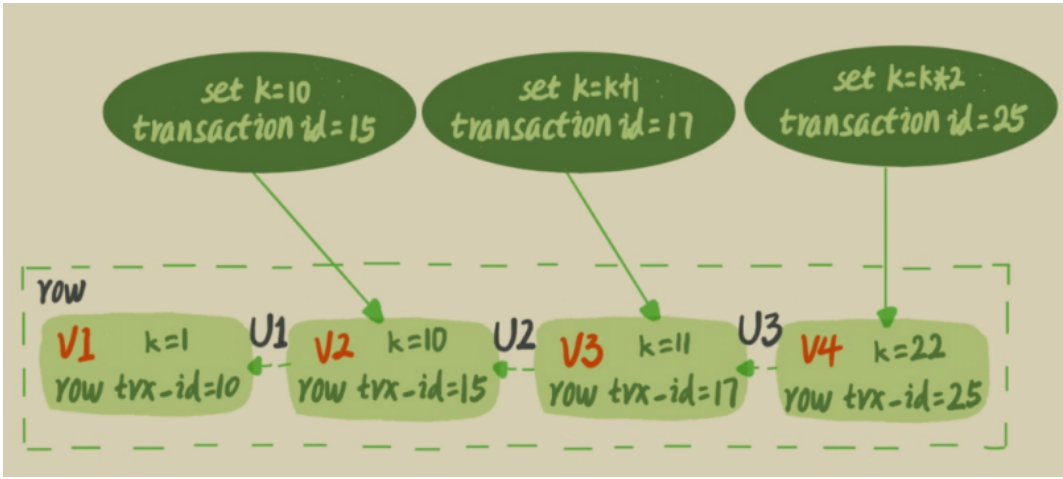
          回滚日志什么时候删除？当系统里没有比之前更早的回滚日志的read-view的时候。

          长事务，会创建比较老的视图，而视图就需要回滚日志来保证数据。占用大量空间。占用锁资源。
        - **事务的启动方式？**
          - 1.显示操作 begin
          - 2.配置好参数
          - 建议 使用显示语句操作事务，避免长事务的使用。
      - `show variables like 'transaction_isolation`
      - **适用场景：需要分别看 银行金钱校对过程不希望新数据的修改印象当前数据的校对工作。**
    - **D(Durability)持久性**
      - 一旦一个事务提交成功之后，那么这种状态就会被固化在硬盘上，除非出现硬件级别的损坏，导致持久性丢失，通常来说，持久性保证事务系统的高可靠性。并不是高可用性。高可靠性在于系统是可靠的。可用是单点故障造成的，所以需要通过集群来进行维护。
    - **原子性 隔离性 持久性 是手段 数据一致性才是目的**
- **MVCC**
  - InnoDB存储引擎实现隔离级别的一种具体方式，用于实现已提交读和重复读这两种隔离级别
  - 版本号
    - 系统版本号 是一个递增的数字，每开始一个新的事务，系统版本号就会自动递增。
    - 事务版本号 事务开始时的系统版本号
  - 快照读
    - 使用MVCC读取的是快照中的数据，这样可以减少加锁，所带来的开销。
  - 当前读
- **事务分类**
  - 扁平事务
  - 带有保存点的偏平事务
  - 链式事务
  - 嵌套事务

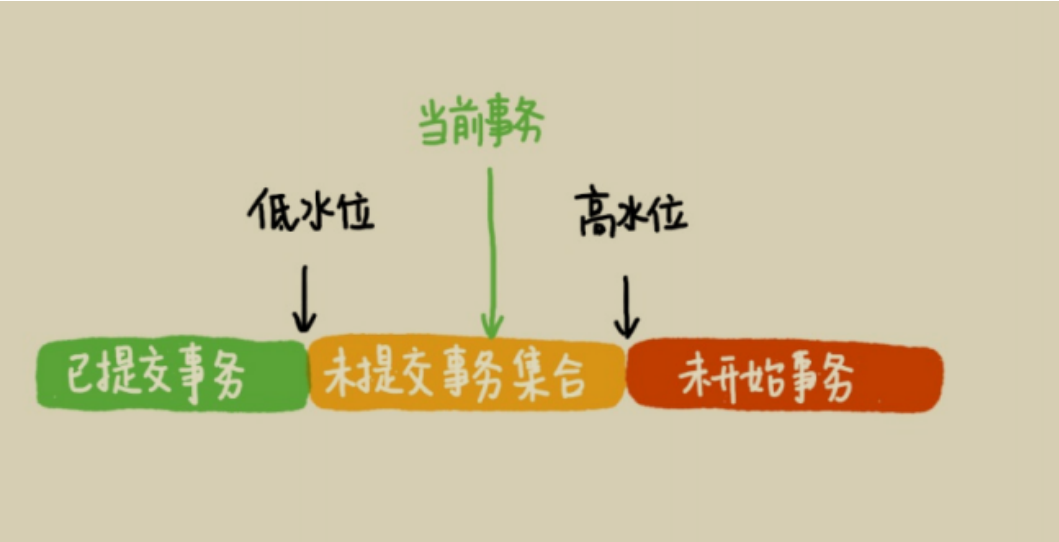
- 分布式事务

- [事务实现原理可以说一下你的理解吗?]
  - 事务实现的效果
    - **可靠性** 对于用户来说，数据前后的状态需要进行记录，日志操作undo log 和 redo log
    - **并发处理** 对于每个用户线程来说，当前处理的过程中，数据必须是不可见的，中间数据是透明的。所以隔离级别就是保证并发处理的要点。
  - 实现事务采取的那些技术以及思想？
    - 原子性：使用 undo log ，从而达到回滚
    - 持久性：使用 redo log，从而达到故障后恢复
    - 隔离性：使用锁以及MVCC,运用的优化思想有读写分离，读读并行，读写并行
    - 一致性：通过回滚，以及恢复，和在并发环境下的隔离做到一致性。
- **事务到底是隔离还是不隔离的？**
  - `start transaction with consistent snapshot` 启动事务并不是`begin/start` 在执行到第一个InnoDB表语句 事务才真正启动。
事务A	事务B	事务C
start transaction with consistent snapshot;		
	start transaction with consistent snapshot;	
		update t set k=k+1 where id=1;
	update t set k=k+1 where id=1; select k from t where id=1;	
select k from t where id=1; commit;		
	commit;	
  - 在Mysql中有两个视图概念 一个是view，通过查询语句 调用生成的结果建立一个虚拟的视图。另一个就是InnoDB在实现MVCC时用到的一致性视图，用于支持读提交和可重复读隔离级别的实现。
  - **快照在MVCC里是怎么工作的？**

    在InnoDB中为每个事务创建一个事务ID Transaction ID，当修改一行数据时，就为这一行数据添加一个数据版本ID，rowx_id，同时，旧的数据版本保留。每一行数据都对应多个版本数据。row trx_id

从图中可以看到V4是由事务tid=25修改的 k=22 所有本行数据就有trx_id=25,我们知道更新数据都会有undo log日志，其实当我们想恢复数据时，k=1时，就需要从v4逐步恢复到v1.



对于第一部分，已提交事务，是开启事务之前一条的事务，或者当前事务生成的。这个数据是可见的。

第二部分，是将来启动的事务生成的，不可见。

第三部，黄色部分。两种情况，a.若`row trx_id`在数组中，表示这个版本由还没有提交的事务生成的，不可见。b.若`row trx_id`不在数组中，表示这个版本是已经提交的事务生成的，可见。

InnoDB利用所有数据都有多个版本，实现了秒级创建的快照的能力。

总结一下：**1.版本未提交，不可见 2、版本已提交。但是是在视图创建后提交的，不可见。**

3.版本已提交，而且是在视图创建前提交的，可见。

更新逻辑

我们分析得出事务B的操作并不是在k=1的基础上进行修改的，而是基于事务C的数据k=2的基础上修改。

更新数据都是先读后写的，而这个读，只能读当前的值，称为当前读（current read）

`select k from t where id=1 lock in share mode;` 读锁

`select k from t where id=1 for update;` 写锁

可重复读 就是一致性读。当事务更新数据时，用当前读，如果当前行数据被其他事务占用锁，需要等锁释放。自己进入锁阻塞状态中。

SQL语句

- **说一下 MySQL 执行一条查询语句的内部执行过程?** ☆☆☆
  - 总体：**1.建立连接 2.查询缓存 有则返回 3.分析SQL 4.SQL优化 5.执行**
  - 客户端先通过连接器连接到 MySQL 服务器；
  - 连接器权限验证通过之后，先查询是否有查询缓存，如果有缓存（之前执行过此语句）则直接返回缓存数据，如果没有缓存则进入分析器；
    - **查询缓存可以大概说一些优缺点吗？**
      - MySQL 查询缓存功能是在连接器之后发生的，它的优点是效率高，如果已经有缓存则会直接返回结果。查询缓存的缺点是**失效太频繁导致缓存命中率比较低**，任何更新表操作都会清空查询缓存，因此导致查询缓存非常容易失效。
  - 总体上分为2层 Server层和存储引擎层。

    - `Select * from T where id = 1;`
    - **Server层**包含**连接器**[管理连接，权限验证]，**查询缓存**[命中则直接返回结果]，**分析器**[词法分析，语法分析]，**优化器**[执行计划生成，索引选择]，**执行器**[操作引擎，返回结果]
    - **存储引擎层**[存储数据，提供读写接口]
        - 连接器
            - 当我们通过软件或者cmd连接mysql时，底层会采用tcp进行通信。而处理连接的就是连接器，当用户名或者密码不符合时，就会抛出异常。`Access denied for user`或者账号正确，连接成功。这里需要注意一点，当我们开启一个新的连接时，如果在开启连接后，去修改权限，在连接过程中，并不会生效，需要我们进行重写连接才可以。连接之后，如果长时间不仅进行操作，`show processlist;`就会显示当前连接空闲，8个小时左右连接就失效。这里我们需要明白长连接和短连接的区别：长连接建立成功之后，客户端会一直持续请求，使用同一个连接，而短连接显而易见，每次建立连接使用之后，会重新创建一个新的连接。所以在实际的开发过程中，应该尽量减少短连接，连接也是比较耗费资源 网络 IO等。当然，在实际中，如果全部采用长连接后，内存会爆满。这是因为mysql执行过程中，临时使用的内存是管理在连接对象里面的。解决方案是重新连接 1.断开重连 2.`mysql reset connection;`重新连接。
        - 查询缓存
            - 当我们建立连接之后，我们就接收到客户端发送的SQL语句了，此时，我们会先去缓存里看一下有没有，有的话 直接返回，没有的话 才进行查询操作。缓存从某种程度上提供了用户的响应速度，那么查询缓存是以一种什么样的结构存储数据呢，其实很简单，用查询SQL做Key，value做数据。**但是，对于大多数情况下，我个人是不建议使用缓存的**，理由 针对于我们系统来说，只要表有更新操作，缓存就会失效。那么缓存的命中率就极低。当然，如果是静态数据，是适合做缓存的，比如系统的相关参数。我们也可以使用参数进行直接显示配置

              `select SQL_CACHE * from user;` **但是MySQL 8之后直接删除了查询缓存的模块，不推荐使用**
        - 分析器
            - 如果缓存中没有找到，就会进行分析器，先做**词法分析**，获取到关键字select知道是一个查询语句，找到对应的列，然后进行**语法分析**，是否符合MySQL语法。如果不对就会出现`You have an error in your SQL syntax`我们应该关注`right syntax to use near`的信息。
        - 优化器
            - 优化器主要要对SQL进行优化，比如SQL中是否使用了索引等。或者多表查询，决定顺序。虽然对于查询结构是一样的，但是执行效率是不同。接着就是执行器执行SQL。
        - 执行器
            - 通过上面一系列分析器知道了做什么，优化器知道了该怎么做，执行器就是进入执行器阶段开始执行。首先，先判断是否有该表的权限，没有 直接报错，有的话。调用存储引擎接口查询数据，没有索引的情况下，需要O(n)查找符合的数据。不断重复，将查询到的数据集添加到结果集中。通过explain可以看`rows`语句执行过程中扫描了多少行，但是有时候执行器调用了一次，在引擎内部扫描了多行，并不是完全对等于引擎扫描数。
- 分析器会对查询语句进行语法分析和词法分析，判断 SQL 语法是否正确，如果查询语法错误会直接返回给客户端错误信息，如果语法正确则进入优化器；

- 优化器是对查询语句进行优化处理，例如一个表里面有多个索引，优化器会判别哪个索引性能更好；
- 优化器执行完就进入执行器，执行器则开始执行语句进行查询比对了，直到查询到满足条件的所有数据，然后进行返回。
- **sql执行过程**
  - `select id,name from abc where id = 1 group by zid = 1 having zid >1 order by id limit 10,2`

    from abc 从表中找出所有数据

    where 进行过滤 在内存中操作

    group by 进行分组过滤， having 进行条件筛选

    order by 进行排序 ， 之后进行 列的选择。
  - ```
    select      查询的数据列
    from        检索的数据表
    where       行级过滤
    group by    分组过滤
    having      在分组的前提上进行数据的过滤
    order by    按照固定字段 输出 desc/asc
    limit       检索的行数

    一个表可以类比成一个集合
    from table -- 就是拿到集合 进行遍历。
    where 就是对集合进行条件遍历、from table全部数据时在硬盘上 where 之后的数据在内存中。 第一次遍历
    举例子: Select user. name，user. age,100 from users user where user. age<20
    select 是对子集和中的结果进行第二次遍历。而where是第一次遍历。
    而这块的100是一个常量值。
    引入group by之后
    group by  ：group by会在select 之前执行。group by之后会得到 where 第一次遍历之后的结果 set<set>
    Having为group by而生 这种想法是片面的，可以直接使用having

    Select * from users where age > 20  ：是在拉到内存的这个过程进行过滤
    Select * from users having age > 20 ：  将全部表的数据拉到内存中 在进行条件判断。

    order by 是最select 最终集合的排序操作。并不是对表中已有的字段进行排序。

    1.子查询
        子查询中只能返回一个字段的数据。
        可以将子查询的结果作为where语句的过滤条件。
        sql: select * from table where col in (select col from table table);
    ```

- **mysql中where groupby having关键字了解他们之间的关系吗？**
  - where子句用来选from子句中指定的操作所产生的行
  - group by 子句用来分组where子句的输出
  - having子句用来从分组的结果中选出
    ```

- eg：`select * from sys_user where id in(1,2,3) GROUP BY status HAVING power = 10;`
- **了解group by 和 order by 有什么缺点吗？**
  - group by 和 order by操作通常需要创建一个**临时表来处理查询的结果**，所以如果查询结果很多的话会严重影响性能。
  - ```
    1.group by clo
        作用:把具有相同的数据值行放到同一组中。
        select clo ,count(*) from table group by clo;//按照clo列进行分组，如果clo值相等 就分成一组。
        如果一个班中有sex = man men 将men man分成两组count(*)进行统计。

    2.group by自动按分组字段进行排序。order by 按照汇总字段来进行排序
        select clo,count(*) as num from table group by clo order by num;

    3.where过滤行， having过滤分组，行过滤应该先于分组过滤
       select col,count(*) as num from table where clo >1 group by col having num >=2;

    分组规定
        * group by 子句出现在where 子句之后，orderby 子句之前。
        *   除了汇总字段外，select语句中的每一个字段都必须在order by 子句中给出。
        * null的行会单独分为一组
    ```

- **各种join操作的区别（left, right, inner join)** ☆
  - left join（左联结），返回左表全部记录和右表联结字段相等的记录；
  - right join（右联结），返回右表全部记录和左表联结字段相等的记录。
  - inner join (求相同的部分)
  - full join (全连接) 左右两表的全部数据
  - ```
    1.连接
        关键字使用：连接用于连接多个表，使用join关键字，并且条件语句使用on而不是where
        执行效率：连接可以替换子查询，并且比子查询的效率一般会更快。
        取别名：可以使用as给列名，计算字段或给表取别名，给表取别名是为了简化sql语句以及连接相同表。

    2.内连接
        2.1 内连接又叫等值连接，使用inner join关键字。
        select A.value,B.value from tableA as A inner join tableA as b on A.key = B.key;
        2.2 不使用inner join关键字 使用普通的多表查询
        select A.value,B.value from tableA as A,tableB as B where A.key = B.key;

    3.自连接
        自连接可以看成内连接的一种，只是连接的表示自身而已。
        eg:一张员工表，包含员工姓名和员工所属部门，要找出与 Jim 处在同一部门的所有员工姓名。
        子查询:
    ```

```
    select employeename from emp where emp.deparment = (select
deparment from emp where employee = 'Jim');

    自连接
    select e1.name from emp e1 inner join emp e2 on e1.department =
e2.deparment AND e2.name = 'Jim';

4.自然连接
    自然连接是把同名列通过等值测试连接起来的，同名列可以有多个。
    内连接和自然连接的区别，内连接提供连接的列，而自然连接自动连接所有同名列。
    select A.value ,B.value from tableA as A NATURAL join tableB as B;

5.外连接
    外连接保留了没有关联的那些行。分为左外连接，右外连接以及全外连接，左外连接就是保
留左表没有关联的行。
```

- **以下 or 查询有什么问题吗？该如何优化?**
  - > select * from t where num=10 or num=20;

    答：如果使用 or 查询会使 MySQL 放弃索引而全表扫描，可以改为：

    > select * from t where num=10
    >
    > union
    >
    > select * from t where num=20;
- **delete、 drop、truncate 的区别（PayPal）**
  - drop 直接删掉表（不再需要一张表的时候，用drop）
  - truncate 删除的是表中的数据，再插入数据时自增长的数据id又重新从1开始（**保留表而删除所有数据的时候用truncate，实际是删除原来的表并重建一张新表**）
  - delete 删除表中数据，可以在后面添加where字句（想删除部分数据行时候，用delete，并且带上where子句）
- **limit分页查询使用方式？**
  - `Select * from tablename orderby liename limit m,n（从 m+1 条开始，取 n 条数据）`
  - Mysql 分页查询：客户端通过传递 start（页码），limit（每页显示的条数）两个参数去分页查询数据库中的数据。 Limit m，n 从 m+1 条开始，取 n 条。1）查询第一条到第十条的是：select * from table limit 0,10;对应的就是第一页的数据。2）查询第 10 条到第 20 条的是：select * from table limit 10,10;对应的就是第二页的数据。3）查询第 20 条到第 30 条的是：select * from table limit 20,10;对应的就是第三页的数据。总结：select * from table limit （页数-1）*每页条数, 每页条数;
- **主键自增设置方式 auto_increment**
- **order by 是怎么工作的？**

存储引擎

- **Mysql的逻辑架构了解吗？说一下？**
  - Server层
    - 包含连接器，分析器，优化器以及执行器
      - 连接器：验证客户端权限，建立和断开Mysql连接
      - 分析器：进行SQL语句的语法分析
      - 优化器：选择索引，生成具体的SQL语句执行计划
      - 执行器：操作存储引擎，执行SQL，返回执行结果

    - 存储引擎层
        - 存储引擎
- **MySql常见的存储引擎有哪些(重点掌握)？**
    - 是否支持事务
        - **MyISAM不支持事务(用于选择密集型，插入密集型)，InnoDB是事务类型的存储引擎**
    - 支持哪种锁
        - MyISAM只支持表级锁，**InnoDB支持行级锁和表级锁，默认为行级锁 使用于更新密集型**
    - 是否支持外键
        - MyISAM引擎不支持外键，InnoDB支持外键
    - 是否支持崩溃后安全恢复
        - InnoDB支持崩溃后安全恢复 myisam不支持
    - 缓存
        - MyISAM只缓存索引，不缓存真实数据
        - InnoDB不仅缓存索引还要缓存真实数据，对内存要求较高，而且内存大小对性能有决定性的影响
    - 性能问题
        - MyISAM 性能比 InnoDB 高
        - InnoDB 主键查询性能高于 MyISAM
    - 对于count(*)查询来说MyISAM更有优势，因为其保存了行数
    - InnoDB是为处理巨大数据量时的最大性能设计的存储引擎
    - MyISAM支持全文索引(FullText) InnoDB不支持
        - Myisam：不支持事务行级锁和外键约束。所以当执行 insert 和 update 时，执行写操作

          时，要锁定整个表，所以效率低。但是它保存了表的行数，执行 select count(*) from table

          时，不需要全表扫描，而是直接读取保存的值。所以若读操作远远多于写操作，并且不需要

          事务，myisam 是首选。

          Innodb：支持事务、行级锁和外键，mysql 运行时，Innodb 会在内存中建立缓冲池，用

          于缓冲数据和索引。不保存表的行数，执行 select count(*) from table 时要全表扫描。写不锁

          定全表，高效并发时效率高。
    - **Memory 将数据存储在内存中，适合做临时表使用，索引支持B+数和Hash索引**
- **如果把一个 InnoDB 表的主键删掉，是不是就没有主键，就没办法进行回表查询了？**
    - 不是，如果把主键删掉了，那么 InnoDB 会自己生成一个长度为 6 字节的 rowid 作为主键。
- **一张自增表中有三条数据，删除两条数据之后重启数据库，再新增一条数据，此时这条数据的 ID 是几？**
    - 如果这张表的引擎是 MyISAM，那么 ID=4，如果是 InnoDB 那么 ID=2（MySQL 8 之前的版本）

日志模块

- **为什么有日志模块呢？**

- 如果对于每一次更新操作先根据条件找到对应的记录，然后将记录更新 在写回磁盘中，那么IO成本以及查询记录的成本很高。通过先将操作记录在日志模块中，在合适的时间才回去写磁盘，日志更新完毕就将执行结果返回给客户端。
- **日志模块有redo log(重做) 和 binlog(归档日志)**

- **Mysql日志说一下你的理解？**

  ① **错误日志**：用来记录 MySQL 服务器运行过程中的错误信息，比如，无法加载 MySQL 数据库的数据文件，或权限不正确等都会被记录在此，还有复制环境下，从服务器进程的信息也会被记录进错误日志。默认情况下，错误日志是开启的，且无法被禁止。默认情况下，错误日志是存储在数据库的数据文件目录中，名称为 hostname.err，其中 hostname 为服务器主机名。在 MySQL 5.5.7 之前，数据库管理员可以删除很长时间之前的错误日志，以节省服务器上的硬盘空间， MySQL 5.5.7 之后，服务器将关闭此项功能，只能使用重命名原来的错误日志文件，手动冲洗日志创建一个新的，命令为：

  ```
  mv hostname.err hostname.err.old

  mysqladmin flush-logs
  ```

  ② **查询日志**：查询日志在 MySQL 中被称为 general log（通用日志），查询日志里的内容不要被“查询日志”误导，认为里面只存储 select 语句，其实不然，查询日志里面记录了数据库执行的所有命令，不管语句是否正确，都会被记录，具体原因如下：

  - insert 查询为了避免数据冲突，如果此前插入过数据，则当前插入的数据如果跟主键或唯一键的数据重复那肯定会报错；
  - update 时也会查询因为更新的时候很可能会更新某一块数据；
  - delete 查询，只删除符合条件的数据；

  因此都会产生日志，在并发操作非常多的场景下，查询信息会非常多，那么如果都记录下来会导致 IO 非常大，影响 MySQL 性能。因此如果不是在调试环境下，是不建议开启查询日志功能的。

  查询日志的开启有助于帮助我们分析哪些语句执行密集，执行密集的 select 语句对应的数据是否能够被缓存，同时也可以帮助我们分析问题，因此，可以根据自己的实际情况来决定是否开启查询日志。

  查询日志模式是关闭的，可以通过以下命令开启查询日志：

  ```
  set global general_log=1

  set global log_output='table';
  ```

  general_log=1 为开启查询日志，0 为关闭查询日志，这个设置命令即时生效，不用重启 MySQL 服务器。

  ③ **慢日志**：慢查询会导致 CPU、IOPS、内存消耗过高，当数据库遇到性能瓶颈时，大部分时间都是由于慢查询导致的。开启慢查询日志，可以让 MySQL 记录下查询超过指定时间的语句，之后运维人员通过定位分析，能够很好的优化数据库性能。默认情况下，慢查询日志是不开启的，只有手动开启了，慢查询才会被记录到慢查询日志中。使用如下命令记录当前数据库的慢查询语句：

  ```
  set global slow_query_log='ON';
  ```

  使用 set global slow_query_log='ON' 开启慢查询日志，只是对当前数据库有效，如果 MySQL 数据库重启后就会失效。因此如果要永久生效，就要修改配置文件 my.cnf，设置 slow_query_log=1 并重启 MySQL 服务器。

  ④ **redo log（重做日志）**：为了最大程度的避免数据写入时，因为 IO 瓶颈造成的性能问题，MySQL 采用了这样一种缓存机制，先将数据写入内存中，再批量把内存中的数据统一刷回磁盘。为了避免将数据刷回磁盘过程中，因为掉电或系统故障带来的数据丢失问题，InnoDB 采用 redo log 来解决此问题。

  ⑤ **undo log（回滚日志）**：用于存储日志被修改前的值，从而保证如果修改出现异常，可以使用 undo log 日志来实现回滚操作。

undo log 和 redo log 记录物理日志不一样，它是逻辑日志，可以认为当 delete 一条记录时，undo log 中会记录一条对应的 insert 记录，反之亦然，当 update 一条记录时，它记录一条对应相反的 update 记录，当执行 rollback 时，就可以从 undo log 中的逻辑记录读取到相应的内容并进行回滚。undo log 默认存放在共享表空间中，在 ySQL 5.6 中，undo log 的存放位置还可以通过变量 innodb_undo_directory 来自定义存放目录，默认值为“.”表示 datadir 目录。

⑥ **bin log（二进制日志）**：是一个二进制文件，主要记录所有数据库表结构变更，比如，CREATE、ALTER TABLE 等，以及表数据修改，比如，INSERT、UPDATE、DELETE 的所有操作，bin log 中记录了对 MySQL 数据库执行更改的所有操作，并且记录了语句发生时间、执行时长、操作数据等其他额外信息，但是它不记录 SELECT、SHOW 等那些不修改数据的 SQL 语句。

- **redolog、undolog、binlog各有什么作用？**
- **一条update语句的执行过程？**
  - 我们知道通过一条SQL语句会经过连接器、查询缓存、分析器、优化器、执行器、存储引擎。但是对于更新语句来说，会有重要的两个日志模块。**一个是`redo log`重做日志和`binlog`归档日志。**
  - `redo log` 用户的每次更新操作都会进行日志的记录，这就是`redo log`，这里需要明确一点，当我们每次更新数据时，都直接将数据更新同步到磁盘数据文件中，磁盘文件在找到对应的行，是比较耗时的。而`MySQL`采用了一定的技巧。存储引擎先将修改记录写到日志`redo log`中，并更新内存。之后空闲的时候在写入磁盘中。**`redo log`是固定大小的，比如可以配置一组4个文件，每个文件的大小是1GB，那么总共就是4GB。从头开始写，写到末尾，从新从头写。** 具体文件格式为`checkpoint` `write_pos`

    那么总共就是二部分，第一部分就是`checkpoint`到`write_pos`之间的位置，表示需要擦除的位置，也就是需要将`redo log`日志写入到硬盘中，并更新硬盘上的数据。第二部分是`write_pos`到`checkpoit` 因为是一个环形链表的结构，所以 需要进行不断轮询。这部分就是当前可以进行写的内存区域空间。那么当`write_pos`追赶上`checkpoint`的时候，说明当前空间已经满，必须进行一部分数据写入到磁盘中。

    那么如果出现宕机情况呢，放心，因为有`redo log` 日志记录在磁盘上，不会出现数据的丢失。这也叫做`crash-safe`
  - `bin log` 我们知道`MySQL`分为存储引擎层和服务层。上面说的`redo log`是存储引擎层的日志，而服务层也有自己的日志。那么这里你就有问题，为什么会有两个日志呢？我们来思考一下。早期使用的存储引擎是`MyISAM`是没有`crash-safe`能力的，`bin log`只能作用于归档。`innodb`使用另一套日志系统也就是`redo log`来实现`crash-safe`能力。

    `redo log`和`bin log`的区别是：

    1.`redo log`是`InnoDB`引擎特有的，`bin log`是`MySQL`的`Server`实现的。所有引擎都可以使用。

    2.`redo log`是物理日志，记录的是在某个数据页上做了什么修改，`binlog`是逻辑日志，记录的是这个语句的原始逻辑，比如`给ID=2这一行的c字段加1`。

    3.`redo log`是循环写的，空间固定会用完毕，而`bin log`是可以追加写入的，当前一个文件写完毕，可以到下一个文件继续写。并不会覆盖以前写的日志。
  - **一条更新语句的执行过程**
    - 执行器会调用执行引擎的接口，查找id=2的数据，如果这个数据在内存中，直接返回。否则，从磁盘中查找，按照B+树结构查找返回。执行器拿到本行数据进行c=c+1，将本条语句调用引擎来执行，存储引擎会先生成对应的`redo log`日志，然后处于准备状态。`preparer状态`，执行器写入`bin log`日志，然后将引擎中提交了事务之后，整个事务就结束了。
  - 两阶段提交

- 上面我们说到了两阶段提交过程，一个是先进行准备，另一个是进行提交操作。那么为什么我们系统要采用两阶段提交呢，好了，我们做一个比喻，`update a set c = c + 1 where id = 1` 之前c=0，如果没有两阶段提交，我们可以先做`bin log`后做`redo log`或者相反，
- 第一种情况 先做`redo log` 当`redo log`执行完毕后，准备写`bin log`的时候，系统宕机了。也就说`redo log`记录了当前c=1的数据，而`bin log`记录了当前c=0的情况，当我们使用`bin log`进行数据的恢复，`c=0`出现了数据不一致问题。
- 第二种情况，先做`bin log`后做`redo log`，`bin log`记录完毕，但是当操作`redo log`时，系统宕机，因为`redo log`失败，事务并不会提交，而`bin log`记录的值是`c=1`而`redo log`记录的值是`c=0`，当使用`redo log`进行数据恢复时，数据不一致。
- **什么场景下会使用数据同步呢？**

  通常， 在工作中除了数据误删的情况下，需要数据的恢复 使用日志，当系统随着业务的拓展，提升读的能力，我们需要进行数据的全量复制外加`bin log`来实现的。这个数据不一致就会导致数据不一致问题。**`redo log`物理日志 和`bin log` 逻辑日志，表示事务的提交状态，两阶段提交是让两个状态保持逻辑上的一致。**

SQL优化、线上故障排查

```
维度：
慢sql定位，链路定位 数据库慢sql日志查询  表结构优化  加索引优化  几种索引方式 索引失效
```

- SQL优化策略 *TODO*
  - 这大概是最全的sql优化方案了
- **查询语句的优化方案有哪些？**
  - 不做列运算，把计算都放入各个业务系统实现；
  - 查询语句尽可能简单，大语句拆小语句，减少锁时间；
  - 不使用 select * 查询；
  - or 查询改写成 in 查询；
  - 不用函数和触发器；
  - 避免 %xx 查询；
  - 少用 join 查询；
  - 使用同类型比较，比如 '123' 和 '123'、123 和 123；
  - 尽量避免在 where 子句中使用 != 或者 <> 操作符，查询引用会放弃索引而进行全表扫描；
  - 列表数据使用分页查询，每页数据量不要太大。
- **合理使用覆盖索引，减少回表操作次数**
- **如何使用explain?**
  - id — 选择标识符，id 越大优先级越高，越先被执行
  - select_type — 表示查询的类型。
  - table — 输出结果集的表
- partitions — 匹配的分区
  - **type — 表示表的连接类型**
    - 其中最重要的就是 type 字段，type 值类型如下：
      - **all — 扫描全表数据**
      - **index — 遍历索引**
      - **range — 索引范围查找**
      - **index_subquery — 在子查询中使用 ref**
      - **unique_subquery — 在子查询中使用 eq_ref**
      - **ref_or_null — 对 null 进行索引的优化的 ref**

        - fulltext — 使用全文索引
        - ref — 使用非唯一索引查找数据
        - eq_ref — 在 join 查询中使用主键或唯一索引关联
        - const — 将一个主键放置到 where 后面作为条件查询， MySQL 优化器就能把这次查询优化转化为一个常量，如何转化以及何时转化，这个取决于优化器，这个比 eq_ref 效率高一点
    - possible_keys — 表示查询时，可能使用的索引
    - key — 表示实际使用的索引
    - key_len — 索引字段的长度
    - ref— 列与索引的比较
    - rows — 大概估算的行数
        - **可以说一下rows是怎么计算的吗？**
            - MySQL认为必须要逐行去检查和判断的记录的条数。比如我们查询一条数据 `select * from t where a = 10 and b = 20` 都没有建立索引，a=10的数据有5条，而a=10 and b = 20 的数据有2条，所以rows显示为5条，需要注意判断。
            - ```
mysql> select * from tt;
+----+----+----+
| id | a | b |
+----+----+----+
1	10	20
2	10	10
3	10	30
4	10	20
+----+----+----+
4 rows in set (0.03 sec)

mysql> select * from tt where a = 10 and b = 20;
+----+----+----+
| id | a | b |
+----+----+----+
| 1 | 10 | 20 |
| 4 | 10 | 20 |
+----+----+----+
2 rows in set (0.03 sec)

mysql> explain select * from tt where a = 10 and b = 20;
+----+-------------+-------+------+---------------+------+---------+------+------+-------------+
| id | select_type | table | type | possible_keys | key | key_len | ref | rows | Extra |
+----+-------------+-------+------+---------------+------+---------+------+------+-------------+
| 1 | SIMPLE | tt | ALL | NULL | NULL | NULL | NULL | 4 | Using where |
+----+-------------+-------+------+---------------+------+---------+------+------+-------------+
1 row in set (0.04 sec)
```
            - 我们可以看出符合条件的数据只有两条，而rows=4 是按照mysql认为需要扫描的数据，因为a=10有4条数据，但是b=20有两条，前者并不能计算出符合条件的数据，没有添加索引的前提下，需要遍历4条数据
    - filtered — 按表条件过滤的行百分比
    - Extra — 执行情况的描述和说明

## 主从复制

- **表的优化策略有哪些？**
    - 读写分离，主库负责写，从库负责读。
    - 垂直分区，根据数据属性单独拆表甚至单独拆库。
    - 水平分区，保持表结构不变，根据策略存储数据分片，这样每一片数据被分散到不同的表或者库中。水平拆分只是解决了单一表数据过大的问题，表数据还在同一台机器上，对于并发能力没有什么意义，因此水平拆分最好分库。另外分片事务难以解决，跨节点 join 性能较差。
    - **数据库分片方案有哪些 了解吗？**
        - 数据库创建的分片方案有两种方式：**客户端代理方式和中间件代理方式。**
            - 客户端代理 — 分片逻辑在应用端，封装在 jar 包中，通过修改或者封装 JDBC 层来实现，比如 Sharding-JDBC、阿里 TDDL 等。
            - 中间件代理 — 在应用层和数据层中间加了一个代理层。分片逻辑统一维护在中间件服务中，比如 MyCat、网易的 DDB 都是中间件代理的典型代表。
```

问题迭代

- 查询数据？
- 执行查询时，若要查询的数据很多，假设要查询 1000 万条，用什么方法提升效率？答案：1）从数据库方面：建立索引、分区、尽量使用固定长度的字段、限制字段长度 2）从数据库 IO 方面：增加缓冲区 3）在 sql 语句方面：优化 sql 语句，减少比较次数、限制返回的条目数 4）在 java 方面，如果是反复使用的查询，使用 preparedStatement。
- **Mysql出现了乱码问题 你如何解决的？**
  - 解决 MySQL 中文乱码的问题，可以设置全局编码或设置某个数据库或表的编码为 utf8。 设置全局编码：

    ```
    set character_set_client='utf8';
    set character_set_connection='utf8';
    set character_set_results='utf8';
    ```

    设置数据库的编码：

    ```
    alter database db character set utf8;
    ```

    设置表的编码：

    ```
    alter table t character set utf8;
    ```

- **MySQL 性能指标都有哪些？如何得到这些指标？**
  - MySQL 最重要的性能指标有以下两个：
    - QPS（Queries Per Second），每秒查询数，一台数据库每秒能够处理的查询次数；
    - TPS（Transactions Per Second），每秒处理事务数。

    这些性能指标可以通过 show status 来查询当前数据库状态的结果信息中估算出来，show status 会有 300 多条状态信息记录，其中以下这些信息 QPS 和 TPS 有关系：

    - Uptime，服务器已经运行的时间，单位秒；
    - Questions，已经发送给数据库查询数；
    - Com_select，查询次数，实际查询次数；
    - Com_insert，插入次数；
    - Com_delete，删除次数；
    - Com_update，更新次数；
    - Com_commit，事务次数；
    - Com_rollback，回滚次数。
- **MySQL 毫无规律的异常重启，可能产生的原因是什么？该如何解决？**
  - 可能是积累的长连接导致内存占用太多，被系统强行杀掉导致的异常重启，因为在 MySQL 中长连接在执行过程中使用的临时内存对象，只有在连接断开的时候才会释放，这就会导致内存不断飙升，解决方案如下：
    - 定期断开空闲的长连接；
    - 如果是用的是 MySQL 5.7 以上的版本，可以定期执行 mysql_reset_connection 重新初始化连接资源，这个过程会释放之前使用的内存资源，恢复到连接刚初始化的状态。
- **影响Mysql的性能因素？**

```
- 定位层面

    - 1.工具定位
    - 2.用户的反馈
    - 3.找某一个层面(大部分都在DB上)
    - 4.
```

- **优化的目标是什么？**

```
- 业务请求流  web->DNS->nginx->tomcat->三方进程->DB (数据流向)
```

```
- 1.根据不同的业务 需要选择合适的结构
- 2.数据的多样化 图片 视频 热点数据 分布存储
- 3.设计对系统的性能
- 4.硬件环境对系统性能的影响
```

- **重写理解数据库**
    - 对于大多数人的理解，数据库就是专门用来存储数据的，其实这只是其中的一点，另一点，数据库也可以用来进行计算，而这个就是靠存储过程来完成的。举一个栗子，比如我们统计10000个员工的平均工资，如果将数据拉到应用服务器上，当然可以计算。但是如果在数据库端直接计算好，那么就能节省一段网络IO的消耗。而Oracle的作用也在于此。Oracle有强大的计算和存储能力。所以我们应该合理利用计算能力。
- **重新理解隔离级别**
    - 事务是一个比较重要的概念，大多数人的理解事务只是停留在原子性，认为要么成功要么失败。在加上持久性。我们来分析一下，如果数据库端只允许同一时刻，一个客户端进行操作，那么就不存在数据库端共享数据的操作，原子性+持久性完全可以保证数据的一致性。但是，在一个系统中，同一时刻是有多个客户端进行并发访问的。如果我们将这种并发访问的操作放入一个工作队列中，那么也是可以保证一致性的。但是问题呢，就是并发度太低，事务本质上就是一个多个读写操作组成的原子包。原子性保证成功或失败，只有一个状态，而持久性保证了数据的可恢复性，而隔离性保证的是多个并发下每个视图数据互不干扰。
    - **串行化：一个事务执行完毕，另一个事务继续执行，但是可串行化，本质就是并发执行，但是最终执行结果是串行化执行结果相同，也就不存在对并发数据的修改。一个例子A事务进行添加数据11 B事务添加数据22** 两个事务之间不存在依赖和关联关系。所以和串行化执行的结果是相同的。
    - **可串行化隔离级别：** 在可串行化隔离级别下，等于多个事务串行执行，可以完全保证数据的一致性。

      事务A读取本表的全部数据，事务B进行插入数据是被阻塞的。对于一些要求数据强一致的业务场景，我们需要使用序列化来保证数据的一致性。另一种方式，可以使用for update。但是for update并不能保证数据的完全一致性。只能保证对于读取的数据行加锁，不能保证新插入的数据。这就是幻读问题。而串行化解决了这个问题，只要我读取到的数据，就不需要进行操作。保证了在读取数据的期间，不会有插入操作。对于数据敏感的业务需要使用可序列化隔离级别。
    - 其他三种隔离级别，从一定程度上放弃了数据的完全一致性，为了性能。需要我们根据业务和性能，找到一个平衡点。
    - **可重复读隔离级别：** 同一个事务中读取到的数据，必须一样。 可序列化保证的数据一致，除了读取到的数据的一致性，还加了不可以有新的插入的数据。也就是解决了幻读的问题。早期mysql是通过加锁的方式来解决的。我读取一行数据，其他事务也不能读取，降低了并发的性能。现在采用MVCC多版本快照技术。大概过程就是建立了一个事务时，立马对数据进

行快照。相当于每个事务独享当前快照数据，而基于快照的操作，对其他数据来说也是不可见的，不仅解决了并发读的问题还解决幻读的问题。

但是基于MVCC+快照 读取的值是过期值，如果我们想拿到最新值呢`select from update`禁用快照。其他其实也无法修改读取到的行数据。可以使用MVCC+快照 以及加锁方式。**MVCC最大的特点，就是其他并发事务允许并发的写，而不是阻塞。大大提高了性能**

- **MVCC过期数据与for update抉择** MVCC读取到的数据可能是过期的，对于一些不允许使用过期数据的业务，我们必须使用for update。虽然MVCC可以基于快照上进行修改，但是幻读问题从根本上没有解决，因此，为了保证数据的一致性 该使用序列化还是应该使用序列化。
- **读已提交隔离级别** 每次读取同一行的数据，都会读取到已经提交的数据。可能两次读取到不同的数据。如果事务A读取的数据行，事务B正在修改，会等事务B提交完成，A之前都是阻塞的。**当一行数据被写锁封锁的时候，这一行数据上的读取会被发生阻塞，直到写锁释放，读已提交限制了读的并发。而可重复读中，当读一行数据之后，这一行被读锁控制，其他所有的并发事务都无法进行写，限制了写的并发。**由于读锁会阻塞，因此可不可以等到该行数据没有提交就可以读。这是就是脏读。
- 在读已提交隔离级别下，使用了多版本技术，read的时候，要么使用旧版本数据，要么使用当前最新的值。在可重复读，使用快照技术，每个事务开始的时候，自动给数据库拍照，每个事务共享一个快照。
- 隔离级别本质上就是加不同的锁，来同步数据的一致性。比如说读未提交，不用加锁，你随意更新和读、读已提交，我读取到的一定是提交的数据。我修改，你别查询。你查我可以改。可重复读模型下。我改，你别查，我查，你别改。随着隔离级别的等级提高。可串行化 我读和该 你都别动。相当于加了表锁。

- **重新理解持久化**
  - 持久化是一个很重要的话题，持久化是任何数据系统的重中之重。先说数据库，数据库中用户加入一条数据，成功的标准是以成功刷入磁盘为准。也就是先写入到内存中，会将持久化到磁盘上。才会返回给用户成功。即使在某一刻宕机，也只会出现两个状态，要么数据持久化成功的状态，恢复一下。要么没有持久化成功。但是对于redis来说，redis的持久化机制有AOF和RDB，加入一条数据，写入成功，是在内存层面的。会fork一个子进程进行异步刷新到磁盘上。说明了。某一个宕机，子进程没有刷新到磁盘上，那么数据就没有被持久化。说明了redis中的持久化和数据库的持久化并不是一个等级。数据库端可以保证只要持久化了一定是成功了。但是redis的持久化只是写入内存中，就返回给用户，异步进行持久化。
  - 不过是有代价的，那就是redis写入内存速度比较快，而数据库需要同步磁盘IO操作完毕才可以。
  - 对比到消息队列中，如果客户端将数据写入到MQ中，MQ立即返回给客户端。会不会存在没有持久化数据，如果此刻宕机，那么会不会出现数据不一致问题。而MQ客户端到MQ中，持久化是一个同步的状态，一旦收到MQ成功，一定知道数据被持久化了，如果因为网络等原因没有接收到，就需要一些定时重发的机制，进行数据的重发。但是如果采用ACID这样的机制，可以保证只会出现两个状态，要么就是持久化成功的状态，要么没有持久化的状态，不会出现持久化了一半，出现脏数据。

    所以说持久化一个维度就是系统崩溃，都不会破坏崩溃前和崩溃后的一致性。

    第二个维护 数据回滚，也就是数据写入到一半的时候，系统宕机了。那么可以保证将数据恢复到没有写入数据的前一刻。保证数据的一致性。**解决系统崩溃之前和系统崩溃重启数据的一致性**
  - 分布式系统中数据持久化是一个很重要的问题。

- **重新理解一致性**

- 整个系统，在逻辑上应该保证一个约束下，始终正确。比如用户支付成功，必定可以查看订单的信息，而不能出现，支付成功，订单没有生成。但是数据库系统就可以保证不会出现这种中间态。**一个企业系统的数据一致性保障，甚至关乎一个系统的生死存亡**，数据库并不仅仅提供了数据的存储和计算功能，还提供了一致性数据的保证。一个事务，包含若干条对数据库的操作语句，原子性保证了一些sql执行，另一些sql没有执行，会回滚到开始阶段。**原子性只是保证一致性的必要条件，** 比如两个事务 分别都对同一行数据进行了操作，原子性只保证了两个事务的执行前后的一致性。但是由于对共享数据的操作，就会出现数据不一致性。线程安全问题。类比到Java线程中，两个线程对共享数据进行操作，那么如果不进行加锁同步操作，会初夏年数据不一致性问题。java端是通过加锁的方式进行的。线程安全问题对比到数据库端就是不同的事务操作数据的不一致性问题。
- 好了，有了上面的基础，我们假设A和B事务对同一行进行操作，A将id=1的数据修改成了10，但是没有提交，B事务将id=1的值修改成了20。那么当事务B提交的之后，是感知不到事务A的存在。因此，也就说出现了更新丢失的问题。数据库是如何解决的呢？ 通过加锁，我在操作的时候某行数据时，不允许你进行操作。但是由于锁会阻塞，导致性能下降，所以都是基于快照和MVCC进行数据的读取和更新的操作，是一种优化技术，目的就是为了提高并发性能。
- 平常开发，我们使用的syn和lock 和 数据库锁是两个层面的锁，一个是用户使用系统提供的锁机制。另一个数据库提供的锁机制。**数据库的隔离级别，是数据库提供给用户的一种更抽象的高层次锁机制**

  说白了，隔离级别就是数据库端提供的一种API级设定不同锁的操作。只不过数据库帮我们将这种复杂的操作进行了内部化的处理。对于程序员来说就是一个API的调用。我具体的使用就是JDBC端进行Connection对象进行设置，而这个隔离级别作用域范围只在本连接中生效。
- 如何使用，数据库最基本的线程安全锁定机制+4种隔离级别，隔离级别越高，并发性能越低。@Transactional 是框架进一步封装到容器中的操作，本质上还是去配置参数设置数据库的事务配置。只在当前事务生效。**好了，有了原子性保证，数据库的线程安全锁定机制，以及 select for update 这样的隐式用户锁，和事务隔离级别的隐式用户锁包**

- **重新理解ACID**
  - 第一个原子性，这个是最简单的，大家都明白.一个事物内的所有操作共同组成一个原子包,要么全部成功，要么全部失败 这是最基本的特性，保证了因为一些其他因素导致数据库异常，或者宕机.这是原子性是大家对事物理解最多的.

    第二一致性，是大家误解最深的,很多老师喜欢用银行转账的例子来讲一致性.所谓一致性是基于原子性的 原子性，只是保证了一个事物内的所有操作的同一性，大家同生死，不会出现你死了，我还活着， 但是，原子性并没有保证大家同一时刻一起生，一起死，计算机指令是有先后顺序的，这样就决定了一个事物的提交，会经历一个时间过程 那么，如果事务提交进行到了一半，我读取了数据库，是否会读到中间结果？为了防止这样的情况，数据库事务的一致性规定了 事务提交的前后，永远只可能存在事务提交前的状态和事务提交后的状态，从一个一致性状态到另一个一致性状态，而不可能出现中间的过程态 也就是说事务的执行结果是量子化状态，而不是线性状态。一致性是什么意思？假如你和你的老婆是穿一条裤子的，任何你知道的东西你都要高速她，她知道的东西都要告诉你 那么，你接受一条信息的时候，她并不知道，这个时候，你们两个人的状态是不一致的，通过一定的同步措施，比如打电话，发短信 他也知道了，最终你们两个人的状态保持一致，这是一种弱一致性，因为，在你正在发短信高速她的同时，如何外部者同时访问你们两个人 得到的结果会不一致，好了，回到数据库事务，数据库提交事务会有一个过程，如果提交的时候，存在一个时间差，在提交的第一秒，一个删过程还没完成 到了第三秒才完成，会不会第一秒访问的人和第三秒当真的人得到不同的结果？出现不一致，状态的混沌？这就是一致性得保证 acid的一致性保证了，只会有前状态和后状态，绝不会出现中间态。但是，分布式事务中，为了保证严格的数据一致性，几乎不可能 因为分布式事务涉及到网络开销，两个系统的数据库操作的分布式事务，如果遇到网络延时，或者一方数据库锁定过长，整个分布式事务会瘫痪 强一致性几乎不可能，中间态出现不可避免，所以，你们现在经常网络购物，钱扣了，但是，会隔很久才会订单生效，或者订单失败了， 会几日后才退款，说明了，他们并不是立即一致的，因为银行系统和

订单系统并不在一个数据库，他们做不到强一致性，但是，他们最终一致了 这就是，著名的最终一致性解决方案，单个数据库系统中，事务很好的保证了强一致性。但是，分布式环境下，这一直是个难题 不过，现在很多人都在做这方面的工作，著名的分布式治理框架。zookeeper，就是这个领域的最强开源产品。

事务的隔离性，基于原子性和一致性，因为事务是原子化，量子化的，所以，事务可以有多个原子包的形式并发执行，但是，每个事务互不干扰 但是，由于事务可能操作同一个资源，不同的事务为了保证隔离性，会有很多锁方案，当然，这是数据库的实现，他们怎么实现的，我们也不必深究

持久性质，当一个事务提交之后，数据库状态永久的发生了变化，这个事务只要提交了，哪怕提交后，宕机，他就确确实实的提交了 不会因为刚刚提交，而在那一刻宕机了而让提交不生效，只要事务提交，他就想洗不掉的纹身一样，永远的固化了，除非你摧毁了硬盘 这四个特性，几乎所有的关系数据库都实现了，可能实现机制各不相同。如果让你去做一个数据库，你会考虑这些可能出现的问题吗 你该怎么设计，保证以上4条？也就是说，上面四个特性，是一种保证，但是，数据库底层如何保证的？锁是一个关键，当然。我们作为数据库的用户 不需要知道复杂的锁怎么实现的，我们只需要理解上面说的4个特性，那是结论。

- **重新理解ACID-2**
  - **1.集合中有线程安全类与线程不安全类** 如果ArrayList被一个线程运行，就不会存在线程问题。但是当多个线程同时执行同一个arrayList的对象的时候，就会出现线程安全问题，同时修改set get() 多个线程操作同一个对象，就会出现数据不一致的问题。

    为了解决这类问题 我们通常的做法的就是加syn关键字，但是 如果每个方法都添加syn 显然 性能与效率是不高。 所以，进一步的演化 我们可以将需要的方法添加syn 不需要的不用添加。

    状态的一致性 表(表级锁)->类(syn) 行级锁<->conhashmap(分段锁) 粒度更细

    2.concurrentHashMap concurrentHash将整个Map进行分段，或者分区，每个分区的状态一致性都是单独维护的， 当一个修改线程进入，对某一个分区进行修改。而另一个线程也要进行修改，就会被阻塞。其实这种思想就类似于行级锁。在锁的粒度上减少了 **3.copyOnwrite的list和get** 每次对集合修改都会创建一个副本，修改都是在副本上进行完成的，只有当修改完成时，将修改后的副本对象 直接与原始的对象进行引用替换 但是 当一个线程在修改时，有另外一个线程区修改就会出现问题。 所以copyOnWrite适合高并发读和少量的写操作场景中。写操作通过乐观机制。 **4.回到数据库层面**

    如果将一个数据库实例类比成一个大对象 那么这个大对象内部包含很多的小对象，小对象恒等于表 我们的web程序是并发执行的 也就是说在数据库层面 数据库系统保证了数据一致，无论何时 我们读取一个数据 都会得到一个正确的数据。而这时数据库做的事情。所以 数据库层面保证数据一致性 和 arrayList保证数据一致性时一样的。 **5.事务就是一系列对该数据库对象的操作步骤，包含crud等操作**。 百度百科：一致性就是指事务使得从一个一致的状态转换到另一个一致状态。 至于其中的原子性，隔离性，持久性都是实现这个目标的手段。一致性是目标，其他三个是手段。比如网上说的一致性就是A向B账户转账1000元 B收到的1000 A的账户也减少了1000元，说这是事务一致性，其实我的理解是这只是一致性最总的业务结果。如果数据库本身就是单线程的话，那么数据的一致性就是没有必要的，单线程的操作数据，并不会出现数据的不一致性问题。但是多线程环境下，我们多个事务同时并行执行，数据的一致性是无法保证的。而锁机制，快照等实现了数据的一致性。而无论是redis mg来说根本都在于保证数据的一致性。

    6.早期数据库中为了保持数据的一致性 使用读锁和写锁来保证。 存在写事务的时候，是否允许read 丢失更新。就是破坏一致性的关键。 原子性，持久性，隔离性(锁机制以及各种并发安全控制机制)是事务一致性的充分必要条件。 回到问题的根本，原子性 持久性 隔离性都是手段，最终的目的 是事务并发下的一致性。

Redis ☆☆☆

0.介绍一下

- Redis(Remote Dictionary Server 远程字典服务)。是一款高性能(key/value)**分布式内存数据库**，基于**内存运行**并支持**持久化**的NoSQL数据库，因为数据都在内存中，所以运行速度快。redis支持丰富的数据类型并且支持事务，事务中的所有命令会被序列化，按顺序执行，在执行的过程中不会被其他客户端发送来的命令打断。

1.持久化 AOF/RDB ☆☆☆

> 为什么会有持久化-数据存储防止丢失

- **1.RDB** 【快照方式】(全量持久化) 一定的时间间隔期间将内存中的数据持久化到磁盘中。**RDBSave<->RDBLoad**
- **Redis会单独创建（fork）一个子进程来进行持久化，会先将数据写入到 一个临时文件中，待持久化过程都结束了，再用这个临时文件替换上次持久化好的文件。** 整个过程中，主进程是不进行任何IO操作的，这就确保了极高的性能 如果需要进行大规模数据的恢复，且对于数据恢复的完整性不是非常敏感，那RDB方 式要比AOF方式更加的高效。**RDB的缺点是最后一次持久化后的数据可能丢失**。
  - 触发方式
    - 1.默认配置 2.save阻塞/bgsave异步同步 3.flushAll
    - **自动触发：**在配置文件中，可以配置执行了多少次save就触发自动持久化。
    - **手动触发：**通过bgsave命令，在后台异步进行生成快照的操作，同时还可以响应客户端的请求，通过redis进程fork操作创建子进程，生成的快照由子进程负责，客户端请求只会在fork阶段被阻塞。
  - **快照恢复**
    - 将备份文件(dump.rdb)移动到redis安装目录并启动服务，redis会自动加载快照文件数据到内存，但redis服务器在载入RDB文件期间，会一直处于阻塞状态，直到载入工作完成为止。
  - 优点
    - 适合大规模的数据恢复
    - 对数据完整性和一致性要求不高
  - 缺点
    - 1.可能会丢失最后一部分数据-单点宕机，如果可以容忍一部分数据的丢失，那么可以采用这种方法
    - 2.Fork的时候，内存中的数据被克隆了一份，大致2倍的膨胀性需要考虑
- **2.AOF**追加日志的形式记录写操作 只记录修改操作。(增量持久化)
- **以日志的形式来记录每个写操作**，将Redis执行过的所有写指令记录下来(读操作不记录)， 只许追加文件但不可以改写文件，redis启动之初会读取该文件重新构建数据，换言之，redis 重启的话就根据日志文件的内容将写指令从前到后执行一次以完成数据的恢复工作
  - **AOF日志重写**
    - AOF文件会随着服务器运行的时间越来越大，可以通过AOF重写来控制AOF文件的大小。
    - AOF重写会首先读取数据库中现有的键值对状态，然后根据类型使用一条命令来替代之前键值对操作的多条命令
    - 使用命令bgrewriteaof来实现AOF重写
  - **AOF重写缓存区**

    - redis是单线程工作，当AOF文件较大时重写时间会比较长，在重写AOF期间，redis将长时间无法处理客户端请求，为了解决这个问题，可以将AOF重写程序放到子进程中执行。
        - 子进程进行AOF重写期间，服务器进程(父进程)可以继续处理其他客户端请求
        - 子进程带有父进程的数据副本，使用子进程而不是线程，可以在避免使用锁的情况下，保证数据的安全性
    - **子进程中AOF重写导致的问题**
        - 子进程在重写AOF期间，服务器主进程任然可以处理用户请求，就会导致数据库的数据和AOF中记录文件数据的不一致。
    - **数据状态不一致解决办法上**
        - AOF重写缓冲区
    - 优点
        - 数据保存比较完整
    - 缺点
        - 数据集比较大
        - 恢复慢于RDB
- 推荐使用
    - 只做缓存-可以不持久化
    - AOF安全，可以及时同步到文件中，需要较多的磁盘IO，AOF文件尺寸较大，文件内容恢复相对较慢也更加完整。
    - RDB持久化，安全性较差。它是正常时期数据备份及master-slave数据同步的最佳手段 。速度快。
    - 同时开启两者 redis重启会优先载入AOF文件，**RDB更适合备份数据库 Redis 默认支持的持久化方式是 RDB 方式。**
- Redis的持久化有两种，一种是RDB 一种是AOF

  RDB

  RDB的操作时在指定的时间间隔内将数据集写入磁盘中，实际操作是fork一个子进程，RDB是Redis默认的持久化方式，会有一个dump.rdb文件，重启后会重新加载这个文件恢复数据。

  优点

    - 只要一个文件dump.rdb,方法持久化
    - 性能最大化，子进程完成写操作，主进程继续处理命令，保证了Redis的高性能
    - 数据集较大的时候，RDB的启动效率比AOF高

  缺点

    - 数据安全性比较低，RDB是每间隔一段时间进行持久化，而在持久化之间，Redis宕机，会出现数据丢失，造成数据不一致性问题。
    - RDB是通过fork子进程来完成持久化，数据较大，会出现服务器间隔停顿。

  AOF

  AOF的操作是通过记录当前操作的记录，追加到一个日志中，而这个日志详细的记录了操作步骤。AOF的出现结局了RDB数据不一致问题。每当Redis重启的时候，都会重新按顺序执行一遍日志文件。

  优点

    - 数据安全性高，AOF持久化可以设置，每操作一个命令就追加到日志中
    - 通过Append模式写文件，即使中途停机，也可以通过redis-check-aof工具解决数据一致性问题。

  缺点

    - AOF比较大，恢复满，启动效率比RDB满
    - 根据同步策略不同，AOF在运行效率上会慢与RDB

2.5种数据类型 场景 ☆☆☆

总体来说 热点key的缓存、分布式session 分布式锁

- **String** (key 对应一个value)
  - 字符串 Json Xml 数字 二进制(图/音/频) 不能超过512MB
  - `nx` **由于Redis的单线程命令处理机制，如果多个客户端同时执行，则只有一个客户端能设置成功，用作实现分布式锁。**
  - 基本命令`set get mget mset incr`
  - **为什么要有mget??**
    - **没有mget 需要n次网络+命令时间 有mget只需要1次网络+命令时间**
  - **内部编码**
    - `int` 8个字节的长整型
    - `embstr` 小于等于39个字节的字符串
    - `raw` 大于39个字节的字符串
  - **应用场景**
    - 缓存功能(数据->redis->DB)
    - **计数**(视频播放数) 一次、商品编号、订单号采用INCR命令生成
      - `incr items:1` 每次加一操作，对于微信喜欢的文章，我们也可以采用这个计数。
    - 共享Session
    - 限速(短信接口 网站IP限制)
    - 分布式锁
      - `set redislock user1 ex 10 nx` 过期时间 10S
- **List**(按照插入顺序 可以在头部或尾部添加数据)
  - 对列表的两端同时操作`push`和`pop`
  - 特点:列表中的元素是**有序的**，可以通过所以下标直接获取 **可重复**
  - **底层:** `ziplist` **压缩列表** `linkedList`
  - 基本命令 `rpush lpush linsert lrange lindex lpop rpop ltrim lset blpop`
  - 应用场景
    - **消息队列** `lpush+brpop` 阻塞队列
    - 文章列表`lpush+lpop栈`
- **ZSet** (double类型的分数，按照这个排序 ZSet成员是唯一的，但是score可以重复)
  - **实现延时队列 ?**
  - 底层:**跳表** ziplist 具体看算法部分。
  - 基本命令`zadd zcard zscore zrank zrevrank zrem zincrby zrange`
  - 应用场景
    - **排行榜系统**
- **Hash**
  - 常用命令`hget hset hdel hlen hmget hmsethexists hkeys hvalshgetall`
  - 内部编码`ziplist` `hashtable`
  - 应用场景
    - 购物车信息

        - `hset shopcar:uid 1024 10 1025 11`
    - 缓存用户信息
        - 简单直观
    - 控制哈希在`ziplist`和`hashtable`两种内部编码的转换
  - **Hash底层**
- **Set**(无序)
  - 保存多个字符串，**不允许重复元素，无序**。可以求 **交/并/差集**
  - 底层:`intset` `hashtable`
  - 基本命令`sadd srem scard sismember srandmember spop smembers sinter sunion sdiff`
  - 应用场景
    - 用户标签 `sadd`
    - 生成随机数 抽奖`spop/srandmember`
    - 社交需求`sadd+sinter`
- **Redis为什么要进行内部编码作用 ??**
  - 第一点 可以改进内部编码，对外的数据结构和命令没有影响
  - 第二点 多种内部编码可以在不同场景下发挥各自的优势，例如ziplist比较节省内存，但在列表元素较多的情况下性能有所下降，这时redis会根据配置选项将列表类型实现转换成linkedList
- **特殊数据类型有哪些？**
  - Bitmap的使用和优点
- **应用场景？**
  - 热点数据
  - 限时业务的运用 exoire 限时的优惠活动信息，手机验证码等
  - 计数器相关，incrby命令可以实现原子性的递增，高并发的秒杀活动，分布式序列号的生成，具体业务还体现一个手机号发多少条短信，一个接口一分钟限制多少请求，接口调用次数的等。
  - 排行榜相关 SortedSet 热点数据的排序。
  - 分布式锁 setnx 进行 防止死锁 可以加一个定期时间。
  - String类型 普通的set和get，可以做简单的kv缓存
    - 应用场景 微博粉丝数
  - hash类型 hset hget 存储用户信息
    - 应用场景 存储用户信息
  - list 有序列表
    - 消息队列
  - set类型
    - set无序集合，自动去重功能
  - 应用常见直接去重
    - sorted Zset类型(跳表)
  - 自动排序
    - 列出前100名畅销的商品

3.缓存相关问题 ☆☆☆

- 1.高性能(用户请求直接查询缓存 性能高)

- 2.高并发(大量请求压垮数据库宕机)
- 3.引入缓存带来的问题
  - 1.**缓存和数据库双写不一致**。
    - 1.先读缓存 缓存没有 读数据库 之后设置到缓存中。【经典的数据库与缓存模式】

      更新的时候 先删除缓存，在更新数据库。【双删延时】
    - 2.将对数据库的数据修改操作 和 缓存设置 放到一个**事务**中。
    - 3.引入MQ MQ针对消息丢失的重传机制 以及消息的无序状态 通过版本号来确保
      - 保证事务操作的顺序性
    - 4.更新数据库产生的binlog订阅(使用canal)，将变化的key记录下来，并且尝试去不断的去删除缓存(如果缓存数据失效)。
    - 如果在一个缓存中，只读取。并不会造成数据不一致性。数据库和缓存更新，就会出现数据不一致性。

      1.先删除缓存 在更新数据库

      线程1删除了缓存，还没有更新数据，线程2去访问缓存，数据为null，则去数据库中更新数据，写入缓存。数据出现不一致现象。

      2.先更新数据库，在删除缓存

      线程1先更新了数据库，在写入缓存的时候宕机了。数据库和缓存数据不一致。

      解决方案

      异步更新缓存（基于订阅binlog的同步机制）

      读在缓存中，写在数据库中，通过数据库的binlog 发送消息队列同步到缓存中，消息队列的消费者只能限制一个，异步的更新缓存。
  - 2.**缓存雪崩 缓存穿透**
    - **缓存雪崩** 当缓存中的key失效后，大量请求都会请求到数据库层面，造成一瞬间的数据库读写瓶颈。
      - 问题解决 **1.缓存失效时间分开 2.缓存不过期 3.二级缓存(双缓存)**
    - **缓存穿透** 用户请求了一个在整个系统中不存在的数据。先请求到缓存层面 没有 向后请求到数据库层 没有 当请求量比较大的时候，数据库很容易宕机。
      - 问题解决 **1.客户端对非法参数过滤 对于不存在的值 设置null+定时间过期 2.加布隆过滤器 在缓存层判断有没有数据，**

    说道雪崩 其实 我们应该区分开 穿透和雪崩之间的关系，穿透是在系统中本来没有这个数据请求打在缓存中 和 数据库上都找不到。而雪崩是由于缓存失效 导致高并发场景下 大量数据请求都落在了数据库中。常见的解决方案 1.设置过期时间 也就是 在同一时间 可以允许部分key的失效 落在数据库中，另一部分数据可以在缓存中查到。 过期时间不一样。2.关于热点key 我们可以通过一些大数据工具 对热点key分析 如果是 可以将热点可以进行多节点存储。在这里 我们应该区分 雪崩 和 穿透的含义。 以及对应两者的不同解决方案。
  - 缓存穿透

    在请求到缓存中，没有找到key，请求转发到了数据库。造成数据库的异常。
    - 缓存失效 造成直接访问数据库

    解决方案
    - 布隆过滤器 判断key是否存在，不存在的key就不同查找数据库了。直接丢弃请求。

    缓存雪崩，

    如果缓存在一段时间内失效，发生大量的缓存穿透，所有的查询都落在数据库上，造成了缓存雪崩。

解决方案

- 分布均匀 不同的key 设置不同的过期时间，让缓存失效的时间点尽量均匀。
- 限制流量 在缓存失效后，通过加锁或队列来控制读数据库写缓存的线程数量

- 3.缓存并发竞争

4.单线程模型 IO多路复用 ☆☆☆

- **Redis相比memcached有哪些优势？**
  - memcached所有的值均是简单的字符串，**redis是其替代者，支持更为丰富的数据类型**。
  - redis的速度比memcached快很多，并且redis支持数据的持久化。
  - redis支持数据的备份，既master-slave模式的数据备份。
  - 使用底层模式不同，他们之间的底层实现方式不同，以及与客户端之间通信的应用协议不一样，redis自己构建了VM管理机制。
  - vlaue大小不同，redis最大可以达到512MB，而memached仅支持1MB。
- **redis单线程模型**
  - Redis是单线程的，redis的单线程是指网络请求模块使用了一个线程，所以不需要考虑并发性。但是对于需要依赖多个操作的复合操作来说，还是需要锁的，可能是分布式锁。
  - 单线程是网络请求模块的所有请求，由一个线程来处理，不需要考虑并发性，而请求的操作会在一个队列中依次执行。

    Redis为什么这么快

    快的原因，我们可以从不同的层面说。

    - 硬件层面 主要是操作内存 (内存的读写和硬盘读写有质的差距)
    - 并发层面 因为Redis是单线程模型，所以不需要考虑线程之间的上下文切换和竞争
    - 非阻塞IO**IO多路复用**
  - **重新理解Redis单线程模型？**
    - Java服务端是多线程的，但是Redis确实单线程模型的，为什么呢？

      redis连接池把Java端的数据发送到redis服务器中。Redis服务器内部的线程模型，和Java线程没有关系，单线程只是说Redis服务器内部的单线程模型，

      这里，我们思考一下。对于服务端Tomcat来说，一定是并发处理请求。为每个用户的请求。创建一个新的线程进行处理，那么我们在调用JDBC连接池和JedisPool时，相当于从一个池中拿一个连接。 每个连接就是一个连接线程对象，到对应的数据库和Redis中。对于数据库来说，是并发处理来自客户端Tomcat的请求的，但是Redis来说，客户端tomcat发送的请求是多个线程，但是内部的处理确实一个单线程进行处理。为什么说Redis就比较快呢，首要原因在于 redis对数据的操作都是基于内存级别，而数据库操作数据的不仅仅需要和磁盘打交道，还需要和网络io传输。

    - 为什么数据库不采用单线程的呢？

      如果通过JDBC进行连接，那么必定网络和磁盘的IO，由于网络和磁盘IO速度问题，一定会阻塞线程，而CPU处于闲置状态，所以必须引入多线程进行处理。**使用多线程会有几个问题。a.线程切换需要开销，b.多线程多共享资源的操作，一定需要进行同步，锁的维护。c.设计上的简洁性。**

- **单线程的redis为什么执行速度如此之快？**
- 第一点 **存内存操作**，数据都存储在内存中。
  - 第二点 **非阻塞IO** 使用epoll作为IO多路复用技术
  - 第三点 **单线程**避免了线程切换和竞争消耗，单线程对于每个命令的执行时间有要求。
  - 第四点 数据结构简单
  - 基于内存实现，完全内存计算。
  - 单线程操作，避免了线程上下文切换操作

    - **多路IO复用的线程模型**，实现了一个线程监控多个IO流，及时响应请求。
    - redis对外部的依赖比较少，属于轻量级内存数据库。
- **多路IO复用机制可以说一下你的理解吗？**
    - Redis的线程模式多路IO复用机制，目前支持的系统调用有select,psselect,poll,epoll等函数。IO多路复用及时就是通过**一个进程可以监视多个描述符**。一旦某个描述符就绪或者写就绪，其能够通知应用程序进行相应的读写操作。
    - 多路IO复用机制与多进程和多线程技术相比比系统开销更小，系统不比创建进程线程，减少了上下文的切换操作，从而大大减少系统开销。
    - **Select函数**
    - **Poll函数**
    - **epoll函数**

5.缓存过期淘汰策略

- **内存相关**
    - redis默认内存多少？在哪里查看? 如何设置修改?
        - 查看Redis最大占用内存 `redis.conf` memory 可以配置
    - redis默认内存多少可以用？
        - 如果不设置最大内存大小或设置最大内存大小为0，64位操作系统不限制大小，32位最多3GB
    - 一般生产上你如何配置？
        - 一般推荐Redis设置内存为最大物理内存的四分之三，也就是0.75 类比hashmap负载因子
    - 如何修改redis内存设置
        - 配置文件 直接修改`redis.conf` memory参数
        - 直接命令 `config set maxmemory`
    - 什么命令查看redis内存使用情况?
        - `info memory`
    - 真要打满了会怎么样? 如果Redis内存使用超出了设置的最大值会怎样?
        - `OOM command not allowed when userd memory > maxmemory`
- **redis缓存淘汰策略**
    - 删除策略
        - 定期删除 过期时间一到就删除，会增加cpu处理时间
        - 惰性删除 一段时候后删除 对内存不好
        - 定期删除
    - **淘汰策略**
        - noeviction: 不会驱逐任何key
        - allkeys-lru: 对所有key使用LRU算法进行删除
        - volatile-lru: 对所有设置了过期时间的key使用LRU算法进行删除
        - allkeys-random: 对所有key随机删除
        - volatile-random: 对所有设置了过期时间的key随机删除
        - volatile-ttl: 删除马上要过期的key
        - allkeys-lfu: 对所有key使用LFU算法进行删除
        - volatile-lfu: 对所有设置了过期时间的key使用LFU算法进行删除
        - LRU LFU TTL RANDOM
- 如何配置
    - `config set maxmemory-policy allkeys-lru`
- LRU(最近最少使用)

- **HashTable+双向链表**

6.高可用和高并发

- 1.单机瓶颈》主从架构》读写分离》支撑10W+读QPS的架构
- 2.redis replication (复制) 以及master持久化
- 3.断线续传 无磁盘化复制 过期key处理
- 4.redis replication的完成流程 全量复制 增量复制
- 5.高可用架构

7.集群、主从复制、哨兵模式

- 算法：一致性哈希算法
- 读写分离 哨兵 持久化

主从复制，大概流程，master-slave节点建立好之后，slave节点会发送一个sync命令到maser节点上

master节点 异步将收集的用于修改数据集的命令，在后台进程中传送给slave节点。完成一次同步。但是这里主要采用两种**复制模式** 第一种是**全量复制** 和 **增量复制**

解决了什么问题，虽然说持久化机制可以保证redis重新启动，也可以重新加载数据，但是如果硬盘出现故障，数据就会丢失。所以，通过主从复制，可以很好的保证一台机器数据丢失了。通过同步数据的方式，保证数据一致性。

复制过程大概如下，

1.slave服务启动，slave会建立和master节点的连接，发送sync

2.master启动一个后台进程将数据库快照报错到RDB文件中，写给slave节点。

哨兵模式在主从架构的redis服务器上，配置了哨兵模式后，当会实时监控master/slave的状态 当master宕机之后，slave就会从新选举出一个新的master节点，当旧的master从新恢复之后 称为一个新的slave节点，并且不可写。主从架构模式 master用来接收客户端的写请求，slave接口客户端的读请求。

主从切换技术：当主服务器宕机后，需要手动将一台从服务器切换成主服务器。人工干预费事，一段时间内服务不可用。

推荐哨兵模式。

Redis提供了哨兵的命令，哨兵是一个独立的进程，会独立运行，原理是哨兵通过发送命令，等待Redis服务器响应，从而监控运行的多个Reids实例。

作用

- 发送命令让服务器返回监控其运行状态，包括主服务器和从服务器。
- 当哨兵机制检测到主服务器宕机后，会将slave切换成master节点，通过发布订阅模式通知其他服务器。

由于一个哨兵 存在单点故障，所以使用多个哨兵进行监控，各个哨兵之间还会进程监控，这样就形成了多哨兵模式。

当一个哨兵检测到主服务器不可用，这个现象被称为**主观下线**。

后面的哨兵也检测到 达到一定的数量之后，哨兵之间发起投票，结果由一个哨兵 通过发布订阅模式，让各个哨兵检测的从服务器切换成主机，这个过程是**客观下线**，这样对于客户端而言一切都是透明的。

这里有一个问题那就是，由于master-slave这种模式，采用的是复制模式,复制模式将写请求到master节点上，在异步将数据到slave节点上，所以这个时间间隔内，我们访问slave节点会出现数据不一致现象。

- Redis集群加进来从节点，主节点如何感知，并同步数据? `todo`
- 故障转移的详细过程？
- 槽位迁移的详细过程？
- **柳大，昨天面试官问我一个问题，说redis 主从节点 ，如果说主节点在用户请求发过来的时候，master节点改变了，那么如何及时让用户知道这个master节点的配置也就是ip地址信息， 我给出的答案是引入zk，进行处理。监听master节点的变化。 watch机制。 不知道 正确否。**
- 主从复制是为了解决分布式系统中的单点问题，通常会把数据复制到多个副本部署到其他机器上，满足了故障恢复和负载均衡等需求。Redis也提供了这样的功能，它可以将主节点数据的改变同步给其他从节点，这样当主节点因为故障不可达，那么从节点就可以作为后备"顶"上来，并保证数据尽量不丢失，也是最终一致性的保证。同时主从复制也扩展了主节点的读能力，对于一致性要求不是那么严格的场景下，一旦主节点不能支撑住大并发量的读操作，可以使用从节点分摊主节点的读压力。

  但是主从复制模式下一旦主节点出现故障，就需要手动将一个从节点晋升为主节点，同时需要修改应用方的从节点地址，还需要命令其他从节点去复制新的主节点，整个过程都需要人工的干预，如果处理不及时，那么系统很长时间都可能处于不可用状态。对于应用方来说，因为无法感知主节点的变化，必然会造成一定写数据的丢失和读数据的错误，甚至可能造成应用放服务的不可用。对于运维来说，整个故障转移过程都需要人工介入，故障转移的实时性和准确性上都无法得到保障。
- Redis主从复制模式下，一旦主节点出现故障不可达，需要人工干预进行故障转移，无论对Redis的应用方还是运维都带来了很大的不便。为了让一切不需要人为干预，在Redis2.8以后提供了Redis Sentinel哨兵机制来解决这个问题。

  所以Redis 的高可用架构就分为Redis数据节点和Sentinel节点，数据节点是主节点和从节点的数据进程，保证数据的最终一致性；Sentinel节点对数据节点进行监控，当它发现节点不可达时，会对节点做下线便是。如果被标识的是主节点，他还会对其他Sentinel节点进行"协商"，当大多数Sentinel节点都认为主节点不可达时，他们会选举出一个Sentinel节点来完成自动的故障转移工作，同时还会将这个变化通知给应用方，整个过程是完全自动的，不需要人工来介入，所以这套方案很有效的解决了Redis高可用问题。

8.事务 ☆☆☆

事务的整体概念就是，将一系列的操作放到一个队列中，而最后一起执行。其中主要有正常执行、撤销事务、主要实现是通过**Watch监控机制**

- **隔离性** redis是单线程的程序，保证在执行事务时，不会对事务进行中断，事务可以运行直到执行完毕所有事务中的命令为止。
- **Redis会将一个事务中的所有命令序列化**，然后按顺序执行，redis不可能在一个事务的执行过程中插入执行另一个客户端发出的请求，可以保证redis将这些命令作为一个单独的隔离操作执行。
- **相关命令**
  - `MULTI` 标记一个事务块的开始
  - `EXEC` 执行所有事务块内的命令
  - `DISCARD` 取消事务，放弃执行事务块内的所有命令
  - `UNWATCH` 取消监视key
  - `watch` 监视key
- **重写理解redis事务**
  - redis中没有并发，将事务写入到一个原子包中 通过串行化的形式传递给redis但是不支持原子性，比如集合执行到了一半，redis宕机了。原子性无法保证。redis的事务 更多的是因为单线程下没有并发干扰，实现的串行化事务。

```
/**
 * 要搞清楚，redis的事务并非是关系数据库那种事务方式，redis客户端通过Multi命令和Exec命令来发送
 * 命令，当服务器接收到一个客户端的Multi命令后，会把之后的所有该客户端的命令都保存到一个队列中，
 * 直到该客户端发送了Exec命令，那么当服务器执行此Multi--Exec命令包的时候，其他竞态条件下的任务都不会执行，而只会执行
 * Multi--Exec命令包中的顺序命令。所以，在执行Multi--Exec命令包的时候，消除了竞态，保证了数据的一致性。
 * 这样就像是实现了事务一样，其实这种方式只是一种模拟原子操作的加锁方式，但是这并不是真正的原子化操作！
 * 原子化操作的基本原则就是要么全部成功，要么全部不成功。但是，Multi--Exec命令包中的某条命令执行出错，并不会整体回滚。
 *
```

9.Redis特点

- 支持数据的持久化(AOF/RDB) 异步将数据保存在磁盘上，重启的时候可以再次加载
- 多种数据类型 List set zset hash
- Redis支持数据的备份，master-slave模式的数据备份
- 事务控制 复制
- **单线程模式**

10.发布订阅

先订阅后发布。

pipeline ?

11.分布式锁

- Redis除了拿来做缓存，你还见过基于Redis的什么用法?
  - 可以用来实现分布式锁
- Redis做分布式锁的时候有需要注意的问题?
  - 过期时间设定 删除锁的原子性 Redis本身的可用行
- 如果是Redis是单点部署的，会带来什么问题?
  - 单点故障
    - 那你准备怎么解决单点问题呢?
      - redis主从模式
- 集群模式下，比如主从模式，有没有什么问题呢?
  - 极端情况下，master节点将数据同步给slave节点时，mater节点宕机，数据也没有同步到slave节点，数据丢失了。可以采用zk进行数据的同步。
- 那你简单的介绍一下Redlock吧?你简历上写redisson，你谈谈。
- 场景设计：我们以一个常见的秒杀为设计。
  - **V1：单机版没加锁** 初始只是单个系统处理用户请求。因为没有加锁控制，会出现同一时间内，多个线程会处理同一个票的情况。出现了同一张票被卖出多次。业务上是不允许的。因此进行加锁，而在单体系统中，我们的程序都是在一个进程内执行的，也既这种锁就是JVM进程内的。一般来说，我们可以采用syn和lock，具体如何权衡需要我们自己来分析。
  - **V2：Nginx分布式微服务架构** 在V1的基础上，加锁。但是一般系统为了更好的提供服务给用户，需要多个模块集群部署。而对于用户来说，具体那一台服务器处理请求是完全透明的，所以需要在前面加一台反向代理服务器Nginx，将用户请求通过某一种策略达到服务器上。但是通过Jmter压测后，还是发现同一个票会被卖多次。我们来分析一下，为什么呢，虽然，我们加锁了。但是这个锁只是进程内的锁，并不能保证跨进程之间共享数据的操作。所以，我们需要进一步采用分布式锁来解决这个问题。`setnx`
  - **V3：释放锁** 虽然采用了分布式setnx进行处理，但是我们无法保证程序执行到最后一定会释放锁，所以将释放锁的code放在finally块中。保证不管程序是否状态与否，一定会释放锁。其他线程可以抢到锁。
  - **V4：单点宕机** 虽然V3可以保证程序一定可以释放锁，但是如果程序执行到一半，此时redis宕机，那么锁就无法释放，而别的线程也无法获取锁。因此需要给一把锁加上一个过期时

间。

- **V5：过期时间与设置key** 对于设置key和过期时间，因为是两个不同的语句，无法保证原子性，因此有可能设置了key，但是过期时间没有设置上，所以需要保证原子操作。直接使用redis提供的命令。
- **V6：只释放属于自己的锁** 虽然我们保证了一定会释放锁，在极端情况下，比如A线程尝试获取锁，获取到RedLock，执行了30S后过期时间到了，但是线程A并没有执行完毕任务。线程B去获取锁，获取到了。线程B往下执行。此时线程A走到释放锁的模块，直接释放锁，B在执行的过程中被A释放锁。就会出现业务数据的不一致。所以 删除锁的之后，应该先判断一下，属于自己的锁才删除。
- **V7：判断锁和删除锁非原子性** 可以使用Redis的事务进行保证，但是推荐使用Lua脚本。
- **V8：过期时间和业务处理时间** 需要进行续期操作，如果没有执行完毕，延长过期时间。
- **V9：Redis的高可用 数据一致性** CAP强制规范下，Redis是一个高可用AP 而Zk是CP,通常，我们搭建一个Redis的主从集群，而数据的一致性用ZK来保证。
- **V10：Redisson** 虽然，我们使用redis相关命令可以实现分布式锁，但是还是推荐使用redisson来实现，编程更加简易化。

11.面试常见题目

1.自己实现一个缓存框架的设计思路

首先来说，如果我们要自己从0到1设计一个缓存框架，那么前提我们要避免自己造轮子，那就是我们需要先参考一下业界相关标准的产品，我们做一个参考，来根据自己独有的业务特性自己设计一个框架。自顶向下的思考设计，我们首先是对于客户端 需要提供一套调用的**API接口**，或者相应的命令接口。这是第一点，第二点我们需要将热点数据存储到一个**缓存层**中,然后对应的缓存层应该有**缓存管理层**，最终就是数据的存储**缓存存储层**，而在缓存存储层中，难免会遇到热点key不断访问，以及相应的非热点key，当出现内存满时，我们需要有相应的数据淘汰策略，业界的标准一般都是LRU/LFU等。总结一下，**API接口->缓存层->缓存管理层->缓存存储层->LRU淘汰策略。**

2.Redis的架构模式

- 单机版 不是高可用 读写压力
- (主从复制) Master-Slave 解决了Master的读压力 不是高可用
- 哨兵模式 sentinel 高可用

3.配置文件了解吗？

- NETWORK：该模块可以配置一些redis服务器地址，端口以及超时时间等
- GENERAL：该模块可以对日志文件的路径和日志级别等进行配置
- SNAPSHOTTING：Redis持久化配置信息等
- REPLCATION：redis集群配置等信息
- MEMORY MANAGEMENT ： 内存管理，包括数据过期删除策略信息的设置
- APPEND ONLY MODE :日志持久化方式信息设置

4.Jedis 和 Redisson 有什么区别？

Jedis 和 Redisson 的区别如下：

- Jedis 是 Redis 的 Java 实现客户端，其 API 提供了比较全面的 Redis 命令的支持；
- Redisson 实现了分布式和可扩展的 Java 数据结构，和 Jedis 相比，功能较为简单，不支持字符串操作，不支持排序、事务、管道、分区等 Redis 特性。Redisson 的宗旨是促进使用者对 Redis 的关注分离，从而让使用者能够将精力更集中地放在处理业务逻辑上。

5.缓存预热，实现方式可以说一下嘛？

缓存预热是指系统上线后，将相关的缓存数据直接加载到缓存系统，这样就可以避免在用户请求的时候，先查询数据库，然后再将数据缓存的问题。

缓存预热的实现方式，可分为以下两种：

- 数据量不大的时候，工程启动的时候进行加载缓存动作；
- 数据量大的时候，设置一个定时任务脚本，进行缓存的刷新。

热点key的问题

- 上述我们实现了缓存预热，但是在实际的生产环境中热点key存在一定问题。比如同一时间，某个热点key，冲击redis集群，造成服务器宕机。借鉴微博热搜。所以，我们需要一个合理机制进行处理。业内解决方案 1.采用本地缓存，二级缓存，直接在应用层面返回。不走缓存。2.采用redis集群承担热点Key问题。 对于热点可以可以进行监控处理，监控热点Key，通知系统做处理。 【热点发现+本地缓存】
    - **热点key失效怎么解决呢？**
        - 缓存过期时间不设置，而是设置在key对应的value里，如果检测到对应key超过过期时间，进行异步更新。
        - 在value中设定一个比过期时间t0 小一点的值t1，当t1过期时 延长key的过期时间。
- **主从不一致怎么办？**
- **redis如何保证原子性?**
- **redis事务和mysql事务的区别？**

0x03-框架篇 ☆

Servlet

- 请求转发
    - forward：请求转发:服务器行为，地址栏不变。
- 请求重定向
    - redirect：请求重定向：客户端行为，本质上为2次请求，地址栏改变，前一次请求对象消失
    - **区别**
    - 1.从地址栏显示来说

      forward是服务器请求资源,服务器直接访问目标地址的URL,把那个URL的响应内容读取过来,然后把这些内容再发给浏览器.浏览器根本不知道服务器发送的内容从哪里来的,所以它的地址栏还是原来的地址.

      redirect是服务端根据逻辑,发送一个状态码,告诉浏览器重新去请求那个地址.所以地址栏显示的是新的URL.

      2.从数据共享来说

      forward:转发页面和转发到的页面可以共享request里面的数据.

      redirect:不能共享数据.

      3.从运用地方来说

      forward:一般用于用户登陆的时候,根据角色转发到相应的模块.

      redirect:一般用于用户注销登陆时返回主页面和跳转到其它的网站等.

      4.从效率来说

      forward:高.

      redirect:低.
- **servlet是什么？**

    - 是用于 java 编写的服务器端程序，其使用 java servlet API，当客户机发送请求到服务器时，服务器可以将请求信息发送给 servlet，并让 servlet 建立起服务器返回给客户机的响应。当启动 web 服务器 or 客户机第一次请求服务时，可以自动装入 servlet，装入后，servlet 继续运行直到其他客户机发出请求。
- **servlet生命周期？**
    - Servlet 生命周期分为 3 个阶段
    - 1）初始化阶段：调用 init()方法
        - 1）初始化阶段：在下列时刻 servlet 容器装载 servlet 1.servlet 容器启动时，自动装载某些 servlet2. 在 servlet 容器启动后，客户首次向 servlet 发送请求 3.servlet 类文件被更新以后，重新装载 servlet。Servlet 被装载后，servlet容器创建一个 servlet 对象并调用 servlet 的 init 方法。在 servlet 生命周期内，init()方法只被调用过一次。Servlet 工作原理：客户端发起一个请求，servlet 调用 service()方法时请求进行响应，service 对请求的方式进行了匹配，选择调用 dopost 或者 doget 等这些方法，然后进入对应方法中调用逻辑层上的方法，实现对客户的响应。
    - 2）响应客户请求：调用 service()
        - 2）响应客户请求：对于用户到达 servlet 的请求，servlet 容器会创建特定于该请求 servletRequest 和 servletresponse 对象，然后调用 servlet 的 service 方法，service 方法从 servletrequest 对象中获得客户请求的信息，处理该请求，并通过 servletresponse 对象向客户返回响应消息。
    - 3）终止:调用 destory().
        - 3）终止：当 web 应用终止 或者 servlet 容器终止或 servlet 容器重新装载 servlet 新实例时，servlet 容器会调用 servlet 对象的第 destory 方法，在 destory 方法中可释放 servlet 占用的资源。
- **数据库连接池？**
    - 数据库连接池的工作机制？答案：J2ee 服务器启动时会建立一定数量的链接，并一直维护不少于此数目的池连接。客户端程序需要连接时，池驱动程序就会返回一个未使用的池连接并将其标记为忙。若当前没有空闲连接，池驱动程序就建立一定数量的连接，新建连接的数量由配置参数决定。当使用的池连接调用完成后，池驱动程序将此连接标记为空闲，其他调用就可以使用这个连接。
- **JSP VS Servlet？**
    - jsp VS servlet：jsp 是 servlet 技术的扩展，本质上是 servlet 的简单方式，jsp 编译后是“类 servlet”。他们最主要额不同在于：servlet 的应用逻辑在 java 文件中，并且完全从表示层中的 html 分离出来。Jsp 是 java 和 html 可以组合为一个扩展名为.jsp 的文件。Jsp 侧重视图，servlet 侧重控制逻辑。
- **重写理解MVC-展现层**

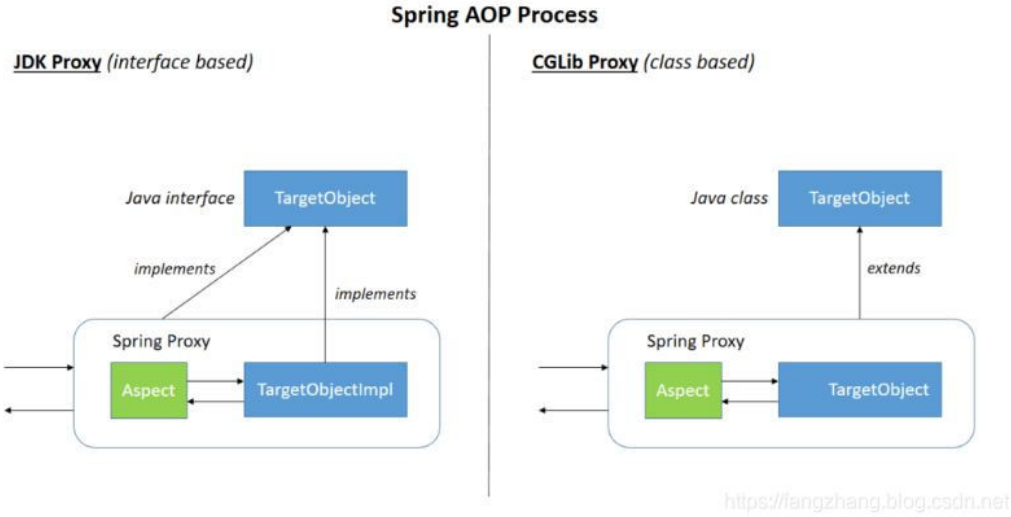
  三层架构在Java2EE领域中是举住轻重。而三层架构之外还有MVC设计模式，MVC具体就是Model VIew 和 Controller。大多数的人的理解就是MVC是展现层的东西，其实这种理解比较片面，其实展现层不仅仅包含Web层还包含用户的浏览器端，用户浏览器端发送http请求到服务器中，tomcat进行解析参数，将参数传递给servlet中，然后servlet将数据处理好，返回给客户端，客户端浏览器进行数据的渲染。所以说，**展现层=用户浏览器端+表现层（VC），Model数据业务模型数据。 展现层等价于Web层**

Spring ☆☆☆

AOP

- **原理：基于动态代理**
    - **JDK动态代理**

        - 只能对实现了接口的类生成代理，而不是针对类，该目标类型实现的接口都将被代理，原理是通过在于运行期间创建一个接口的实现类来完成对目标对象的代理
        - 大概步骤如下
            - 定义一个实现接口InvocationHandler的类
            - 通过构造函数，注入被代理类
            - 实现invoke(Object proxy,Method method,Object [] agrs);方法
            - 在主函数中获得被代理类的类加载器
            - 使用Proxy.newProxyInstance()产生一个代理对象
            - 通过代理对象调用各种方法
    - **CGLib**
        - 对是否实现接口无要求，原理是对指定的类生成一个子类，覆盖其中的方法，因为是继承，所以被代理的类或方法**不可以声明为final类型 因为需要继承 一旦final了就不能继承了**
        - 大概步骤如下
            - 定义一个实现了MethodInterceptor接口的类
            - 实现其Intercept()方法，在其中调用proxy.invokeSuper();



    - **可以说一下Spring AOP对这两种代理方式的选择嘛？**
        - 如果目标对象实现了接口，默认情况下会采用JDK的动态代理实现AOP，也可以强制使用cglib实现AOP
        - 如果目标对象没有实现接口，必须采用cglib库，Spring会自动在JDK动态代理和cglib之间转换。
- 表现切面编程-AspectJ
    - 前置通知@Before
    - 后置通知@After
    - 环绕通知@Around
    - 返回通知@AfterReturning
    - 异常通知@AfterThrowing
- **描述Spring的AOP**
    - AOP的理念
        - 将分散在各个业务逻辑代码中 相同的代码 通过 **横向切割** 的方式抽取到一个独立的模块中（模块化）
        - 将相同逻辑的重复代码横向抽取出来，使用动态代理技术将这些重复代码**织入**到目标对象方法中，实现和原来一样的功能
    - 基本概念

        - 通知（advice） --> 切面的工作被称为通知，定义了切面是什么以及何时被使用
        - 连接点（join point） --> 应用执行过程中能够插入切面的一个点，可以是方法调用时，抛出异常时等等
        - 切点（pointcut） --> 需要应用切面的方法（具体定位的连接点）
        - 切面（aspect） --> 切面是 通知 和 切点 的结合，共同定义：是什么，在何时和何处完成其功能
        - 织入(weaving) --> 把切面应用到目标函数的过程
    - 好处
        - 显示地声明在何处如何应用该行为，有效减少代码冗余，让类更加关注自身主要功能，**降低模块间的耦合度**
    - **可以说一下AOP 5的顺序吗？**
        - 正常情况下环绕通知->前置通知->**业务处理->返回通知**->后置通知->环绕通知之后
        - 异常情况下环绕通知->前置通知->**异常通知**->后置通知
        - 而Spring4和5之间是有区别的
            - 正常情况下环绕通知->前置通知->**业务处理->环绕通知**->后置通知->返回通知
            - 异常情况下 环绕通知->前置通知->后置通知->**异常通知**
        - 最根本在于**spring4默认用的是JDK的动态代理，spring5默认动态代理用的是cglib,不再是JDK的动态代理，因为JDK必须要实现接口，但有些类它并没有实现接口，所以更加通用的话就是cglib**，

- **什么是Spring注解，Spring中有哪些常用的注解，以及注解自身是如何实现的（阿里/字节跳动）** `TODO`
    - 注解 --> 减少配置文件内容
    - Java annotation Wiki
    - 秒懂，Java 注解 （Annotation）你可以这样学（简单理解： 注解 --> 标签）
    - 精进Spring—Spring常用注解（常见注解）
- **可以说一下AOP应用场景？**
    - 日志系统，安全统一检验
- **Spring AOP自调用问题？**
    - 注入代理类Bean到自己 循环依赖使用@Lazy
    - 获取当前代理对象 `AopContext.currentProxy()`
        - 缺点 1. 代理模式的限制 2.ThreadLocal的限制，不能跨线程了 3.bean设置的限制，比如@Async代理创建方式不同其他
- **如何理解AOP编程？**
- 我们知道，在做业务处理的时候，会有很多事物 日志 安全等非核心业务处理的代码，如果系统比较少，这是可以接受到，或者你独立抽取出一个类、但是还是少不了需要方法的调用，对于业务代码来说是被污染的，所以Spring AOP设计理念应运而生，为了解决重复与业务无关的代码，抽取成一个切面来进行环绕在业务方法的四周。不在是代码层面的调用。本质上就是减少业务代码的处理。举一个生活中的例子，那就是 食堂大妈和餐厅窗口来说是只负责自己的事情，食堂大妈只负责自己卫生相关的事情，而餐厅窗口人员只需要负责卖饭，其他一些后勤蔬菜 食品的运输，也都是由后勤部来保证的。如果说没有切面的类是一个普通类，那么有切面的类就是一个增强类。
    - **如何对于非核心业务与核心业务进行划分界限呢？**
    - 通常来说，针对于一些优惠活动，有打折的。我们可以将它做成一个切面，AOP到业务代码中。当活动取消，而这个优惠活动不会影00响到核心业务。那么就可以做。
    - **面向切面编程代替了OOP编程吗?**
    - 这是不对，面向切面编程是面向对象编程的拓展，丰富了面向对象编程的技术，给面向编程对象新的生命力。面向对象编程核心在于通过封装 继承 多态，创建出一个对象，通过对象之间的调用完成一系列业务处理相关逻辑，

IOC

- **描述Spring的IoC**
  - IoC是一种思想，并非一个具体技术 **组件之间的耦合性 解耦**
    - 基于 **依赖倒置原则（Dependency Inversion Principle）**
      - 把原本的高层建筑依赖底层建筑“倒置”过来，变成底层建筑依赖高层建筑。高层建筑决定需要什么，底层去实现这样的需求，但是高层并不用管底层是怎么实现的。这样就不会出现前面的“牵一发动全身”的情况
    - 将原本在程序中手动创建对象的控制权，交由Spring容器来管理
    - 反转 --> 由 **IoC容器** 来帮忙创建及注入依赖对象（Spring 提供 BeanFactory 和 ApplicationContext 两种容器）
      - BeanFactory 和 ApplicationContext 的区别（依图）
        - BeanFactory 为含有 bean 集合的工厂类
        - ApplicationContext 接口是由 BeanFactory 接口派生出来的，所以提供了 BeanFactory 的所有功能
    - 实现 --> 依赖注入（相对 IoC 而言，“依赖注入” 明确描述了 “被注入对象依赖IoC容器配置依赖对象”）
- **ioc 容器**——BeanFactory 是最原始的 ioc 容器，有以下方法 1.getBean2.判断是否有 Bean，containsBean3.判断是否单例 isSingleton。BeanFactory 只是对 ioc 容器最基本行为作了定义，而不关心 Bean 是怎样定义和加载的。如果我们想要知道一个工厂具体产生对象的过程，则要看这个接口的实现类。在 spring 中实现这个接口有很多类，其中一个xmlBeanFactory。xmlBeanFactory 的功能是建立在 DefaultListablexmlBeanFactory 这个基本容器的基础上的，并在这个基本容器的基础上实行了其他诸如 xml 读取的附加功能。xmlBeanFactory（Resource resource）构造函数，resource 是 spring 中对与外部资源的抽象，最常见的是文件的抽象，特别是 xml 文件，而且 resource 里面通常是保存了 spring 使用者的Bean 定义，eg.applicationContext.xml 在被加载时，就会被抽象为 resource 处理[我自己理解, resource 就是定义 Bean 的 xml 文件]。
- **Ioc 容器建立过程：**

  1）创建 ioc 配置文件的抽象资源，这个抽象资源包含了 BeanDefinition的定义信息。

  2）创建一个 BeanFactory，这里使用的是 DefaultListablexmlBeanFactory。

  3）创建一个载入 BeanDefinition 的读取器，这里使用 xmlBeanDefinitionReader 来载入 xml 文件形式的 BeanDefinition。4）然后将上面定义好的 resource 通过一个回调配置给 BeanFactory。

  5）从资源里读入配置信息，具体解析过程由 xmlBeanDefinitionReader 完成。

  6）ioc 容器建立起来。
- BeanDefinition 类似于 resource 接口的功能，起到的作用就是对所有的 Bean 进行一层抽象的统一，把形式各样的对象统一封装为一个便于 spring 内部进行协调管理和调度的数据结构。BeanDefinition 屏蔽了不同对象对于 spring 框架的差异。
- **依赖注入 (DI)**，就是 **把底层类作为参数传入上层类，实现上层类对下层类的“控制”**
  - 谁依赖于谁：当然是**应用程序依赖于IoC容器 组件之间依赖关系由容器在运行期决定的**
  - 为什么需要依赖：应用程序需要IoC容器来提供对象需要的外部资源
  - 谁注入谁：很明显是IoC容器注入应用程序某个对象，应用程序依赖的对象
  - 注入了什么：就是注入某个对象所需要的外部资源（包括对象、资源、常量数据）
  - **依赖注入的思想时通过反射机制实现的**。在实例化一个类时，它通过反射调用类中的 set方法将事先保存在 hashmap 中的类属性注入到类中。

    - 好处
        - 让你脱离对依赖对象的维护，只需要随用随取，不需要关心依赖对象的任何过程
        - 可以有效地改善模块之间的紧耦合问题

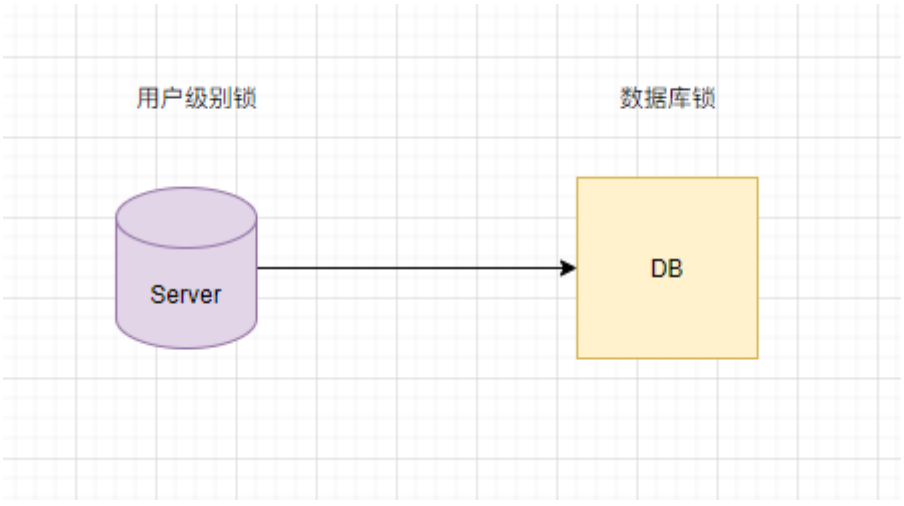
- **IOC是如何体现的?**
    - 依赖注入
        - **[构造注入，setter注入，接口注入]**
        - **如何注入 先按照类型还是按照名字？说一下**
            - 自动扫描(component-scan)
            - 注解
                - @Autowired **优先ByType 后ByName**
                - @Resource **优先ByName 后ByType**
    - 依赖查找。 目的：解耦
- **如何理解IOC？**
    - **主线：为什么需要IOC 以及DI实现？**
    - **A.没有IOC之前的样子 B.有了IOC容器之后的样子 C.整体视角**
    - 整体 无论对于Spring、Dubbo、Linux、Maven、以及Jenkens来说 **都是在一套流程中进行插件化的**

      流程化之后 加一些自定义框架的行为、拓展点 对标业务。【一整套生命周期】
    - 刚开始接触java的时候，我们通过在main方法中，直接进行方法的调用，学习了OOP思想后，知道了可以通过创建不同的对象来进行合作处理业务逻辑，而所有的对象在main线程之外，都会被终结。为了更近一步，我们可以创建一个全局的静态类，提前将系统需要的对象创建在map中，当程序需要的时候，在进行getBean()操作。但是其中有一个问题，ServerA依赖了DaoB，而这个关系的依赖，我们无法做，即使可以做会有无数的set操作。一个应用程序就是**一个对象图**，通过一个顶层入口，将对象进行创建出来，并设置对应依赖关系，但是如果我们去做这件事情，就违背初衷，此时你就会发现我们在重复造轮子了。这就是SpringIOC容器做的事情。
    - IOC 控制反转就是原先我们在main中创建对象的方式，对象的依赖关系，由程序员转换为Spring框架替我们做，对象之间关系我们只需要在一个pojo中声明这是一个Bean就可以。而剩余的工作Spring会在启动的时候通过扫描配置文件或者相关注解，进行对象的创建，以及对象之间的依赖关系。如果没有Spring 那么我们可能需要自己去做这个工作。手动维护对象的创建以及依赖关系。创建-装配-提供？ 创建和装配说明了，但是提供是如何做，如果只是单纯的使用Spring，我们可以通过IOC容器来获取对象getBean()，但是当别的框架集成到Spring中，一个web请求如何做的，其实这就是SpringMVC 映射器做的事情了。找到对应的Controller 进入这个入口，Controller通过IOC容器来配置对应的依赖的对象。在说一个框架中很多人说Spring整合MVC等，其实整合不准确，Spring更像一个主板，提供了较多接口，其他框架只需要集成到Spring中，Spring就会管理对应的Bean组件生命周期。
    - **思考：是不是所有企业级业务对象应该被容器管理呢？**
        - 并不是 比如说一个订单，只是临时性的，这些都是系统中出现的不同对象，有不同的角色 定义。

事务

- **事务类别**
    - 编程式事务 顶级接口`PlatformTransactionMananger`
    - 声明式事务 `@Transactional` 建立在AOP之上，本质通过AOP功能实现。

        - **声明式优点**
            - 就不需要在业务逻辑代码中掺杂事务管理的代码，只需在配置文件中做相关的事务规则声明或通过@Transactional注解的方式，便可以将事务规则应用到业务中去。**粒度级别为方法级别。编程式事务可以具体到代码块级别。**
- **事务相关接口介绍：**
    - `PlatformTransactionManager` 平台事务管理器，Spring事务策略的核心
    - `TransactionDefinition` 事务定义信息(**事务隔离级别**，**传播行为**，**超时**，只读，回滚规则)
        - **事务隔离级别**
            - default mysql默认repeatable_read oracle默认read_commited
            - read_uncommited 脏读、幻读、不可重复读
            - read_commited 幻读、不可重读
            - repeatable_read 幻读
            - serializable
        - **事务的传播行为** ☆ 七大
            - required (**支持当前事务，没有创建事务**)[工作中用到]
            - supports(**如果当前没有事务，就以非事务运行**)
            - mandatory(**如果没有事务，就抛出异常**)
            - requierd_new(**创建事务 无论当前是否有事务**)
            - not_support（**不支持事务，如果存在事务，则挂起**）
            - never (**非事务运行 如果当前有事务直接抛出异常**)
            - nested (**存在则嵌套事务，没有，按requided属性执行**)
        - **可以说一下什么异常会回滚吗？**
            - `by default a transaction will be rolling back on RuntimeException and Error,but not on checked exceptions (business exceptions)` **阅读源码 可以看到默认遇到运行时异常和错误会回滚，但是受检异常不会回滚。**
    - `TransactionStatus` 事务运行状态
    - **三者关系**
        - `PlatformTransactionManager`是事务上层的管理者，`TransactionDefinition`和`TransactionStatus` 事务接口的描述
- **事务失效的本质在于只有代理类具有AOP功能，但是通过目标对象调用其他兄弟方法，this是目标对象，并不是代理对象，想让事务生效必须调用代理类进行调用兄弟方法，4.x 可以 5.x 好像直接使用了代理？**
    - 首先看数据库是否使用了支持事务的存储引擎，InnoDB支持，MyIASM不支持事务。
    - AOP代理
        - **基于接口的jdk动态代理。**
            - 一般A方法使用Transaction注解，会使用事务的。但是调用了一个也有事务的方法B() 如果是this.B() 是不会有事务效果，因为我们知道AOP代理的本质是根据目标对象生成对应的代理对象，在调用A（）方法的时候，是有代理对象，但是A（）方法内部调用方法B（）的对象是this对象，并不是代理类对象。所以这就是事务没有效果的原因，但是如果我们通过`AopContext.currentProxy()`去获取代理对象，然后用代理对象调用B（）B是有事务功能的。
            - eg1: A()没有事务，调用了B()加了事务，由于基于jdk接口代理，生成的代理类对象与目标对象提供了一样的接口，但是实际接口的调用是通过在目标对象生成前后左右通知进行事务处理。A()没有事务，所以A()去调用B() 是通过this调用 并没有走外层的拦截。也就是本类对象，所以事务不会有作用。
                - 解决方案：通过

        - **基于CGlib的动态代理**
            - 因为是采用基于父类，所以目标类对象会生成对应的实现方法，内部持有super引用。所以会生效。
    - 事务传播机制等，我们可以配置相关的传播机制。具体不展开。
    - 回滚异常 看上面
- **可以说一下Transaction注解的相关使用吗？**
    - **作用范围**
        - **方法**：推荐将注解使用于方法上，不过需要注意的是：**该注解只能应用到 public 方法上，否则不生效。**
        - **类**：如果这个注解使用在类上的话，表明该注解对该类中所有的 public 方法都生效。
        - **接口**：不推荐在接口上使用。
    - **常用配置参数**
        - transactionmanager 事务管理器
        - propagation 事务传播行为 默认required
        - isolation 隔离级别 default
        - timeout 超时时间 默认-1
            - **为了使应用程序很好的运行，事务不能运行太长时间，因为事务可能涉及对后端数据库的锁定，所以长时间的事务会不必要的占用数据库资源，事务超时就是事务的一个定时器，在特定时间内事务没有完成，那么就会自动回滚，而不是一直等待其结束**
        - readOnly 是否只读 默认false **默认读写事务**
        - rollbackFor 触发事务回滚的异常类型
            - 默认情况下，事务只会遇到`RuntimeException`才回滚。
    - **@Transaction原理了解吗？可以说一下嘛？**
        - 基于AOP实现，AOP使用动态代理实现，如果目标对象有接口，默认采用JDK的动态代理，目标对象没有接口，使用Cglib动态代理实现。`DefaultAopProxyFactory`的`createAopProxy`方法。会根据是否有接口而使用JDK动态代理和Cglib动态代理。
- guide说事务
- **Spring事务和数据库事务有什么区别和联系吗？**
    - 事务是数据库的概念，和Java没有关系。我们通过JDBC操作事务<->DB。 框架中的事务只不过是一个客户端而已，通过设置最终会发送到数据库端。而事务操作只是通过javaAPI设置了一下。Spring的内部的事务处理 还是依托于数据库事务的。是对于jdbc的封装，是一种客户端代理对象，将数据库层面的事务抽象到java进程中。事务是企业级应用开发中最重要的东西，保持数据的正确性。
    - 

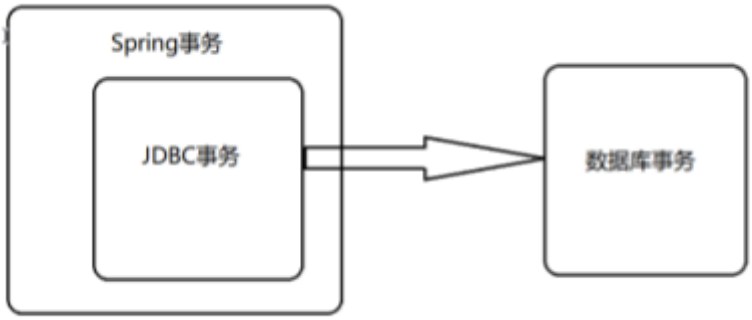
spring事务管理器与数据库事务的关系

事务，这是数据库的概念，和java没有关系。jdbc是数据库客户端，通常，我们是通过客户端来操作真实的数据库。Spring的事务管理器，并不是说spring内部进行是事务处理，事务还是依托数据库事务，我们在jdbc中进行executeUpdate，并不是在java进程中操作的。

在java端的东西 都是客户端。在jdbc中获取一个数据库连接，然后提交sql执行。默认情况下，每个sql都是独立的，也就是自动提交，相当于每条sql 一个事务，又称为单操作事务，如果我们希望在一个连接上开启多操作事务，提供一段原生代码参考，就需要把autocommit设置为false,这样，我们可以在一个事务中包含多条sql语句。实际上，事务任何时候 都是开启的，设置为autocommit为true就是没开事务，这是无稽之谈。

Jdbc中设置这个自动提交，他只是java端的客户端代理，这个设置最终会通过网络，传输到真实数据库，真实数据库通过这些配置，决定 连接上的操作是否开启自动提交。

spring的事务管理器，是对jdbc的封装，所谓的事务，都是一种客户端代理对象，将数据库层面的事务，抽象到java进程。



读、写事务

如果一个事务中所有的sql语句，都不会对数据库进行修改，那么，这是一个只读事务。

为什么要开启事务，假如说 我们开启了一个事务，进行了100次读操作，其实，这个事务是没有必须要的，每条sql可以自动提交。数据库对自动提交有优化，因为只读事务中存在共享锁，你一个事务开很久，挂的锁就很多，如果确定一个多条的sql都是只读的，我们可以对每个sql设置成自动提交。这样对数据库来说 性能最好。在jdbc中开启 自动提交为false 就证明了在这个连接上，一定有写操作。

在看spring的事务管理器，我们可以配置它，告诉他这个方法是只读的。这样，spring事务管理器就明白了，不需要将你所用的底层jdbc设置autocommit为false。在事务的可重复隔离级别，每次重复读取，都读到一样的值，实际上，这是第一次读取的值的快照，如果，你设置了自动提交为false.那么，这个读就会建立快照，自动提交的事务，每条sql操作结束就自动提交了，不存在多条sql，多轻量级，开启一个多行事务，是沉重的，如果，我们不配置只读。

Select for update 是不是只读的

读取来，接着后边的sql语句，在该事务中。要进行写操作。

事务，是企业级应用开发中最最重要的东西，保证数据的正确性，一定要理解透彻。

Spring事务(基础JDK,CGLib动态代理，基于jdbc这个桥梁，基于数据库)

java中的事务

实际上，我们通常是依赖了数据库提供的事务，但是，有一个东西，叫JTA。

Java中对事务操作的API,对于这个规范，有一些厂商的实现，我们普通用的，都是用了数据库自带的事务，

Bean

- **Bean作用域**

    - **singelon** 默认单例，[安全问题]->threadlocal set get[每个线程独享]
    - **prototype** 多例
    - **request** 每次http请求都会创建一个Bean，该作用域仅在基于web的下有效。
    - **session** 在一个http Session中 一个Bean定义对应一个实例。
    - **global session** 在一个全局的http session中 一个bean对应一个实例
- **bean实例化**
    - 无参构造
    - 静态工厂
    - 实例工厂
- **Bean的注入方式**
    - `autowired`
    - `resource`
    - `injection`
    - `value与配置文件一起使用`
- **Bean循环依赖的问题**
    - 问题描述：A依赖了B B也依赖了A Bean进行初始化的时候 不能选择应该先加载哪一个。
- 通常来说，如果问spring容器内部如何解决循环依赖， 一定是指默认的单例Bean中，属性互相引用的场景。也就是说，Spring的循环依赖，**是Spring容器注入时候出现的问题**
    - **构造器方式无法解决，只能抛出异常 A B创建的时候 都需要对方先创建**
        - **多例方式无法解决，只能抛出异常**
            - 因为Spring容器不缓存"prototype"作用域的bean，因此无法提前暴露一个创建中的bean。
        - **单例模式可以解决+setter**
            - 通过提前暴露一个单例工厂方法，从而使其他bean能够引用到该bean/提前暴露一个正在创建中的bean，**使用单例+setter没有问题**
    - **为什么单例可以解决，而多例是无法解决的呢？**
    - **默认的单例(singleton)的场景是支持循环依赖的，不报错.原型(Prototype)的场景是不支持循环依赖的，报错** 根本原因在于单例可以通过提前暴露的三级缓存来解决循环依赖问题，单例只有一份，可以复用。
- **了解三级缓存吗？**
    - 第一层`singletonObjects`存放的是已经初始化好了的Bean,经历完整生命周期的Bean对象
    - 第二层`earlySingletonObjects`存放早期暴露出来的Bean对象，Bean的生命周期未结束（属性还未填充完整）
    - 第三层`singletonFactories`存放的是FactoryBean。假如A类实现了FactoryBean,那么依赖注入的时候不是A类，而是A类产生的Bean

      根本：所谓的三级缓存其实就是spring容器内部用来解决循环依赖问题的三个map
    - **可以说一下生命周期重要的方法吗？**
        - **getSingleton**：希望从容器里面获得单例的bean，没有的话
    - **doCreateBean**: 没有就创建bean
        - **populateBean**: 创建完了以后，要填充属性
    - **addSingleton**: 填充完了以后，再添加到容器进行使用
- **实例化和初始化的区别？**
    - 实例化是对象创建完毕，分配了堆内存空间，但是还没有进行属性的相关配置，而初始化需要进一步属性的相关配置。这也是循环依赖的问题所在，在对象实例化之后进行初始化的过程中。
- **整体流程**

1 A创建过程中需要B，于是A将自己放到三级缓存里面，去实例化B

2 B实例化的时候发现需要A，于是B先查一级缓存，没有，再查二级缓存，还是没有，再查三级缓存，找到了A 然后把三级缓存里面的这个A放到二级缓存里面，并删除三级缓存里面的A

3 B顺利初始化完毕，将自己放到一级缓存里面（此时B里面的A依然是创建中状态） 然后回来接着创建A，此时B已经创建结束，直接从一级缓存里面拿到B，然后完成创建，并将A自己放到一级缓存里面。

总体来说，先去单例池找A，没有直接返回，创建A，实例化完毕A之后，填充A的相关属性时，需要B对象，接着去三级缓存中找，没有找到，创建B，填充B的相关属性，但是需要依赖A，因为之前由一个实例化过但是没有初始化的对象A，拿到之后B初始化完毕，A获取到B的对象，A也初始化完毕。

Spring解决循环依赖依靠的是Bean的"中间态"这个概念，而这个中间态指的是已经实例化但还没初始化的状态……>半成品。 实例化的过程又是通过构造器创建的，如果A还没创建好出来怎么可能提前曝光，所以构造器的循环依赖无法解决。

- **解决方案**：
  - 重新设计
  - 延迟加载 依赖的时候通过代理生成一个对象 只有在第一次真正使用的时候，才会进行初始化。
  - **使用setter注入**

- **Bean生命周期**
  - 一次加工 BeanPostProcessor
  - 依赖注入 BeanWare接口
  - **什么是Bean以及描述Bean的生命周期（美团）**
    - 在 Spring 中，构成应用程序主干并由Spring IoC容器管理的对象称为Bean。一个Bean是一个由Spring IoC容器实例化、组装和管理的对象
    - 生命周期
      - 创建Bean
        - 实例化 Bean 对象
        - 设置属性
        - 检查 Aware 相关接口并注入依赖（具体包括 BeanNameAware、BeanFactoryAware 和 ApplicationContextAware，分别注入Bean ID，Bean Factory 或 ApplicationContext）
        - 调用 BeanPostProcessor 的前置初始化方法 postProcessBeforeInitialization
        - 如果实现了 InitializingBean 接口，则会调用 afterPropertiesSet 方法
        - 调用 Bean 自身定义的 init 方法
        - 调用 BeanPostProcessor 的后置初始化方法 postProcessAfterInitialization
        - 创建过程完毕
      - 销毁
        - Spring Bean 的销毁过程会依次调用 DisposableBean 的 destroy 方法和 Bean 自身定制的 destroy 方法
    - 第37讲 | 谈谈Spring Bean的生命周期和作用域？
    - Spring 中 Bean 的生命周期如下：
      - ① **实例化 Bean**：对于 BeanFactory 容器，当客户向容器请求一个尚未初始化的 Bean 时，或初始化 Bean 的时候需要注入另一个尚未初始化的依赖时，容器就会调用 createBean 进行实例化。对于 ApplicationContext 容器，当容器启动结束后，通过获取 BeanDefinition 对象中的信息，实例化所有的 Bean；

        - ② **设置对象属性**（依赖注入）：实例化后的对象被封装在 BeanWrapper 对象中，紧接着 Spring 根据 BeanDefinition 中的信息以及通过 BeanWrapper 提供的设置属性的接口完成依赖注入；
        - ③ **处理 Aware 接口**：Spring 会检测该对象是否实现了 xxxAware 接口，并将相关的 xxxAware 实例注入给 Bean：
        - 如果这个 Bean 已经实现了 BeanNameAware 接口，会调用它实现的 setBeanName(String BeanId) 方法，此处传递的就是 Spring 配置文件中 Bean 的 id 值；
        - 如果这个 Bean 已经实现了 BeanFactoryAware 接口，会调用它实现的 setBeanFactory() 方法，传递的是 Spring 工厂自身；
        - 如果这个 Bean 已经实现了 ApplicationContextAware 接口，会调用 setApplicationContext(ApplicationContext) 方法，传入 Spring 上下文；
        - ④ **BeanPostProcessor**：如果想对 Bean 进行一些自定义的处理，那么可以让 Bean 实现了 BeanPostProcessor 接口，那将会调用 postProcessBeforeInitialization(Object obj, String s) 方法；
        - ⑤ **InitializingBean 与 init-method**：如果 Bean 在 Spring 配置文件中配置了 init-method 属性，则会自动调用其配置的初始化方法；
        - ⑥ **如果这个 Bean 实现了 BeanPostProcessor 接口，**将会调用 postProcessAfterInitialization(Object obj, String s) 方法；由于这个方法是在 Bean 初始化结束时调用的，因而可以被应用于内存或缓存技术；

      以上几个步骤完成后，Bean 就已经被正确创建了，之后就可以使用这个 Bean 了。

        - ⑦ **DisposableBean**：当 Bean 不再需要时，会经过清理阶段，如果 Bean 实现了 DisposableBean 这个接口，会调用其实现的 destroy() 方法；
        - ⑧ **destroy-method**：最后，如果这个 Bean 的 Spring 配置中配置了 destroy-method 属性，会自动调用其配置的销毁方法。
- [Spring中的Bean是线程安全的吗？]
    - 不是线程安全的。
        - 对于原型Bean，每次都会创建一个对象，所以不会存在共享数据的问题，线程安全。
        - 单例Bean，所有线程共享一个单实例Bean，存在资源的竞争。
        - 解决方案
            - 使用threadLocal
    - controller service dao安全吗？
        - 本身是不安全的，但是由于都是无状态Bean(不保存数据)。所以可以理解为线程安全。

设计模式

- Spring 中使用的设计模式如下：
    - 工厂模式：通过 BeanFactory、ApplicationContext 来创建 bean 都是属于工厂模式；
    - 单例、原型模式：创建 bean 对象设置作用域时，就可以声明 Singleton（单例模式）、Prototype（原型模式）；
    - 观察者模式：Spring 可以定义一下监听，如 ApplicationListener 当某个动作触发时就会发出通知；
    - 责任链模式：AOP 拦截器的执行；
    - 策略模式：在创建代理类时，如果代理的是接口使用的是 JDK 自身的动态代理，如果不是接口使用的是 CGLIB 实现动态代理。
    - 代理模式：AOP代理
    - **模板模式：**`PlatformTransactionManager`定义了一个接口，通过抽象类`AbstractPlatformTransactionManager`将一些公共的基础实现定义到抽象类中，`HibernateTransactionManager`定义不同的实现 对于`Hibernate` `jdbc` `jpa`

问题迭代

- **可以说一下BeanFactory和FactoryBean的区别吗？☆**
    - **BeanFactory**：Bean工厂，是一个工厂(Factory) 是Spring IOC容器的最顶层接口，他的作用是管理Bean，既实例化、定位、配置应用程序中的对象及建立这些对象间的依赖。
    - **FactoryBean**：工厂Bean，是一个Bean，作用是产生其他的Bean实例，需要提供一个工厂方法，该方法用来返回其他的Bean实例。
- **BeanFactory和ApplicationContext有什么区别？**
    - **BeanFactory**是Spring里面最顶层的接口，包含了各种Bean的定义，读取Bean配置文档，管理Bean的加载、实例化、控制Bean的生命周期、维护Bean之间的依赖关系。
    - **ApplicatinContext**接口是BeanFactory的派生，除了提供BeanFactory所具有的功能外，还提供了更完整的框架功能
        - 继承了MessageSource，支持国际化。
        - 提供了统一的资源文件访问方式。
        - 提供在Listener中注册Bean的事件。
        - 提供同时加载多个配置文件的功能。
        - 载入多个上下文，使得每个上下文都专注于一个特定的层次，比如应用的web层。
        - **ApplicationContext三种常见的实现方式可以说一下嘛？**
            - **FileSystemXmlApplicationContext** :容器从一个XML文件中加载Bean的定义，XMlBean配置文件的全路径命名必须提供给他的构造方法。
            - **ClassPathXmlApplicationContext**:容器从一个XMl文件中加载Bean的定义，需要正确设置classpath 因为这个容器将在classpath里找Bean的配置
            - **WebXmlApplicationContext** 此容器加载一个xml文件，定义了一个Web应用的所有Bean。
        - **创建Bean和内存占用方面的区别：**
            - **BeanFactory采用的是延迟加载形式来注入Bean的**，既只有在使用到某个Bean时，才对Bean进行加载实例化，这样，就不能发现一些存在于Spring配置中的问题。如果Bean的某一个属性没有注入。BeanFactory加载后，直至第一次使用调用getBean方法才会抛出异常。
            - **ApplicationContext 它是在容器启动时，一次性创建了所有的Bean**，可以提前检测到错误。
- **可以说说常用注解吗？**
    - @Component和@Bean区别？
        - Bean作用于类上 Component作用于类上
- **事务失效可能什么原因？**
    - 从数据库存储引擎、基于JDK还是CGLib动态代理、传播事务级别、回滚异常。
- **为什么要进行分层？**
    - 在学习到SSM阶段的时候，我们会学习到一个概念，那就是分层。数据层只负责和数据进行交互，而业务逻辑层只需要进行相关业务的处理。如果对于一个小型系统来说，可以通过servlet进行搭建一套系统，不进行分层，但是对于一定规模系统来说，分层从一定程度上可以减少模块之间的耦合性，否则你的代码中都是一堆业务和数据 纷繁复杂，后期的维护也是很困难的。好比说，当你进入一家银行营业厅，前台有专门接待你办理相关业务的人员，也有相关基金服务人员。以及保安大叔。为什么要这么做，一个人直接做所有事情不可以嘛，在相对较少的规模的营业厅确实可以，但是稍有规模的，一定是人员之间的职责划分相当明确，你是做什么的，做好份内的事情就可以。所以这就是分层，将每一部分进行划分，你只管相关业务的处理，或者是数据交互。
- **Spring和SpringBoot有什么区别？**

    - Spring技术栈致力于将三方框架进一步整合，但是在整合的过程中，出现了很纷繁复杂的配置，虽然从最基础的XML到注解配置，简化了配置，但是从一定程度上还是需要配置。因此SpringBoot应运而生，Boot从某种角度只是为了简化Spring框架开发、测试、部署的简易型。是一种拓展。
- **Spring解决的问题域**
    - 1.升级了对象的概念 SpringBean统一使用了BeanDefinition来封装。BeanDefinition是对Class更上层的封装，不仅仅包含元信息，还包含一系列是否懒加载，抽象类，单例类等。Spring就是一个平台。目的是为了更好的发展，Spring给我们提供的资源是Bean(IOC/DI) 更好的资源就更容易，简单的拓展方式(AOP)
    - **站在程序员角度，以前类需要自己new，现在不用了，Spring帮我们完成了DI。**
    - **站在Spring角度，以前类都是程序员操作，现在是IOC了。**
- **重新理解Shiro**

  shiro是一个专门处理这类问题的框架，他通过吧认证和权限处理等分离出商业逻辑，放在一个独立的地方进行处理，以前我都是自己手工处理一些认证和权限的事情，我觉得很累，现在，我外包了一个保安队，让保安队负责所有的这些工作，我不再管理，所以我雇佣了一个安全经理，SecurityManager，请到了我的项目中，所有的权限，认证相关的，从现在起，全部交给这个经理负责了，他刚刚来，肯定不熟具体的业务，所以你需要告诉她哪些地方要保护，怎么保护，之后，你就高枕无忧啦，你可以想象一下，那些机构大院的保安亭，他们就是shiro，使用这类框架，可以让我们更专注商业逻辑的开发，否则，我们的商业代码中会散步大量的和商业逻辑无关的权限验证等处理，这样，开发会变得非常杂乱无章，使用专业化的框架，框架内部已经完成了90%的逻辑，这个securityManager带来了一帮小弟和打手，他们个个摩拳擦掌，他们怎么打人，怎么驱赶人，怎么拦人，都有自己的方式，现在，你只需要告诉我，你让我怎么做，这就是10%的需要配置的选项，你只需要提供权限名单，权限详细的信息给我，其他的，怎么打人，怎么驱赶人，怎么拦车，我自己有自己的方式，我只需要基础权限数据和你们的少量配置信息。

  如果你不用shiro，你就会自己写很多代码，拦车的，打人的，驱赶人的，这些代码你都要写，框架，90%的东西都是各个项目中通用的，shiro还是很简单上手的，spring那个安全框架 就有点复杂。每个项目提供自己不同的配置，也许，这个保安队被你解雇了，他们去了另外一家公司，配置了另外的选项，但是他们打人，拦车，驱赶人的部分，还是那一套，这就是框架！

Spring MVC

- **可以说一下mvc执行流程**
    - 前端控制器 `dispatchServlet`
    - 处理器映射器 `HandlerMapping`
    - 处理器适配器 `HandlerAdapter`
    - 视图解析器 `viewResolver`
    - Spring MVC 的执行流程如下：
        1. 客户端发送请求至前端控制器（DispatcherServlet）
        2. 前端控制器根据请求路径，进入对应的处理器
        3. 处理器调用相应的业务方法
        4. 处理器获取到相应的业务数据
        5. 处理器把组装好的数据交还给前端控制器
        6. 前端控制器将获取的 ModelAndView 对象传给视图解析器（ViewResolver）
        7. 前端控制器获取到解析好的页面数据
        8. 前端控制器将解析好的页面返回给客户端
    - 源码分析
        - 1.先进入`FrameworkServlet`的`service`(); 根据多态 调用 `dispatcherServer.doService()`

    - 2.`dispatcherServlet.doService()` 调用`doDispatch`
    - 3.获取当前请求的执行链`getHandler(processedRequest);`
    - 4.获取当前请求的适配器`getHandlerAdapter(mappedHandler.getHandler());`
    - 5.遍历所有定义的 interceptor，执行 preHandle 方法
    - 6.调用handle()
    - 7.处理成默认视图名，添加前缀和后缀
    - 8.拦截器postHandle方法进行处理
    - 9.渲染最后结果
- **原理**
    - sprigMVC 运行原理 1）客户端请求提交到 DispatcherServlet。 2）由 DispatcherServlet控制器查查询一个 or 多个 handlermapping，找到处理请求的 controller。3）DispatcherServlet将请求提交给 controller。4）controller 调用逻辑处理完后，返回 methodAndView。5）DispatcherServlet 查询一个 or 多个 ViewResolver 视图解析器，找到 modleAndView 指定的视图。6）视图负责将结果显示到客户端。
- **核心组件**
    - Spring MVC 的核心组件如下列表所示：
        1. **DispatcherServlet**：核心处理器（也叫前端控制器），负责调度其他组件的执行，可降低不同组件之间的耦合性，是整个 Spring MVC 的核心模块。
        2. **Handler**：处理器，完成具体业务逻辑，相当于 Servlet 或 Action。
        3. **HandlerMapping**：DispatcherServlet 是通过 HandlerMapping 将请求映射到不同的 Handler。
        4. **HandlerInterceptor**：处理器拦截器，是一个接口，如果我们需要做一些拦截处理，可以来实现这个接口。
        5. **HandlerExecutionChain**：处理器执行链，包括两部分内容，即 Handler 和 HandlerInterceptor（系统会有一个默认的 HandlerInterceptor，如果需要额外拦截处理，可以添加拦截器设置）。
        6. **HandlerAdapter**：处理器适配器，Handler 执行业务方法之前，需要进行一系列的操作包括表单数据的验证、数据类型的转换、将表单数据封装到 POJO 等，这一系列的操作，都是由 HandlerAdapter 来完成，DispatcherServlet 通过 HandlerAdapter 执行不同的 Handler。
        7. **ModelAndView**：装载了模型数据和视图信息，作为 Handler 的处理结果，返回给 DispatcherServlet。
        8. **ViewResolver**：视图解析器，DispatcherServlet 通过它将逻辑视图解析成物理视图，最终将渲染结果响应给客户端。
- **父子容器**
    - 开发中我们会遇到web层的bean 不需要Spring容器进行组件的扫描，但是被Spring MVC进行扫描了。主要是因为Spring是一个大容器，而SpringMVC是一个小容器，MVC只管理Web的Bean，如果MVC加载了service的Bean，MVC依赖Service的Bean的时候会优先加载MVC小容器中的service。【亲疏优先级】而这个小容器中加载的service是没有事务功能。所以会造成事务失效。但是MVC小容器加载Spring大容器的ServiceBean，会有事务功能。就近原因，MVC有servceiBean 就用但是没有事务功能。
    - Spring父子容器

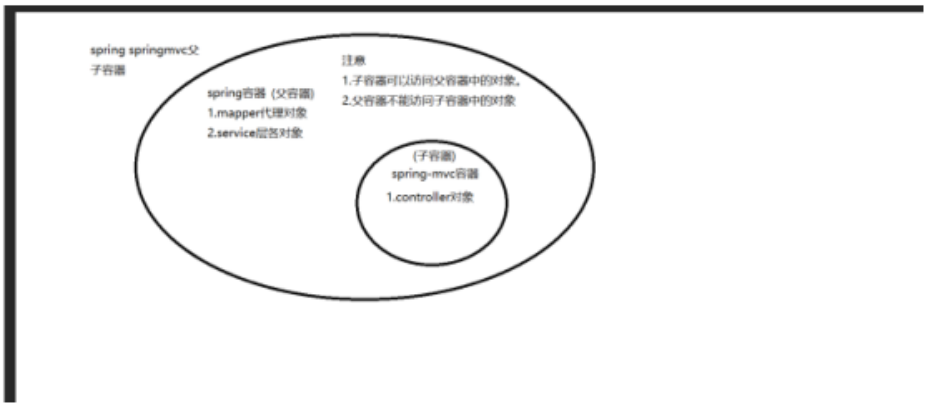
      父子容器，他们都是WebApplicationContext.本质上来说，他们没有什么区别。事务上没有什么差别。我们完全可以使用spring mvc容器，而不是用spring容器来做应用，一点问题都没有。这样，整个应用就和springmvc耦合了。父子容器分离，是一种很好的去耦合的设计，我们将web层的组件放在springmvc容器中，将其他的application组件放在spring根容器，因为web层的mvc框架，也有可能是其他的，比如structs。但是此处会有一个service层事务失效的问题。如果spring和springmvc两个容器都对service层的组件都进行了扫描，那么会这两个容器中都部署service对象，由于 我们配置的事务一般都在spring容器中，在spring容器中生成的service对象，比如有事务注解的都是最终事务aop代理后的对象，但是

springmvc容器扫描到的service对象，由于springmvc容器并没有配置事务aop,因此，这个springmvc容器中的service对象是没有被事务代理的。service对象一般是注入到controller对象中，而controller对象是springmvc管理的，虽然父子容器中都具有符合要求的注入的service对象，但是注入的时候会有亲疏优先级，优先注入本容器中的，如果本容器不存在，才注入父容器的，这样就最终导致你们的autowirde注入的是springmvc中扫描的的service对象，是没有被事务aop代理，这是事务失效的根本原因。

两个容器的作用和范围不同，一个项目中，可以配置多个springmvc DispacherServlet,但是他们可以共用同一个root父亲，这样我们可以

很方便的对应用分模块，但是工共用相同的父模块组件，springmvc是web层框架，他更多的是处理web层的组件。还有一个问题，就是如果我们的应用程序，面对的web层，前端层是多重的。比如，移动端，PC，REST接口。暴露服务什么的。我们可以定义多个DispacherServlet。





- **重定向与转发的区别？**
  - forward 和 redirect 区别如下：
    - forward 表示请求转发，请求转发是服务器的行为；redirect 表示重定向，重定向是客户端行为；
    - forward 是服务器请求资源，服务器直接访问把请求的资源转发给浏览器，浏览器根本不知道服务器的内容是从哪来的，因此它的地址栏还是原来的地址；redirect 是服务端发送一个状态码告诉浏览器重新请求新的地址，因此地址栏显示的是新的 URL；
    - forward 转发页面和转发到的页面可以共享 request 里面的数据；redirect 不能共享数据；
    - 从效率来说，forward 比 redirect 效率更高。
- **注解**
  - @Controller：用于标记某个类为控制器；
  - @ResponseBody ： 标识返回的数据不是 html 标签的页面，而是某种格式的数据，如 JSON、XML 等；
  - @RestController：相当于 @Controller 加 @ResponseBody 的组合效果；
  - @Component：标识为 Spring 的组件；

- @Configuration：用于定义配置类；
- @RequestMapping：用于映射请求地址的注解；
- @Autowired：自动装配对象；
- @RequestHeader：可以把 Request 请求的 header 值绑定到方法的参数上。

- **重新理解三层架构？**
  - 在早期学习框架，MVC作为一种前端与后端数据交互的模式。逐渐被大家所认知，但是随着微服务的盛行。View也逐渐由前后端分离而消失。之前由Model View Controller MVC组成，View移步前端。后端层架架构大家也熟悉，但是我想说的是其实Controller只是一个前端的一个代理层，并不属于实际的后端三层架构，通常一个用户发送一个表单提交，通过网络进行传输，SpringMVC接收到参数，需要进行解析，本质上和Servlet来说以一致的，只不过MVC框架封装的更加严密，功能更加丰富。参数的解析，本质上来说，属于前端数据的接收与判断，所以Controller属于前端的一个代理层，并不是属于后端三层架构，通过这个前端代理层，我们调用对应的服务层处理，而对于一些分布式系统来说，也都是通过在前端代理层和服务层之间进行业务的RPC调用。高度接触耦合，而服务层之前我们都是通过BeanServiceXXX进行定义，比如有一个订单就直接OrderService这样，其实这种方式是不好的，我们需要将高内聚的相关模块封装成一个服务，比如用户登录服务，用户信息服务，用户登录服务。而不是按照传统意义上的按照Bean进行划分。
  - **中台** 当服务层提供的服务较多，需要将系统进行一些公共基础服务进行向下的沉淀，就需要把一些相关基础模块进行更下维度的沉淀，比如对于整个系统来说，安全，日志。用户等相关模块。以提升整个系统的复用能力
  - **网关** 刚开始我们在一个系统中，但是随着业务发展需要将系统进行微服务化，1个系统划分成10个子系统，但是对于前段，你就需要记录这些相关的信息，但是如果1000个 1万个，那么我们就需要一个代理。而这个网关就是一个客户端的一种代理。前后发送的请求直接请求到网关上，网关进行下不步的服务调用。

Mybatis

- **重要组件**
  - **Mapper 配置**：用于组织具体的查询业务和映射数据库的字段关系，可以使用 XML 格式或 Java 注解格式来实现；
  - **Mapper 接口**：数据操作接口也就是通常说的 DAO 接口，要和 Mapper 配置文件中的方法一一对应；
  - Executor：MyBatis 中所有的 Mapper 语句的执行都是通过 Executor 执行的；
  - SqlSession：类似于 JDBC 中的 Connection，可以用 SqlSession 实例来直接执行被映射的 SQL 语句；
  - SqlSessionFactory：SqlSessionFactory 是创建 SqlSession 的工厂，可以通过 SqlSession openSession() 方法创建 SqlSession 对象。
- **可以大概说一下执行流程？**
  - 首先加载 Mapper 配置的 SQL 映射文件，或者是注解的相关 SQL 内容。
  - 创建会话工厂，MyBatis 通过读取配置文件的信息来构造出会话工厂（SqlSessionFactory）。
  - 创建会话，根据会话工厂，MyBatis 就可以通过它来创建会话对象（SqlSession），会话对象是一个接口，该接口中包含了对数据库操作的增、删、改、查方法。
  - 创建执行器，因为会话对象本身不能直接操作数据库，所以它使用了一个叫做数据库执行器（Executor）的接口来帮它执行操作。
  - 封装 SQL 对象，在这一步，执行器将待处理的 SQL 信息封装到一个对象中（MappedStatement），该对象包括 SQL 语句、输入参数映射信息（Java 简单类型、HashMap 或 POJO）和输出结果映射信息（Java 简单类型、HashMap 或 POJO）。
  - 操作数据库，拥有了执行器和 SQL 信息封装对象就使用它们访问数据库了，最后再返回操作结果，结束流程。
- **# $区别**

    - **“#”是预编译处理，“$”是字符替换**。 在使用“#”时，MyBatis 会将 SQL 中的参数替换成“?”，配合 PreparedStatement 的 set 方法赋值，这样可以有效的防止 SQL 注入，保证程序的运行安全。
- **分页方式？**
    - 逻辑分页，使用 MyBatis 自带的 RowBounds 进行分页，它是一次性查询很多数据，然后在数据中再进行检索；
    - 物理分页，自己手写 SQL 分页或使用分页插件 PageHelper，去数据库查询指定条数的分页数据形式。
- 1.实现原理 【动态代理 根据接口生成Sql对象】
- **一级缓存、二级缓存**
    - **一级缓存是 SqlSession 级别的**，是 MyBatis 自带的缓存功能，并且无法关闭，因此当有两个 SqlSession 访问相同的 SQL 时，一级缓存也不会生效，需要查询两次数据库；
    - 二级缓存是 Mapper 级别的，只要是同一个 Mapper，无论使用多少个 SqlSession 来操作，数据都是共享的，多个不同的 SqlSession 可以共用二级缓存，MyBatis 二级缓存默认是关闭的，需要使用时可手动开启，二级缓存也可以使用第三方的缓存，比如，使用 Ehcache 作为二级缓存。
- 4.执行器`todo`
- 5.常见标签
- 6.分页插件 pagehelper `todo` 自己实现一个思路
- 7.动态标签怎么写`todo`
- **有哪些拦截器，什么作用？**
    - Executor：拦截内部执行器，它负责调用 StatementHandler 操作数据库，并把结果集通过 ResultSetHandler 进行自动映射，另外它还处理了二级缓存的操作。
    - StatementHandler：拦截 SQL 语法构建的处理，它是 MyBatis 直接和数据库执行 SQL 脚本的对象，另外它也实现了 MyBatis 的一级缓存。
    - ParameterHandler：拦截参数的处理。
    - ResultSetHandler：拦截结果集的处理。
- **Dao和Mapper的区别？**
    - Dao是属于JavaEE三层架构的范畴，而Mapper只是Mybatis中的一个概念 操作数据库的。并不是一个层次。

Hibernate

Spring Boot

- **简述Spring Boot框架**
    - 核心
        - 核心思想是约定优于配置
        - 自动配置
            - **@EnableAutoConfiguration @Configuration @ConditionalOnClass**
        - 配置顺序
        - 起步依赖
        - 如何自定义springboot starer
    - 从本质上来说，Spring Boot就是Spring，它做了那些没有它你自己也会去做的Spring Bean配置
    - 《Spring Boot实战》前三章

- **SpringBoot的核心注解？**
  - 启动类上面的注解是**@SpringBootApplication** 也是SpringBoot的核心注解
    - **SpringBootConfiguration** 组合
    - **EnableAutoConfiguration** 打开自动配置的功能，也可以打开某个自动配置的选项，如关闭数据源自动配置功能：@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
    - **ComponentScan** Spring组件扫描
- **描述一个Spring Boot项目的启动过程（阿里）**
  - @SpringBootApplication --> 复合注解（@SpringBootConfiguration / @EnableAutoConfiguration / @ComponentScan）
  - [SpringBoot 应用程序启动过程探秘]
- **自动配置的实现原理以及boot的启动流程？**
- **常用注解？**
- **springboot中同类型bean的区分？**
  - **唯一标识符id来区分，防止系统报错**
- **从SpringBoot看世界**
- 百年前，普通人可以自己尝试创造一个汽车，但是随着工业水平的提高，越来越封装。从一个懂汽车原理的人到不懂汽车原理的人来说，都可以开车。只要一脚油门的事情，但是你知道发动机原理吗，不知道，所以，这就是封装将一些默认的配置直接配置好，你可以直接使用，但是如果你要自定义，那好自己去从写花钱改造一下就可以。再来看Boot，为什么说从一开始SSH SSM 没有推广Boot约定大于配置的思想呢，早起的需求还不足矣复杂，通过大多是比较小的企业应用，都可以应付，随着近几年 互联网技术不断发展，第一 业务的复杂性不断提高，需要我们高效率解决实际开发问题 第二 开发周期短 一个月上线，所以不断衍生出对应更加快速的技术。来服务于业务。
  - 我们应该持有的心态，不是因为简单了，而不学习，而是越简单替代性越高，我们应该更加好好专研自己的技术。而无非就是基本功+编程功底。

微服务

难点 单体服务->微服务(演进)

小公司 业务单一 单个项目 单tomcat

原因：业务拓张

- 微服务遇到的难点
  - 1、分布式事务、分布式锁
  - 2、**开发的时候会依赖下游，开发协调沟通测试问题**
  - 3、运维挑战、容器云、Docker、用k8s还是挑战。服务进程上千个。对小公司的挑战。
  - 4、**性能问题，调用链太长、网络IO太长**

Spring Cloud

- 是什么？
  - Spring Boot 是在 Spring 框架的基础上开发而来，让更加方便使用 Spring；Spring Cloud 是依赖于 Spring Boot 而构建的一套微服务治理框架
  - 资料参考
- **单体应用存在的问题？**
  - 随着业务发展，开发越来越复杂。
  - 修改、新增某个功能，需要对整个系统进行测试、重新部署。

- 一个模块出现问题，可能导致整个系统崩溃。
- 多个开发团队同时对数据进行管理，容易产生安全漏洞。
- 各个模块使用同一种技术开发，各个模块很难根据实际情况选择更合适的技术框架，局限性很大。

- **分布式和集群的区别？**
  - **集群：**一台服务器无法负荷高并发的数据访问量，就设置多台服务器一起分担压力，是在**物理层面解决问题。**
  - **分布式：**将一个复杂的问题拆分成若干简单的小问题，将一个大型的项目架构拆分成若干个微服务来协同完成，**在软件设计层面解决问题**。
- **微服务的优点？**
  - 各个服务的开发、测试、部署都相互独立，用户服务可以拆分为独立服务，如果用户量很大，可以很容易对其实现负载。
  - 当新需求出现时，使用微服务不再需要考虑各方面的问题，例如兼容性、影响度等。
  - 使用微服务拆分项目后，各个服务之间消除了很多限制，只需要保证对外提供的接口正常可用，而不限制语言和框架等选择。

服务注册与发现

Eureka

- **自我保护模式**
  - 当服务者启动注册到Eureka Server上之后，会保持一定的心跳检测机制。但是因为网络是不稳定的，所以，在极端情况下，由于只是网络出现问题，但是注册在Eureka上的服务提供者并没有宕机，如果直接删除，那么当再次连接的时候，就会造成资源的一种浪费。所以Eureka提供一一种自我保护模式，说白了，就是将当短时间内造成过多客户端请求丢失，那么会启动自我保护模式。这期间不会删除注册在Eureka Server中的服务列表，当再次服务提供者再次连接上会通过一定的次数来保证。取消自我保护模式。
  - 如何配置`eureka.server.enable-self-preservation = false` 禁用自我保护模式。
  - 某时刻某一个微服务不可用了，eureka不会立刻清理，依旧会对该微服务的信息进行保存
- **Eureka集群**

Zookeeper

Consul

负载均衡

Ribbon

- **是什么？**
  - Spring Cloud Ribbon是基于Netflix Ribbon实现的**一套客户端 负载均衡的工具**。

    简单的说，Ribbon是Netflix发布的开源项目，主要功能是**提供客户端的软件负载均衡算法**，将Netflix的中间层服务连接在一起。Ribbon客户端组件提供一系列完善的配置项如**连接超时**，重试等。简单的说，就是在配置文件中列出Load Balancer（简称LB）后面所有的机器，Ribbon会自动的帮助你基于某种规则（如简单轮询，随机连接等）去连接这些机器。我们也很容易使用Ribbon实现自定义的负载均衡算法。
- **负载均衡是什么 可以说一下嘛？**
  - LB，即负载均衡(Load Balance)，在微服务或分布式集群中经常用的一种应用。 负载均衡简单的说就是将用户的请求平摊的分配到多个服务上，从而达到系统的HA。 常见的负载均衡有软件Nginx，LVS，硬件 F5等。 相应的在中间件，例如：dubbo和SpringCloud中均给我们提供了负载均衡，SpringCloud的负载均衡算法可以自定义。

    - **分类**
        - **集中式LB**

          即在服务的消费方和提供方之间使用独立的LB设施(可以是硬件，如F5, 也可以是软件，如nginx), 由该设施负责把访问请求通过某种策略转发至服务的提供方；
        - **进程内LB**

          将LB逻辑集成到消费方，消费方从服务注册中心获知有哪些地址可用，然后自己再从这些地址中选择出一个合适的服务器。

          Ribbon就属于进程内LB，它只是一个类库，集成于消费方进程，消费方通过它来获取到服务提供方的地址。
    - **Nignx和Ribbon有什么区别？**
        - Nginx是服务端的负载均衡，所有客户端请求都
- **Ribbon有哪些核心组件？可以说一下吗**
    - IRule：根据特定算法中从服务列表中选取一个要访问的服务
        - 轮询 RoundRobinRule
        - 随机 RandomRule
        - 自定义

OpenFeign

- **是什么？**
    - Feign是一个**声明式WebService客户端**。使用Feign能让编写Web Service客户端更加简单, 它的使用方法是定义一个接口，然后在上面添加注解，同时也支持JAX-RS标准的注解。Feign也支持可拔插式的编码器和解码器。Spring Cloud对Feign进行了封装，使其支持了Spring MVC标准注解和HttpMessageConverters。**Feign可以与Eureka和Ribbon组合使用以支持负载均衡**。

Hystrix 断路器

- **分布式系统面临的问题？**
    - **服务雪崩** 多个微服务之间调用的时候，假设微服务A调用微服务B和微服务C，微服务B和微服务C又调用其它的微服务，这就是所谓的“**扇出**”。如果扇出的链路上某个微服务的调用响应时间过长或者不可用，对微服务A的调用就会占用越来越多的系统资源，进而引起系统崩溃，所谓的“**雪崩效应**”.

      对于高流量的应用来说，单一的后端依赖可能会导致所有服务器上的所有资源都在几秒钟内饱和。比失败更糟糕的是，这些应用程序还可能导致服务之间的延迟增加，备份队列，线程和其他系统资源紧张，导致整个系统发生更多的级联故障。这些都表示需要对故障和延迟进行隔离和管理，以便单个依赖关系的失败，不能取消整个应用程序或系统。

备注：一般情况对于**服务依赖的保护**主要有3中解决方案：

（1）**熔断模式**：这种模式主要是参考电路熔断，如果一条线路电压过高，保险丝会熔断，防止火灾。放到我们的系统中，如果某个目标服务调用慢或者有大量超时，此时，熔断该服务的调用，对于后续调用请求，不在继续调用目标服务，直接返回，快速释放资源。如果目标服务情况好转则恢复调用。

（2）**隔离模式**：这种模式就像对系统请求按类型划分成一个个小岛的一样，当某个小岛被火少光了，不会影响到其他的小岛。例如可以对不同类型的请求使用线程池来资源隔离，每种类型的请求互不影响，如果一种类型的请求线程资源耗尽，则对后续的该类型请求直接返回，不再调用后续资源。这种模式使用场景非常多，例如将一个服务拆开，对于重要的服务使用单独服务器来部署，再或者公司最近推广的多中心。

（3）**限流模式**：**上述的熔断模式和隔离模式都属于出错后的容错处理机制**，而限流模式则可以称为预防模式。限流模式主要是提前对各个类型的请求设置最高的QPS阈值，若高于设置的阈值则对该请求直接返回，不再调用后续资源。这种模式不能解决服务依赖的问题，只能解决系统整体资源分配问题，因为没有被限流的请求依然有可能造成雪崩效应。

- **是什么？**
  - Hystrix是一个用于处理分布式系统的**延迟和容错**的开源库，在分布式系统里，许多依赖不可避免的会调用失败，比如超时、异常等，Hystrix能够保证在一个依赖出问题的情况下，**不会导致整体服务失败，避免级联故障，以提高分布式系统的弹性。**

    “断路器”本身是一种开关装置，**当某个服务单元发生故障之后，通过断路器的故障监控（类似熔断保险丝），向调用方返回一个符合预期的、可处理的备选响应（FallBack）**，而不是长时间的等待或者抛出调用方无法处理的异常，这样就保证了服务调用方的线程不会被长时间、不必要地占用，从而避免了故障在分布式系统中的蔓延，乃至雪崩。
- **服务降级**
  - 备注：Hystrix服务降级，其实就是线程池中单个线程障处理，防止单个线程请求时间太长，导致资源长期被占有而得不到释放，从而导致线程池被快速占用完，导致服务崩溃。 Hystrix能解决如下问题： 1.请求超时降级，线程资源不足降级，降级之后可以返回自定义数据 2.线程池隔离降级，分布式服务可以针对不同的服务使用不同的线程池，从而互不影响 3.自动触发降级与恢复 4.实现请求缓存和请求合并
  - **服务降级处理是在客户端实现完成的，与服务端没有关系**
- **服务熔断**
  - 备注：熔断模式：这种模式主要是参考电路熔断，如果一条线路电压过高，保险丝会熔断，防止火灾。放到我们的系统中，如果某个目标服务调用慢或者有大量超时，此时，**熔断该服务的调用，对于后续调用请求，不在继续调用目标服务，直接返回，快速释放资源**。如果目标服务情况好转则恢复调用。
- **服务限流**
  - 备注：限流模式 主要是提前对各个类型的请求设置最高的QPS阈值，若高于设置的阈值则对该请求直接返回，不再调用后续资源。这种模式不能解决服务依赖的问题，只能解决系统整体资源分配问题，因为没有被限流的请求依然有可能造成雪崩效应。

路由网关

Zuul

- **是什么？**
  - Zuul包含了对**请求的路由**和**过滤**两个最主要的功能： 其中**路由功能负责将外部请求转发到具体的微服务实例上**，是实现外部访问统一入口的基础而过滤器功能则**负责对请求的处理过程进行干预**，是实现请求校验、服务聚合等功能的基础.Zuul和Eureka进行整合，将Zuul自身注册为Eureka服务治理下的应用，同时从Eureka中获得其他微服务的消息，也即以后的访问微服务都是通过Zuul跳转后获得。

注意：**Zuul服务最终还是会注册进Eureka**

提供=**代理+路由+过滤**三大功能

GateWay

分布式配置中心

Config

- **分布式系统遇到的问题？**
  - 微服务意味着要将单体应用中的业务拆分成一个个子服务，每个服务的粒度相对较小，因此系统中会出现大量的服务。由于每个服务都需要必要的配置信息才能运行，所以一套集中式的、动态的配置管理设施是必不可少的。SpringCloud提供了ConfigServer来解决这个问题，我们每一个微服务自己带着一个application.yml，上百个配置文件的管理。
- **是什么？**
  - SpringCloud Config为微服务架构中的微服务提供**集中化的外部配置支持**，配置服务器为各个不同微服务应用的所有环境提供了一个**中心化的外部配置**。

    怎么玩 SpringCloud Config分为**服务端**和**客户端**两部分。

    服务端也称为**分布式配置中心**，它是一个独立的微服务应用，用来连接配置服务器并为客户端提供获取配置信息，加密/解密信息等访问接口

    客户端则是通过指定的配置中心来管理应用资源，以及与业务相关的配置内容，并在启动的时候从配置中心获取和加载配置信息配置服务器默认采用git来存储配置信息，这样就有助于对环境配置进行版本管理，并且可以通过git客户端工具来方便的管理和访问配置内容。

消息总线

SpringCloud Bus

消息驱动

SpringCloud Stream

分布式请求链路追踪

问题迭代

- **什么是微服务？**
  - **微服务架构是一种架构模式或者说是一种架构风格**，它提倡将单一应用程序划分成一组小的服务，每个服务运行在其独立的自己的进程中，服务之间互相协调、互相配合，为用户提供最终价值。服务之间采用轻量级的通信机制互相沟通（通常是基于HTTP的RESTful API）。每个服务都围绕着具体业务进行构建，并且能够被独立地部署到生产环境、类生产环境等。另外，应尽量避免统一的、集中式的服务管理机制，对具体的一个服务而言，应根据业务上下文，选择合适的语言、工具对其进行构建，可以有一个非常轻量级的集中式管理来协调这些服务，可以使用不同的语言来编写服务，也可以使用不同的数据存储。
  - 理解：微服务架构是一种架构模式，之前我们是一个单一的应用程序，通过业务划分成多个服务，服务之间都部署在一个单独的进程中，通过RPC或者MQ进行调用或者HTTP等。并且每个服务拥有一个独立的数据库。核心在于实现高内聚，低耦合。属于同一业务进行划分。
  - **微服务和微服务架构的区别？**
    - 微服务架构一种架构模式，单一应用进行划分。
    - 微服务 关注的是某一个点，是具体解决某个问题

    - **优点**
        - 内聚，聚集在某一个业务上。开发简单，有自己的存储能力、
- 微服务之间是如何独立通讯的？
- **springCloud和Dubbo有哪些区别？**
    - 通信方式不同：SpringCloud抛弃了Dubbo的RPC通信，采用的是基于HTTP的REST方式。
- **SpringBoot和SpringCloud，请你谈谈对他们的理解？**
    - **SpringBoot专注于快速方便的开发单个个体微服务。**

      SpringCloud是关注全局的微服务协调整理治理框架，它将SpringBoot开发的一个个单体微服务整合并管理起来， 为各个微服务之间提供，**配置管理、服务发现、断路器、路由、微代理、事件总线、全局锁、决策竞选、分布式会话**等等集成服务

      SpringBoot可以离开SpringCloud独立使用开发项目，但是SpringCloud离不开SpringBoot，属于依赖的关系.

      SpringBoot专注于快速、方便的开发单个微服务个体，SpringCloud关注全局的服务治理框架。
- 什么是服务熔断？什么是服务降级？
- 微服务的优缺点分别是什么？说下你在项目开发中碰到的坑？
- 你所知道的微服务技术栈有哪些？请列举一二？
- **eureka和zookeeper都可以提供服务注册与发现的功能，请说说两个的区别？**
    - 作为服务注册中心，Eureka比Zookeeper好在哪里 著名的CAP理论指出，一个分布式系统不可能同时满足C(一致性)、A(可用性)和P(分区容错性)。由于分区容错性P在是分布式系统中必须要保证的，因此我们只能在A和C之间进行权衡。 因此 **Zookeeper保证的是CP, Eureka则是AP。**

      1 Zookeeper保证CP 当向注册中心查询服务列表时，我们可以容忍注册中心返回的是几分钟以前的注册信息，但不能接受服务直接down掉不可用。也就是说，服务注册功能对可用性的要求要高于一致性。但是zk会出现这样一种情况，当master节点因为网络故障与其他节点失去联系时，剩余节点会重新进行leader选举。问题在于，选举leader的时间太长，30 ~ 120s, 且选举期间整个zk集群都是不可用的，这就导致在选举期间注册服务瘫痪。在云部署的环境下，因网络问题使得zk集群失去master节点是较大概率会发生的事，虽然服务能够最终恢复，但是漫长的选举时间导致的注册长期不可用是不能容忍的。

      4.2 Eureka保证AP Eureka看明白了这一点，因此在设计时就优先保证可用性。Eureka各个节点都是平等的，几个节点挂掉不会影响正常节点的工作，剩余的节点依然可以提供注册和查询服务。而Eureka的客户端在向某个Eureka注册或时如果发现连接失败，则**会自动切换至其它节点，只要有一台Eureka还在，就能保证注册服务可用(保证可用性)，只不过查到的信息可能不是最新的(不保证强一致性)**。除此之外，Eureka还有一种**自我保护机制**，如果在15分钟内超过85%的节点都没有正常的心跳，那么Eureka就认为客户端与注册中心出现了网络故障，此时会出现以下几种情况：

      1. Eureka不再从注册列表中移除因为长时间没收到心跳而应该过期的服务
      2. Eureka仍然能够接受新服务的注册和查询请求，但是不会被同步到其它节点上(即保证当前节点依然可用)
      3. 当网络稳定时，当前实例新的注册信息会被同步到其它节点中

      因此， Eureka可以很好的应对因网络故障导致部分节点失去联系的情况，而不会像zookeeper那样使整个注册服务瘫痪。

SpringCloud Alibaba

Nacos服务注册和配置中心

Sentinel实现熔断与限流

Seata处理分布式事务

0x04-科班篇 ☆

设计模式

面向对象六大原则

- **单一职责原则**
  - 对于每个类来说，只能允许有一个功能，如果依赖过多的。会出现高度耦合。比如对于用户来说，用户有相关的属性以及方法，我们应该将属性和方法进行分离。按照不同粒度来进行划分。从耦合度上来说，可以降低不同类，模块之间的依赖。本质上还是降低耦合度。降低类的复杂度，每一个类都有一个清晰明确的职责。程序的可读性和可维护性提高。降低业务逻辑变换导致的风险。
- **开闭原则**：**对扩展开放,对修改关闭**，多使用抽象类和接口。
  - 比如代理模式，不修改原有的类，而是在基础上做扩展
- **里氏替换原则**：基类可以被子类替换，使用抽象类继承,不使用具体类继承。
  - **子类可以扩展父类的功能，但不能改变父类原有的功能。**可以使用子类对象当做父类的对象来使用。屏蔽了具体的实现细节，程序灵活性高，通用的编程，通用的代码。适应需求的变化。实现了传递不同的子类来完成不同的业务逻辑、
- 依赖倒转原则：要依赖于抽象,不要依赖于具体，针对接口编程,不针对实现编程。
  - 面向接口编程
- **接口隔离原则**：**使用多个隔离的接口,比使用单个接口好，建立最小的接口。**
- **迪米特法则：一个软件实体应当尽可能少地与其他实体发生相互作用，通过中间类建立联系。**
- 合成复用原则：尽量使用合成/聚合,而不是使用继承。

P1.单例模式 ☆☆☆

- 恶汉
- 懒汉->双检查
  - **为什么要使用syn关键字?**
    - 使用syn关键字可以解决多线程并发下的安全性问题
  - **为什么要加volatile呢?**
    - DCL(双检锁)机制不一定是安全的，有指令重排序的问题，加入volatile可以禁止指令重排序
    - 对于`Instance instance = new Instance()`
      - 在堆内存上分配对象的内存空间
      - 在堆内存上进行初始化对象
      - 将instance指向刚才分配的内存地址
- 应用场景
  - 创建单独对象时，比如一些基础的类库直接newInstacne()
- 静态内部类
- 枚举类

P2.装饰者模式

接口形式 抽象类形式

- 应用场景
  - IO流，当需要使用一个类进行封装时，可以基于原有对象进行装饰一下。

P3.代理模式 ☆☆☆

- **静态代理**
  - **JVM角度来看**：静态代理在编译时警将接口、实现类、代理类变成一个实际的.class文件
  - 主要实现
    - 1.定义一个接口和其实现类
    - 2.创建一个代理类并实现这个接口
    - 3.将目标对象注入到代理类中，通过调用代理的方法，在目标方法前后做相关逻辑。
- **动态代理** ☆☆☆
  - **应用场景**
    - Spring中AOP编程 Transaction事务和Autowired 都是使用动态代理实现的。
  - **解决的问题点**
    - 首先它是一个代理机制，如果熟悉设计模式中的代理模式，我们会知道，**代理可以看作是对调用目标的一个包装，这样我们对目标代码的调用不是直接发生的，而是通过代理完成，通过代理可以让调用者与实现者之间解耦**。比如进行 RPC 调用，通过代理，可以提供更加友善的界面；还可以通过代理，做一个全局的拦截器。
  - **jdk Proxy(接口形式)**
    - 1.定义一个接口和实现类
    - 2.实现invokehandler接口 重写invoke方法 做相关的逻辑处理
    - 3.通过Proxy.newProxyInstance() 类加载器，接口，实现handler的对象
  - **cglib(父类形式)**
    - 定义一个类
    - 自定义一个`MethodInterceptor`并重写`intercept`方法，用于拦截增强的方法。
    - Enhancer类的create()创建代理类
    - cglib 底层是通过子类继承被代理对象的方式实现动态代理的，因此代理类不能是最终类（final），否则就会报错 java.lang.IllegalArgumentException: Cannot subclass final class xxx。
    - 缺点
      - 因为 cglib 只能代理可以有子类的普通类，对于像最终类（final），cglib 是不能实现动态代理的，**因为 cglib 的底层是通过继承代理类的子类来实现动态代理的**，所以不能被继承类无法使用 cglib。通过父类生成一个代理类 而这个代理类就是父类的子类，内部包含了父类。
  - 静态代理与动态代理的区别
    - 静态代理通过本地`.class`字节码进行加载 内部持有一个被代理对象
    - 动态代理 基于接口或者父子类 形式来实现的。需要在运行时期动态创建代理类对象。进行前后逻辑处理。
    - JVM层面:静态代理在编译时就确定了 生成一个字节码文件，而对于动态代理来说，是在运行时期生成字节码文件的，并加载到JVM中的。
  - **1. JDK动态代理**

    （1）创建被代理对象的接口类。

（2）创建具体被代理对象接口的实现类。

（3）创建一个InvocationHandler的实现类，并持有被代理对象的引用。然后在invoke方法中利用反射调用被代理对象的方法。

（4）利用Proxy.newProxyInstance方法创建代理对象，利用代理对象实现真实对象方法的调用。

1. 上层接口和实现类

```
public interface Subject {
    void request();
}

public class RealSubject implements Subject {
    @Override
    public void request() {
        System.out.println("request invoke");
    }
}
```

2. 实现InvocationHandler接口

```
public class ConcreteInvocationHandler implements InvocationHandler {

    private Subject subject;

    public ConcreteInvocationHandler(Subject subject) {
        this.subject = subject;
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args)
            throws Throwable {
        return method.invoke(subject, args);
    }
}
```

3. 测试

```
public class JDKDynamicProxyTest {
    public static void main(String[] args) {
        Subject subject = new RealSubject();
        InvocationHandler handler = new
ConcreteInvocationHandler(subject);
        Subject proxy =
(Subject)Proxy.newProxyInstance(RealSubject.class.getClassLoader(),
                RealSubject.class.getInterfaces(), handler);
        proxy.request();
    }
}
```

2. CGlib动态代理

1. 原有类

```
public class Target {
    public void request() {
        System.out.println("执行目标类的方法");
    }
}
```

2. 实现拦截器

```
public class TargetMethodInterceptor implements MethodInterceptor {
    @Override
    public Object intercept(Object obj, Method method, Object[] args,
                            MethodProxy proxy) throws Throwable {
        System.out.println("方法拦截增强逻辑-前置处理执行");
        Object result = proxy.invokeSuper(obj, args);
        System.out.println("方法拦截增强逻辑-后置处理执行");
        return result;
    }
}
```

3. 测试

```
public class CglibDynamicProxyTest {
•
    public static void main(String[] args) {
        Enhancer enhancer = new Enhancer();
•
        // 设置生成代理类的父类class对象
        enhancer.setSuperclass(Target.class);
•
        // 设置增强目标类的方法拦截器
        MethodInterceptor methodInterceptor = new
TargetMethodInterceptor();
        enhancer.setCallback(methodInterceptor);
•
        // 生成代理类并实例化
        Target proxy = (Target) enhancer.create();
•
        // 用代理类调用方法
        proxy.request();
    }
}
```

3. 静态代理和动态代理的区别

> 既然有了静态代理的方式，为什么还要使用动态代理的方式？

你会发现每个代理类只能为一个接口服务，这样程序开发中必然会产生许多的代理类所以我们就会想办法可以通过一个代理类完成全部的代理功能，那么我们就需要用动态代理。

> 静态代理和动态代理不同的特点？

普通代理模式，代理类Proxy的Java代码在JVM运行时就已经确定了，也就是在编码编译阶段就确定了Proxy类的代码。而动态代理是指在JVM运行过程中，动态的创建一个类的代理类，并实例化代理对象。因为实际的代理类是在运行时创建的。

4. JDK和CGlib实现方式对比

- 字节码创建方式：JDK动态代理通过JVM实现代理类字节码的创建，cglib通过ASM创建字节码。
- **JDK动态代理强制要求目标类必须实现了某一接口，否则无法进行代理。而CGLIB则要求目标类和目标方法不能是final的，因为CGLIB通过继承的方式实现代理。**【InvocationHandler和MethodInterceptor】
- CGLib不能对声明为final的方法进行代理，因为是通过继承父类的方式实现，如果父类是final的，那么无法继承父类。

5. JDK代理方式如何代理多个方法

另外，被代理类可以实现多个接口。从代理类代码中可以看到，代理类是通过InvocationHandler的invoke方法去实现被代理接口方法调用的。所以被代理对象实现了多个接口并且希望对不同接口实施不同的代理行为时，应该在ConcreteInvocationHandler类的invoke方法中，通过判断方法名来实现不同的接口的代理行为。

P4.工厂模式

- 工厂方法模式适合，凡是出现了大量的产品需要创建，并且具有公共的接口，可以通过工厂方法模式进行创建，一个工厂里，不同方法创建不同的类。
- **1.简单工厂**
- **2.抽象工厂**
  - 工厂方法模式有一个问题就是，类的创建依赖于工厂，也就是说想要扩展程序，必须对工厂类进行修改，用抽象工厂，创建多个工厂类，这样一旦需要增加新的功能，直接增加新的工厂类就可以了，不用改以前的代码，一个工厂生产一个具体对象。
  - 优点 对于客户端来说不需要知道创建的细节，通过封装出一个抽象类。可以实现代码之间的松耦合。当需要创建一个新的产品，可以很容易实现。
  - **使用场景**
    - 对比于单例来说，本质上就是创建对象，而单例只需要创建一个对象，工厂方法可以实现批量的生产，类似于工厂，可以批量生成手机。而使用的就是同一个模板。所以，我们需要去权衡是否值得引入工厂方法创建对象。
    - 2.第二点 如果系统中确实需要类产生对个对象，那就可以使用工厂方法，其实对比于框架Spring，默认使用单例，当需要多个对象时，一定也是通过类动态创建对象的。而这个创建对象的过程一定是在Spring生命周期的过程中，所以就需要用到发射和动态代理。无外乎Spring也是一个业务框架，只不过节点的业务点是不同的。
- **3.方法工厂**
  - 对比
    - **工厂方法和抽象工厂的区别**
      - 如果产品单一，适用于工厂模式，业务产品比较多，使用抽象工厂。
      - 工厂方法针对的是面向同一个产品等级结构。
      - 抽象工厂针对的是不同系列产品的结构

P5.策略模式 ☆

- 替代 if else
- 思考，在实际项目中，不可避免会出现代码逻辑的判断，if else 我们可以将代码进行抽取，通过面向接口编程，比如对于CD来说，内部持有CD磁带，但是磁盘是灵活的，我们通过创建不同的磁盘，CD机就可以使用了。因此，不仅仅实现可拓展。

P6.命名模式

P7.模板方法

- **如何由子类决定模板方法内的执行顺序**
- 基本思想
  - 基本方法：抽象方法，由子类实现，并在模板方法中调用
  - 模板方法：可以有一个或几个，一般是具体的方法。确定一个框架；
  - 总结 在模板方法中调用基本方法类执行业务逻辑。
- 优点
  - 封装不变部分，拓展可变部分。
  - 提取公共部分代码，便于维护。
  - 行为由父类控制，子类实现。

P8.适配器模式

-

P9.责任链模式

- 一个接口，可能有多个实现类，对应不同的策略实现，一般来说项目可能在初始化时就指定了策略，那假如是动态的呢？比如说，当num大于100小于200，用A策略。num大于200小于300 用B策略。

P10.建造者模式

建造者模式，对于一个复杂产品，建造的过程一样，但过程可以有不同的表示。

P11.观察者模式

问题迭代

- **代理模式和装饰者模式的区别？**

  代理是在内部生成一个代理对象，构造函数参数为空。装饰的构造函数参数有一个对象，就是对这个对象进行装饰。说白了代理是基于目标类做.用代理模式，代理类可以对他的用户隐藏一个对象的具体信息。因此，使用代理模式的时候，我们常常在一个代理类中创建一个对象的实例。当我们使用装饰模式时，我们通常的做法是将原始对象作为一个参数传给装饰者的构造器。

  装饰者必须在本体上完成操作，而代理不一定调用本地。

  代理模式强调的是控制，装饰者模式强调的是增强。

  代理模式强调的是透明访问，装饰者模式强调的的自由构建。

- **Java源代码中常见的设计模式有哪些？**
  - jdk 中的设计模式：
    - 1）单例，比如 Runtime 类；
    - 2）静态工厂 Interger a=Integer.valueOf(int or String);
    - 3)迭代器模式 Collection.interator();
    - 4)原型设计模式,clone 方法；
    - 5）适配器inputStreamReader 和 outputStreamWriter；
    - 6）桥接模式，jdbc，抽象部分与实现相分离；
    - 7）装饰模式 Reader 和 bufferedBeader；
    - 8）代理，jdk 动态代理；
    - 9）观察者 observable 和 observer；
    - 10）责任链，classloader 的双亲 委 派 模 型 ；

        - 11）组合，某个类型的方法同时也接收自身类型作为参 java.util.list.addall(collection);
        - 12)抽象工厂，一个创建新对象的方法，返回的是接口或抽象类Connection c=DriverManager.getConnection();
        - 13)工厂方法，返回一个具体对象的方法Proxy.newProxyInstance;
        - 14)解释器模式，该模式通常定义了一个语言的语法,java.util.pattern。
- **重学dp**
    - 设计模式的目的是实现代码的可拓展性和可维护性。为什么，我们在实际编写业务代码的时候，很少去使用设计模式去设计代码。我认为其实主要有两点，第一点的话，本身我们编写的业务就是单一，固定的，极少情况下，会存在，本项目可以被复用。第二点的话，框架中到处都是设计模式的身影，在于框架的目的是为了适应不同的业务，将一些设计实现写的很灵活，比如，为了实现事务Spring中采用动态代理，进行目标对象的方法拦截处理。框架在于要实现可拓展性，微内核+可插件化。
    - 我们如何平衡业务编码与精心使用设计模式呢，对于业务来说，大多数时候，会直接修改源码解决，这带来了一定的修改成本，比如，为了解决一个项目中不明确的未知业务，精心设计拓展性的架构，成本也是显而易见的，**这是一个取舍**。

计网

网络分层

- **简单说一下OSI七层协议模式？**
    - OSI七层模型包括应用层，表示层，会话层，传输层，网络层，数据链路层以及物理层。
    - **每层作用可以具体说一下吗？**
        - **应用层** 由用户自己规定，**规定各个应用之间消息传递的形式**等，常见的协议有Http FTP协议等
        - **表示层** 在满足用户需求的基础上，尽可能的节省传输费用而设置的，比如传输压缩文件，jpeg或加密文件等格式。
        - **会话层** 建立和拆除会话
        - **传输层** 负责将来自会话层的消息传递给网络层 常见的传输层协议有TCP和UDP协议等
        - **网络层** 规定通信网内的路由选择方式，建立用户间的信息报传输设施，常见的网络层协议有IP ICMP ARP
        - **数据链路层** 与建立数据传输链路相关
        - **物理层** 规定一些机电性能，也包括工作方式，双工 单工 半双工 建立通信的启动和终止等
    - OSI中自顶向下分别是 应用层 表示层 会话层 传输层 网络层 数据链路层 物理层
    - TCP/IP四层中分别是 应用层 传输层 网际层 物理层
    - 5层网络体系结构 应用层 传输层 网络层 数据链路层 物理层一般
    - 我们使用的是TCP/IP 5层网络体系结构
    - 为什么要进行分层呢？
        - 在我看来，分层在整个计算机领域中是随处可以的。比如说操作系统中 应用层软件开发中，三层模型。计算机网络中的分层的出现主要是解决了网络中各种纷繁复杂的网络协议所带来的问题，分层更加专注于本层业务逻辑的处理，相对于其他上下层来说，是一个透明的存在。
    - 下面说一下各层的出现解决了什么问题？
        - 物理层主要解决了 物理层考虑的是怎样才能在连接各种计算机的传输媒体上传输数据比特流，而不是指具体的传输媒体。（实现相邻计算机节点指教比特流的透明传送，尽可能屏蔽具体传输介质和物理设备的差异）数据链路层 解决了 源自网络层来的数据可靠地传输到相邻节点的目标机网络层网络层只提供简单灵活的 无连接的 尽最大努力交付

的数据报服务运输层 进程之间通信的可靠性（两台主机之间的通信提供通用的数据传输服务）
    - 好了上面说了分层的问题 接下来 我们谈一谈各层之前对应的各种协议
        - 网络层 主要是IP协议 主要提供定位在哪个地方（选择合适的网间路由和交换节点，确保数据及时传送）运输层 主要是TCP、UDP y应用层 有常见的DNS 域名解析 服务 HTTP HTTPS FTP文件传输服务 SMTP简单邮件传输协议综上所述 就是我的理解
- **TCP/IP了解吗?**
    - TCP/IP协议是一系列网络协议的总称，是网络通信的基本骨架，TCP/IP协议模型在OSI七层模型的基础上，通过合并的方式，简化为四层 **应用层 传输层 网络层 链路层** 当我们通过应用层程序进行发送一个HTTP请求时，在传输层的TCP中在Http数据头部加上传输层的信息 在网络层的TCP数据中添加Ip地址 在链路层中添加对应的数据 传递到服务器端，进行相应的解析。合并->分解
    - **IP协议**
        - 根据IP地址能够区分两个主机是否同属于一个网络中
    - **ARP(地址解析协议)** 到达某个局域网 根据IP地址找对应的MAC机器
        - 根据IP地址获取MAC地址，将目的IP地址在链路层进行包装，生成以太网数据包，在同一个子网内进行广播出去，各个主机拿到IP地址和自己的IP地址对比，若一样，则返回自己的MAC地址，注意，MAC地址与对应的IP地址存入本机ARP缓存中并保留一定时间
    - **路由协议**
        - ARP寻址必须在一个局域网内，我们可以根据IP协议判断是否在一个子网中，如果不是在一个子网中，就需要进行转发到对应的子网中，完成这个路由协议的物理设备就是路由器。
- **一个网络数据包包含有哪些说一下？**
    - 网络数据包一般包含头部和数据部分。在TCP协议中，在发送的数据中加上TCP部分(主要是应用层上的端口号信息，以及TCP相关信息，)，IP模块添加上IP(IP+port) 确定一个唯一目标。MAC头部。最前面加上报头、起始帧分界以及末尾加入FCS(帧校验序列) 就构成一个完整的数据包。
    - 报头/起始帧分解符->MAC头部->IP头部->TCP->头部->数据->FCS(帧校验序列)

数据链路层

- 以太网帧
- MAC地址
    - 实现本地网络设备间的直接传输
- ARP 【广播请求-单播响应】
    - 从IP地址找MAC地址
    - 动态地址解析：广播
    - **了解ARP欺骗吗？**
- RARP【广播请求-单播响应】
    - 从MAC地址找ip地址

网络层 ☆

- IP
    - 实现大型网络间的传输
    - **为什么分类IP地址呢**
        - 简单明了
        - 具有3个级别的灵活性

        - 选路(基于网络地址)简单
    - CIDR子网掩码
    - 0 和 127 不作为 A 类地址。子网掩码只有一个作用，就是将某个 ip 地址划分为网络地址和主机地址两部分。两台计算机各自的 ip 地址和子网掩码进行与操作运算后，若得出相同结果，则说明这两台计算机处于同一个子网中。
- ICMP
- NAT
    - 网络地址转换，常用于私有地址与公有地址的转换，以解决 ip 地址匮乏的问题
- MTU 最大传输单元

传输层 ☆

- **多路复用与多路分解**
    - 多路分解：运输层报文段中的数据交付到套接字的工作
    - 多路复用：套接字从进程中获取的数据交付到运输层的工作
- **如何确定一条唯一的请求**
    - 源ip+源端口 目标ip+目标端口
- **可靠传输的基本原理**
    - 停止等待协议
    - 连续ARQ协议
    - 超时重传计时器
    - 重传与确认
        - Sequene 序列号、ACK序列号
            - 设计目的 解决应用层字节流的可靠传输
                - 跟踪应用层的发送端数据是否送达
                - 确认接收端有序的接收到的字节流 ACK
            - 序列号的值针对的是字节而不是报文
- **TCP** ☆
    - **思考：TCP解决的问题点是什么？**
        - 在Ip协议之上，解决网络通讯可依赖问题
- 特点、首部各个字段+可靠传输的原理
    - **三次握手**
        - TCP协议是一个可靠的协议，在正式传输数据之前必须通过三次握手建立连接并且互相交换窗口大小，在传输结束之后，通过四次挥手来确认双方都结束数据交互。
        - 具体流程
            - 首先服务端必须先开启Listener状态，监听接收来自客户端的请求。然后客户端通过请求发送了一个SYN请求，当服务端接收到请求后回送一个ACK确认消息，而在这一过程中，极有可能客户端发送的消息是丢失的，所以需要等待对方的回应。客户端接收到后返回一个ACK，我可以建立了，基本上三次握手可以很好的确保双发的手法能力。第一次连接 确定客户端的发送能力 和服务端的接收能力，第二次连接 确定服务端的发送能力 和 客户端的接收能力 第三次是确认。而客户端经历了Close->SYN_SENT->ESTABLISHED的状态变化，而服务端经历了Close->Listen->SYN_RCYD->EATABLISHED的状态变化
            - 第一次：client : syn = 0
            - 第二次：server : ack=1 syn = 122
            - 第三次：client : ack = 123

        - **为什么需要三次握手，两次可以吗？【掌上先机】** ☆☆☆☆☆
            - 如果两次握手的话，那么当服务端发送SYN+ACK的时候，很可能在发送完毕之后，服务端就处于工作状态，而SYN+ACK如果在过程中数据丢失，那么客户端以为服务端没有准备好，而服务端一直处于工作状态，就会造成资源的一种浪费，而网络资源是非常宝贵的。
    - **四次挥手** ☆☆☆☆☆
        - **为什么断开需要四次连接？** ☆☆☆☆☆
            - 首先我们应该梳理一下，四次挥手流程，客户端发送一个FIN终止信号，表示客户端想要断开连接。当客户端FIN信号发送到服务端后，服务端接收到客户端的终止请求，服务端先发送一个ACK信号对客户端的FIN信号的确认。当服务端也想断开连接的时候，服务端也需要发送一个FIN信号给客户端，客户端响应客户端的FIN的确认。所以断开需要分别客户端发送一个FIN和ACK，服务端也需要发送一个FIN和ACK。
        - **四次挥手主动方为什么需要等待2MSL？**
            - 首先我们先说一下，主动断开方接收到被动关闭方的FIN的终止信号，会立马发送一个ACK对于被动关闭方的确认，进入TIME_WAIT时间，等到2MSL关闭连接。
            - **你可以解释一下什么MSL嘛？**
                - MSL表示最大报文生存周期，任何报文在网络上存在的最长时间，超过这个时间报文将被废弃。主动关闭方需要等待2MSL是为了 **防止最后一次ACK没有被正确的传输到被动关闭方，而被动关闭方，需要发送第三次的FIN信号。**
- **流量控制机制**- 解决速度不匹配问题
    - **滑动窗口**
        - 采用滑动窗口进行流量控制，滑动窗口大小可变，窗口大小的单位是字节
            - 发送窗口**在连接建立时由双方确定**，但在通信过程中，接收端可以根据自己的资源情况，

              随时动态的调整对方的发送窗口上限制。
            - 接收端窗口：这是接收端根据其目前的接收缓存大小所许诺的最新窗口值，是来自接收端的流量控制。接收端将此窗口值放在 tcp 报文的首部中的窗口字段，传送给发送端，**是来自接收端的流量控制。**
        - 糊涂窗口综合征
            - SWS避免算法
                - 接收方 窗口边界移动值小于min MSS 通知窗口为0
                - 发送方 Nagle
                    - 没有已发送未确认报文段时，立即发送数据
                    - 存在未确认报文段时，直到 1 数据长度到达MSS在发送
        - 位于传输层的TCP协议是面向连接的，可靠的传输协议，拥有这确认机制，理论上，每发一个数据包都会收到其对应的确认包，然后才可以继续发送数据。
    - 在三次握手阶段，双方互相将自己的最大可接受的数据量告诉对方，也就是自己的数据接收缓冲池的大小，这样对方可以根据已发送的数据量来计算是否可以接着发送，在处理过程中，当接收缓冲区池的大小发生变化时，要给对方发送更新窗口的大小的通知，利用滑动窗口机制可以有效提高通信效率。
- **拥塞控制-【全局性，防止过多的数据注入网络】**
    - 拥塞避免的算法主要有两种
        - 慢启动+拥塞避免
        - 快重传+快恢复

    - 拥塞窗口：这是发送端根据自己估计的网络拥塞程度而设置的窗口值，**是来自发送端的流量控制**。
    - 防止过多的数据注入网络中，使得网络中的路由器或链路不至过载
    - **网络拥塞的判定？**
        - 当网络发生拥塞时，路由器就会丢掉分组，因此，只要发送端没有按时收到应当到达的确认报文 ack，就可认为网络出现了拥塞
    - 慢启动(2的N次增加)-拥塞避免(回退到一半窗口大小 +1操作) 当出现丢包 快速重传和快速恢复解决
    - **慢启动**
        - 由小到大逐渐增加发送数据量 **发送端的拥塞窗口数值**，每收到一个报文确认，就加一。**指数级增加**
        - 窗口 cwnd 的增长引起网络拥塞，还需要慢开始门限 ssthresh。
        - 当 cwnd<ssthresh,使用慢开始算法; 当 cwnd>ssthresh,，使用拥塞避免算法；cwnd=ssthresh,既可用慢开始算法也可用拥塞避免算法。
        - 无论是慢开始还是拥塞避免，只要发送端发现网络阻塞，就将慢开始门限设为出现拥塞时的发送窗口值的一半，然后拥塞窗口为一，并执行慢开始算法。这样做的目的是迅速减少主机发送到网络中的分组数，使得发生拥塞的路由器有足够时间把队列中积压的分组处理完毕
    - **拥塞避免**
        - 当网络中的资源供应不足，网络的性能就要明显变坏，整个网络的吞吐量随之负荷的增大而下降，也就是说对资源的需求超过了可用的资源，因为传输数据是需要资源的。
    - **快重传**
        - 快重传算法规定：发送端只要一连收到三个重复 ack，即可断定有分组丢失，就应该立即重传丢失的报文，而不需要等待为该报文设置的重传计时器超时。
        - 与慢开始不同，拥塞窗口不设为 1，,而设为慢开始门限+3*mss（mss：最大报文段）。
    - **快恢复**
    - **选择性重传算法**
- 超时重传机制
- 数据包分片与重组功能
- **你能说一下流量控制和拥塞控制的理解吗？**
    - 流量控制考虑点对点的通信量的控制，也就是客户端和服务端直接数据传输数据量的大小。
- 拥塞控制考虑的问题是整个网络，是全局性的考虑。
- **说一下你对TCP的理解？**

  tcp 如何保证传输的可靠性？tcp 是面向连接，可靠的字节流服务。

  面向连接意味着两个使用 tcp 的应用（通常是一个客户端和一个服务器）在彼此交换数据之前必须先建立一个 tcp 连接。在一个 tcp 连接中，**仅有两方进行彼此通信，广播和多播不能用于 tcp。**

  Tcp 通过下列方式提供可靠性：

  1）将应用数据分割为 tcp 认为最合适发送的数据块；

  2）**超时重传**：当 tcp 发出一个段后，他启动一个定时器，等待目的端确认收到这个报文段。若不能及时收到一个确认，将重发这个报文段。

  3）当 tcp 收到发自 tcp 链接另一端的数据时，它将发送一个确认（对于收到的请求，给出确认响应）。这个确认不是立即发送，通常将推迟几分之一秒（之所以推迟，可能是要对包做完校验）；

  4）若 tcp 收到包，校验出包有错，丢弃报文段，不给出响应，tcp 发送端会超时重传；

5）对于失序数据进行重新排序，然后交给应用层（tcp 报文段作为 ip 数据报进行传输，而 ip 数据报的到达会失序，因此 tcp 报文段的到达也可能失序。若必要，tcp 将对收到的数据进行重新排列，以正确的顺序交给应用层）。

6）对于重复数据，直接丢弃。

7）tcp 可以进行**流量控制**，防止较快主机致使较慢主机的缓冲区溢出。

字节流服务：两个应用程序通过 tcp 连接，tcp 不在字节中插入记录标识符，我们将这种为字节流服务。

> Tcp 对字节流的内容不做任何解释，tcp 不知道传输的字节流数据是二进制数据还是 ascii字符或其他类型数据，对字节流的解释由 tcp 连接双方的应用层。

- **UDP**
  - 应用场景：DNS
  - 特点
    - 发送数据更为灵活
    - 无须建立连接
    - 无连接状态
    - 分组首部开销小
  - 报文段结构
    - 源端口号 目的端口号 长度 检验和 应用数据
  - UDP检验和
  - **怎么用 udp 实现 tcp？**
    - 由于在传输层 udp 已经是不可靠的，那就要在应用层自己实现一些保证可靠传输的机制，简单来说，要使用 udp 来构建可靠的面向连接的数据传输，就要实现类似于 tcp 的超时重传（定时器），拥塞控制（滑动窗口），有序接收（添加包序号），应答确认（ack 和 seq）。目前已经有了实现 udp 可靠运输的机制——udt：主要目的高速广域网海量数据传输，他是应用层协议。
- **TCP与UDP对比**
  - TCP协议进行数据通信之前需要建立三次握手建立连接，UDP协议不需要建立连接即可发送数据
  - TCP有确认机制，丢失数据包可以重发，保证数据的正确性，UDP不保证正确性，只是单纯的负责发送数据包。
  - TCP协议包可能会对大数据包进行拆分，并且在接收方进行重组数据包操作。UDP只是面向报文的，不会进行分片和重组，所以需要注意传输的报文大小。
  - 网络包中的TCP头部为20字节，UDP头部为8字节
  - TCP是点对点，全双工。
  - **特点**

    是否面向连接 TCP 面向连接 UDP 无连接

    可靠性 TCP可靠传输 UDP不可靠传输

    传输形式 TCP 字节流 UDP 报文

    性能

    传输效率 TCP慢 UDP快

    所需资源 TCP多 UDP少

    应用场景

TCP 要求通信数据可靠 （文件 传输 邮件传输

UDP 要求通信速度 (域名 转发 游戏数据包传送 QQ消息)

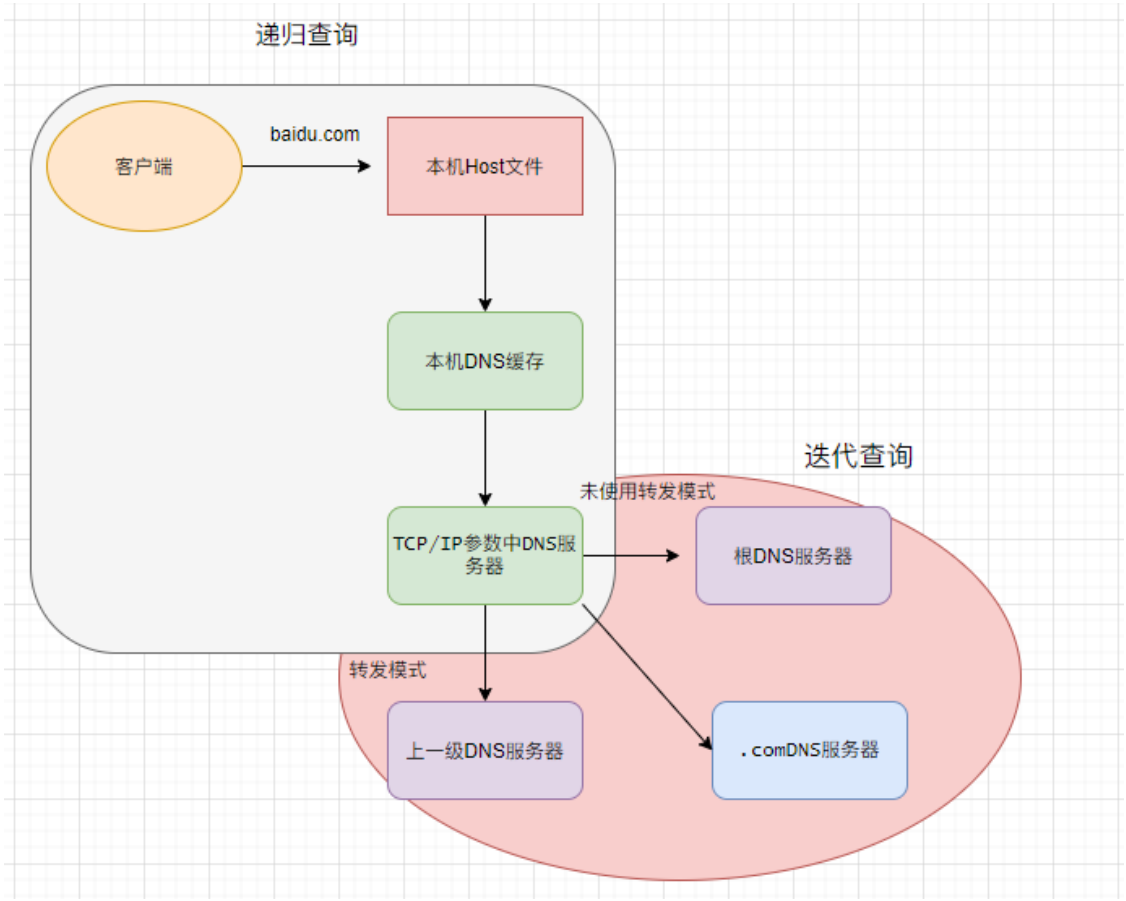
首部字节大小

TCP 多 20/60

UDP 8 字节

应用层 ☆

- **应用层协议原理**
  - 网络应用程序体系结构
    - 客户-服务端体系结构
    - 对等P2P体系结构
  - 进程通信
    - 进程(应用)与计算机网络之间的接口 -**socket**
    - 进程寻址-1.主机地址 2.进程标识符-端口 (IP+PORT)
- **域名解析系统DNS**



DNS 是一个分布式数据库系统，**存储了域名和 IP 地址的映射关系。**

主机向本地域名服务器的查询采用递归查询：如果本地域名服务器不知道被查询域名的 IP 地址，就会以 DNS 客户的身份向其他根域名服务器继续发出查询请求。

本地域名服务器向根域名服务器查询采用迭代查询：根域名服务器会告知顶级域名服务器的地址，顶级域名服务器给出 IP 地址，或者告知下一步应该向哪个权限域名服务器进行查询。

  - **层次结构**
    - 客户机->Host文件->本地DNS服务区->根DNS服务器->TLD DNS服务器->权威服务器
  - **为什么要进行DNS缓存？**
    - 改善时延性能并减少在因特网上到处传输的DNS报文数量。

- **迭代查询和递归查询？**
  - **从请求主机到本地DNS服务器的查询是递归查询，其余的查询是迭代查询。**
- **DNS有哪些功能？**
  - 主机名到IP地址的转换
  - 主机别名 baidu.com ->www.baidu.com
  - 邮件服务器别名
  - 负载均衡
- **单点DNS的问题？**
  - 单点故障
  - 通信容量
  - 远距离的集中式数据库
  - 维护
- DNS劫持?

- **HTTP 1.0 1.1 2.0之间的区别？**
  - **1.0 是一种无状态的协议，浏览器每次和服务端交互都要发送先建立连接，使用完毕之后关闭连接。服务器不跟踪每个客户端也不记录过去的请求。也就是默认的Connection:Close**
  - 1.1 使用了保持连接`Connection:keep-alive` 避免了在连接建立和释放的开销，服务器按照客户端的请求先后顺序来处理客户端的请求，通过`Content-length` 来判断当前请求的是否已经接受。不运行存在并行的响应。
  - 持续连接
- **HTTPS (详细的握手过程、摘要算法、数字签名、数字证书的原理和过程)**
  - https 其实是由两部分组成：http+ssl/tls，也就是在 http 上又加了一层处理加密信息的模块，服务端和客户端的信息传输都会通过 tls 加密，传输的数据都是加密后的数据。加解密过程：

    1）客户端发起 https 请求（就是用户在浏览器里输入一个 https 网址，然后连接到 server的 443 端口）

    2）服务端的配置（采用 https 协议的服务器必须要有一塔数字证书，可以自己制作，也可以向组织申请，这套证书就是一对公钥和私钥）。

    3）传输证书（这个证书就是公钥，只是包含了很多信息）

    4）客户端解析证书（由客户端 tls 完成，首先验证公钥是否有效，若发现异常，则弹出一个警示框，提示证书存在问题，若无问题，则生成一个随机值，然后用证书对随机值进行加密）

    5）传输加密信息（这里传输的是加密后的随机值，目的是让服务端得到这个随机值，以后客户端和服务端的通信就可以通过这个随机值来进行加密了）

    6）服务端解密信息（服务端用私钥解密后得到了客户端传来的随机值，then 把内容通过该值进行对称加密。所谓对称加密就是，将信息和私钥通过某种算法混在一起，这样除非知道私钥，不然无法获取内容，而正好客户端和服务端都知道这个私钥，所以只要加密算法够彪悍，私钥够复杂，数据就够安全）

    7）传输加密的信息

    8）客户端解密信息，用随机数来解。

    问：在客户端抓包抓到的是加密的还是没加密的？不知道是哪个面试官问的，我就乱说是加密的，然后面试官说错了，是没有加密的

- **说说你对HTTP与HTTPS的理解 区别？**
  - HTTP是超文本传输协议，数据明文传输，HTTPS在HTTP的基础上加入了SSL协议，实现数据的加密传输。
  - HTTPS需要去申请证书，一般是收费的。

- HTTP默认使用80端口，HTTPS默认使用443端口。
- **那可以接着说一下HTTP中Get和POST方法有哪些区别吗？** ☆
  - Get一般用于从服务器上查询获取资源，Post一般用来更新服务器上的资源。
  - Get方法将参数直接拼接在URL后边，明文显示，可以通过浏览器地址栏直接访问
  - Post请求用于提交订单，数据不是明文的，安全性更高。
  - Get请求有长度限制，Post请求没有
  - **常见的HTTP状态码可以说一下吗？**
    - 1XX(临时响应)
    - 2XX(成功)
    - 3XX(重定向) 表示要完成请求需要进一步操作
    - 4XX(错误) 表示请求可能出错，妨碍了服务器的处理
    - 5XX(服务器错误) 表示服务器在尝试处理过程时发生内部错误
  - **常见HTTP协议的状态码**
    - 200(成功)
    - 302(重定向) 请求重定向到执行网页
    - 304(未修改) 自从上次请求后，请求的网页未修改过，服务器返回此响应过，不会返回网页内容
      - **详细说一下304的理解？== 如何解决客户端缓存与服务器数据数据的一致性？**
        - 当用户请求一个Index页面，第一次请求到服务器端，服务器返回该页面并将一个`last-modified`最后修改时间返回， 客户端显示给用户并将该文件缓存在客户端缓冲中，当第二次用户请求该页面，还是会请求服务端带着`if-modified-since`头部，和服务器端文件修改时间做一个对比，如果没有修改，直接返回一个304。客户端就知道文件没有修改，从缓存中直接获取，直接返回给用户。主要是及时响应用户的吞吐率。【技术架构-缓存】
    - 401(未授权) 请求要求身份验证
    - 403(禁止) 服务器拒绝请求(死循环，一直访问)
    - 404(资源找不到) 服务器找不到请求的网页
    - 405(方法禁用) post请求当成了get请求直接访问
    - 500(服务器内部错误) bug
    - 502(错误网关) 服务器从上游接到了无效请求
    - 504(网关超时) nginx请求超时，请求一直没有返回。
      - **可以说一下301和302的区别吗？** ☆
        - 302重定向只是暂时的重定向，搜索引擎会抓取新的内容而保留旧的地址，**因为服务器返回302，所以，搜索搜索引擎认为新的网址是暂时的。**而301重定向是永久的重定向，搜索引擎在抓取新的内容的同时也将旧的网址替换为了重定向之后的网址。
- **Session和Cookie的区别 session_id 是怎么 返回给客户端的 谈谈你的理解？** ①
  - 当用户发送一个请求到服务端，是没有cookie的，服务端接收到请求后，会生成一个Cookie并设置上一个sessionid，并存储到cookie中，返回给客户端。等用户下一次请求，cookie中带着sessionId，就可以和服务端的session中的id进行匹配，查看用户的信息。Session的相关信息是存储到服务端中，而cookie是存储在客户端中的。**一个session 对应一个sessionID**
  - **区别：**

        - 默认情况下 cookie 是会话级别，存储在浏览器内存中，用户退出浏览器之后被删除，若希望浏览器将 cookie 存在磁盘上，则要使用 maxage，并给一个以秒为单位的时间，表示cookie 的存活时间
        - Cookie 作用范围：可以作用当前目录和当前目录的子目录，但不能作用与当前目录的上一级。

          Cookie:在客户端保持 http 状态信息的方案，会话跟踪。

问题迭代

- **描述三次握手四次挥手以及原因（阿里/腾讯）**
    - 三次握手 [发送能力-接受能力]
        - 发送端 --> SYN
        - 接收端 --> [SYN/ACK]
        - 发送端 --> ACK
        - SYN（Synchronize Sequence Number） 是 TCP/IP 建立连接时使用的握手信号，置1表示有效
        - 需要三次握手原因：信息对等 && 防止超时（《码出高效》 第一章）
        - 接收端回传SYN --> 告诉发送端我接收到的信息确实就是你所发送的信号
    - 四次挥手
        - 发送端 --> FIN (发送端进入 FIN-WAIT-1 （终止等待1）状态)
        - 接收端 --> ACK (发送端收到接收端的确认请求后，进入 FIN-WAIT-2 （终止等待2）状态)
        - 接收端 --> FIN（接收端进入半关闭状态 CLOST_WAIT）
        - 发送端 --> ACK（发送后，发送端进入 TIME-WAIT 状态）
    - TCP的三次握手与四次挥手（详解+动图）

      假设A、B两方进行TCP握手 如果是两次握手A->B(第一次握手因为某些原因迟到了很久) A->B(A再次进行一次握手请求) B->(响应握手建立连接) 而此时第一次握手请求这个时候才到了B端，那么B端也要响应这个握手连接啊，于是就返回响应，但是这个时候的A已经将第一次的握手请求作废，所以这一条连接就纯属于浪费服务器的资源。 如果是三次握手 A->B(请求握手)、B->A(同意握手 但是不确定A能否收到这个同意握手的回应)、A->B(已收到回应 TCP连接建立)，如果A将第一次握手请求作废，那么就不会有第三次的握手，而TCP连接也不会建立起来
- **三次握手**

  客户端发送 syn标志的数据包 一次握手-服务端

  服务端 发送syn ack标志的数据包 二次握手 客户端

  客户端-发送带有带有ack标志的数据包 三次握手 服务端

  客户端向服务端报告我要和你建立连接，顺便把我自己的一个发送的能力发给服务器，让服务器知道。服务器判断我是否可以给你创建连接，把我的一个接收的能力返回给客户端，只有三次握手才能保证 双方的发送能力和接收能力都能达到一个协商的过程，但是因为协议没有100%可靠的，三次就够了，四次也不能保证100%可靠。

  为什么三次 四次不行吗 或者 两次不行吗

  如果是二次握手。我们可以假设这样一种情况。当第一次请求因为网络延迟而没有及时到达。客户端(超时重发)再次发送建立连接的请求，服务端接收到客户端的请求，响应给客户端。此时 连接建立成功，发送数据结束后关闭连接，但是因为第一次请求因为网络延迟 现在才到达，也就是存在一个时间差，服务端会误以为 客户端会再次建立一个链接。所以会创建一个新的连接。
- **三次握手中SYN/ACK包的具体内容（腾讯）**
    - SYN --> 同步序列，用于建立连接过程

    - FIN --> Finish标志，用于释放连接
    - ACK --> 确认接收到的数据，确认序号标志
    - TCP三次握手中SYN，ACK，Seq三者的关系
    - Understanding TCP Sequence and Acknowledgment Numbers

- **用Socket描述TCP的三次握手（腾讯）** *TODO*
    - Socket过程详细解释（包括三次握手建立连接，四次握手断开连接）（C++实现，未看）
- **网络间的进程如何表示（腾讯）** *TODO*
    - 网络中进程之间如何通信
- **TCP和UDP的特点和区别（腾讯）**
    - TCP --> 面向连接 / UDP --> 无连接（发送数据前不需要建立连接）
    - TCP 提供可靠的服务（数据传输）/ UDP 无法保证
    - TCP --> 字节流 / UDP --> 数据报文段
    - TCP和UDP的区别
    - 常见面试题整理--计算机网络篇（每位开发者必备）
- **TIME_WAIT状态产生（腾讯）** *TODO*
    - 理解TIME_WAIT，彻底弄清解决TCP: time wait bucket table overflow
- **从客户端输入一个URL，该请求是如何传到服务端的（爱奇艺）**
    - DNS解析
        - 递归查询 --> 该模式下DNS 服务器接收到客户机请求，必须使用一个准确的查询结果回复客户机。
        - 迭代查询 --> DNS 服务器并不直接回复查询结果，而是告诉客户机另一台DNS 服务器地址，客户机再向这台DNS 服务器提交请求，依次循环直到返回查询的结果
    - 建立TCP连接
    - 发送HTTP请求
    - 服务器处理请求并返回HTTP报文
    - 浏览器解析并渲染页面
    - 连接结束
    - 从输入URL到页面加载发生了什么（依次逐步解释）
    - 在浏览器地址栏输入一个URL后回车，背后会进行哪些技术步骤？（更具体的解释）
    - DNS递归查询和迭代查询的区别
    - http 请求过程——当我们在浏览器输入 www.baidu.com，然后回车之后的详解。1）域名解析（域名 www.baidu.com 变为 ip 地址）。2）发起 tcp 的三次握手，建立 tcp 连接。浏览器会以一个随机端口（1024-65535）向服务端的 web 程序 80 端口发起 tcp 的连接.这个请求（原始的 http 请求，经过原始的 tcp/ip 四层模型层层封装），到达服务器端后，进入网卡，然后进入内核的协议栈（一层一层拨开），然后到达 web 应用程序，最终建立了 tcp/ip 链接。3）建立 tcp 连接后发起 http 请求。4）服务器响应 http 请求，客户端得到 html 代码。服务器 web 应用程序收到 http 请求后，就开始处理请求，处理之后就返回给浏览器 html文件。5）浏览器解析 html 代码，并请求 html 中的资源。6）浏览器对页面进行渲染，并呈现给用户。
    - 1.DNS域名解析

      通常来说baidu.com 只是为了用户更好的记录网站地址，而这个域名会绑定不限于1个IP地址，而将域名解析的工作就要由DNS来做.一般先去查找浏览器缓存，没有 去路由器缓存 没有 去 ISP DNS，没有的话去进行递归搜索，先从顶级域名.com搜索。

DNS 是一个 IP : 域名的关系的分布式数据库，域名并不是真实资源所在的位置，将输入的域名解析成IP地址

2.TCP建立连接

三次握手

3.发送HTTP请求

当连接创建成功之后，客户端发送数据到服务端，服务端接收到请求 响应。

4.服务器处理请求和返回HTTP报文

5.浏览器解析渲染页面

1.根据html解析出DOM树

2.根据css解析生成css规则树

3.结合dom树和css规则树 生成渲染树

4.根据渲染树计算每一个节点的信息

5.根据计算好的信息绘制页面

6.断开连接

四次挥手。

- **HTTP 与 HTTPS 的区别**
  - HTTP协议以明文方式发送内容，不提供任何方式的数据加密
  - HTTPS其实就是建构在SSL（Secure Sockets Layer） / TLS之上的 HTTP协议
    - **SSL握手协议使用的加密算法，非对称加密算法？**
  - HTTPS密文传输 / HTTP 明文传输
  - HTTPS协议需要到CA申请证书
  - 详细解析 HTTP 与 HTTPS 的区别
- **描述HTTPS的加密过程 / 对称加密&非对称加密在HTTPS加密过程中如何实践（字节跳动/阿里）** *TODO*
  - HTTPS加密过程和TLS证书验证
  - HTTPS中的TLS（密码学角度）
- **HTTP代理如何实现（阿里）** *TODO*
  - 如何实现一个HTTP/HTTPS代理客户端
- **描述SSO的原理（拼多多）** *TODO*
  - CAS实现单点登录SSO执行原理探究(终于明白了)
  - How does single sign-on work? *TODO*
- **介绍 OSI 七层模型**
  - 应用层（应用层 --> 表示层 --> 会话层） --> 传输层 --> 网络层 --> 数据链路层 --> 物理层 (7层)
  - 应用层 --> 传输层 --> 网络层 --> 数据链路层 --> 物理层 （5层）
  - 并非标准，一个概念性框架
  - OSI七层模型详解
  - 干货：计算机网络知识总结.md（逐层术语解释）
- 描述Session的实现原理（或者如何设计一个Session）（拼多多） *TODO*
- **Ping的原理可以说一下嘛？**

```
* ping 基于什么协议？答案：icmp，是 tcp/ip 协议族的额自协议，在网络层，用于在 ip
```

```
    主机、路由器之间传输控制消息。控制消息是指网络通不通，主机是否可达、路由是否可用

    等网络本身的消息。这些控制信息虽然并不传输用户数据，但是对于用户数据的传递起着重

    要的作用。Ping，tracert 都是基于 icmp。可以利用 os 规定的 icmp 数据包的最大尺寸不超过

    64k，向主句发起 ping of deathgongji 该攻击原理：若 icmp 数据包的尺寸超过 64k 上限，
主
    机就会出现内存分配错误导致 tcp/ip 堆栈奔溃，致使主机死机。防范方法：1）在路由器上

    对 icmp 数据包进行带宽限制，将 icmp 占用的带宽控制在一定的范围，这样即使有 icmp 攻

    击，他所占用的带宽也非常有限。2）在主机上限制，设置 icmp 数据包的处理规则，最好是

    设定拒绝 all 的 icmp 数据包。
```

- **URL 中可以存在中文吗？**

  A:可以，先将中文进行编码，tomcat 默认解码为 iso8859-1，这样就会出现乱码，我们可以再用 iso8859-1 进行编码，再用指定码表解码（post 和 get 都可以）。对于 post，可以使用requset 的 setCharacterEncodeing 方法设置指定的解码表。

- **TCP的可靠传输原理、滑动窗口、流量控制、拥塞控制**

  1.应用数据被分隔成TCP认为最合适发送的数据块

  2.对数据包进行排序，接收方对数据包排序，按有序数据交付给应用层

  3.**检验和**(如果数据在传输的过程中 出现错误 会直接弃包)

  4.TCP 接收端会丢弃重复的数据

  5.利用**滑动窗口**实现**流量控制**

  6.**拥塞控制**(网络拥塞时，会减少数据的发送)（**慢开始、拥塞避免、快重传和快恢复**）

  7.**ARQ协议**(对每一个分组数据 进行ack)

  8.**超时重传(**有一个定时器 TCP发送一个数据包后，启动一个定时器，等到目的端对数据的确认，如果在规定时间内不能确认，重新发送数据)

- **Get和Post的区别？**

  1.POST相对来说比GET安全， GET在地址栏上可见

  但是从传输层角度来说，两者都是不安全的，HTTP协议是明文传输，通过抓包工具 都可以获取到数据报文。要安全推荐使用HTTPS

  2.GET长度 HTTP协议中没有Body和URL长度限制 ，对URL限制是浏览器和服务器端原因

  服务器端处理比较长的URL是比较耗费资源的，为了安全和性能两方面的考虑 会给URL长度加限制。

  3.Get用于搜索请求 POST用于表单请求

  4.get请求在url中发送，post请求在http消息主体中发送。

- **HTTPS的实现原理 、数字签名原理**

  1.端口号 HTTP 端口号 80 HTTPS 端口号 443

  2.https协议需要ca申请证书 需要一定费用

  3.http是超文本传输协议 信息是明文传输 https则是具有安全性的ssl加密传输协议

4.http连接简单 无状态 https协议是SSL+HTTP协议构建的可进行加密传输，身份认证的网络协议，比http协议安全

对称加密 (一个密钥可以加密 也可以解密 会在网络上传输 所以有一定风险)

非对称加密

1.客户端发起一个服务端请求 服务端返回一个公钥

2.客户端本地生成一个随机的密钥 用公钥进行加密传输给服务端

3.服务端接收到客户端传送的加密后的密钥 用公钥解出。

4.后续数据**客户端用公钥加密****服务器使用私钥解密****

明文+加密算法+私钥 = 密文

密文+解密算法+公钥 = 明文

HTTPS加密过程

1.客户端发送一个https请求 连接到服务器的443端口

2.服务器保存着一对 公钥 和 私钥 服务器将公钥发给客户端

3.客户端接收到公钥 进行校验，客户端本地生成一个随机数。用公钥加密 我们叫做客户端key 。发送一次http请求给服务端。

4.服务端 利用私钥进行解密 客户端key 解密之后的明文就是客户端秘钥了。然后用客户端秘钥对数据 进行对称加密 发送给客户端

5.客户端 用客户端秘钥对数据进行解密 获取到数据。

总的来说**对数据进行对称加密，对称加密所要使用的秘钥通过非对称加密传输。**

- HTTP的长连接 短连接

  http 1.0 默认使用短连接 1.1默认使用长连接

  短连接 每次打开同一个服务器的页面请求 都会创建一个链接

  长连接 会一直保持TCP连接 在一定时间断开。

  本质 HTTP协议的长连接和短连接 实质上是TCP协议的长连接和短连接

  https://www.cnblogs.com/gotodsp/p/6366163.html

  资源角度 可以多分析

- **Cookie和Session?**

  Cookie一般用来保存用户信息应用场景 比如用户注册一个账号后 保持登录章台 会将账号 密码存储到cookie里，但是为了安全起见，服务器端在首次被客户端访问的时候 生成一个Token 回馈给客户端 ，客户端将Token存储到Cookie中，下次登录直接根据Token查找用户即可。

  Session的主要作用通过 服务端 记录用户的状态

  应用场景 购物车，服务端给特定的用户创建特性的Session 可以根据标识这个用户并且跟这个用户、

  第二点 就是 Cookie数据保存在客户端 Session数据保存在服务端。

  HTTP的无状态

  因为HTTP是不保存状态的协议，Session机制解决了这个问题，Session通过服务端记录用户的状态，典型的场景就是购物车。(定时销毁)

  Session跟踪 一般在Cookie中添加一个Session ID。

  但是如果Cookied被禁用，就重写URL 在URL路径上直接添加Session ID

  http 请求报文由请求行请求头部空行和请求数据四部分。请求行：请求方法-url-http 协议版本。请求头：通知服务器有关客户端请求的信息，每行一个键值，键值用“：”分开

- **URI 和 URL**

  URI 统一资源标识符(唯一标识一个资源)

  URL统一资源定位符(提供该资源的路径)

- **DNS解析过程？**

  1.查询本地DNS

  2.查询根域名服务器 . 得到地址

  3.查询顶级域名服务器 .com 得到顶级域名服务器地址

  4.查询二级域名服务器 baidu.com 得到二级域名服务器地址

  5.查询权威服务器www.baidu.com得到ip地址

  是一个递归过程

  假设在一个以太网内 要查看某一个网站的内容

  1.先查询域名对应的ip地址(DNS服务器)

  DNS采用的运输层使用过的是UDP协议。UDP协议会将HTTP报文端 添加到UDP的首部

  目标服务器 192.168.0.2 目的端口 53 源目的ip192.168.0.1 源端口为 12345

  通过交换机，查找到DNS域名服务器。根据ip地址查找到对应的服务器。 目的端口号53的 该进程会接收到该请求，触发一次查询操作.xxx.com的ip是192.168.0.3 。DNS的该进程会将数据封装到UDP报文段中，通过以太网发送到目标服务器。这时候 目标服务器 192.168.0.1 目标端口 12345 源ip 192.168.0.2 源端口 53

  用户接收到xxx.com的ip是什么后，然后根据上面一样发送数据。其实本质上就是一个源ip+源端口 和目的ip+目的端口的转变。

  DNS 劫持：通过某些手段获得某域名的解析控制权，修改此域名的解析结果，导致对该域名的访问由原 ip 地址转入到修改后的指定 ip。其结果是对特定的网址不能访问或访问的是假网址。

思考

- **网络基础概念**

  一开始我们接触到了互联网这个概念，但是有没有想是谁在一直维护互联网呢，其实，没有一个专门的组织进行维护，刚开始高校出现，企业也接入，但是为了数据的交互和通信 大家都建立了一条通信通道，而后随着用户不断的增加过多，所以出现了中介，中移动 电信 进行铺设大量的基础设施通信设备，我们如果要接入网络只需要缴纳一定的费用，就可以享受网络的便利。

  基础设计物理结构 服务提供商也叫ISP，ISP是按照层级进行划分的，比如 在一个小区 学校 属于局域网 一个城市ISP 省级ISP 全国骨干网ISP，每个上层ISP对其下层ISP提供服务。**实现了端到端系统的数据通信。**设计到路由器 链路层交换机

  分布式应用程序角度 互联网是构建分布式应用程序的基础设施，而提供给我们的的API是什么，其实就是每个端系统的操作系统封装起来的，Socket接口，只需要填写目的ip地址+目的端口号，就可以唯一确定目的地。而具体的通信细节都被系统屏蔽了。

  分组交换网 VS 电路交换网 我们知道电话之间通信 需要在物理上建立一个真实存在的网络，而在此时只能你和对方进行通信，这条线路是不允许别人进行占用，这就是，别人打你电话号码 打不通占线中。而分组交换，很有意思，我不需要建立一个真实的线路，我复用基础服务通信路线，比如 我传递一个数据包 但是大，我可以进行划分成多个小的数据包，那么就可以进行传输了。而数据包首部 标记 目的地在哪里，也就类似于我们的快递，只需要标记发货地址和接收地址。至于中途怎么传递的，我们不需要管，快递公司会根据自己公司建立的快递网络进行传输，在这个过程中，虽然不同的数据包会分散到不同的地方，但是没事，我目的地址都是一样的，最终都会到达同一个目的。**这就是分组交换 和 电路交换。** 优缺点 就是分组交换需要等到数据到一定量之后，才会进行数据的传输，而电路交换 只要建立好连接就会传输 。

ISP对于互联网公司怎么搞？ 1.自建专属网络 阿里云 2.租用大型商用数据中心的托管服务 IDC机房。

- **应用层**

计组

- **补码**

正数原码，反码，补码相同 负数反码除了符号位不变，其他位取反，补码=反码+1；

- **编码**

字节是一种计量单位，表示数据量多少，他是计算机信息技术用于计量存储容量的一种单位。字符：计算机中使用的文字和符号。不用编码里，字符和字节对应关系不同。

- 1）ASCII中，一个英文字母（不论大小写）占一个字节，一个汉字占 2 个字节。
- 2）UTF-8，一个英文1 个字节，一个中文，3 个字节。
- 3）unicode ，中文、英文都是两个字节。

操作系统

进程 ☆

- 进程间的通信
- 进程状态
- 进程控制块-PCB
- 进程调度策略
- **进程和程序的区别？**
  - 进程 VS 程序——程序，一段代码，一组指令的有序集合。进程：程序的一次动态运行，通过进程控制块唯一的标识这个进程。进程：动态，有资源，有唯一标识，有并发性；程序：静态，无资源，无唯一标识，无并发性。

线程 ☆

- 线程同步方式
- win32 环境中，线程有 3 种线程模式，单线程（一个线程完成 all 工作，一个人搬房子），单元线程（all 线程都在主应用程序内存中各自的子段范围内运行，此模式允许多个代码实例同时单独立运行，找朋友搬房子，每个朋友在单独的房间工作，并且不能帮助在其他房间工作的人），自由线程 （多个线程可以同时调用相同的方法和组件，与单元线程不同，自由线程不会被限制在独立的内存空间，找朋友搬房子，all 朋友可以随时在任何一个房间工作）
- 同步：一个进程在执行某个请求时，若该请求需要一段时间才能返回消息，那么这个进程将会一直等下去，直到收到返回信息才继续执行下去。异步：进程不需要一直等下去，而是继续执行下面的操作，不管其他进程的状态，当有消息返回时，系统会通知进程进行处理，这样可以提高效率。

死锁 ☆

- 死锁的四个条件
  - 互斥条件：一个资源每次只能被一个进程使用，即在一段时间内某 资源仅为一个进程所占有。此时若有其他进程请求该资源，则请求进程只能等待。
  - 请求与保持条件：进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源 已被其他进程占有，此时请求进程被阻塞，但对自己已获得的资源保持不放。

    - 不可剥夺条件:进程所获得的资源在未使用完毕之前，不能被其他进程强行夺走，即只能 由获得该资源的进程自己来释放（只能是主动释放)。
    - 循环等待条件: 若干进程间形成首尾相接循环等待资源的关系
    - **这四个条件是死锁的必要条件，只要系统发生死锁，这些条件必然成立，而只要上述条件之一不满足，就不会发生死锁。**
    - 自己的理解：首先造成死锁一定是多个线程多共享资源的占有，而在占有资源的前提下，去获取对方的资源，就出现了死锁。**在并发程序中，避免了逻辑中出现复数个线程互相持有对方线程所需要的独占锁的的情况，就可以避免死锁。**
- 若系统有 5 台绘图仪，有多个进程均需要使用 2 台，规定每个进程一次仅允许申请 1台，则至多有多少个进程参与竞争而不发生死锁？答案：4 个，哲学家就餐问题，最多允许4 个哲学家同时进餐，以保证至少有一个哲学家能够进餐，最总总会释放出他们所使用过的2 只筷子，从而可以使更多的哲学家进餐。

存储器

系统级IO

- epoll/poll/select区别 epoll实现

用户态内核态

中断

虚拟内存

- 分页、分段、段页
- 页面置换算法

内存管理 ☆

fork函数

- **进程/线程的概念和区别？**
    - 进程 --> 计算机中已运行的程序，具有一定独立功能的程序关于某个数据集合上的一次运行活动，进程是系统进行**资源分配和调度的一个独立单位**
    - 线程 --> **操作系统能够进行运算调度的最小单位**，它被包含在进程之中，是进程中的实际运作单位 线程之间共享地址空间和其他资源(所以通信和同步等操作，线程比进程更加容易) ThreadLocal 线程上下文比进程上下文切换快
    - 切入点 【由来-》区别》结合java进程和线程的区别。 三个方法来回答】
        - **可以说一下线程切换比进程快的原因吗?**
            - **进程切换时**，涉及到当前进程的CPU环境的保存和新被调度运行进程的CPU环境的设置
            - **线程切换时**，仅需要设置和修改少量寄存器的内容，不涉及存储管理方面的操作。
        - **线程可以共享进程的资源和共享空间，线程可以拥有独属于自己的资源吗？**
            - 可以，通过ThreadLocal可以存储线程的特有对象，也属于当前线程的对象，
        - **多线程和单线程的关系可以说一下吗？**
            -
    - 关系
        - 进程和线程都是一个时间段的描述，是CPU工作时间段的描述，不过是颗粒大小不同
        - 一个程序至少有一个进程,一个进程至少有一个线程

    - 腾讯面试题04.进程和线程的区别？
    - 进程与线程的一个简单解释
    - 线程和进程的区别是什么？
    - 线程 Wiki
    - 进程 Wiki
- **线程同步（通信）的方式（字节跳动）**
    - Inter-thread Communication in Java
- **进程间通信的方式**
    - 进程间通信 --> 在不同进程之间传播或交换信息
        - 管道（数据只能单向流动）
        - 系统IPC（InterProcess Communication）(包括消息队列, 信号量, 共享存储)
        - Socket（可用于不同机器间的进程通信）
    - 进程间的几种通信方式
    - 进程间8种通信方式详解
- **描述操作系统的启动过程（字节跳动）**
    - 步骤
        - BIOS（Basic Input / Output System）
        - 主引导记录（Master boot record）
        - 硬盘启动
        - 操作系统
    - 计算机是如何启动的？
- **用一门编程语言（如Java）实现一个死锁（PayPal）**
    - 死锁产生的条件
        - 禁止抢占 no preemption --> 系统资源不能被强制从一个进程中退出
        - 持有和等待 hold and wait --> 一个进程可以在等待时持有系统资源
        - 互斥 mutual exclusion --> 只有一个进程能持有一个资源
        - 循环等待 circular waiting --> 一系列进程互相持有其他进程所需要的资源
    - 死锁 Wiki
    - 第18讲 | 什么情况下Java程序会产生死锁？如何定位、修复？（死锁代码实例）
    - JAVA实现的一个简单的死锁（附解释）
- **如何判断内存泄漏（腾讯）**
    - 因为长生命周期持有它的引用而导致不能被回收，这就是Java中内存泄漏的发生场景
    - Java内存泄漏分析和解决
- **什么是系统调用（爱奇艺）**
    - 指运行在用户空间的程序向操作系统内核请求需要更高权限运行的服务
    - linux内核中设置了一组用于实现系统功能的子程序，称为系统调用。系统调用和普通库函数调用非常相似，只是系统调用由操作系统核心提供，运行于**内核态**，而普通的函数调用由函数库或用户自己提供，运行于**用户态**
    - 系统调用 Wiki
    - Linux系统调用详解（实现机制分析）--linux内核剖析（六）
    - 用户态和内核态的区别
- **select 和 epoll 的区别** `TODO`
    - select和epoll区别

- **CPU时间片调度有哪些方式？**

Linux

- 1.整机性能 top **uptime 内存 free -m 磁盘 df -h** cpu **vmstat** -n 2 3 磁盘 io iostat -xdk 2 3
- top，load 指令，free 中 cached和buffers的区别（阿里）
- 找出某目录下文件中带电子邮箱的文件（爱奇艺）
- 杀死所有Java进程
  - `ps -ef | grep java | grep -v grep | awk '{print $2}' | xargs kill -9`
  - Linux 杀掉所有Java进程
- 查看进程信息 `ps -ef | grep tomcat` 杀死 `kill pid`
- 查看全局文件 `find`
- 查看tomcat日志文件
  - `tail -fn catalin` `n`末尾100行
- 查看当前系统是多少位
  - `getconf LONG_BIT`
- `grep`
  - `# grep "qxlxi" file.log` 查看file.log 中是否包含qxlxi字符串
  - 统计出现次数
    - `grep "qxlxi" file.log -c`
    - `grep "qxlxi" file.log | wc -l`
- `awk` `todo`
- `sed` 利用脚本来处理文本文件，sed可以依照脚本的指令来处理，编辑文本文件
  - `sed -n '2,3' file.log` 裁剪2、3行数据
- `sort` 对文件进行排序
  - 正序排序
    - `sort -n test.txt`
  - 反序排序
    - `sort -nr test.txt`
- tar
  - -s 还原文件的顺序和备份文件内的存放顺序相同。
  - -t 列出备份文件的内容。
  - -v 显示指令执行过程。
  - -f 指定压缩文件
  - -x 从备份文件中还原文件。

1.linux查看文件内容的命令

1.cat从第一行开始显示内容，并将所有内容输出

2.tac 从最后一行开始显示内容，并将所有内容输出

3.head 只显示头几行

4.tial 只显示后几行

5.nl 和 cat一样 要显示行号

2.linux常用命令

cd 打开文件夹

cd . 后退

mkdir 创建目录

rm 删除文件

head tial 显示文件头尾

hostname 显示主机名

ipconfig 查看网络情况

ping 测试网络连通

netstat 显示网络状态信息

clear 清屏

pwd 现在当前路径

useradd 用户名 添加用户

who am i 当前用户时谁

ls -a 显示目录下的所有文件，包括隐藏文件

ls -l 显示长列表格式

rmdir 删除空目录

touch 建立空文件

cp 复制命令

top 进程资源占用情况

df 磁盘使用情况

关机命令

shutdown -h now 立即进行关机

shutdown -r now 现在重新启动计算机

reboot 现在重新启动计算机

进入桌面

startx

用户注销

logout

vim

输入i 进入插入模式

输入esc键 进入命令模式

输入 :wq 表示退出保存

:q! 退出不保存

linux主要目录结构

root 存放root用户的相关文件

home 存放普通用户的相关文件

bin 存放常用命令的目录，如vi su

sbin 要具有一定权限才可以使用命令

mnt 挂载光驱

etc 配置文件

var 变化的文件

boot 引导系统启动的相关文件

usr 软件的默认目录

3.显示最近登录的5个用户名

```
[root@localhost test]# last -n 5
[root@localhost test]# last -n 5 | awk '{print $ 1}'  //如  果只显示最近登录的5个账号
```

```
[root@localhost test]# ps -ef | grep java  查看进程
```

4.生产环境服务器变慢，诊断思路和性能评估谈谈?

top 整机

vmstat cpu

free 内存

df 硬盘

iostat 磁盘io

ifstat 网络io

5.假如生产环境出现CPU占用过高，请谈谈你的分析思路和定位

1.先用top命令找出cpu占比最高的。

2.ps -ef或者jps进一步定位，得知是一个怎么样的一个后台程序

3.定位到具体线程或者代码 ps -mp 进程 -o THREAD,tid,time

4.将需要的线程ID转换为16进制格式(英文小写格式) printf "%x\n" 有问题的线程ID

5.printf "%x\n" 有问题的线程ID

6.查找特定文件

```
find ~ -name "xxx" 精确查找文件
find ~ -name "target*" 查找所有target开头的文件
find ~ -iname "target*" 查找所有target开头的文件 忽略大小写
```

7.检索文件内容

```
grep "xx" target*   检索包含xx的文本
```

8.awk

```
语法  awk [options ] 'cmd'  file
一次读取一行文本 按输入分隔符进行切片，切成多个组成部分
将切片直接保存在内建的变量中，$1 $2 $0 表示行的全部
支持对单个切片的判断，支持循环判断，默认分隔符为空格
awk '{print $1,$4}' netstat
```

1、绝对路径用什么符号表示？当前目录、上层目录用什么表示？主目录用什么表示？切换目录用什么命令？

绝对路径：如/etc/init.d当前目录和上层目录：/./

主目录：~/

切换目录：cd

2、怎么查看当前进程？怎么执行退出？怎么查看当前路径？

查看当前进程：ps

执行退出：exit

查看当前路径：pwd

3、怎么清屏？怎么退出当前命令？怎么执行睡眠？怎么查看当前用户id？查看指定帮助用什么命令？

答案：

清屏：clear

退出当前命令：ctrl+c彻底退出

执行睡眠：ctrl+z挂起当前进程fg 恢复后台查看当前用户id:“id”：查看显示目前登陆账户的uid 和gid及所属分组及用户名查看指定帮助：如man adduser这个很全而且有例子；adduser-help这个告诉你一些常用参数；info adduesr；

4、Ls命令执行什么功能？可以带哪些参数，有什么区别？

答案：

ls执行的功能：列出指定目录中的目录，以及文件哪些参数以及区别：a所有文件l详细信息，包括大小字节数，可读可写可执行的权限等

5、查看文件有哪些命令

vi文件名#编辑方式查看，可修改

cat文件名#显示全部文件内容

more文件名#分页显示文件内容

less文件名#与more相似，更好的是可以往前翻页tail文件名#仅查看尾部，还可以指定行数head文件名#仅查看头部，还可以指定行数

6、列举几个常用的Linux命令

列出文件列表 ls [-al]

创建目录 移除目录 mkdir rmdir

用于显示文件后几行内容：tail，例如：tai l-n 1000：显示最后1000行

打包：tar -xvf

打包并压缩：tar-zcvf

查找字符串：grep

显示当前所在目录：pwd

创建空文件：touch

编辑器：vim vi

7、你平时是怎么查看日志的？

Linux查看日志的命令有多种：tail、cat、tac、head、echo等，本文只介绍几种常用的方法。

1、tail最常用的一种查看方式

命令格式：tail[必要参数[选择参[文件]

-f 循环读取

-q 不显示处理信息

v 显示详细的处理信息

-c <数目>显示的字节数

-n <行数>显示行数

-q，-quiet，-silent从不输出给出文件名的首部

-s，-sleep-interval=S与f合用，表示在每次反复的间隔休眠S秒例如：

```
tail-n 10 test.1og 查询日志尾部最后10行的日志；
tail-n+10 test.1og 查询10行之后的所有目志
tail-fn 10 test.1og 循环实时查看最后1000行记录（最常用的）
```

一般还会配合着grep搜索用，例如：

```
tail-fn 1000 test.1og I grep'关键字’
```

如果一次性查询的数据量太大，可以进行翻页查看，例如：

```
tail-n4700 aa.1og lmore-1000可以进行多屏显示（ctrl+f或者空格键可以快捷键）
```

8、利用ps怎么显示所有的进程？怎么利用ps查看指定进程的信息？

ps-ef（system v输出）

ps-aux bsd 格式输出

ps-ef l grep pid

9、查看当前谁在使用该主机用什么命令？查找自己所在的终端信息用什么命令？

查找自己所在的终端信息：who ami

查看当前谁在使用该主机：who

10.使用什么命令查看磁盘使用空间？空闲空间呢？

df -h1

11.查看各类环境变量用什么命令

```
[root@localhost logs]# env
[root@localhost logs]# env $java  某个具体的
```

文件操作

- `cat` 从第一行开始显示内容，并将所有内容输出。
- `Tac` 从最后一行开始显示内容，并将所有内容输出。
- `nl` 显示行号
- `tail 和head`
  - `tail -n 2 file.log`可以查看文件的最后2行
  - `tail -f file.log` 可以实时查看文件的后边追加的部分
  - `head -n 2 file.log` 可以查看文件的开始2行
- `top`和`ps`
  - ps命令 默认只显示运行在当前控制台下的属于当前用户的进程
    - `ps -A`和`ps -e` 可以显示所有进程
    - `ps -ef` 显示完整格式的所有进程
    - 指定进程名 `ps -ef | grep "java"` 找出进程中包括java的所有进程
  - **top 可以实时检测进程** ☆
    - 第一行显示了当前时间，系统的运行时间，登录的用户数和系统的平均负载(1min,5min,15min)
    - 第二行显示了进程的概要信息，有多少进程处于运行，休眠，停止或者僵化状态
    - 第三行是cpu的概要信息
    - 第四行是系统内存的状态
  - **可以说一下top和ps 两者区别嘛？**
    - ps看到的是命令执行瞬间的进程信息，而top可以持续的监视
    - ps只是查看进程，而top还可以监视系统性能，如平均负载，cpu和内存的消耗
    - top可以操作进程，如改变优先级(r)和关闭进程(k)
    - ps主要是查看进程的，关注点是查看需要查看的进程
    - top主要看cpu，内存使用情况，及占用资源最多的进程由高到低排序，关注点在于资源占用情况

- ```
  # service network status //查看网络状态
  # service network stop //停止
  # service network restart //重启
  # service network reload  //重新加载
  # service network start //启动
  通过chkconfig命令设置自启动
  # 查看服务 chkconfig --list | grep xxx
  # chkconfig --level 5 服务名  on
   vim /etc/sysconfig/network-scripts/ifcfg-eth0
  ```

问题迭代

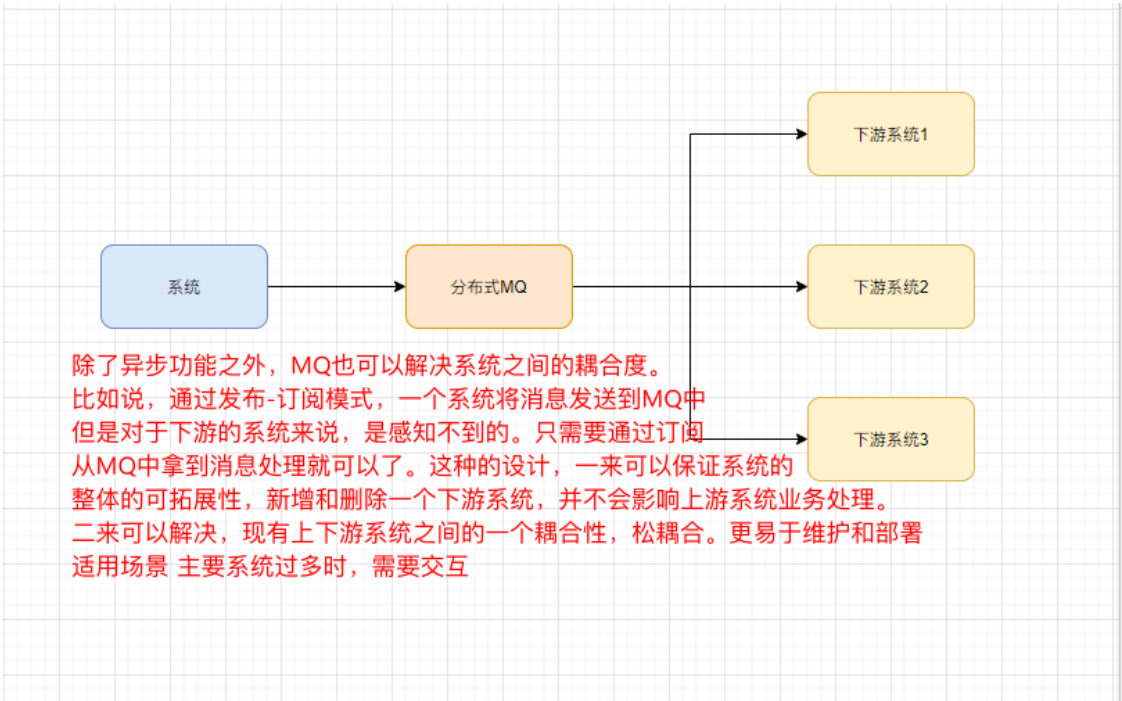
- linux 32 位 os 下，一个应用程序最多分配和访问的内存大小？ 答案：3G，32 位可以映射为 4G，有 2G 的用户模式虚拟地址空间位于 4G 地址空间的一半，而与之相应的另一半 2g地址空间由 os 内容使用。

软件工程

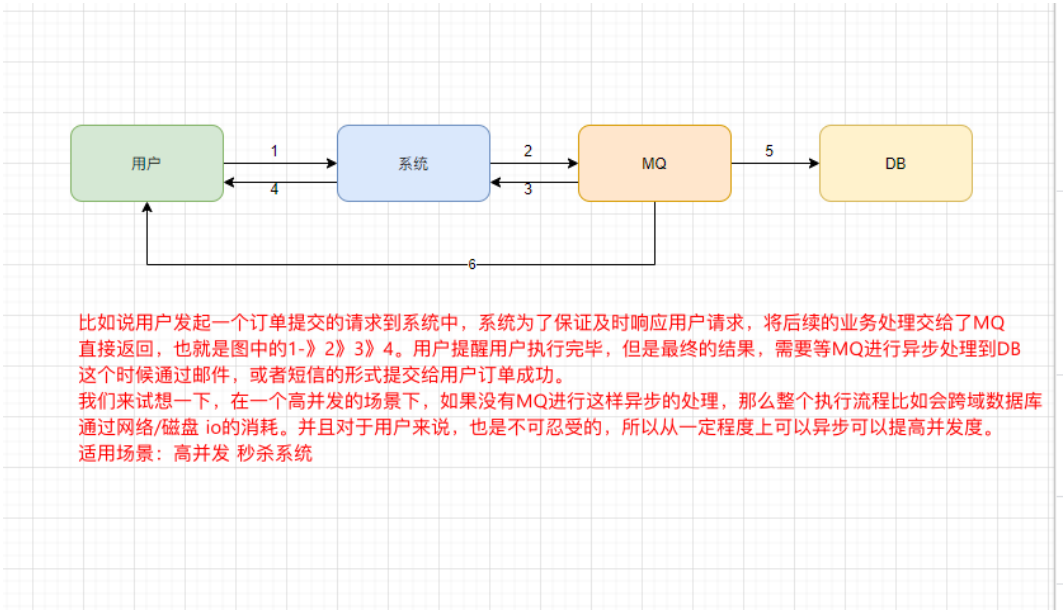
0x05-中间件

MQ

1.为什么使用MQ

- 削峰
- 解耦
- 

- 异步
  ```



比如说用户发起一个订单提交的请求到系统中，系统为了保证及时响应用户请求，将后续的业务处理交给了MQ直接返回，也就是图中的1-》2》3》4。用户提醒用户执行完毕，但是最终的结果，需要等MQ进行异步处理到DB这个时候通过邮件，或者短信的形式提交给用户订单成功。
我们来试想一下，在一个高并发的场景下，如果没有MQ进行这样异步的处理，那么整个执行流程比如会跨域数据库通过网络/磁盘 io的消耗。并且对于用户来说，也是不可忍受的，所以从一定程度上可以异步可以提高并发度。
适用场景：高并发 秒杀系统

2.MQ的优点和缺点

1.系统可用行降低 MQ宕机 2.系统复杂性提高 (重复消费、消息丢失、消息无顺序、消息堆积) 3.一致性问题

- 引入一个组件，系统就会带来一定的问题，我们分别来说一下，第一点的话，就是系统可用行降低，MQ单点故障是难以避免的，一旦MQ单点故障之后，整个系统就会崩溃。这是不可忍受的。可以搭建集群来配置使用。第二点的话，系统的整体复杂度提高了，消息的丢失问题(三个维度) 消息的重复消费 消息的顺序性。第三点的话，数据的不一致性。

3.四大MQ的有优缺点。

4.可用行

- RaibbitMQ
  - 1.单机模式
  - 2.普通集群模式
  - 3.镜像集群模式
- Kafka分布式模式
  - 1.数据分区存储，leader.follwer保证数据的可用行。

5.MQ重复消费问题

- 1.MQ重复消费问题。
  - 同一条数据 因为宕机等原因，造成数据会插入多次。
- 2.幂等性问题
  - 一个数据 重复多次，可以保证数据不会改变。
- 3.怎么解决幂等性问题
  - 1).redis
  - 2).全局唯一id
  - 3).数据库唯一主键保证 结合业务谈

6.消息数据丢失

- 1.生产者方数据丢失
- 2.mq数据丢失

- 3.消费者数据丢失(3个维度展开讲)
- **消息丢失三个方面来说 同步落盘**
  - 客户端 把自动提交ack改为手动提交
  - kafka端 收到消息并持久化，并且将消息同步到其他的备份节点，才算消息到达。
  - 生产者 提交给kafka时，会收到kafka端的成功回调 没有收到就一直发。
- **如果消息重复了，会做幂等**

7.消息数据顺序消费

-

8.消息积压

1.紧急扩容 .....

9.设计一个架构

- 1.可伸缩性
- 2.持久化消息数据
- 3.可用性
- 4.数据丢失问题

10.应用场景

- 分布式事务

ZK

- **ZK是什么？**
  - ZooKeeper 是一个开源的分布式应用程序协调服务，是一个典型的**分布式数据一致性解决方案**。设计目的是将那些复杂且容易出错的分布式一致性服务封装起来，构成一个高效可靠的系统，并以一系列简单易用的原子操作提供给用户使用。
- **ZK提供了那些功能？**
  - **分布式服务注册与订阅**：在分布式环境中，为了保证高可用性，通常同一个应用或同一个服务的提供方都会部署多份，达到对等服务。而消费者就须要在这些对等的服务器中选择一个来执行相关的业务逻辑，比较典型的服务注册与订阅，如 Dubbo。
  - **分布式配置中心**：发布与订阅模型，即所谓的配置中心，顾名思义就是发布者将数据发布到 ZooKeeper 节点上，供订阅者获取数据，实现配置信息的集中式管理和动态更新。
  - **命名服务**：在分布式系统中，通过命名服务客户端应用能够根据指定名字来获取资源、服务地址和提供者等信息。
  - **分布式锁**：这个主要得益于 ZooKeeper 为我们保证了数据的强一致性。
- **ZK有几种搭建方式？**
  - **单机模式**：zoo.cfg 中只配置一个 server.id 就是单机模式了，此模式一般用在测试环境，如果当前主机宕机，那么所有依赖于当前 ZooKeeper 服务工作的其他服务器都不能进行正常工作；
  - **伪分布式模式**：在一台机器启动不同端口的 ZooKeeper，配置到 zoo.cfg 中，和单机模式相同，此模式一般用在测试环境；
  - **分布式模式**：多台机器各自配置 zoo.cfg 文件，将各自互相加入服务器列表，上面搭建的集群就是这种完全分布式。
- **ZK有哪些特性？**

    - **顺序一致性（Sequential Consistency）**：来自相同客户端提交的事务，ZooKeeper 将严格按照其提交顺序依次执行；
    - **原子性（Atomicity)**：于 ZooKeeper 集群中提交事务，事务将“全部完成”或“全部未完成”，不存在“部分完成”；
    - **单一系统镜像**（Single System Image）：客户端连接到 ZooKeeper 集群的任意节点，其获得的数据视图都是相同的；
    - **可靠性**（Reliability）：事务一旦完成，其产生的状态变化将永久保留，直到其他事务进行覆盖；
    - **实时性**（Timeliness）：事务一旦完成，客户端将于限定的时间段内，获得最新的数据。
- **ZK相关组件可以说一下嘛？**
    - **ZNode节点**

      Znode维护了一个stat结构，这个stat包含**数据变化的版本号**、访问控制列表变化、还有时间戳。版本号和时间戳一起，可让Zookeeper验证缓存和协调更新。**每次znode的数据发生了变化，版本号就增加**。

      例如，无论何时客户端检索数据，它也一起检索数据的版本号。并且当客户端执行更新或删除时，客户端必须提供他正在改变的znode的版本号。如果它提供的版本号和真实的数据版本号不一致，更新将会失败。 类似版本号更新操作一样。 CAS
    - **ZNode = Path+data+Stat**
    - **节点可以说一下分类吗？**
        - PERSISTENT-**持久化目录节点** (客户端与zookeeper断开连接后，该节点依旧存在)
        - PERSISTENT_SEQUENTIAL-持久化顺序编号目录节点 (客户端与zookeeper断开连接后，该节点依旧存在，只是Zookeeper给该节点名称进行顺序编号)
        - **EPHEMERAL-临时目录节点** (客户端与zookeeper断开连接后，该节点被删除)
        - EPHEMERAL_SEQUENTIAL-临时顺序编号目录节点 (客户端与zookeeper断开连接后，该节点被删除，只是Zookeeper给该节点名称进行顺序编号)
        - 注意**不能基于临时节点创建子节点，临时节点只能是叶子节点。**
- **ZK的应用场景**
    - 1.分布式协调 A-》MQ-》B A通过注册到ZK上一个节点监听节点的变化。
    - 2.zk实现分布式锁 节点临时和唯一性 以及监听节点的数据变化。
    - 3.配置信息管理 dubbo
    - 4.高可用 主备节点直接感知通过zk实现
    - 5.DNS服务
    - 不适用于大数据量的存储，适用于存储和协同相关的关键数据
    - **数据发布、订阅**
        - 发布者将数据存储在ZNode节点上，订阅者通过Watch机制进行数据订阅，达到动态获取的目的。比如说，我们系统中的配置文件，可以动态维护。 配置存储->配置获取+watch->配置变更
    - **负载均衡**
    - **命名服务**
    - **分布式协调/通知**
        - 分布式系统机器间通信方式
            - 心跳检测 基于ZK临时节点来判断机器是否存活
            - 工作进度汇报
            - 系统调度
    - **Master选举**
- **分布式协调服务原理**

- **读写机制**
- **高可用**
- **leader选举机制** -源码分析
- **崩溃恢复**
- **持久化机制** -源码分析
- **Watch机制**
  - 组件部分：客户端线程，客户端WatchManager，ZK服务器
  - 机制：首先ZK Server启动，创建一个`/zklock`节点，客户端监听这个节点的变化，从watcherManager中获取变化，等节点发生变化时，zk通知客户端 会发送一个状态，类型，节点路径，而具体的信息还是需要客户端发起一个请求到ZK中。说白一点就是客户端无法直接从事件中直接获取对应数据节点的原始数据以及变更后的数据内容，需要客户端再次发送一个请求到ZK中获取。**客户端注册Watcher 服务端处理Watcher 客户端回调Watcher**
- **ZK实现分布式锁？**
  - **为什么需要分布式锁？**
    - 分布式系统之间同步访问共享资源的一种方式，通过互斥手段，来保证数据一致性。
  - 客户端连接 ZooKeeper，并在 /lock 下创建**临时的且有序的子节点**，第一个客户端对应的子节点为 /lock/lock-10000000001，第二个为 /lock/lock-10000000002，以此类推。
  - 客户端获取 /lock 下的子节点列表，判断自己创建的子节点是否为当前子节点列表中序号最小的子节点，如果是则认为获得锁，否则Watch机制监听刚好在自己之前一位的子节点删除消息，获得子节点变更通知后重复此步骤直至获得锁；
  - 执行业务代码；
  - 完成业务流程后，删除对应的子节点释放锁。
  - 相关cutor
  - **对于ZooKeeper的羊群效应，分布式锁实现应该如何优化？**
    - 什么是羊群效应？
      - 简单说，当一个客户端创建成功节点，当临时节点被删除后，会通知被监听的客户端，如果客户端过多，会导致多个客户端争抢锁(创建节点)，造成资源的浪费，网络的开销。
    - 如何解决：创建一个临时顺序节点。按照顺序进行进行监听。
  - **如果遇到ZooKeeper脑裂问题，分布式锁应该如何保证健壮性？**
    - 什么是脑裂？
      - 分布式系统中，有一个Master节点，因为网络不可连接，其他节点误以为这个节点不可用，就会选举一个新的Master节点，集群中会出现两个Master节点。
    - 解决方案：
- **ZK实现分布式事务？**
  - ZooKeeper 实现分布式事务，类似于两阶段提交，总共分为以下 4 步：
    - 客户端先给 ZooKeeper 节点发送写请求；
    - ZooKeeper 节点将写请求转发给 Leader 节点，Leader 广播给集群要求投票，等待确认；
    - Leader 收到确认，统计投票，票数过半则提交事务；
    - 事务提交成功后，ZooKeeper 节点告知客户端。
- **集群中为什么要主节点的存在呢？**
  - 在分布式环境中，有些业务逻辑只需要集群中的某一台机器进行执行，其他的机器可以共享这个结果，这样可以大大**减少重复计算，提高性能**，这就是主节点存在的意义。

-
- **8.回调的线程机制**

Paxos

- **解决的问题点**
  - 在分布式系统中，如何就某个值达成一致。基于消息传递模式的一致性算法。
  - 应用场景：
    - 分布式数据库中副本之间读写请求的一致性
  - **节点通信：**
    - 共享内存
    - 消息传递
- 可以类比投票选举一个董事长。生活中，我们会通过在董事会进行选举投票。有一个投票箱，当结束之后，统计票数最多的人就当选为董事长。

  问题一：我们应该何时停止投票，开始投票。结束的条件是什么？

  方案1：我们可以设定总人数比如10人。但是也存在一个问题，那就是无法预知，如果有一个人出现了情况，而来不了，我们应该怎么解决呢。导致在一段较长的时间内没有办法选举出一个董事长。

  a.没有投票，就没有帮主 b.有投票并产生了一个帮主，帮主只能有一个 c.其他人获取帮主只能是选举出来的帮主。我们知道在分布式系统中，

  进一步推演，我们出一种策略，那就是当有一半以上的票数得票人，就可以当选董事长。因为另外一部分绝不可能大于一半。允许投票进程网络出现问题或者宕机。但是，很多时候，我们无法获知总人数是多少。

  在进一步推演， 我们假设不限定投票人，也不想必事先知道投票人的信息。那么通常生活中的网络投票就是这样的策略，通过一个限定时间来决定，最终人数较多者就是老大。但是新的问题会出现，如果在一定的时间内没有出现投票，或者投票数多的人是多个比如A是10票 B也是10票 怎么办？

  这个时候，我们想到可以用加时赛，但是如果一直不出现，那就没有办法解决。

  算法： 一种解决问题的思路和步骤、基于我们提出的问题，我们是无法解决问题，而如果一个算法不能解决实际问题，那这个算法就是不可行。

  针对于上面，我们是无法解决的。因此，我们提出一个思路，那就是我们当只要出现第一个投票，那么这个人就是帮主。

  小结：我们可以看到，我们有两种方法，一种是基于第一个就是帮主，如果一直没有出现第一个，那么，我们就会无限期等待，第二种 设定期限，

  Paxos最大用处就是分布式锁，大量的客户端请求锁服务器，那么第一个到达之后，就直接获取锁。

  加餐一 Paxos第一个前置条件就是，必须有一个投票行为来确定帮主的行为。为了解决投票箱单点问题，我们必须加入多个投票箱进行高可用。因此Paxos的前提必定是有一个确定的数。算法知道投票箱的个数，不知道投票人数量，假设 是5个投票箱。

  基于5 个投票箱的问题，正常情况下，我们会接收到5个投票，只要出现大于一半的票数的人，就可以当选帮助，但是有可能A去访问得知帮助是乔峰 B去获取的的时候，有一个节点 投票箱挂掉了，那么 本来是 乔峰3票，全关青 2票。 乔峰有一个投票箱节点宕机了 都是2票 无法获取。但是可以确定的一点是，只要选出了一个帮主，那么就必定不会出现第二个唯一值。要么是拿到确定的帮主，要么是不知道。

  所以**该算法有一个重要的规定，重启任何已经死掉的投票线程，必须能够记住此前已经获取的投票信息。**

问题 a.可能没有任何一张选票过半的，这样算法无法得到结果。b.也可能中间获取过了结果，但是投票箱节点宕机了。

Paxos论证大幕的第一幕

基本构成：proposer,acceptor learner

先来说一下proposer 称为投票人，proposer会提出一个提案(相当于选举哪一个人，投哪个票) 该提案有一个value值，value对应的就是人名。

acceptor 投票箱 实际上是一个集群，也可以说是一个acceptor数组，acceptor接收proposer的提案进行批准。在之前我们推导的,只会接收到第一个提案进行批准(接受) 。

learner 对应之前提到的路人，这些路人是没有权利投票。但是有权利知道帮主是谁。

为什么交learner?

- 因为帮主是谁不能直接拿到，需要learner向acceptor获取。询问所有的learner。是一个过程。

其中，难免会有一些即使proposer 也是acceport 。

任何一个acceptor投票箱，都可以批准不止一张的选票。

这其实是颠覆了我们之前的认识，之前只允许一个投票箱接收一张投票，然后选择出过半的票数就可以。那么当acceptor投票箱能接收到的选票不止一张的时候，我们就需要有一种策略进行分配。因为会出现多个过半的票数情况。比如说learn1学习acceptor集群中获取到鲁肃是帮主，learn2在学习的过程中 acceptor中突然宕机了几台，导致accepotr选择的帮主是乔峰，那么learn1和laren2之间就会出现数据的不止一次问题。唯一性和一致性就有问题。既然有问题，那么我们看paxos算法是如何解决的？

paxos算法中，采用唯一递增的选票id,对每个投票人的投出的选票进行编号。

比如每个投票人都需要从一个中间件自动生成id 投票时获取一个全局id 进行选票比如1 乔峰 2.全关青

paxos算法：

P1：每个acceptor必须接受他们收到的第一张选票。

为什么这么说呢，因为当只有一个proposer的时候，如果acceptor集群不能而接受第一个投票的人，那么就一直无法选择出帮主。但是基于p1来说也是无法保证可以选择出帮主，因为必须接受第一章选票，因为网络故障 id=2 先完成了半数票的选举 而之后是id=1 半数票的选举，那么多张不同的人命的选票同时被批准。一致性无从谈起

p2:当某个编号的选票被选定(这里选定的意思，就是过半批准) 那么任何大于该编号且被选定的选票都和该编号被选定的选票 都有一样的value值

这是什么意思呢，比如3号选票被选定了，那么之后的4,5号选票的选票一定是拥有和3号一样的票数。

如果满足p1和p2 就可以解决问题了。

我们来重写审视一下p2这个理论，当3号被第一次选定，那么p2只要求在之后的必须票数一样，但是之前的没有限定呀，也就是2号选票在3之后，那么就出现了两个帮主 你如何解决不一致问题。

所以这就意味着在比3号小的都直接拒接。保证3号是最小的且第一个票数过半的人。

P2A：当某个编号的选票被选定(这里选定的意思，就是过半批准)，那么，任何大于该编号且被接收的选票，

都和该编号被选定的选票，都有一样的 value 值

P2和p2A 只是一词之差，P2强调的是选定的票(过半接收的意思) 而接收 只是被某个接收箱接收。

因为p2要求，所以大于该编号的被选定的选票，都和该编号有一样的值，而p2A是保证了所有大于该编号的被接收的选票，都和该编号的有一样的值。 从某种意义上来说P2A降低了P2的实现难度

但是P2A也是比较难实现的，因为当我们选定了3号，而之前4号接收了1票 无法实现在之后将4号的票数变成半数。

P2B：当某个编号的选票被选定(这里选定的意思，就是过半批准)，那么，任何大于该编号且被投票人投出 的选票，都和该编号被选定的选票，都有一样的 value 值

但是P2B也是有问题的，也就是说，比如从ID生成器中获取到8号投票，去accepotr查询的时候 没有选定，但是当投递选票的时候，有一个窗口期，选定了一个id为7的，这就出现了问题。所以应该选定什么值，这就违背了P2B。因此只要推导出P2C可以推导出P2B即可

P2B是无法保证通过查询可以正确获取到被选定的编号。

- **分布式事务 和 分布式一致性**
  - **分布式一致性**，主要在于集群中，单点问题一直是分布式中不可避免的问题。所以集群在一定程度上可以保证高可用，但是集群中一定有Master slave节点，而Master节点的写数据需要实时同步到slave节点中，会引发一个数据同步。也就是在某一时间内，对于客户端A来说读取 slave A节点和slave B节点的数据必须是一致的，不能出现两义性。分布式数据一致性就无法保证。 **重点在于数据同步中数据一致性，是一种数据层面的一致性。**
  - **分布式事务** 分布式事务更多的是业务上的一致性，通常 我们一个下订单操作，可能会涉及到减少库存，订单生成，余额扣减 但是我们必须保证这些互相关联的操作，必须处于一个原子性。保证他们数据业务上的一致性。 **重点在于数据业务上的一致性。**
- **问题描述**
  - **在这些被提出的提案中，只有一个会被选定** 唯一确定性
  - **如果没有提案被提出，那么就不会有被选定的结果**
  - **当一个提案被选定后，进程应该可以获取被选定的提案信息**
- 对于一致性来说，
  - 只有被提出的提案才能被选定
  - 只能有一个值被选定
  - **如果某个进程认为某个提案被选定了，那么这个提案必须是真的被选定的那个**

Netty

思考三个问题

1.为什么需要Netty

2.Netty的出现解决了那些现有技术点的痛点

3.如何理解Netty的技术架构和业务抽象设计架构 相关的组件

- **链式编程**

相关资料

- 图解tcp/ip wireshark网络分析
- java网络编程 java tcp/ip socket编程
- netty
  - netty权威指南
  - netty实战
  - netty进阶之路 跟着案例学netty
- **netty是什么**
  - 本质 ： 网络应用程序框架
  - 实现：异步、事件驱动
  - 特性：高性能 可维护 快速开发
  - 用途：开发服务器和客户端

- **为什么会使用netty**
    - 支持常用应用层协议
    - 解决传输问题:粘包、半包现象
    - 支持流量整形
    - 完善的断连 Idle等异常处理等
- **Reactor模式**
    - **是什么？**
        - 模式的核心流程
            - 扫描感兴趣的事件->扫描是否有感兴趣的事件发生->事件发生后作出相应的处理
        - Channel做的事情
            - Client
                - SocketChannel : **Connect**、**Write**、**Read**
            - Server
                - ServerSocketChannel : **Accept**
                - SocketChannel : **Write**、**Read**
    - **Reactor单线程** 一人做饭 接待 包揽所有
        - 客户端发送请求到服务端，Reactor单线程的按照客户端的请求类型 如果是连接请求就创建连接，之后创建handler进行后续的业务处理，否则不是连接直接转发到handler进行业务处理，说到底就是一个人在做事。
    - **Reactor多线程** 一个前台，多个厨师做饭
        - 在前面是一个线程做，而这块Reactor只负责连接的建立和将请求转发到handler中，剩余创建一个新的线程执行。从某种程序上来说，是实现了多线程的处理的。
    - **主从Reactor多线程模式** 多个前台处理请求，多个厨师处理业务
        - 为了解决单点Reactor的问题，所以需要多个Reactor进行处理程序，比如程序中，我们有一个主Reactor负责连接的创建，而剩余的就转发到具体的subReactor中去处理，subReator按照对应的请求执行Handler请求。具体的业务处理。当然，也可能一个主Reactor对应多个子subReactor。而当subReactor执行完毕，直接返回给客户端，而无需通知主Reactor。
    - **Netty模式可以说一下吗？**
        - Netty模型中有两个Reactor，一个BossGroup和一个WorkGroup。前者主要进行处理连接请求，后者处理相关业务处理。BossGroup和WorkGroup都是一个NIOEventLoopGroup，而一个NIOEventLoopGroup对应多个NIOEventLoop,BossGroup中NIOEventLoop都是一个事件轮询处理线程，处理来自客户端的请求，每个NIOEventLoop都有一个Selector，监听绑定在其上的socket通信。轮询acctpt事件，处理accept事件，与client建立连接，生成NIOSocketChannel,将其注册到Work中 NIOEventLoop中，执行write/read相关事件。每个work NIOEventLoop处理业务时，会使用pipeline包含了channel，既通过pipeline可以获取到对应通道，管道中维护了很多的处理器。
    - 相关题目题目？
        - **Netty如何支持主从Reactor模式的?**
            - 两种Channel绑定到两个不同的Group模式上。
        - **为什么说Netty的main Reactor大多并不能用到一个线程组，只能线程组里的一个？**
        - **Netty给Channel分配NIO Event Loop的规则是什么?**
            - `GenericEvent` 轮询
            - `PowerOfTwoEvent` 2的幂次方 进行&运算 效率高效
        - **通用模式的NIO实现多路复用器是怎么跨平台的？**

        - `WindowsSelectorProvider` 不同的平台不同的实现类
- 三种模式
    - BIO NIO AIO
- **netty设计到的那些设计模式？**
- **ByteBuf的理解？**
- **粘包 半包问题** `todo`
    - **什么是粘包 半包问题？**
        - 客户端发送一个AB CD 可能服务端接收到的数据可能有三种情况
        - 第一种情况就是 AB CD 这是属于正常的数据包，没有问题。
        - 第二种情况 ABCD 出现了粘包问题 AB数据包粘了后续的数据包
        - 第三种情况ABC D第一个数据包接收到不是完整的数据包。半包问题。
    - **为什么TCP应用中会出现粘包 半包问题现象？**
        - **粘包的主要原因：**
            - 发送方每次写入数据<套接字 缓冲区大小 希望多发送数据给接收方 但是两个数据包连接在一块了
            - 接收方读取套接字缓冲区数据不够及时。 接收方读取数据时，没有及时拿走缓冲区中的数据，导致下一次读取时，可能读取到两个数据包。
        - **半包的主要原因**
            - 发送方写入数据 > 套接字缓冲区大小
            - 发送的数据大于协议的MTU(最大传输单元) 必须拆包
        - 换个角度看
            - 收发 一个发送可能被多次接收，多个发送可能被一次接收
            - 传输 一个发送可能占用了多个传输包 多个发送可能公用一个传输包
        - 根本原因
            - TCP是流式协议，消息无边界
    - **几种常见解决办法？**
        - TCP连接改成短连接，一个请求一个短连接，建立一次连接就传输一次数据，但是缺点就是消耗连接资源，效率低下。
        - 封装成帧
            - 固定长度 每次传输固定的长度，空间浪费 不推荐
            - 分隔符 不浪费空间，但是需要特殊字符需要进行转移 推荐
            - 固定长度字段存个内容的长度信息 先解析出固定长度的字段获取长度，然后读取后续内容，
            - 其他格式 JSON
    - **Netty对常用封帧方式的支持？**
        - 具体看脑图
    - **解读Netty处理粘包、半包源码**
- **Keepalive与idle监测 ？**
    -
- **问题迭代？**
    - **重学ByteBuf？**
        - **读问题：**在BIO时代中，我们读取数据主要是存储到一个byte数组中，byte [] bytes = new bytes[1024];而只有出现while (bytes.read != -1) 才会返回，这里的返回有两种情况，一个是刚好读满了1024字节另一种情况就是 数据读取完毕，直接返回. 也就是-1。当连接中没有读取完毕，会一直阻塞读取数据。所以，我们可以理解为bytes基本就就是满载而归。NIO中，非阻塞的读取数据。bytebuf，读取不到数据时，不会出现阻

塞，而是返回一个0，假如，我们自己还是定义一个bytes字节数据，因为NIO是不会阻塞的，所以，我们每次读取的数据的长度是不固定的，可能上一次是10个数据，下一次是20个数据。我们可以使用offset进行记录当前的位置，显然，我们需要一种数据结构，那就是将这种工作进行封装起来。当一个数组具有流动性，在字节数组是做不到这个的，比较机器化。

在BIO中，比如，我们定义一个数据的大小为12个字节，而每次传输20个字节，我们需要将前12个字节取出来，因为整个字节数组是固定，如果不记录下，会被后面的数据覆盖，所以，需要将剩余的8个数据也记录下。等后边的4个字节接收到后，我们进行拼接。但是如果在NIO中，我们使用高级的缓冲区ByteBuffer，比如，读取了20个字节，先将12个字节拿到，readindex到这个位置了，拿到一个完整的数据，而readindex记录当前读取到的位置，下一次读取的时候，下标位置也就是11这个位置。因为write是不能超过read区域的，所以，write只能写12个字节。第二次读取，就可以拿到后半部分的4个字节了。而这就是形成了一个write和read 追赶的过程了。而不需要担心数据覆盖的问题了。比如我们读取9个字节，并不进行操作，而是等12个字节足够，才进行操作。相对于最基础的字节数组，功能是不是强大很多。

写问题： 在BIO中，我们写一个数据，一定是写操作完毕才会返回，否则就会一直阻塞中。但是在NIO中，我们写一个数据，如果没有写完毕，因为是非阻塞的，所以直接返回，但是需要记录当前写到那个位置。万一下一次还没有写完毕，还需要记录这个值。因此，我们需要一个更加高级的buf，写的时候，自动记录当前写入的位置，下一次，直接从最后直接插入就行。并且是一边写一遍读，互相不干扰。相对于原生NIO中的ByteBuffer大小是固定的，不能实现这样的功能。但是netty中做了相关的优化，自动扩容。更加高级的数组结构，专门服务于网络通信传输。一旦发现数据读取到的不是一个完整的数据，属于半包。就不读取直接放弃，等下一次数据完整后，才读取。所有那些handler解码器都是这样的，从解码器的handler内置的buf中取走数据，解析协议。组成一个更上层的对象数据。传递到下一个buf，解码器的buf可以一直重复使用，并连续工作。handler中的msg，是netty从bytebuf池中拿去一个空的对象，添加对象数据后。从中读取你想要的数据，然后放回到netty的buf，很大程度上节省了创建字节数组和销毁字节数据的开销。本质就是一个byte数组。

- **可以谈谈你对netty的理解吗？** 网络IO 协议解析 业务处理
  - **异步:**在谈到netty中，我们一定会遇到异步这个概念，异步 对于大多数人来说 认为NIO中的就是，比如从一个缓冲区中读取数据，没有 直接返回，没有阻塞在哪里 **这是非阻塞**。其实这个点没有理解透彻，异步基本上就是**多线程、异步事件驱动 还会使用事件队列+回调**
  - Netty是建立在NIO的基础上，框架对上层提供一个接口。

    一个网络框架必须要解决几个问题？

    1.封装网络IO的部分，程序操作数据的时候，框架会自动读写数据。只需要API级别的操作。程序员不需要关注和TCP交互的细节。

    2.NIO和Netty的处理方式不同 NIO基于Selector 记录每个线程的SelectorKey，当有read/write 操作，就进行操作。而这个过程需要不断轮询。而Netty则是 当一个将read/write 写到缓冲区中，生成一个事件加入到eventLoop中，然后按照先进后出的顺序进行处理当前事件，
  - **Netty本身就是异步事件驱动的**，其实异步事件编程的本质，就是使用一个事件队列，触发事件之后，丢到事件队列中，然后一个回调线程执行不断取出事件执行对其的回调。**异步事件通知和单纯的异步还是不一样的，** 异步通常来说，就是多线程机制，从开始执行，陷入到一个点开启一个新的线程执行，可能不存在通知上层线程机制，比如Spring async。但是有一些相关的场景需要回调上层通知，这种机制就是异步事件驱动，一般需要使用事件队列。**异步就是多线程，异步事件驱动 多线程+回调+事件队列**，通常所属的比如客户端从服务端读取数据，没有获取到数据，直接返回。这并不是异步，而是非阻塞，所以，我们要从根本上理解异步的概念。

- 举个栗子，异步在ajax中体现的淋漓尽致，当客户端调用一个ajax方法后，当前线程并不会阻塞，而是继续向下执行，而创建一个新的线程继续执行ajax方法进行网络Io的处理。而这个新的线程和当前线程并不是所属关系。所以，ajax方法调用后，在这个点上陷入进入，创建一个新的线程执行网络操作。多线程出现了。等到服务端处理完毕通信之后，往事件队列中添加一个事件。然后js线程会从事件队列中取出事件。并执行其回调，相当于，底层的线程调用了js线程，发起了一个通知。两种机制合并起来就是异步事件驱动了。而在NIO中，客户端给服务端发送50个字节，而缓存区只分配了30个字节，一次请求只发送了30个字节，而直接返回了，那么这能说是异步吗。因为多线程都没有使用到，何谈异步？异步写的含义，是开启一个新的线程，执行write的操作，而主线程不管write线程的操作，直接返回，等被回调就可以了。
- NIO整体流程，通过select去查询read write就绪，要么进行读 或者 写 操作 一个网络框架，一定可以将这些网络编程相关操作的给封装起来，
- 一个网络框架 **第一。一定要解决封装网络IO的部分，Netty 一是网络IO，二是协议解析，三是业务处理。**只需要给上层的程序员提供高度抽象化的API，就可以使用。比如框架会自动发送数据到服务端，会自动从tcp缓存区解析好给程序员。程序员不需要关注TCP交互的细节。而Netty也是基于NIO这一套做的，只不过封装的更好。比如，当netty一个线程执行select操作的时候，发现了两个socket上的读信号，会对应两个selectionKey，这都没有问题，netty接收帮我进行网络IO，读取到数据放到一个Bytebuf中，netty帮我们把数据读取好了之后，放到bytebuf中，netty生成一个事件，叫做read事件，将这个事件和Bytebuf关联，把这个event放入事件队列中。队列是先进先出的。**read事件->事件队列**

- **Dubbo同步调用与netty异步事件驱动？**
  - Dubbo中是将提供者和消费者进行划分的，消费者通过同步调用提供者的服务。但是dubbo又是netty实现的，那么netty的异步到低用在了哪里呢？
  - RPC调用都是同步调用的，类比一下JDBC调用数据库数据，没有数据是不会返回的，一直阻塞状态。对于客户端来说，我们可以使用BIO NIO 或者netty都能实现，BIO的实现就是同步阻塞的。先写，然后读阻塞，直到读取到数据响应，在这种模式中，一般来说，业务和网络通信处于一个线程，使用BIO实现客户端，也可以无可厚非的，毫无问题。JDBC底层的连接都使用的是BIO进行连接。远程服务端无论是BIO 还是NIO都没有问题，本质上就是基于TCP之上的网络通信数据传输。
  - 如果我们非要使用netty进行网络通信，怎么办，netty中有很多第三方的业务组件，http 等各种三方的handler模块，客户端可以直接使用。**netty是异步的，rpc调用是同步的**，netty是如何获取一个连接的，(channel)，是不是通过一个sync方法获取一个channelFuture，然后这个channel调用close的sync() 实现阻塞，`ChannelFuture channelFuture = bootstrap.bind(6688).sync();` 这个方法说明底层tcp连接建立之后，方法才会返回。返回的channelfuture包含了一个channelfuture，如果不加sync().这个方法是异步的，sync是等三次握手成功之后，才返回的。下一行才获取到真正的对象。而第二行代表的就是等关闭的时候，返回，不关闭就会一直阻塞。
  - 客户端线程启动发起了一个远程通信，创建Bootstrap，设置好参数。两个sync() 客户端等待服务端的响应，但是这个时候，客户端被阻塞了，只有等close.sync()关闭的时候，才会返回，这个时候，我们需要保证一点sync阻塞的时候，一定可以拿到服务端响应。**那么真实的网络通信和handler处理是在客户端线程处理的吗？** 其实并不是，是在netty专用的eventloop中。在是自己设定的eventloop线程组，**客户端线程目前被你的sync阻塞的，**我们只需要保证在netty处理过程中，将响应相关的协议业务等处理完毕之后，关闭channel。eventloop的handler执行channel关闭。这个时候syn就结束了。客户端线程就可以接着往下走。那么等关闭之后，我们需要保证将handler中处理的完整响应传递到客户端线程，我们的handler在哪里创建的，是不是通过bootstrap配置创建出来的。如何传递呢，需要在handler中设置一个成员变量。从客户端线程设置到hanlder实例中，handler业务都是在eventloop中被处理，也就是其他线程中，当eventloop完成后，这个变量也被填充了。syn也结束阻塞了。

    - netty的异步，指的是eventloop中的异步，并不是客户端线程自身的异步。那么和bio中有什么区别吗？bio编程中，所有业务逻辑， 而netty中，第一，你可以直接复用一些通用的业务组件。第三方通用的handler。第二，网络协议的业务，和客户端线程的客户业务相分离。把底层的网络协议封装到一系列的handler，无论是扩散还是客户端线程，移植修改，都是方便的。第三点，也就是这些引导对象，是可以相互clone.并且eventloop是共享的。
    - **异步编程** 人就是一个异步线程，而异步模式中，需要保护好上下文现场。人脑是基于跳跃式思维，电子计算机，cpu模型，从根本上来说，就是同步的。只能在大的同步背景下实现局部小范围的异步性。
    - ajax异步调用中，1、我们可以确定，那就是当客户端调用一个ajax后，一定会创建一个后台线程进行处理当前ajax请求，然后当前主线程继续执行向下的任务，这里一定要明确一点，那就是当前往下执行的相关逻辑，一定与ajax返回值无关。如果有关系，那么这部分代码，我们必须封装到一个回调中执行。等待ajax执行完毕，回调这部分代码。

Dubbo

- **是什么？**
    - Dubbo 是一款高性能、轻量级的开源 Java RPC 框架，它提供了三大核心能力：**面向接口的远程方法调**用，**智能容错和负载均衡**，**以及服务自动注册和发现**。
- **Dubbo有哪些相关特性?**
    - 面向接口代理的高性能 RPC 调用：**提供高性能的基于代理的远程调用能力**，**服务以接口为粒度**，为开发者屏蔽远程调用底层细节；
    - 智能负载均衡：内置多种负载均衡策略，智能感知下游节点健康状况，显著减少调用延迟，提高系统吞吐量；
    - 服务自动注册与发现：支持多种注册中心服务，服务实例上下线实时感知；
    - 高度可扩展能力：遵循微内核+插件的设计原则，所有核心能力如 Protocol、Transport、Serialization 被设计为扩展点，平等对待内置实现和第三方实现；
    - 运行期流量调度：内置条件、脚本等路由策略，通过配置不同的路由规则，轻松实现灰度发布，同机房优先等功能；
    - 可视化的服务治理与运维：提供丰富服务治理、运维工具：随时查询服务元数据、服务健康状态及调用统计，实时下发路由策略、调整配置参数。
- **Dubbo有哪些相关核心组件？**
    - Provider：服务提供方
    - Consumer：服务消费方
    - Registry：服务注册与发现的注册中心
    - Monitor：主要用来统计服务的调用次数和调用时间
    - Container：服务的运行容器
- **Dubbo有哪些负载均衡策略？**
    - **随机负载均衡（Random LoadBalance）**：按权重设置随机概率，在一个截面上碰撞的概率高，但调用量越大分布越均匀，而且按概率使用权重后也比较均匀，有利于动态调整提供者权重；
    - **轮询负载均衡（RoundRobin LoadBalance）**：按公约后的权重设置轮询比率，存在慢的提供者累积请求的问题，比如：第二台机器很慢，但没挂，当请求调到第二台时就卡在那，久而久之，所有请求都卡在调到第二台上；
    - **最少活跃调用数负载均衡（LeastActive LoadBalance）**：使用最少活跃调用数，活跃数指调用前后计数差；
    - **哈希负载均衡（ConsistentHash LoadBalance）**：使用哈希值转发，相同参数的请求总是发到同一提供者。
- 核心部分
    - 远程通信

    - 集群容错
    - 自动发现
- 负载均衡算法
    - **一致性哈希**
        - 一致性含义 上一次打在哪台机器上，下一次也会打在哪台机器上
    - 加权Hash 性能好的机器 打在这个机器上
    - 轮询

Tomcat

- Tomcat常用的配置有哪些
- Tomcat调优一般都调哪些参数

Nginx

- **负载策略**
    - Nginx：可以根据客户端的 ip 进行负载均衡，在 upstream 里设置 ip_path，**负载均衡五种配置**：
        - 1）**轮询（默认）**，每个请求按时间顺序逐一分配到不同的后端服务器，如果后端服务器 down 掉，能自动剔除；
        - 2）**指定权重**，指定轮询几率。权重越大，轮询几率越大，用于后端服务器性能不均的情况。
        - 3）**ip 绑定 ip_path**，每个请求按访问 ip 的哈希结果分配，这样每个客户固定访问一个服务器，可以解决 session 问题。
        - 4）fair（第三方）按后端服务器的响应时间来分配请求，响应时间短的优先分配。
        - 5）url_hash 按访问的 url 结果来分配请求，使每个 url 定位到同一个后端服务器。后端服务器为缓存时比较有效。具体的配置自己网上百度

ES

MongDB

思考

- **从新理解中间件？**
    - 理解一个业务，比如，我们有一个商品系统，有对应的数据库在MySQL中，搜索系统ES，通过es查询对应的商品信息，但是商品信息的添加数据需要同步到ES中。当然我们可以借助于MQ来解决。从一个更加宏观的视角看问题，分布式系统中有很多组件，MQ并不是一个完全独立的专用系统，而mysql来说是一个可以独立运行的专用系统，分布式中很多组件也是这个范畴内的。文件系统，数据库系统，缓存，集群。这些系统之间需要集成起来，需要粘合剂，而MQ就是这样的角色。**中间件的作用是集成分布式系统的两个独立组件，并对双端提供接口**，tomcat和mysql需要集成，需要使用jdbc和数据库驱动。jdbc是java端的一个中间件，因为非常了解mysql相关处理 通过数据库驱动进行连接mysql。如果说没有jdbc，那么我们需要和mysql对外暴露的tcp相关协议进行处理，而这部分代码就需要在jdbc中由程序员自己来编写，所以可以确定的就是中间件从某种程度进行了一种更向上一层的封装。对标中间件MQ，Dubbo来说都是分别给消费者和提供者一个固定的接口，消费者和提供者直接内嵌到接口就可以直接使用。很多人对于中间件没有自己清晰的理解，中间件吗 就是MQ ，其实这种想法是狭隘的。为什么说大厂有很多中间件研发，对于有一定规模的公司来说，系统

之间一定需要远程的交互，成千上万的交互，而中间件就是做这个事情的，将分散的组件，组合起来完成更上层的应用逻辑。
    - 对于上面提到的问题，我们可以采用另外一个思路来解决，当数据插入到mysql中，通过触发器来定时扫描新添加的数据到临时表中，记录一个Id 当5分钟时间，通过ids 从临时表的ids中查找数据，添加到es中。回到mysql,我们知道针对于Mysql的分库分表，有很多方案，比如sharingjdbc，mycat，而对于客户端java端来说，需要这些中间件进行屏蔽相关的细节，直接通过java端进行操作数据。所以，一定要从根本上理解中间件的含义，而不是人云亦云。

0x06-工具篇

Git/SVN

- **git基本概念**
    - **工作区** `git add` -> 暂缓区
    - **暂缓区** 可以在工作区和版本库之间进行数据交互 `git commit`
    - **版本库** Git本地版本仓库 -> `git push`
    - **远程仓库**
- **git rebase 和 git merge 有什么区别 ☆**
    - 同：都是用于从一个分支获取并且合并到当前分支
    - 异：rebase --> 会合并之前的commit历史（带有破坏性的修改 commit 历史的命令）
    - 一个干净的，没有merge commit的线性历史树 --> git rebase
    - 保留完整的历史记录，并且想要避免重写commit history的风险 --> git merge
    - git rebase 和 git merge 的区别
- **git fetch 和 git pull 有什么区别（PayPal）**
    - pull = fetch + merge
    - 详解git fetch与git pull的区别
- **描述 git 中的 cherry-pick 指令（小红书）** *TODO*
- **git reset和git revert的区别是啥？**
    - git reset是通过回退HEAD 来完成版本回退，git revert是生成了一次新的提交

```
创建分支
·git branch<分支名>
·git branch-v  查看分支
>切换分支
·git checkout<分支名>
·一步完成：git checkout-b<分支名>
>合并分支
·先切换到主干git checkout master
·git merge<分支名>
删除分支
·先切换到主干 git checkout master
·git branch-D<分支名>
```

- **Github基本操作**

```
git clone xx //从github下载
git add . //添加到本地仓库
git commit -m "xxx"  //提交
git push origin master //push到github上
```

- 版本冲突如何解决
  - git版本冲突
  - 版本冲突

Maven

- **dependencyManagement 和 dependencies 区别?**
  - **dependencyManagement** 只是声明依赖，并不实现引入，子项目需要显示的用依赖。
  - 父项目规定版本号，子模块继承后,提供作用:锁定版本+子module不用groupId和version
- **常用命令**
  - `mvn clean`删除上次打包生成的target文件夹
  - `mvn compile` 对源代码进行编译
  - `mvn clean package` 清除并打包
  - `mvn clean install` 删除并安装包
  - `mvn clean deploy` 删除并发布到私服

IDEA

- 检索

Ctrl + F	**在当前文件进行文本查找**
Ctrl + R	在当前文件进行文本替换
Ctrl + Shift + F	全局进行文本查找（若与搜狗输入法冲突重置即可）
Ctrl + Shift + R	全局进行文本替换
连按两次Shift	采用Search Everywhere检索
Ctrl + N	通过类名查找（类名:行号可以跳转到指定行）
Ctrl + Shift +N	通过文件名查找

- 编辑

Ctrl + Alt + O	**优化import（未使用的会自动删除）**
Alt + Insert	代码自动生成（构造方法，Get/Set方法，ToString等方法）
Ctrl + Z	撤销
Ctrl + Shift + Z	恢复Ctrl+Z 撤销掉的内容
Ctrl + /	添加或去除光标所在行代码注释
Ctrl + Shift + /	添加或去除指定代码块注释
Ctrl + Alt + T	针对选中代码自动生成提示语句
Ctrl + Shift + Enter	自动补全代码（如if while ";"等自动补全）
变量名.if	快速生成if(布尔值)
变量名.notn	快速判断对象是否为空
变量名.return	快速return
Alt+Shift+↑/↓	上移或下移一行
Ctrl +Alt +M	抽取代码为方法
Ctrl + Shift +Alt	**多行同时编辑 （同时按住按键后鼠标选择要修改处）**
Ctrl + C	**复制光标所在行**
Ctrl + X	剪贴光标所在行
Ctrl + Y	删除光标所在行
Ctrl + D	复制光标所在行到下一行
Ctrl + Delete	删除光标后面的代码
Ctrl + BackSpace	删除光标前面的代码
Ctrl + Alt + L	格式化代码（可以在Code Style中设置规则）
Ctrl+W	逐渐扩大选中光标位置的代码
Ctrl+Shift+W	逐渐缩小选中光标位置的代码
Ctrl+Shift+C	复制当前文件的绝对路径
Ctrl+Alt+Shift+C	复制当前文件的引用路径
Ctrl + F4	关闭当前编辑文件
Alt + Enter	代码补全提示

：针对Alt + Enter可以在以下情况选择自动提示：

①针对光标所在的接口，生成对应接口实现类。

②接口类中添加方法后，接口实现类自动实现。

③对调用的方法生成返回值

④对光标所在对象自动导包

- 查看

Alt +7	**弹出当前文件结构的窗口**
Alt+F7	弹出工程或库中类、方法、变量等被使用处的窗口
Ctrl +F7	逐一显示当前文件中类、方法、变量等被使用处（可以用F3切换）
Ctrl+Shift+F7	高亮显示当前文件中类、方法、变量等被使用处
Ctrl +O	查看可重写或实现的方法
Ctrl +I	查看可实现的方法
Ctrl + Shift+"+"	展开代码
Ctrl + Shift +"-”	折叠代码
Ctrl + E	查看最近打开的文件列表
Ctrl + Shift + E	查看最近编辑的文件列表（若与搜狗输入法冲突重置即可）
Shift + ↑	滚轮上下滚动

- 导航

Ctrl+H	**查看当前类的层级关系**
Ctrl +F12	查询当前文件的所属成员（可以在弹出层进行检索）
Ctrl+Alt+←/→	回退/向前(上一次浏览的地方)
Ctrl +Shift+Backspace	回退到上一次修改的地方（向前可单独设置快捷键）
F2	跳转到下一个错误高亮放的地方
Shift+F2	回退到上一个错误高亮的地方
Alt + ↑/↓	将光标定位到代码所在方法或前/后的方法
Ctrl+Shift+数字	快速创建与取消书签
Ctrl + 数字	跳转到指定的书签
Shift+F11	弹出全部书签窗口

- 模板

Ctrl+J	**快速插入动态模板**

例如：

psf：生成“public static final” 语句（加i或s选择数据类型）

psvm：生成main方法语句

thr : 生成“throw new” 手动抛出异常语句

sout：生成System.out.println 控制台输出语句;

itli： 生成遍历List的for循环

itco： 生成遍历Collection的for循环语句

iter： 生成增强型for循环语句

itit： 生成遍历Iterator的while循环语句

inn：生成 if not null 语句

inst：生成”if instance of ”实例化判断语句

- 重构

Alt+Delete	**安全删除，删除前会提示调用的地方**
Ctrl+F6	重构方法名称、参数、返回值等
Shift+F6	重命名
F5	复制类
F6	移动类

- 编译运行

Ctrl+F9	**编译工程**
Ctrl+Shift+F9	重新编译选中的模块，文件等
Alt+Shift+F10	选择配置后启动运行
Alt+Shift+F9	选择配置后启动调试
Shift+F10	启动运行（相当于Run按钮）
Shift+F9	启动调试（相当于Debug按钮）
Ctrl+F2	停止调试（相当于Stop按钮）

- 断点调试

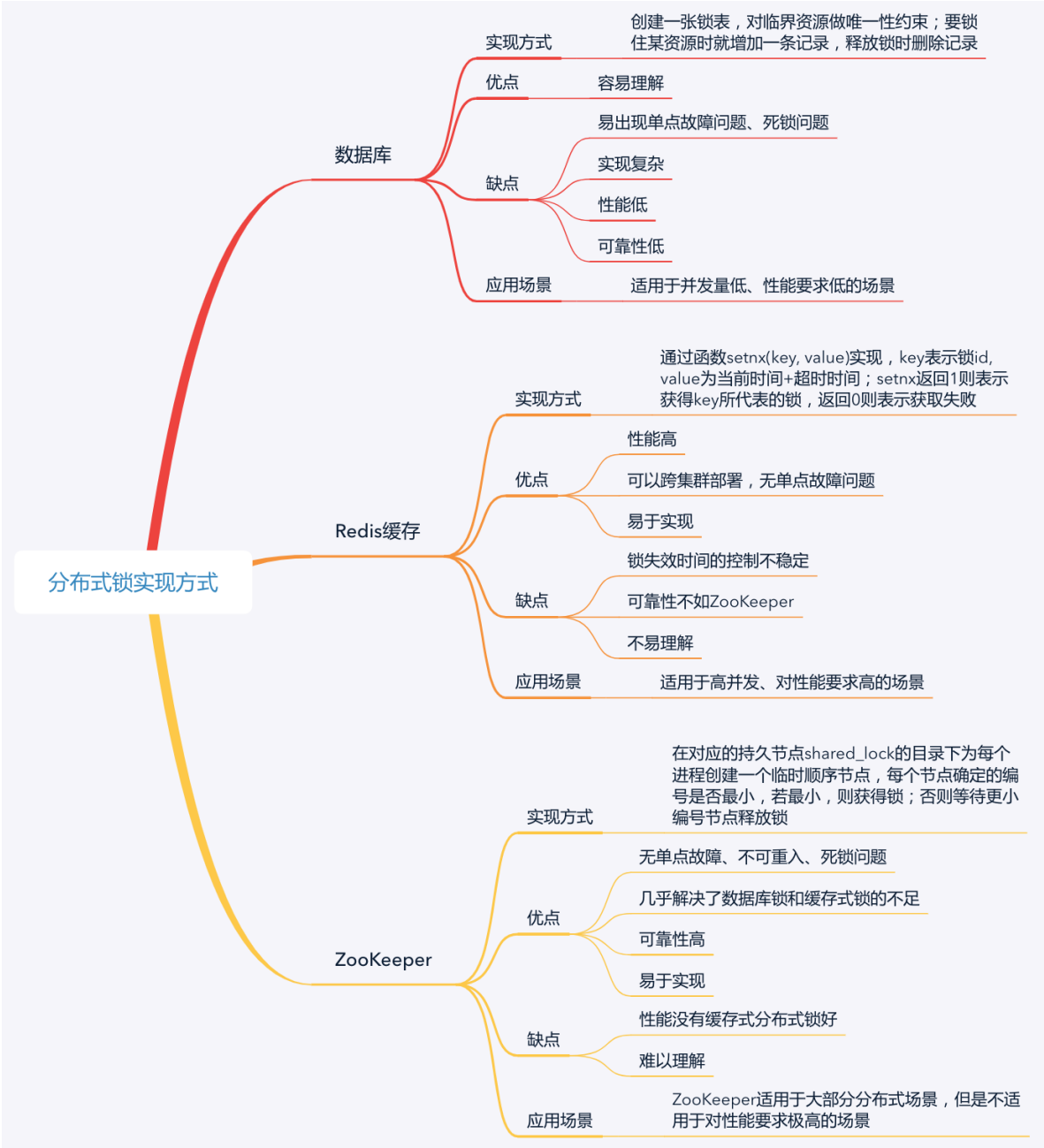
F8	**单步跳过**
F9	跳过（进入下一个断点或执行完程序）
F7	进入函数内部
Shift+F8	跳出函数
Alt + F8	执行表达式查看指定变量的值
Ctrl + F8	给光标所在行添加与取消断点
Ctrl+Shift+F8	查看全部断点

0x07-分布式篇

D1.CAP BASE

- 分布式系统的理解
  - **多台计算机和通信的软件组件通过计算机网络连接(本地局域网或广域网)组成。分布式系统是建立在网络之上的软件系统。**
  - 分布式和集群的区别
    - 分布式：不同的多台服务器上面部署不同的服务模块，模块之间通过RPC/Rmi通信和调用，对外提供服务和组内协作。
    - 集群：不同的多台服务器上面部署相同的服务模块，通过**分布式软件进行统一的调度(Zk)**，对外提供服务和访问。
- CAP
  - C(Consistency) **强一致性**
    - 通俗来说，在分布式系统中，一致性是**数据在多个副本之间能否保证数据一致性的特性**。在一致性需求下，当一个系统在数据一致的状态下执行更新操作后，应该保证系统数据仍处于一致的状态。
  - A(Availablity) **可用性**
    - **系统提供的服务必须一直处于可用的状态，对于用户的每一个操作请求总是在有限的时间内返回结果。**
  - P(Partition tolerance) **分区容错性**
    - **分布式系统在遇到任何网络分区故障的时候，任然需要能够保证对外提供满足一致性和可用性的服务，除非是整个网络环境都发生了故障。**
    - CA **单点集群**，满足一致性，可用性，但是可拓展性不是太强大。
      - RDBMS
    - CP 满足一致性，分区容错性，通常性能不是特别高
      - Redis MongDB ZK(满足一半以上的节点 就可用) MySql复制集(需要全部都保证一致)
    - AP 满足可用性，分区容错性，通常数据不能保证实时一致。
      -
    - 在分布式存储系统中，CAP是基础条件。但是由于网络是不可靠的，虽然有TCP一些可靠传输，但是也会出现**数据延迟和丢包**的问题，因此，分布式系统中必须在满足分区容错性的前提下，在一致性和可用性之间进行选择。
    - 实际使用，对于单机版的Oracle数据库，采用的是数据强一致性，以及可用性。也就是**CA**
    - 而Redis、MongoDB 在可用性(**A**)作出妥协，在保证数据的一致性 并且分区容错性 **CP**
    - 而对于大多数的网站来说，我们可以容忍数据在一段时间间隔内 可以出现数据的不一致情况，但是在最终数据必须保证一致性，也就是**数据最终一致性。** 所以是**AP**
- BASE
  - 为什么提出
    - 为了解决关系数据库强一致性引起的问题而引起的可用行降低而提出的解决方案。
  - 是什么
    - **基本可用(Basically Available)**
    - **软状态(Soft state)**
    - **最终一致性(Eventually consistent)**
  - 为什么
    - **它的思想是通过让系统放松对某一时刻数据一致性的要求来换取系统整体伸缩性和性能上改观。**为什么这么说呢，缘由就在于大型系统往往由于地域分布和极高性能的要求，不可能采用分布式事务来完成这些指标，要想获得这些指标，我们必须采用另外一种方式来完成，这里BASE就是解决这个问题的办法

D2.分布式锁 ☆☆☆



> 场景问题 超卖问题

- **mysql的乐观锁实现 syn reentrantLock**
- **redis方式**
  - **单机redis**
  - **redisson**
  - 相关问题
    - 锁释放：Redis如果在加锁期间遇到程序宕机，没有超时机制的话，那么锁永远不会被释放

      因此必须加**超时机制**
- **zk实现**
  - 1.node的临时性 唯一性 顺序性
  - 超时机制 等待锁

◆**不推荐编写的分布式锁** 推荐Redisson和Curator实现的分布式锁

- **你们的分布式锁做过高并发优化吗？能抗下每秒上万并发吗？**
  - 将库存分布在多个服务器上。分而治之。分段加锁+合并库存



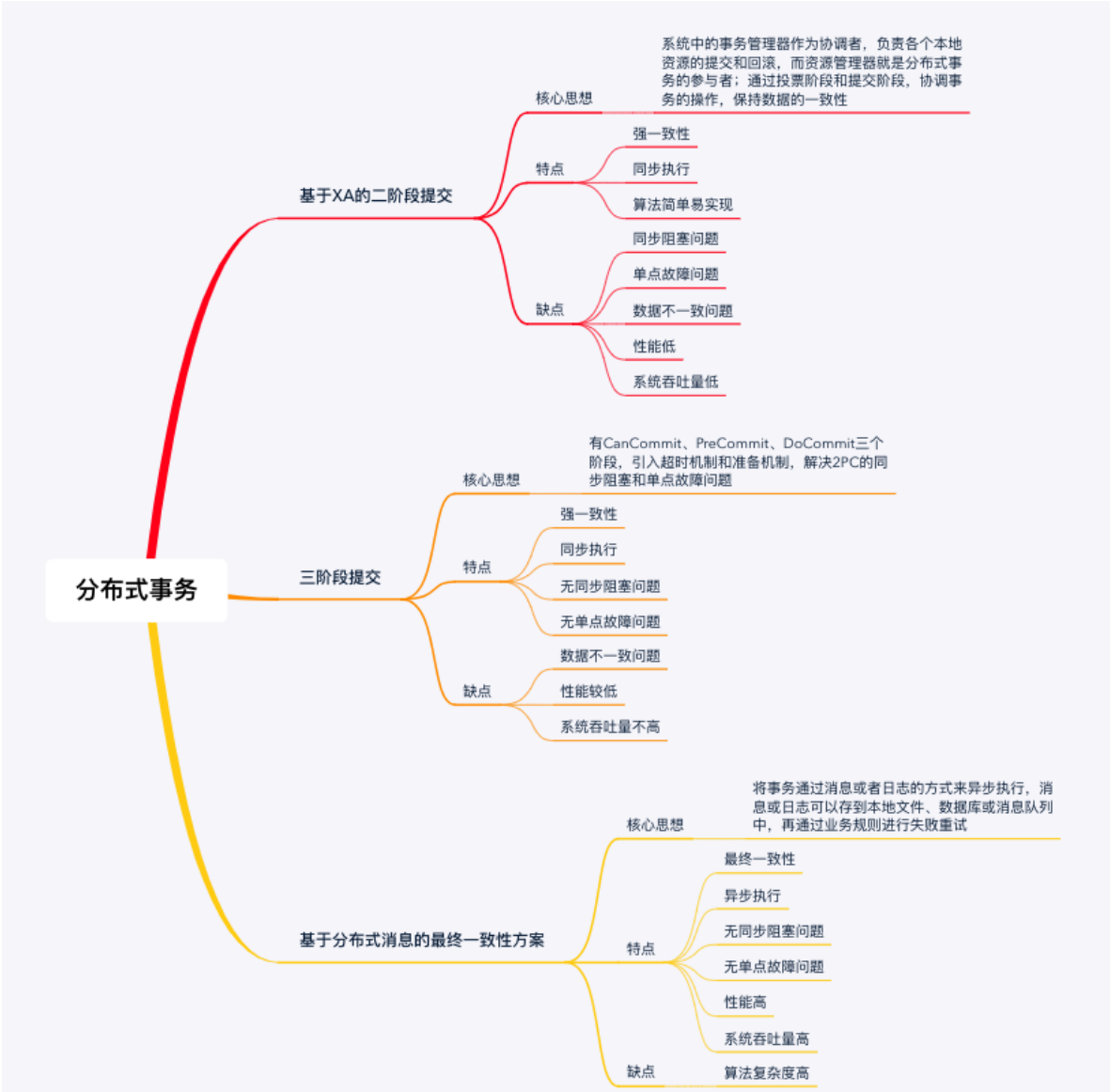
方式	优点	缺点
数据库	实现简单、易于理解	对数据库压力大
Redis	易于理解	自己实现、不支持阻塞
Zookeeper	支持阻塞	需理解Zookeeper、程序复杂
Curator	提供锁的方法	依赖Zookeeper，强一致
Redisson	提供锁的方法，可阻塞	

- **谈分布式锁的理解？**
  - ZK容错性比较高，而Redis编程简单化 对于Redis来说，加锁期间 程序宕机，如果没有超时机制，永远不会释放锁。而ZK是强一致性 CP 保证课一致性，牺牲了可用性。
  - **ZK是强一致性的分布式系统，ZK同时兼备可用性和分区容错性 岂不是违背了CAP理论？**
    - **MySQL复制集** 保证了可用性和分区容错性，但是丢失了一致性，也就是不同节点直接数据的一致性需要同步，而这个同步操作需要有一个时间段。虽然我们可以通过一种方式来实现数据之间的一致性，那就是采用配置同步更新策略，当update一条数据时，需要等所有集群节点都部署同步完成后，才可以操作。这就比较有意思，你保证了一致性，但是可用性呢，比如在同步的过程中，一个节点宕机，整个update操作被阻塞了，如何做。你可能会说设置一个超时时间，如果设定了超时时间，那么可用性就无法满足了。
    - **可用性**：不是说数据同步开销多态节点开销很大，如果网络正常那么2/3 ms 是无可厚非的，而是说，在网络过程中，出现了意外。进程僵死。而是描述节点是否在一段时间内可以持续提供服务给客户端。
    - **ZK** ZK是强一致性系统，分区容错性也可以满足，死一台没有关系 只要一半以上的节点存活就可以。**而ZK满足的可用性和分区容错性是有条件的**，无条件的分区容错性是说，无论网络发生什么，只要节点能连接上，系统依旧可以运行。但是，ZK的网络分区是有要求的那就是不能有一半以上的节点出现网络分区问题，同时可用性也是由条件的，必须提供半数以上才能提供可用性，所以，**ZK中一致性是硬性指标，而可用性和分区容错性是待着约束条件的，**
    - **对比ZK和mysql复制集** mysql需要保证数据的同步复制 而zk只需要保证一半以上的节点就可以，可用性不是集群规模变大，大致网络开销变大，系统不可用。**ZK是不适合大规模集群**，因为当节点过多 数据集的复制 RPC交互 需要耗费较多的时间消耗。如果大规模集群考虑eureka。Zookeeper主业是分布式系统中的协调，所谓的注册中心。eureka是一个专业的注册中心。

D3.分布式session

- 1.tomcat+redis
- 2.spring session+redis

D4.分布式事务 ☆☆☆



- 数据的一致性问题
  - 数据的并发访问、修改
  - 不同请求之间的数据隔离
  - 多个服务共同完成一个业务请求，保证都完成或失败
  - 发生异常时的数据回滚
- **单体应用-订单提交失败**-（Spring事务）
- **两阶段提交事务** (事务管理器 第一阶段 先询问 第二阶段执行)
  - 优点 原理简单，实现方便
  - 缺点 同步阻塞，单点问题，脑裂
  - **解决方案**
    - XA方案
    - seata方案
- **TCC方案**
- **本地消息表**
- **可靠消息最终一致性方案**
  - 最大努力
- **可以详细说一下你对seata解决分布式事务的理解吗？**

- 一次业务操作需要跨多个数据源或需要跨多个系统进行远程调用，就会产生分布式事务问题
- 一个典型的分布式事务过程
  - **分布式事务处理过程的-ID+三组件模型**
  - **Transaction ID XID (全局唯一的事务ID)**
  - 3组件概念
    - Transaction Coordinator(TC) **事务协调器**，维护全局事务的运行状态，负责协调并驱动全局事务的提交或回滚;
    - Transaction Manager(TM) 控制全局事务的边界，负责开启一个全局事务，并最终发起全局提交或全局回滚的决议;
    - Resource Manager(RM) 控制分支事务，负责分支注册，状态汇报，并接收事务协调器的指令，驱动分支（本地）事务的提交和回滚；
- **分布式事务的执行流程**
  - a.TM开启分布式事务(TM向TC注册全局事务记录)
  - b.换业务场景，编排数据库，服务等事务内资源（RM向TC汇报资源准备状态）
  - c.TM结束分布式事务，事务一阶段结束（TM通知TC提交/回滚分布式事务）
  - d.TC汇总事务信息，决定分布式事务是提交还是回滚
  - e.TC通知所有RM提交/回滚资源，事务二阶段结束。
- **seata分布式思路**
- 本质上还是二阶段提交，在业务sql之前和之后保存了对应的前置和后置镜像。如果当时事务执行成功，删除镜像和对应的行锁，如果失败的话，就执行前置镜像回滚数据。回滚的时候，需要先判断当前数据和后藕汁镜像中的数据是否是一致的。避免出现脏写数据。**分布式事务的解决方案，是最终一致性**

D5.单点登录SSO、分布式会话

D6.分布式全局ID

- **为什么要使用分布式ID？**
  - 在分库分表中，由于不能保证ID唯一性 因此引入。通俗一点来说，在一个单体应用中，只会存在一个数据库，而表id按照自增的策略完全可以保证唯一性，但是引入分库分表之后，由于表之间是横向切的，所以id会出现重复，必须有一个唯一的ID进行保证。
- **使用UUID作为id实现主键全局唯一性保证**
- **通过统一ID序列表，实现全局ID**
- **雪花算法** 具体见脑图
- **重新学习分布式ID**
  - zookeeper是集群高可用，有一些人通过redis的递增api来控制全局id，我认为并不会，因为redis如果单点宕机，服务就不可持续。就算是redis集群，也无法保证集群间的数据一致性。但是Zookeeper可以做到集群一致性数据视图，任何时候，访问任何一个节点，都可以得到一致的数据。Zookeeper最强大的就是他的zab协议，如何保证复制集之间的数据一致性。这是什么mysql主从复制，redis主从复制不具备的能力。

    如果你的主redis服务器挂了，这个时候，和从服务器之间的复制还存在一个时间差，你的哨兵会将从服务器提升为主服务器，这个过程中，从服务器有可能和宕机的主服务器数据不一致，那么，产生的全局id会有影响，如果你不用主从redis，redis又会存在单点问题，而Zookeeper，这一切都不复存在了。分布式锁也同样如此，不影响，zab协议会选举一个新的leader，如果他解决不了这个问题，他就不是分布式一致性的产品了，这个协议内容本身很复杂，也是基于paxos，有兴趣的同学可以了解下，在分布式节点上保证数据一致性，paxos的作者，获得了图灵奖，可以想象这个领域的分量有多重，不是你想的那么简单的，主从同

步之类的直观逻辑，Zookeeper是一个伟大的产品，有了它，我们可以基于他做一些分布式系统的上层逻辑，而不用考虑底层的网络，节点故障等造成的数据不一致的影响，在没有Zookeeper的岁月里，很多分布是软件的开发者，要花很多精力去处理节点故障，网络故障造成的影响，会有很多和业务无关的代码，专门处理这些故障，并且，还不一定处理的好，有了它，分布式软件的开发者，精神上解放了，所以你们会看到很多分布是软件系统，她们都会依赖Zookeeper给他们提供一致性保障，相当于说，现在，你有了Zookeeper作为支撑，你也可以轻松开发一个分布是软件了，因为框架帮你解决了那些问题，你可以专注于应用逻辑层面的处理，Zookeeper的底层采用了zab协议，zab协议是在paxos的协议之上借鉴发展而来，而paxos协议的作者，对分布式领域做了巨大的贡献，因此获得了2013年的计算机最高奖，图灵奖。因为我们访问一个url的资源，

不会白白给你访问，你需要带上认证信息，但是，现在我们很少通过这样的方式去认证，我们最常见的认证，是通过一个表单post提交用户名密码，然后把认证信息存储在session中，下次访问，对比session来确认，我们常见的手工处理认证和授权的操作，都是在程序中进行，

访问一个资源，动态获取该用户是否具有权限，但是这类需求在每个项目中都存在，如果手工去处理这些东西，是不是有必要抽象出一个框架？

D7.接口幂等性

D8.分布式限流

D9.正反代理

- **反向代理**
  - 代理的是服务端，比如接收用户请求Nginx 然后转发到服务器端。
  - client 不知道 server，并不是 URL 中请求的那个资源，而是不知道从什么地方来的。以代理服务器来接收 internet 上的请求，然后将请求转发给内部网络的服务器，并将从服务器上得到的结果返回给 internet 上请求的客户，此时代理服务器对外就表现为一个服务器。
  - 1）保证内网安全、
  - 2）负载均衡，nginx 通过 `proxy-pass-http` 配置代理站点，`upstream` 实现负载均衡。
- **正向代理**
  - 代理的是客户端，比如我们想访问google，可以通过一个代理服务器，将我们的请求转发到国外服务器。

    国外服务器将请求转发到google.com 中 并将处理结果返回给客户端。
  - 正向代理作用：1）访问原来无法访问的资源。2）可以做缓存，加速访问资源。3）但代理可以记录用户访问记录（上网行为管理），对外隐藏用户信息。4）客户端访问权限，上网进行验证。

D10.一致性哈希·233-·66

在移除 or 添加一个 cache 时，他能够尽可能小的改变已经存在 key 映射关系。

- **Hash**:把任意长度的输入，通过散列算法，变换成固定长度的输出。不同的输入可能导致相同的输出。作用：文件校验，数字签名，hash 表查找 O(1)。
- 一致性算法：
  - 1）平衡性：哈希的结果能够尽可能分布到所有缓存中去。
  - 2）单调性：如果已经有一些内容通过 hash 分派到了相应的缓冲中，又有新的缓冲区加入到系统中，那么hash 的结果应该能保证原有已分配的内容可以被映射到新的缓冲区中，或原来的缓冲区中。

- 3）分散性：在分布式环境中，终端有可能看不到所有缓冲区，而只能看到其中一部分，当终端希望通过哈希过程将内容映射到缓冲区上时，由于不同终端所见的缓冲范围有可能不同，可能导致相同的内容被不同的终端映射到不同的缓冲区上，这种情况应该避免。
- 4）负载：既然不同的终端可能将相同的内容映射到不同缓冲区中，那么对于一个特定的缓冲区而

言，也可能被不同的用户映射为不同的内容。

- **如何计算？**

  在分布式集群中，对机器的添加和删除或机器故障后自动脱离集群，这些操作是分布式集群管理最基本的功能，若采用常用的 hash(object)%N 算法，那么在有机器添加或删除以后，很多原有的数据就无法找到，所以出现一致性哈希算法——

  - 1）环形 hash 空间：按照常用的hash 算法来将对应的 key 哈希到一个具有 2 32 个桶的空间，即（0-2 32 -1）的数字空间中，现在我们将这些数字头尾相连，想象成一个闭合的环形。
  - 2）把数据通过一定的 hash 算法映射到环上。
  - 3）将机器通过一定的 hash 算法映射到环上。
  - 4）节点按顺时针转动，遇到的第一个机器，就把数据放在该机器上。

D11.分布式算法`todo`

D12.问题迭代

- **重写理解分布式**

  分布式从名字上就可以看很容易去理解，分布式顾名思义就是一个系统按照多种维度的划分，将服务部署在不同的机器上，可能是按照业务或者是模块等功能划分。本来一个单体系统可以独立部署运行开发测试，但是当业务逐渐增加，模块之间的耦合关系紧密，最终为了更好的提供服务，进行拆分，将一个系统 按照 业务模块划分，什么订单系统，支付系统，交易系统划分，部署在不同的机器上，这就是分布式。为了更好的沉淀业务，可以将一些通用的模块进行下沉，走中台战略。有些服务需要支撑起比较高的并发量，将系统水平拓展做成集群，提升系统的高并发能力。

  我们来看下，当系统进行划分成多个子系统时，就需要子系统之间交互 调用，就需要RPC这种框架，配合注册中心来进行服务调用发现、提供。缓存的相关的功能 中间件缓存 以及系统之间异步的通信 MQ，搜索系统 等，而这就需要我们不断解决，衍生于从单体事务 单体锁 上升到一种分布式角度 ，分布式锁 分布式事务 分布式ID 分布式Session。

  分布式的本质也为了高内聚，低耦合。单体系统中，类与类之间的耦合度是比较高的，依赖是随处可见，程序员直接一个import就可以导入包。而三层架构目的也是为了降低耦合度，专注于某一层要做的事情，对比生活中，水房大爷的职责是定时烧水，而学生只需要在指定时间内去打水。这本上就是一个职责明确的工作。你不能说 学生不要上课了，水房大爷我帮你干吧，显然不符合逻辑。

  单体系统也是一个分布式，广义上分布式是业务上的一个划分，好了，我们来分析一下，单体系统对于用户来说一定是通过网络进行交互，而tomcat 部署在在一套服务器上，而mysql 也是 需要通过网络交互。而这种本身就跨网络交互的，只不过不是业务分布式的范畴。客户端作为用户交互的唯一接口。**现在的分布式是以业务进行划分，而不是客户端 服务端这种大粒度的分布式**

  为什么要进行细粒度的划分呢？

  1.项目过于庞大，难以维护。

  2.降低依赖，降低可维护性,增强可维护性

  3.复用能力

  关于系统划分的界限？

  划分子系统是一个艺术，按层 模块 子系统都是可以的，毫无定论。

采用用例划分。

广义上的分布式理解？

负载均衡+tomcat复制集+后端数据库、

分布式数据库中，不同数据分片到不同的节点。分片之间不会进行交互，而是通过前端路由到不同的分片节点上，显然，这些分片之间没有交互RPC通信，但是都连接到了路由节点，路由节点间接维护了整个分布式集群的整体性，**a.第一种 分布式系统的形式 节点之间不需要进行交互** ，两个系统共享同数据库，可以一种分布式系统。如果仅仅将分布式理解成组件RPC交互是比较狭隘的。分布式中zk承担了分布式协调角色，不同节点之间的通信和交流，可以通过zk进行间接进行。**b.第二种 可以多个系统通过内存，共享数据进行交互 第三种 通过MQ进行异步通信，两个系统通过MQ进行间接通信。** 前后端通信本质上rpc调用也是没有本质区别的。**dubbox，spring cloud解决他们的问题域，但是他们只是分布式概念的一个子集，我只是想表达这个观点**

功能，性能，计算，存储，这些都是系统的主题

复制集的前端路由，一般有两种形式。第一种就是对于客户端来说，毫无感知，直接通过前端路由定位到物理节点，第二种 客户端这一方面维护物理节点的配置信息，而配置信息就存储在ZK上，通过ZK进行交互。**单台tomcat 不是整个系统，只是系统的一部分 ，每个节点承担的部分不一样，可能是计算 存储 功能 性能等的集合**

0x08-容器篇

Docker

- **解决的问题点是什么？**
  - 解决了**运行环境和配置问题**软件容器，方便做持续集成并有助于整体发布的容器虚拟化技术。
- **Docker 和传统虚拟化方式的不同之处：**

  传统虚拟机技术是虚拟出一套硬件后，在其上运行一个完整操作系统，在该系统上再运行所需应用进程； 而**容器内的应用进程直接运行于宿主的内核**，容器内没有自己的内核，而且也没有进行硬件虚拟。因此容器要比传统虚拟机更为轻便。

  每个容器之间互相隔离，每个容器有自己的文件系统 ，容器之间进程不会相互影响，能区分计算资源。
- **DevOps**
  - 更快速的应用交付和部署

    - 更便捷的升级和扩缩容
    - 更简单的系统运维
    - 更高效的计算资源利用
- **Docker的基本组件**
    - 镜像
        - Docker 镜像（Image）就是一个只读的模板。镜像可以用来创建 Docker 容器，一个镜像可以创建很多容器。
    - 容器
        - 容器是用镜像创建的运行实例
    - 仓库
        - 仓库（Repository）是集中存放镜像文件的场所。
- **常用命令**
    - `service docker start` 启动
    - 帮助命令
        - `docker version` `docker info` `docker --help`
    - **镜像命令**
        - `docker images`
        - `docker search 某个XXX镜像名字`
        - `docker pull 某个XXX镜像名字`
        - `docker rmi 某个XXX镜像名字ID`
            - 删除镜像
            - `docker rmi -f 镜像ID`
    - **容器命令**
        - `docker run xx` 新建并启动容器
        - `docker ps [OPTIONS]` 列出当前所有正在运行的容器
        - `docker start 容器ID或者容器名` 启动容器
        - `docker restart 容器ID或者容器名` 重启容器
        - `docker stop 容器ID或者容器名` 停止容器
- **Docker容器数据卷**
- **docker是怎么工作的？**
    - Docker是一个Client-Server结构的系统，Docker守护进程运行在主机上， 然后通过Socket连接从客户端访问，守护进程从客户端接受命令并管理运行在主机上的容器。 **容器，是一个运行时环境，就是我们前面说到的集装箱。**
- **docker为什么比VM快？**

  (1)**docker有着比虚拟机更少的抽象层**。由于docker不需要Hypervisor实现硬件资源虚拟化,运行在docker容器上的程序直接使用的都是实际物理机的硬件资源。因此在CPU、内存利用率上docker将会在效率上有明显优势。

  (2)docker利用的是宿主机的内核,而不需要Guest OS。因此,当新建一个容器时,docker不需要和虚拟机一样重新加载一个操作系统内核。仍而避免引寻、加载操作系统内核返个比较费时费资源的过程,当新建一个虚拟机时,虚拟机软件需要加载Guest OS,返个新建过程是分钟级别的。而docker由于直接利用宿主机的操作系统,则省略了返个过程,因此新建一个docker容器只需要几秒钟。
- **问题迭代**
    - **你知道run之后的原理吗？**
        - 首先docker按照镜像名字在本机中找到直接以镜像为模板创建一个容器，没有，去配置的仓库进行查找对应的镜像，找到下载到本机。然后运行容器。找不到否则报错。

K8S

0x09-项目篇 ☆

面试官 你好 我叫贾宝宝 ，目前华北理工大学大三在读生，在学校期间 跟随学院老师 ，参与3个项目的研发。分别是雄安新区 宝利鑫光伏发电 白洋淀票务 三个系统，白洋淀票务我主要整体负责，剩余两个项目是部分模块的开发。通过三个项目的开发，让我对整体的开发流程有一个比较深入的理解。其中遇到比较典型的两个问题。第一个问题 1.在文件下载的过程中由于数据量过大，导致需要2分钟的时间才可以下载完毕，通过使用线程池技术分化线程并发执行，将下载时间缩短在10秒内。第二个问题 1.为了保证项目的安全，使用了BJCA进行安全加密。第三个问题 1.为了保证与旧系统之间数据的一致性，使用多数据源解决了数据的一致性问题。

- 如何更好的介绍自己的项目
    - 系统架构，数据流程，业务价值
    - 你的重点是要能够把这个项目干了啥，**业务流程**是怎么样 的，**对接的数据流**是怎么样的，商业价值是怎么样的 （反例:不是我做了一个支付模块 调用了wxpay）
    - 注意点？
        - 1.面对的用户群体是谁？
        - 2.软件做什么业务，提供什么服务？
        - 3.数据流程是什么样子的？项目核心流程说一下
        - 4.使用的技术 框架
        - 5.遇到的坑 怎么解决？
            - 开始没有索引，-》索引-》
        - 6.负责那一块？ 详细谈一下？
            - **提前准备好 剧本。**
        - 面试官目的
            - 了解以前做过什么，项目规模，难易程度 来间接考察你的角色，软件开发能力。
- 项目中遇到的问题 如何解决？
    - 白洋淀
        - 新旧系统之间数据库一致性问题
            - 亮点 分布式事务
        - 订单支付->定时框架
    - 光伏发电
        - XML文件上传。原样式显示。前端->XML配置文件
        - 数据量较大-使用Oracle数据库-多线程技术下载文件操作
        - Hibernater Session连接释放，资源的占用
    - 雄安拆迁测算平台
        - BJCA数据安全问题
        - MQ异步发送
        - 微信支付 流程 `todo`
        - 自定义注解`TODO`
- 为什么这样做？优缺点分析
    - 画系统架构图->数据流向
- 项目过程中遇到的问题，以及如何解决的
    - 实习初期不适应，加班学习解决 请教别人。

- 遇到棘手的问题 如何解决
  - 分析法，将流程进行走一遍，分析是那一层出现的，然后进行细节定位。
- 你是如何学习的
- 对于职业生涯的规划
  - 目前技术的夯实阶段，经验积累，技术提升，然后选择一个技术方向专供，比如存储方向。
  - ps:项目描述，尽量不要摆自己做了什么功能和模块 遇到了什么问题？你做了哪些工作？遇到了哪些困难？最后达到了什么效果？
- 在工作中给你分配超出你能力范围的任务，你应该怎么做
  - 先庖丁解牛，按自己解决，如果确实遇到问题，及时反映自己的进度，让同事帮自己解决问题。

1.白洋淀票务系统

2.宝利鑫光伏发电

3.雄安新区

- 数字证书
  - 验证证书是否有效。1）验证证书是否在有效期内：证书中会包含证书的有效期的起始时间和结束时间。2）验证证书是否被吊销。被吊销的证书是无效的，验证吊销有两种：CRL和OCSP。CRL：吊销证书列表，证书被吊销后会被记录在 CRL 中，CA 定期发布 CRL，应用程序可以依靠 CRL 来验证证书是否被吊销。有两个缺点：1）可能会很大，下载很麻烦。2）有滞后性，就算证书被吊销了，应用也只能等到发布最新的 CRL 才能知道。OCSP：在线证书状态检查协议，应用按照标准发送一个请求，对某张证书进行查询，之后服务器返回证书状态。OCSP 可认为是即时的（实际实现中可能有延迟）。3）验证证书上是否有上级 CA 签名：每一级证书都是由上一级 CA 证书签发的，上级 CA 证书还可能有上级，最后会找到根证书，根证书即自签证书，自己签自己。以上三点只要有一个没通过，这张证书就是无效的，不该信任。

4.轻量级RPC

- **RPC和http对比？**
  - 速度来看 rpc比http速度更快 底层都使用tcp协议，http协议信息比较臃肿
  - 难度 rpc实现难度高 http比较简单
  - 灵活性 http让用户不用关系具体细节，跨平台，跨语言。
  - 都有不同的场景 需要进行权衡

5.问题迭代

- 你觉得你在项目中印象比较深刻的地方是什么？或者说你的两个项目里，关键点是什么，说一下。

6.HR问题

1. 为什么选择我们公司

这个问题就是在考察我们对公司的了解了，所以要对投递的公司业务情况、发展历程了解清楚，可以和面试官多聊聊自己对公司目前主营业务的想法，以及对公司文化的向往，这些回答都不会踩坑的。

参考答案**：**公司从成立到现在，业务不断增长，每个领域的成绩都在前列，大家有目共睹，公司对人才的渴望以及待遇都让我看到对人才的重视，企业文化也很吸引我，一直想来贵公司工作。

2. 为什么做这个岗位

这个问题就是谈谈你对这个工作岗位的认识以及了解具体的工作是做什么的，还有个人能力对于这个岗位的匹配度，一定要把岗位职责以及岗位要求了解清楚哦~

3. 简历上的某个项目有什么困难，怎么解决的

一般来说这个问题提出的时候面试官会让我们对这个项目做一下介绍，首先要把自己的个人项目经历梳理好，明确简历上写的每个项目的细节，这个项目具体要做什么，是要实现什么结果，自己其中负责了什么，有什么进展，目前达到的结果如何，其次是谈在做项目的时候遇到的困难，比如经验少对工作不熟悉，或者项目中用的工具不了解等等，怎么解决就可以说自己花花时间通过什么渠道用什么工具进行了学习，以及如何向有经验的人讨教之后，有了怎样的提升，这样就会给面试官留下你是一个能独立思考并且勤奋好学的同学的形象。

参考答案：我组织身边的同学参加了数学建模比赛，并对每个人做了分工，我负责资料搜集，A同学负责编程，B同学负责论文拟写，在比赛期间，我们通过对编程软件的学习以及以往资料的总结，最后获得了三等奖，参加比赛遇到的困难主要是大家的课程安排不一样，能在一起商量的时间很少，但是我们还是尽量多沟通，多抽时间一起学习来完成各自负责的内容，在这期间我们三个人结下了深厚的友谊，也积累了比赛经验。

4. 对于实习的工作的成果的看法，思考

有实习工作过的同学会被问实习工作，我们在回答的一般都会说自己的某项能力有了提升，对具体工作成果有了明确的数据表达，其实面试官主要是想知道你负责的工作对工作整体而言是有怎样的作用，比如你负责的一部分工作内容在你上手之后有了怎样的提高，对于工作整体而言有什么推动作用，如果进展不大可以说在慢慢摸索，和上司也在探索分析，并告诉面试官你们分析的结果以及以后的策略，这样会给面试官留下考虑周到并喜欢思考的印象。

参考答案：在我入职的这段时间里，负责我们部门的人才招聘工作，工作内容有简历筛选，和候选人以及业务主管沟通面试时间，以及后续进程的完善，在这期间完成了300+的简历筛选，并为我们部门带来了10+优秀人才的工作，我个人的沟通能力也得到了提升。

5. 个人的优点缺点

优点的回答主要是针对投递的岗位的，投销售岗位可以说自己外向善于沟通表达，投技术岗可以说自己稳重细心有责任感，投职能岗可以说自己细心认真负责，投产品运营可以说自己细心，善于思考分析总结，擅长沟通和团队合作，有时候面试官会让我们举例子来证明自己的这些特质，记得要准备好这些例子哦~

参考答案：投销售岗可以说我是个乐观外向喜欢交朋友的人，平时在生活中不喜欢抱怨，愿意积极想办法去解决，我周围的朋友也说我很仗义，经常帮助他们，跟我在一起很欢乐，遇到问题我都愿意给他们建议帮他们分析，他们也愿意听我说，我相信我能做好这份工作，给公司带去更多的客户。

缺点的回答最好不要说太追求完美，脾气大这类，回答的思路应该是说一些不会影响大局的缺点但是能用自己的方式在工作中积累去改变以及提高的。

参考答案：我是一个喜欢规划的人，但是在做决策之前会顾虑很多，导致会浪费一些时间错过一些时机，其实实践多了有了经验加上把握以往的规律就能够快速准确地做出决策的，以后在工作中以后让自己快速学习做好规划的。

6. 职业规划是什么

职业规划的问题是为了明确你的稳定性，一旦答了工作两三年准备再辞职创业或者去考研考博都不会给面试官留下好印象，作为应届生应该拿出谦虚的态度，表明自己在本公司本岗位深耕发展的决心。

参考答案：我会在刚入职的1年里尽快熟悉业务，向前辈学习，提升自己的专业能力，在之后的几年里能够做到独立负责某个项目某项工作，成为这个领域里的专业型人才，后续可能会根据工作内容的变动做细微的调整，但大方向是不变的。

7. 你还有什么问题要问我吗

这个问题最好不要说没有，要抓住这个机会向面试官展示我们的进取心以及对于这个工作的向往，也最好不要问薪资福利这类问题。

参考答案：我们这个岗位以后的工作日常可以说一下吗，新人培训制度如何，以后的晋升发展是怎样的。

0X10-方法迭代

『要获得什么，就需要牺牲什么』

- e.g. 比如TCP协议和UDP协议的对比学习：TCP协议能够保证传输数据的完整、有序，UDP协议是不可靠的，不需要接收方确认。但是，从另一方面来说，UDP协议的效率更高。
- e.g. 整个学习过程也是如此，要熟悉算法题，就需要付出足够多的时间&精力刷题总结。
- e.g. 所有的操作都是有代价的，有得有失，完美的解决方案往往难以奢求。

『新技术的起源是首先关注的』

- 相对于old things，新技术的出现是为了解决以前技术的问题
- e.g. Spring框架的出现是为了让程序员更加专注于业务逻辑，摒弃原生技术对于代码形式上的约束。
- e.g. Redis数据库在MySQL数据库之后出现，是为了解决解决高并发场景下的性能问题。
- e.g. JDK1.8的更新，提出了函数式编程，lambda表达式是为了让代码更加clean，更加提现业务逻辑。

『从实际问题出现，搭建知识系统』

- 对于每一种技术或者体系，往往会有一套主要的解决问题的体系，但是往往需要耗费大量的时间去实践。
- 从实际问题出发，"面经"表示面试官比较感兴趣的知识点：
    - 说明是实际项目中比较需要的point
    - 说明是能够提现被面试者专业知识积累的point
- 从实际问题出发，比如使用"RabbitMQ"，就要考虑到在分布式系统中消息队列经常需要面对的问题：宕机如何处理，如何实现数据的持久化

『脱离形式主义的大坑』

- 保证笔记的良好风格，方便复习以及形成知识体系
- 但是笔记最终的目的是为了提高效率，不是为了复制整理，形式不是一切
    - 一级标题：核心内容，二级标题&三级标题：主题，四级标题：常见应用和细节

『面试的引导性』

- 你了解Java中的集合框架吗
- 你谈到了ArrayList和LinkedList，那么有线程安全的List实现类吗
    - 既然提到了线程安全，你知道Java里的锁吗
        - 既然提到了Java里的锁，那你知道乐观锁吗

                - 既然提到了乐观锁，那你知道ABA问题吗
                    - 既然提到了版本控制机制，那你知道MySQL MVCC吗
                        - 既然提到了MySQL MVCC，那你知道MySQL隔离级别吗
- 你谈到了HashMap，那你说说HashMap的实现原理
    - 你谈到了红黑树，那你说说红黑树的定义
        - 红黑树是AVL树的改进，你知道其他改进吗
            - 你谈到了B+树，你说说B+树的应用吗
                - 你谈到了索引，那你说说为什么不使用B树吗
                    - 你谈到了B树，能说说操作系统的文件系统为什么要用它吗
                        - 我们谈到操作系统，那你说说内存管理吧
                            - ...
    - 你知道Hashmap的扩容机制吗
        - 既然谈到了扩容，你知道缩容机制吗
    - 你知道哪些安全的hashmap吗
        - 为什么不使用hashtable concurrenthashmap的区别

Redis相关

- 谈谈你对缓存击穿、穿透和缓存雪崩的理解
    - 布隆过滤器是什么
- 说一下相关的数据类型
    - zset底层什么原理
        - 什么是跳表

『What matters most leading first』

- 面经常考 > 面经出现 > 知识系统细节

0x11-场景、系统设计

1.购物车

购物车：

1. 购物车跟用户的关系?

a) **一个用户必须对应一个购物车**【一个用户不管买多少商品，都会存在属于自己的购物车中。】

b) 单点登录一定在购物车之前。

2. 跟购物车有关的操作有哪些?

a) **添加购物车**

i. **用户未登录状态**

1. 添加到什么地方?未登录将数据保存到什么地方?

a) Redis? --- 京东

b) **Cookie**? --- 自己开发项目的时候【如果浏览器禁用cookie】

ii. **用户登录状态**

1. Redis 缓存中 【读写速度快】

a) Hash ： hset(key,field,value)

i. Key:user:userId:cart

ii. Hset(key,skuId,value);

2. 存在数据库中【oracle，mysql】

b) **展示购物车**

i. 未登录状态展示

1. 直接从cookie 中取得数据展示即可

ii. 登录状态

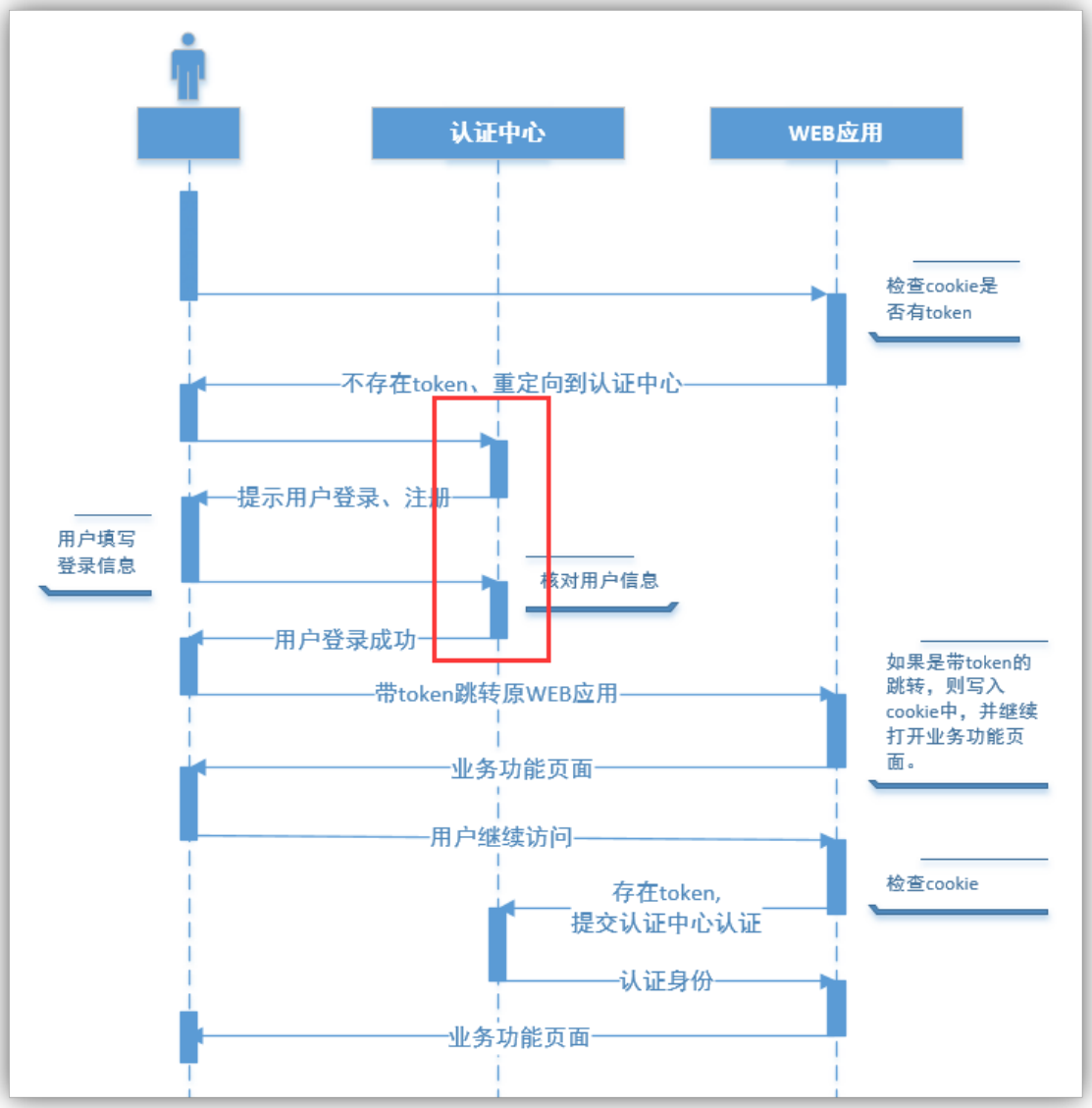
1. 用户一旦登录：必须显示数据库【redis】+cookie 中的购物车的数据

a) Cookie 中有三条记录

b) Redis中有五条记录

c) 真正展示的时候应该是八条记录

2.单点登录



单点登录：一处登录多处使用！

前提：单点登录**多使用在分布式系统**中。

单点登录流程

- 用户首次访问web站点，cookie中没有token，所以返回**重新请求认证中心** ，当用户名和密码正确获取到token，用户访问web站点就可以从cookie中获取到token，后续就不需要再次让用户再次登录，所以实现了在一个分布式系统中，用户不需要重复登录，只需要登录一次，其核心就是只需要第一次登录认证中心获取token。

京东：单点登录，是将token放入到cookie中的。

案例：将浏览器的cookie禁用，则在登录京东则失败！无论如何登录不了！

3.秒杀系统

- 超卖问题如何解决？
- 业务流程 扣减库存->生成订单->减少账户余额
- 超卖现象1
  - 系统中库存为1，但是产生了两笔订单。卖家在商品发货时，发现只有1件商品，但是有2笔订单。AB同时查看库存为1，都同时下了订单，原因在于扣减库存的时候，并发下产生了。
  - 解决方案：将扣减库存操作下沉到数据库中。update修改解决并发。(行锁-悲观锁)
- 超卖现象2
  - **解决超卖的方法**1）乐观锁 2）队列，建立一条先进先出的队列，每个请求加入到队列中，然后异步获取队列数据进行处理，把多线程变为单线程，处理完一个就从队列中移出一个，因为高并发，可能一下将内存撑爆，然后系统又陷入异常状态。或者设计一个极大的队列，但是系统处理速度和涌入速度根本没办法比，平均响应时间小。

4.大数据问题

- **在5000w的ip数据数据中找到被拉黑ip？**
  - 布隆过滤器，将拉黑的Ip存入到布隆过滤器中。
- **在1000w的整形数据流中获取每次加载数据的top90？**
  - 分而治之
- **两个10G文件存着int类型数字，8G内存，求交集（求出现了两次以上的）**
  - 分而治之

5.设计一个分布式锁 需要注意哪些事项。

- **互斥性**：在任意时刻，只有一个客户端能持有锁。
- **安全性**：即不会形成死锁，当一个客户端在持有锁的期间崩溃而没有主动解锁的情况下，其持有的锁也能够被正确释放，并保证后续其他客户端能加锁。
- 可用性：就 Redis 而言，当提供锁服务的 Redis master 节点发生宕机等不可恢复性故障时，slave 节点能够升主并继续提供服务，支持客户端加锁和解锁；对基于分布式一致性算法实现的锁服务，如 ETCD 而言，当 leader 节点宕机时，follow 节点能够选举出新的 leader 继续提供锁服务。
- 对称性：对于任意一个锁，其加锁和解锁必须是同一个客户端，即，客户端 A 不能把客户端 B 加的锁给解了。

6.限流问题

7.线上环境有遇到过OOM场景吗，说一下问题排查思路

8.线上CPU遇到过100%吗？说一下排查思路？

9.分布式事务的情景

- **面试问你分布式事务并不是要你解决方案，而是一种思想**
- 比如我们有一个前台微服务订单系统，和中台交易系统，都需要创建订单。在jd上点击提交后，我们应该衡量哪一部分的权重高，优先级高。而先做这一部分。而这也是数据一致性的问题？我们如何保证。大多数人会说我可以采用分布式事务来解决呀，其实，大多数的分布式事务，我们可以借助于MQ进行传输数据。先创建中台交易系统的订单在创建前台微服务订单系统的订单，而通过MQ进行传输。
- 我们来分析一下失败的原因 2.网络IO 系统宕机等问题，这部分我们无法保证。具有不可控性。
- 1.MQ本身的机制 我们可以使用ACK来保证，其实这部分是可以通过业务去解决。**不要觉得非要什么阿里的seata，2PC啥的，才是分布式事务的解法**，依赖于公司的体系体系， **那些理论应该学，但是实际的公司都是具体的业务情景** 因为可以通过业务的解决来规避 即使无法规避，也需要去看业务的重要性，成本

10.SQL练练连

- 写sql语句（统计一个班里男生和女生的人数）

11.高并发抢票系统设计思路

针对高并发逻辑处理

比12306的火车票业务，成千上万的人抢票，京东的热门商品，同一个数据下并发修改很多

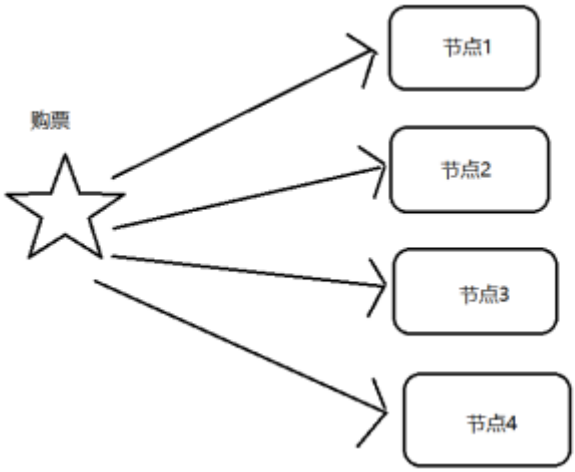
其实我们可以根据具体的业务，设计出各种各样的解决方案，这个要结合具体业务分析

，比如很多银行转账有日限额。举个例子，12306 g65高铁票，有2000张。很多人在抢票，每一次修改余票，事务操作会大大降低系统的性能。因为，会加锁。那么用户体验肯定很糟糕

，我分成20个子账户，每个账户100张票，然后，抢票的时候，程序中在1到20中随机取一个数，可以负载均衡到不同的子账户，这样就可以把群分散了，提高系统并发写的能力，如果某个子账户余票为零，可以重新计算一次随机数，然后路由到另外的子账户，concurenthadhmap的分段锁，是一个类似的思想，超过了余额上限，比如。普通用户一次性最多转账5万，vip一次性最多20万，我们有了这个限额，就可以很轻松的分子账户，并且，不同的子账户级别不同

。比如普通子账户100个，最低余额是5万，所有普通用户的转账都路由到普通子账户，vip子账户20个，分配的余额肯定比普通子账户更大，最后一个就是总余额的显示问题，这个好办，直接用redis缓存，每次成功转账后，同步修改redis缓存，如果所有大部分子账户余额不足

，这样，可以临时从程序中删除该子账户的引用，当然，这些做起来肯定不轻松，特别是要做到自动化，可以结合zk等，来自动化管理客户端的重连，路由等逻辑。



12.附近人，打车

用户表

uid-0	x	y
uid-1	x	y
uid-2	x	y

地区-用户关联表

R0	uid-1	uid-3
R1	uid-4	uid-7
R2	uid-0	uid-2

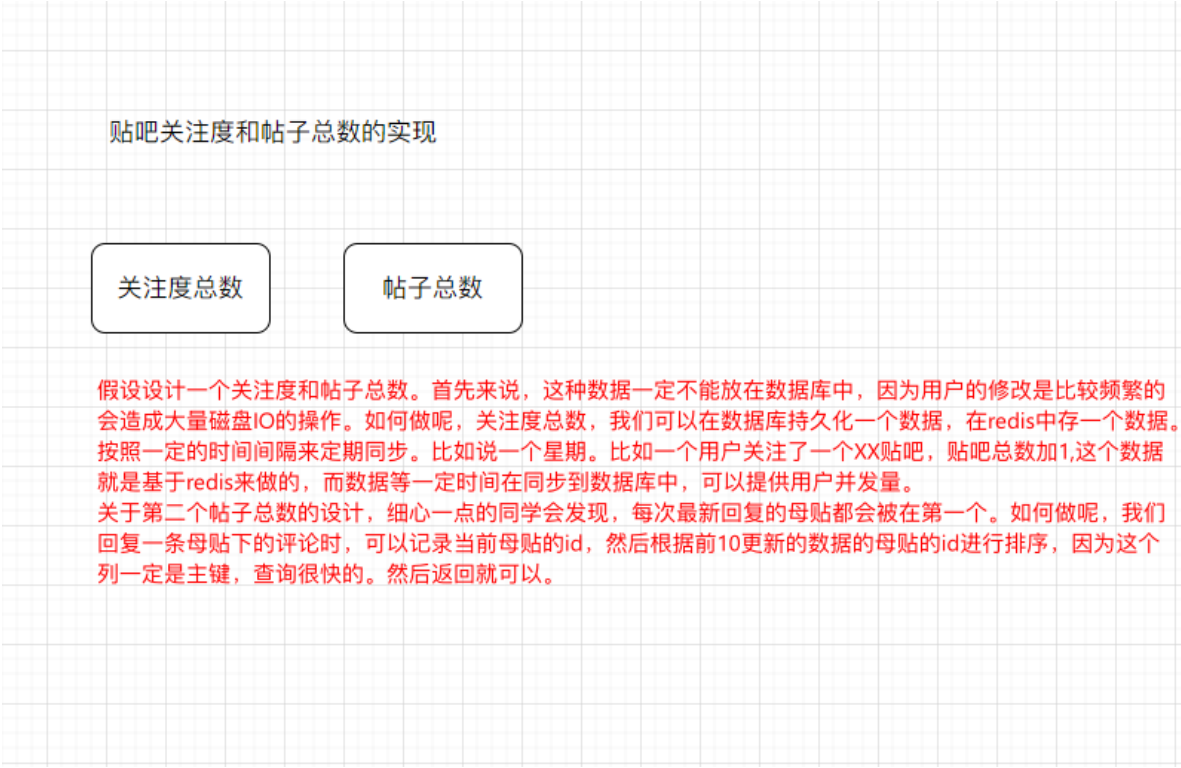
分区表

R0	R1	R2
R3	R4	R5
R6	R7	R8

1.建立一张用户表，存储到对应的Redis中
2.建立一个分区表
3.建立一个地区-用户管理表
4.根据用户的地理信息 计算出所在的区域，然后添加到对应的关联表中。
根据关联表查找即可

不推荐使用Mysql的原因
1.频繁的读写，IO瓶颈
2.速度问题
3.用户数据允许丢失

13.百度贴吧



0x12-白板编程

0.快排

```
    private static void quickSort(int[] arr, int start, int end) {
        if(end <= start){
            return;
        }

        int piovt = partition(arr,start,end);//获取到piovt下标
        quickSort(arr,start,piovt-1);
        quickSort(arr,piovt+1,end);
    }

    private static int partition(int[] arr, int start, int end) {
        //piovt 标杆位置， counter 小于piovt的元素的个数
        int piovt = end;
        int counter = start;//为什么是start位置呢，因为每次partition过程中。
        //counter位置是不确定的，不能从0开始
        for (int i = start; i < end; i++) {
            if (arr[i]<arr[piovt]){  //如果小于Arr[piovt] 移动到piovt前面 否则不动
                int temp = arr[counter];
                arr[counter] = arr[i];
                arr[i] = temp;
                counter++;
            }
        }
        //标识位和最后的
        int temp  = arr[piovt];
        arr[piovt] = arr[counter];
        arr[counter] = temp;
        return counter;
    }
```

1.归并

```
    private static void mergeSort(int[] array, int left, int right) {
        //条件判断
        if (right <= left) {
            return;
        }
        //中间下标位置
        int mid = (left + right) >> 1;

        //左边排序
        mergeSort(array, left, mid);//care mid
        //右边排序
        mergeSort(array, mid + 1, right);
        //合并
        merge(array, left, mid, right);
    }

    private static void merge(int[] array, int left, int mid, int right) {
        //临时数组
        int[] temp = new int[right - left + 1];

        int i = left, j = mid + 1, p = 0;

        //比较大小 后存储
        while (i <= mid && j <= right) {
            temp[p++] = array[i] <= array[j] ? array[i++] : array[j++];
        }
        //剩余的存储
        while (i <= mid) {
            temp[p++] = array[i++];
        }

        while (j <= right) {
            temp[p++] = array[j++];
        }

        //赋值给原来的数组
        for (int k = 0; k < temp.length; k++) {
            array[left + k] = temp[k];  //care
        }
    }
```

2.请手写一个死锁

```
        Object obj1 = new Object();
        Object obj2 = new Object();
        //线程t1
        new Thread(new Runnable() {
            @Override
            public void run() {
                synchronized (obj1) {
                    try {
                        Thread.sleep(1000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
```

```
                }
                synchronized (obj2) {
                    System.out.println(Thread.currentThread().getName());
                }
            }
        }
    },"t1").start();

    //线程t2
    new Thread(new Runnable() {
        @Override
        public void run() {
            synchronized (obj2) {
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                synchronized (obj1) {
                    System.out.println(Thread.currentThread().getName());
                }
            }
        }
    },"t2").start();
```

3.两个程序依次输出 11/22/33 ？

```
    Thread t1 = new Thread(new Runnable() {
        @Override
        public void run() {
            for (int i = 0; i < 4 ; i++) {
                System.out.println("t1:"+i);
                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    });

    t1.start();

    Thread t2 = new Thread(new Runnable() {
        @Override
        public void run() {
            for (int i = 0; i < 4 ; i++) {
                System.out.println("t2:"+i);
                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    },"t2");
```

```
        t2.start();
```

4.单例

```
    private static volatile SingleDemo instance = null;

    private SingleDemo(){
        System.out.println(Thread.currentThread().getName()+" new instance!");
    }

    //dcl 双端检查
    public static SingleDemo getInstance(){
        if (instance == null){
            synchronized (SingleDemo.class){
                if (instance == null){
                    instance = new SingleDemo();
                }
            }
        }
        return instance;
    }
```

5.生产者消费者模型

1.synchronized+notify+wait

```
/**
 * @author i
 * @create 2019/12/29 14:57
 * @Description 生产者消费者 1 版本
 */
class MyShareData{

    private Integer count = 0;//资源

    public synchronized void increment()throws  Exception{
        while (count != 0){
            //如果count 不等于0 当前线程等待
            this.wait();
        }
        count++;
        System.out.println(Thread.currentThread().getName()+" 生产了一个"+count);
        this.notify();
    }

    public synchronized void decrement()throws  Exception{
        while (count == 0){
            //如果count 不等于0 当前线程等待
            this.wait();
        }
        count--;
        System.out.println(Thread.currentThread().getName()+" 消费了一个"+count);
        this.notify();
    }

}
```

2.lock+Condition+awiait+singal

> Lock 替代了 synchronized 方法和语句的使用，Condition 替代了 Object 监视器方法的使用

```
/**
 * @author i
 * @create 2019/12/29 15:04
 * @Description 生产者 消费者 2版本
 *  Lock 替代了 synchronized 方法和语句的使用，Condition 替代了 Object 监视器方法的使用
 *
 */
class MyShareData {

    private Integer count = 0;

    private Lock lock = new ReentrantLock();
    private Condition condition = lock.newCondition();

    //生产
    public void increment() throws Exception {
        try {

            lock.lock();
            while (count != 0) {
                condition.await();
            }
            count++;
            System.out.println(Thread.currentThread().getName() + " 生产了一个" +
count);
            condition.signal();

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            lock.unlock();
        }
    }

    //消费
    public void decrement() throws Exception {
        try {
            lock.lock();
            while (count == 0) {
                condition.await();
            }
            count--;
            System.out.println(Thread.currentThread().getName() + " 消费了一个" +
count);
            condition.signal();

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            lock.unlock();
        }
    }
```

```
}
```

3.volatile/BlockingQueue/AtomicInteger

> 版本1中通过syn来保证原子操作，但是锁的粒度比较大。重量级。使用notify和wait 并且不确定唤醒哪一个线程有一定的随机性。
>
> 版本2中使用Lock 和 condition来进行控制
>
> 版本3 使用volatile BlockingQueue AtomicInteger 来实现组合使用。

```
/**
 * @author i
 * @create 2019/12/29 16:42
 * @Description 生产者消费者 volatile/BlockingQueue/AtomicInteger
 *
 */
public class ProdConsumer_BlockQueueDemo {

    private volatile boolean flag = true;//标志位
    private BlockingQueue blockingQueue = null;
    private AtomicInteger atomicInteger = new AtomicInteger();

    //构造
    public ProdConsumer_BlockQueueDemo(BlockingQueue blockingQueue) {
        this.blockingQueue = blockingQueue;
    }

    //生产者
    public void pro() throws InterruptedException {
        String str = null;
        while (flag) {
            str = atomicInteger.incrementAndGet()+"";//原子操作增加值
            System.out.println(Thread.currentThread().getName() + "\t 生产了
"+str);
            blockingQueue.offer(str,1,TimeUnit.SECONDS);//生产
            TimeUnit.SECONDS.sleep(1);
        }
        System.out.println(Thread.currentThread().getName()+" boss叫停了服务 生产者
退出。。。");
    }

    //消费者
    public void con()throws  Exception{
        String str = null;
        while (flag){
            str = (String) blockingQueue.poll(2L, TimeUnit.SECONDS);
            if (null == str || str.equalsIgnoreCase("")){
                flag = false;
                System.out.println(Thread.currentThread().getName()+"消费者退
出。。。");
                return;
            }
            System.out.println(Thread.currentThread().getName()+" \t 消费了
"+str);
```

```
        }
    }

    public void stop(){
        flag = false;
    }
}
```

6.个线程依序打印ABC

> 创建线程3个线程 B线程执行前a.join()
>
> 大体思路:借助juc中Lock+Condition实现，通过一个变量来标志执行哪一个线程 condition中唤醒对应的线程。

```
/**
 * @author i
 * @create 2019/12/29 16:03
 * @Description  ABC线程每次依序打印 ABCABC循环 10次
 */
class MyShareData{

    private Integer count = 0;
    private Lock lock = new ReentrantLock();
    private Condition cA = lock.newCondition();
    private Condition cB = lock.newCondition();
    private Condition cC = lock.newCondition();

    public void printA(){
        try {
            //1.判断
            lock.lock();
            while (count != 0){
                cA.await();//等待
            }
            //2.工作
            System.out.println(Thread.currentThread().getName()+"\t A");
            //3.唤醒其他线程
            count = 1;
            cB.signal();
        }catch (Exception e){
            e.printStackTrace();
        }finally {
            lock.unlock();
        }
    }

    public void printB(){
        try {
            //1.判断
            lock.lock();
            while (count != 1){
                cB.await();//等待
            }
            //2.工作
            System.out.println(Thread.currentThread().getName()+"\t B");
            //3.唤醒其他线程
```

```
                count = 2;
                cC.signal();
            }catch (Exception e){
                e.printStackTrace();
            }finally {
                lock.unlock();
            }
        }

        public void printC(){
            try {
                //1.判断
                lock.lock();
                while (count != 2){
                    cC.await();//等待
                }
                //2.工作
                System.out.println(Thread.currentThread().getName()+"\t C");
                //3.唤醒其他线程
                count = 0;
                cA.signal();
            }catch (Exception e){
                e.printStackTrace();
            }finally {
                lock.unlock();
            }
        }
    }
```

7.手写LRU算法

具体看algo模块

0x13-心得

简历相关

简历不要摆功能，而是体现一个架构和思想, 你可以着重讲你做的某一个点，对系统的支撑, 要直击要害, 你摆再多的crud功能，真的意义不大

举个例子，做的淘淘商城，最好准备一张图 , 能够把架构画出来, 如果面试官问你一些问题，你说表述不好, 我画个图, 然后圈出图中最核心，最困难的地方, 包括你们遇到的问题，和怎么解决的, 淘淘商城讲到了一个静态化, 你们不能只在简历里面，说我会freemaker, 就完事，你要说我们最开始没有采用静态化后来发现数据库瓶颈很高, 性能一直上不去，上一段时间公司的数据库大规模宕机，因为业务的发展, 后来我们重新设计了一个静态化的方案, 前端采用5台nginx做静态化缓存, 巴拉巴拉的, 然后采用异步的方式, 用activeMQ推静态化数据到nginx, 你要这样说，所以，你要准备好你的图, 让面试官跟着你的思维走, 而不是让面试官拿着你的简历不断地对你攻击,

你们在简历中可以把淘淘商城按照京东的样子, 分成5个子系统, 采用了5个域名, 至于是什么，你们自己想, 然后最重要的是，把图画出来, 要让面试官知道，你们做的是一个大型系统, 你们简历就写京东商城或者国美商城什么的, 但是，不要说你是负责架构的, 你看看，京东有多么大, 比如，你做了solr那一块, 你说，你做了整站的搜索子系统, 你不要看京东的搜索框就一个框框, 那只是京东的搜索子系统的一个页面接口, 真正庞大的东西都在后面, 其实你不需要研究的多深, 你只需要准备的多么充分, 一个系统，架构

是怎么样的，用到了什么技术，开发或者部署的时候遇到了什么问题，最后怎么解决了这些问题 , 其实你们写的2到3年互联网工作经验的, 至于说细节的东西，本质上还是javaweb那一套了, 如果你真想让你的商城简历出彩, 你就把京东的这100个子系统全部过一遍,

-

软件架构

- **架构设计的目标一定是解决当下或者未来短期内的痛点问题，**

重复代码

- 在编写业务时，很有可能出现相同的一段代码逻辑，我们其实不能单纯的任务将相同的逻辑代码设计到一块，因为本身来说，不同的业务只是出现了耦合性的冗余代码，但是业务之外，我们还需要另外进行判断，所以说，我们应该正视冗余代码的存在，如果说一段代码抽取出来，可以为整个组件 设计一个系统使用，那是必然的。

框架设计

- 类比Spring MVC设计。
    - HandlerMapping抽象的是Handler的映射，比较基础的映射包含 a.静态资源的映射 b.动态资源的映射。动态资源分为视图和非视图两种形式。 非视图可以理解为字符串、JSON、XML等格式。视图可以理解为JSP、FM、THF等模板引擎，可以看到设计一套基础的MVC框架是比较困难的。在Servlet时代，我们需要去编写对应的servlet来进行处理，而参数的解析，获取、封装，都需要程序员自己完成，但是Spring体系中MVC框架，从根本上解决了这一难题。程序员更加专注于业务逻辑的处理。从某种角度看Spring也是一套业务框架，只不过J2EE解决的问题点在于如何利用现有的框架，进行业务逻辑的商业业务的处理。而Spring解决的技术业务问题点。我们应该正视框架的存在。**最难的点就是 动态代理，Java事务 Spring异步 多线程 动态代理这种比较难理解，动态代理，反射，泛型框架的大杀器**
- **自动生成框架理解？**
    - 针对于一些自动生成技术，快速开发平台。在一定程度上，对于公司现有开发有一定的便捷性，但是对于程序员本身来说，我们应该理性看待这样的技术。因为这种技术的出现思想是想一劳永逸的解决 现有的问题，模板化。但是 业务是会改变的，随着业务 技术 架构理念的改变，不能可一层不变的解决，这些自动生成技术都会注销没落于历史的舞台之中。所以，为了解决现有问题的开发效率，可以使用，但是没必须深究。所以 紧跟时代潮流，学习那些顶级的开源框架和平台。

项目依赖

- 前倾概述 在一个maven工程中 有不同的模块 都有对应的spring配置 ，但是项目的依赖在pom中直接添加了。 实际上不能说一个模块就相当于与是一个Spring容器了，只是说引入了spring jar包了。好比说，我们编写的应用程序在tomcat中，webapp依赖于服务器环境，但是我们编写的servlet需不需要serlvet的包。只是为了编译通过，实际上Spring容器只有一个，web监听器启动的哪一个。引入项目导入了service包，dao包。放在了classpath下，spring启动的时候，会扫描service的包，dao的包上注解。
- 其实就算没有maven，我们项目导入hibernate项目 spring项目 导入的jar都是别人开发好的项目，所以，我们的项目其实就是多模块的，拿来主义就可以了。maven在这块做的很好，可以按照维度进行拆分。而是将整个项目拆分成子项目，最后集成。模块拆分，spring做的比较好。
- 微服务的各个服务可以独立运行，如何解释呢 独立进程服务，如果说每个微服务都有Spring，那微服务就是另一种形式，容器都是独立的。微服务就是多个Spring容器。 对比的话就是远程间调用和进程内调用。拆分子项目的时候，我们可以多一种思路，实际上，大多数的软件是没必要微服务化的。根本就没有复用。一般来说 微服务的目的是为了进程间调用，共享服务。很多公司并没

有做到服务的下沉 通用 复用。而远程调用，是性能和分布式事务的敌人。

- **什么样的项目需要使用微服务作为支撑呢？**

  大型系统，比如很多系统之间是公用的，说一个简单的例子，比如说淘宝的订单系统 购物车系统，可以被手机淘宝 pc淘宝 等其他系统 直接公用，而无需开发一套订单系统 而这套系统完全可以交付给一部分人来做这个功能，抽离出去。 这是SOA吗 其实只是比较像 但是具体还是有区别的？ SOA的核心是系统总线，集成多跨语言平台的项目，SOA类似于主板，服务之间的调用必须依赖于SOA才可以，CPU想要和磁盘打交道必须通过总线才可以。另一个特点就是可插拔性。所有服务进驻到总线注册，由总线实现所有服务间的协调，Spring不是在成千上万的项目中复用吗？不过他是以进程依赖的方式进入到对方的进程中。对比redis和cache 只是交互方式不同 一个是进程间 一个是进程内。从一种宏观角度看待 其实软件开发不久那么回事吗。所以，我们通过从不同的维度看待技术，微服务的本质就是从进程内升级到进程间的交互。

中台

- 中台解决的问题点：企业内部多种类似的前台业务中共享的模块
- 服务化架构
  - **微服务分层和中台**

    微服务设计中一共三层 网关层 前台层 后台层，而我们熟知的中台是独立于这三层的。如果非要给中台下一个定义，那就是系统级能力复用平台，系统级说明并不是在某一个应用系统中，而是一个更上层次的级别，能力复用。这样类比一下，如果说对于一个企业 会有一些公共的基础服务设施 比如电商系统 安全 物流 等 是可以对多个系统进行服务，而这种服务就可以抽取出来，作为公共服务。而上面说的网关层 前台层 后来层 都是依赖于中台进行迭代开发。因此中台是一个更加宏观的 全局的概念。

    好了，中台我们大概有一个了解后，说一下微服务设计中的三层 一般来说 网关层是对于整个系统的api入口，对于用户来说。我说通过网关进行访问某个具体的前台服务，网关进行路由到某一个前台服务。一个应用中 只会有一个网关。而网关路由的前台服务对应多个。前台处理调用真正的api 即后台服务。总结 但是在软件开发中 对具体应用 具体业务 具体考虑。

业务架构

- 架构是一个比较宏观的词汇，但是对于不同人有不同的理解，无法去用一个标尺去衡量，因为架构是多种维度的，就像看一个三角形立体，每个人所看到的的是不一样的。首先来说第一个维度，那就是业务架构，不管对于任何系统来说，我们都是在用技术不断的解决业务中出现的问题。架构设计一定是在解决问题，如果为了解决而技术，那么是一个不合格的架构师。
- 通常，我们知道一个程序是由一个**对象图**进行构建出来，每个对象依附于别的对象，进行行为操作，处理业务。但是划分依据是什么。也很难衡量。我们可以将粒度从小到大不断进行粗化。最小单元就是方法 然后是对象，会根据不同业务划分成一个模块，比如用户模块，登录模块。再次不断粗化就是子系统，子系统群，最后是网关。而大多数架构师所要做的事情，也是从宏观到微观的不断细化。很多人的理解，就是架构就是将应用程序中设计到的业务，都进行分门别类的划分出，具体到类、方法级别。其实这是一个误区。**大多数业务架构师所要做的一定是解决核心业务**。比如我们做一个打车滴滴系统，那一定是路线规划的算法是核心。对应到生活中，我们装修一个单元房，厕所的核心就是上厕所，你不可能说，我要安装到客厅中，这是背道而驰。需要根据业务分析出业务中核心点在于什么。在实际开发中需求是变化莫测，我们应该设计出符合当前业务发展的架构，以及对于远期有一个软件架构上的可拓展性(新功能的添加，不会与旧应用进行高度耦合)，可维护性。
-

技术架构

缓存视角

客户端-Http缓存

在一次http请求中，response中响应中包含有`cache-control`这样的属性，用于建议浏览器客户端进行数据缓存。但是仅仅是建议。因为对于浏览器客户端来说，缓存了数据。如果相同url的数据发生了变化，那么我们就无法及时获取到相同的数据。虽然说静态文件中属于固定的数据，(顺便提一下，我们所有静态文件的处理 都会经过defaultServlet来处理静态数据的 404都会经过他) 但是难免会发生修改。所以当客户端缓存了数据，使用了相同的url访问，只是获取到了缓存的数据。**那么这个是怎么解决浏览器缓存和服务端数据一致性问题呢?**

其实挺简单，一般来说，服务端返回一个相应都会返回一个带有`last-modified`最后一个更新的时间，而客户端请求每次都会带有一个`if-modified-since`。然后服务端接收到请求 解析该字段，判断当前文件的的修改时间和当前时间是否相同。如果相同说明数据没有发生改变。返回`304`(服务端已经得到了Get，但是文件没有发生变化)。否则，就更新数据。

其中，不同的业务可以驱动不同的技术创新。归根结底，我们要理解技术的本质。

总结，我们需要根据自己业务的特性对一些长时间不会修改的请求数据进行缓存。这样才能很好的提升整个系统的性能瓶颈。这也是靠近客户端最近的缓存机制。

CDN-HTTP正向代理缓存-》HTTP反向代理缓存

代理，我们可以理解，那就是我打官司，我请律师帮我打，这就是代理。

正向代理和反向代理的区别就是 正向代理代理的是客户端，而反向代理的代理是服务端

正向代理需要在客户端配置，而反向代理对于客户端来说是透明的。

首先 我们来分析一下正向代理，如果对于每个用户都使用正向代理，那么每个用户都必须在本地浏览器中配置正向代理服务器。这样一来 是很麻烦的。如果用户强制不配置，那么服务端是无能为力的。因此，我们换一个思路，那就是反向代理。一把实现 我们是通过CDN 内完成 反向代理缓存。而这个服务对于 中小企业，只需要购买相关的云服务，就可以实现。将一些静态文件存储到用户最近的CDN服务器上。用户访问一下静态资源 例如 视频 图片 文档等就可以直接返回响应。无序再次请求服务端。而对于web动态资源 需要走服务器才可以请求。

反向代理服务器缓存

行业标准就是nginx，一般来说nginx既可以做反向代理服务器，也可以当做正向代理服务器。一般缓存的数据都是动态的json数据。

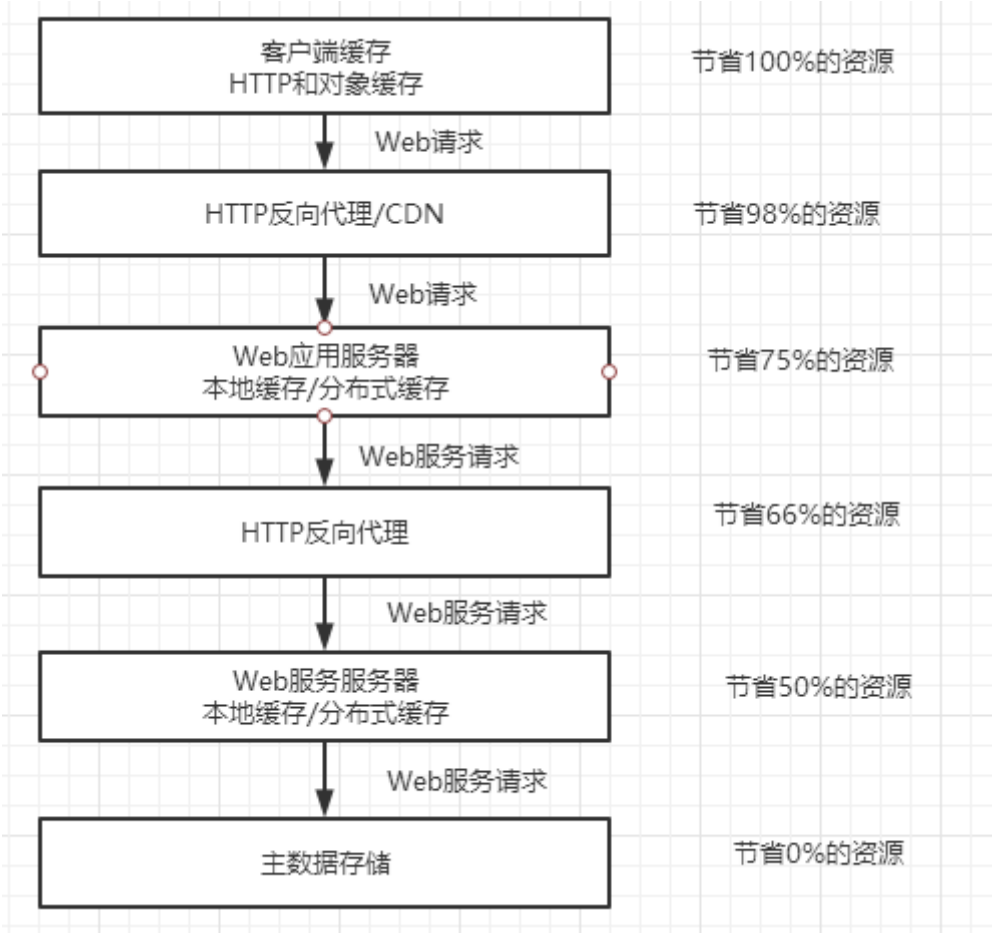
应用服务器localCache

应用服务器缓存 相比较分布式缓存 来说，性能更快。首次请求分布式缓存，将查询到的数据缓存到应用服务器缓存中。

分布式缓存

分布式缓存是最后一层保护数据库层面的缓存，当然还有数据库的查询缓存，但是这个缓存对于读写有瓶颈的数据库来说，我们应该慎用。

总结一下。**HTTP缓存>正向代理缓存->CDN缓存->正向代理缓存->应用服务器缓存->本地服务器缓存->分布式缓存->数据库查询缓存**



软件设计

- 软件很大，这个大是指业务上的大，还是并发访问很大，还用用户量大还是日活高。我觉的，在不同的方面，可以指不同的东西，并且，我们要对具体不同的情况，来分析。聚一个例子，微博、业务模式很简单，业务并不复杂，但是微博显然是一个大软件。用户量大，数据量大，日活并发大。我们思考一下，如果业务量比较小的话，我们可以采取头脑风暴的的形式开发软件，但是当业务架构大之后，显然这种形式是不足以支撑这种变化。那么如何一步步设计呢。**不同的系统有不同的设计方式**
- 比如说传统企业改造的项目，以中国平安为例子，开发保险业务。首先，平安的用户量已经很大，很多线下积累了一定的用户量，数据，以及内部员工使用系统都是已知的。这类系统在规划的时候，应该量化各种指标，比如，多少客服和日均业务。完全可以量化。qps,tps，日活，高可用的情况，都是可以得到的。**然后根据量化指标去设计系统组件，中间件** 进行压测，达到已经规划好的量化值。
- 但是如果对比一个初创互联网公司呢? 用户和业务发展是不明朗的。这种项目，我们应该短平快，不需要太多的头脑风暴，太多的架构设计，快速试错。所有的性能指标不明朗，从0开始，边走边看，边开发边重构。业务量如何上来，推到重来，细化精化。
- 总结，因为，我认为 任何设计，都应该关注一个很重要的点，那就是目前面临的情况，需求是什么？团队有对少人，项目资金多少，公司技术栈如何？包括老板的意志和高层规划，业务部分，市场部分对项目的预期和投入，这些应该作为设计的考量。所以说，架构设计，既权衡和取舍。我们不应该只看技术层面，技术层面的约束和考虑，是一个方面，但不是全部，我的观点是**没有一个放之四海而皆准的法则去设计** 每一个系统，都有不同的设计思路，这取决于这个系统锁所面临的各种影响力和约束条件。
- 比如，钱多 可以向阿里 成立中间件团队，研究mq,tfs,oss,建设私有云。大数据平台都上来，但是，我们前不多，公司搞不起来这样的投入，所以，我们要改思路，假如，业务的发展非常迅猛，市场竞争也很激烈，架构师更加应该关注系统底层技术架构，来适应业务的快速发展，并反哺业务。每个项目，面临的处境和条件都不一样，

- 总结，互联网模式 都要遵循演化模式，即使是巨头上线，也要如此。 **a.演化模式 b.直接收购**

工作

- 工作中如何快速上手项目？
  - 先宏观在微观
  - 1，这个项目是做什么的？商业目标？为谁服务，业务的大范围。 2，项目的基础架构，使用的技术栈，数据库，编程语言，前端技术，部署环境，关于架构的越详细越好。 3，我的角色是什么，领导要求我干什么，我的工作内容是什么。 4，把我要负责的项目下载到本地，可能会有很多红叉叉，跑不起来，那么，要跑起来，尽一切可能。这个过程中遇到了哪些困难，全部记录下来，一个不漏。
  - 如何做。将目前不清楚的罗列出Todo，按照优先级一步一步完成就可以。庖丁解牛。
- **工作中的思维方式**
  - 首先将需求理清楚，只有理清楚需求才可以动手做下一步，然后站在用户的角度思考问题，比如前段的页面是什么样子，具体的页面布局，以及需要哪些数据。然后打通之后就用假数据和Controller层进行交互，后面在一步步思考如何业务的编写，数据的交互。要学会拆解问题。 **无论如何功能，复杂也好，简单也好，总是离不开客户，服务，数据，逻辑，**交付给领导，你要记住，任何复杂的，都可以分解成简单的步骤，我们通过分解来隔离复杂性，而不让所有的复杂性相互扩散，

做项目一年的思考

1.技术角度

　这一段时间我都是一个需求的接受者，Leader拆解任务，被分配需求，与PM沟通。详设，完成开发。

在这个过程中，我的工作态度是，把简单的事情做好。视角是考虑事情的角度在系统的局部模块，并且更多的是技术角度，而不会取考虑系统本身的价值。

这里的局限在于，考虑问题太片面化，比如我考虑的视角是系统内的一个模块，而且更关注的是技术角度。

2.

参加了一些传统项目的开发，可以去从整体以技术角度把控一个小型系统。

去年12月开始回归互联网公司，12月-2020年2月，我一直是在从0到1开发新系统。这个阶段，我似乎可以技术与商业业务并行

去思考一个系统 我会写代码至深夜，也会和PM沟通用户体验，用户到底需要什么，无论是TO C还是TO B。

当我们向前走的时候，这个时候我们会发现，领导已经开始让你负责项目了，我可以再一次负责整体的项目，带着组里面的同学前行

这个时候，我开始关注业务线，我们这个业务线到底是在解决现实的什么问题？

业务线里面各个系统之间，每个系统的职责到底是什么？

我也会关注，每一次的项目，开需求建立，评审，详设，排期，测试，上线，全流程的跟进。我依然在继续。

这个时候，PM的Leader或者技术总监，或者业务侧的Leader，甚至运营

他们都会和你沟通，因为我思考问题，可以从他人的角度来看了。

PMleader关心业务的需求，我可以谈。业务侧的leader关心是个系统是符合他们的需求。

为什么需要和这些人沟通，因为，我们的系统就是为了解决某些问题

这些问题是来自现实，而不是冰冷的屏幕。

所以，总结一下，**从单一的技术模块-->系统技术角度->业务与技术并行-->现实问题与系统并行。**

2.工作中和领导以及同事的合作，同部门以及跨部门协作

1.与领导合作

我一直是一个时刻保持反馈的状态。比如我现在负责一个项目，我们给他每个大节点的情况反馈，每天的进度（可量化），我会时刻让他清楚

我在做什么，项目做到什么地步，是否具有风险。

尤其是项目汇报的时候，很多同学不发表一个可量化的数字。

领导：进度多少了。同事：由于难点，***，进度未知。

领导内心什么感受？他只想清楚项目的进度，他会关心你的某一个所谓的难点？细化到一个模块？可是这一点，很多同事意识不到。 工作要以结果为导向，可量化。

不管和任何同事，千万不要抱怨，抱怨的事情，留在自己心里，去找合适的人说，没有就烂在肚子里！！！！

2.部门内部的合作

首先是聆听，然后沟通，风险点即使反馈。公司就是工作，不要扯其它。

跨部门协作，最重要的一点，换位思考

这个举个例子

我们要上传一个Excel，其中有一列是城市。我们的程序要解析这一列城市进行存储。

程序员会思考：那么这个excel列必须符合一个特定的规范，比如陕西省***市**区，正则解析还是什么解析，性能，一次传多少条。

PM会思考：传excel是为了存储一下具体的业务信息。

可以为其余业务做什么铺垫，或者提供用户效率。

用户：每次需要花大量的时间，去制造这个Excel，去符合城市这一列的规范。

比如，程序员规范，这里必须传陕西省**市。

而实际用户侧的城市信息是不规则的，几万条数据，他们花费无数的时间，为了满足这一列的程序运行规范。

这个需求上线了，可以用。

可是，这个需求的价值在于哪里呢。毫无意义。

就是因为用户一条一条录数据太麻烦，这件事交给程序来做，才提高效率。

而现在，用户反而花大量的时间去制造excel，直到眼瞎。

这个需求是希望，我们智能解析城市这一列，进行智能匹配解析。

用户不需要花时间去Excel一条一条校验城市列，交给程序做，1秒反馈你。

生产力得到了极大的解放。

这是一个简单的小例子，我想表达的意思是，做一个哪怕在简单的需求，我们要换位思考，去解决真正的问题。

总结一下:与领导沟通，及时反馈，进度可量化，结果为导向。

与部门内同事沟通：先去聆听，做到心中有数，然后再去沟通。尽可能不要抱怨，这是忌讳！！！

这就是大部分程序员缺乏的

跨部门协作：换位思考是核心。打破程序员本位思考。

思考软件技术真正的意义，不要局限。

还有一点，我补充一个，就是要懂得妥协和取舍，权衡

就是，每个人心里都有完美的解释

但是，可能并不一样

程序员总想用最牛逼的技术，最牛逼的架构，最优雅的设计

但是，这种东西，可能会伤害另一些人的利益

在其他人眼里，换个角度，比如成本，人力，你这个并不是最好的办法

这个时候，要提升眼界

用更综合的角度去思考问题

可能要砍一些

可能要做的看上去并不难的完美的办法

有时候，你内心可能是不愿意的

但是，你一定要明白，你是为了大局

所以，权衡，取舍，考虑其他的因素，比如，站在产品的角度，站在公司程序员的角度，包括业务，技术栈的角度

比如，这个业务明显用mongo比较好

但是，公司没有人接触过

对mysql比较熟

你是个架构师，你是要强行推momgo还是妥协，用mysql

肯定还是MySQL

这个要多方面权衡和考虑，而不能说自己觉得怎么样

架构师，每天的工作都是衡权利弊

而不是说，技术好，推高大上的东西

就能做好架构师

这就是刘凡说的

你要站在其他人的角度，去思考，跳出自我的局限性

嗯，有时候一个方案的选择，可以节省很多工作量

如果你发现自己总处在一个很偏激的思维中，听到不同的声音，肾上腺素立马升高，准备对喷，战斗

这是很不好的

你一定要听别人的想法，为什么他要这样想

静下来去思考

让自己变得更平和，更能集思广益

而不是只在自己的“我就是正确的”这样的思维牢笼中

你的正确在别人眼里，可能很不正确

考虑更多的相关因素，综合考虑不同群体的利益关系，然后做出折中的设计

这才是好的架构设计

3.优势与缺陷。

3.优势与缺陷。（对自身而言）

优势，是我们自己擅长做的事情。缺陷，是我们自身排斥的事物。

先说优势，我可能对一些编程思想技术，以及业务会付出多一些，这也导致，工作的下限我是具备的。

就像我毕业以前实习的那一年，我天天都在写代码，学技术，工作。

可是我的缺陷在于哪里呢？

与人沟通，不愿意聆听他人，去尊重他人，思考问题非常局限。说白了

学生思维，以自我为中心。谁会愿意与我一起工作？

我们一定要认识自己的问题，勇敢去面对自我的缺陷，我不追求完美，但我会尽可能打碎自己的缺陷。

及时我无法打碎这些缺陷，我也会向走一步，尽一份力

打碎这些东西，可以让我去尊重他人，学会聆听他人，学会换位思考，这对我的人生，有积极的作用。

比如做技术分享，群里说多了

我在公司分享技术和业务，也是滔滔不绝。

4.自我反思

会思考问题的人，才会真的进步

人和人能力的差距，体现在这点上，就是，你会不会真的去思考当下的各种环境。很多人不会进化，3年前怎么样，现在还是怎么样，10年后也还是那样，我称这样的人，是固化状态的人。有一些人不同。他长期处于不断的进化和调整中，不断的自省，随着环境和自身发展调整自己思考问题的方式和角度，今年之后，整个人就完全不一样了。

你可以回看你身边的同事，包括你自己

是不是很多固化状态的人？

那些30多岁稀里糊涂的老哥，25，6岁的时候，和现在也基本上差不多，观念和思维

这样的人，从来不会静下来自省，去思考和总结。更不会有任何改进自己生活工作的计划，他们好像总是那样，从来没变化过。

人和人的差距，就是头脑的差距

我觉得，群里任何一个感觉自己总是止步不前的人，职位或者薪水，都可以去思考

是什么让你总处在一个低下的位置

真的是你的技术不够好吗？

刘凡这几年的经历，我们看得到，他也总是分享自己的心得，总在调整自己的状态，从不好到好，愿意去思考更多更广，不会总停在某个地方坚守自己旧的观念。

并且总在去行动

这就是榜样，我们都应该去学习。

如果你三年前，持有什么观念，你现在依然没有变化，那说明你没有任何进步，这是一件很危险的事

随笔

- 系统架构师是一个既需要控制整体，又需要洞悉局部瓶颈，并依据具体的应用场景给出解决方案的人。确定和评估需求，给出开发规范。**在需求阶段**，负责理解和管理非功能性需求（可维护性、可复用性），还需要确定客户及市场人员所提出的需求，确定开发团队所提出的设计。在**软件设计阶段**，负责对软件体系结构关键构件、接口。在编码阶段，详细设计者和编码者的顾问，并且经常举行技术研讨会。随着集成和测试，集成和测试支持将成为软件架构师的工作重点。
- **如何进行区分技术和业务型开发的边界？**
    - 通常来说，每个企业一定会沉淀自己的相关数据、业务中台，而中台的出现一定是为了提升企业的复用能力。而对标这一类，是技术业务想融合，用技术解决业务问题，而也沉淀出相当一部分的中台。而中台之下，一般就是基础设施服务，和大数据平台等相关的。而这个类就是相关技术驱动为主，主要为整个企业提供整体的支撑体系。而中台之上就是前台，利用下层的。不断解决用户需求。业务驱动业务型。大多数我们是无法去衡量业务与技术相关的比重，更多的是自己经历相关。有一点可以确认的就是，我们必须专注一个小的领域，做到行业顶尖。纵深发展，T字形是一个技术人的永远标杆。前进方向。
- **业务VS技术**
    - 商业业务的成功，不一定就是技术最牛逼的，而是通过技术去驱动业务。很烂的代码，效益可能更好，很好的代码，效益可能不好。技术是手段，为了实现商业业务的手段，技术是手中的剑，如何使用需要靠自己去磨练。对技术的思考深度直通本质是大多数人缺少的，技术不一定要会的多，一定要精。思考才是根本力。
- **技术人的个人价值如何体现？**

    - 在工作中，我们难免发现，每个员工技术水平都是有限的，但是确可以创造出一个好的系统，使用。我们不应该过分强调个人的价值应该无限大。一个人写一个操作系统。写一个淘宝。一定要明白，随着社会的发展，个人的作用对于整个公司来说是很弱小的。将一个人放入一个孤岛生存。可能都不如一个叫花子的生存能力。如果说在原始社会，我们可以依靠自己的力量进行斗争，但是社会是在不断进步的。展望真个历史发展的进程，逐步从个人到整体。不断的工业化。汽车工业化。从马车到汽车。正是工业化中，每个人相互协作才能构建出一辆在高速上奔驰的汽车。而不可能有一个人，对于汽车的全部部件，每个细节都掌握到极致。对于工程师来说，只负责自己的一部分，对于总架构师来说，需要负责整体的全局部署与框架的设计。对于领导来说，需要高瞻远瞩 规划公司的发展方向。对于软件开发也来说。同样也是如此。我们应该有一个态度，那就是**正视个人的价值**。
    - **技术狂人症**。对于程序员来说，可能对于技术的追求是极致的，需要正视黑盒的价值的存在。更加专注于自己的领域才是根本。心态上要放好，永远的学习者。大部分的程序员都是在做业务，无论对于微软。facebook 来说，window是一个操作系统应用相关的业务。facebook是交友相关业务。而javaer针对的是javaee的业务。基于基础，现成的东西做上层的产品。做现代软件的缔造者。比如七牛云架构师，对于AOP的理解，可能不如你，但是他对于自己专注的事情，一定是行业顶尖级别。
    - **违底层论思想** 很对程序员的认知就是，越做底层的，就越牛逼，我们应该放弃这种想法，不要人云亦云。如果说相对于J2EE领域来说，操作系统是底层，那么对于操作系统来说电路就是核心，所以，我们要容忍对于某些技术的透明性，对于他们来说，做自己专注的事情，对于我们来说做自己专注的事情。在一个学校里，食堂大妈收拾卫生，店里卖自己的饭，你能说做饭的厨师厨艺技术一定比大妈强吗。所以不要违底层论，只不过每个人做专注于自己领域的事情罢了。
- **技术学习应该有的心态，不应有术，还应有道**
    - 学生时代，我们会不断拼命学习一些开发技术，SSM Boot等，但是随着参加工作后，我们逐步会发现用技术可以解决工作中问题，会出现对于技术的追求性。逐渐降低。对于技术人来说，只有苦练技术，才是出身。
    - 对于造轮子的看法：1.遇到实际问题，先看有没有现成的解决方案，2.看相关技术 解决的问题点在于哪里。3.如果再次之上没有，才会造轮子。(被逼的)，不一定要在项目中一定使用某个技术。局限自己的视野。
    - **无论 什么岗位上，做事情都做到最好 -极致**，主动编写学习文档，免费解答他们问题。主动。**做人做事优秀，比技术好更重要**，积极沟通，积极说话，下班晚点，上班早点，**1.熟悉更多业务，更多代码。**

      需求分析的时候更加准确，能够在需求阶段识别风险，影响，难点。

      问题处理的时候更加快速，因为相关的业务和代码都熟悉，能够快速地判断问题的原因并进行排查处理

      方案设计的时候考虑更加周全，由于有对全局业务的理解，能够设计出更好的方案。
- **对于比较复杂的业务或者流程？**
    - 建议自己做一个流程图，理清楚每一个分支，文档细无巨细，流程图详细。类比 刷算法题，不要求急，一点一点理明白流程逻辑。重在质量。
    - **对于阅读源码的心态？**
        - 说道最终Spring就是一个项目，只不过我们平时面对的是业务项目，而Spring解决的点是工具
- **工作中的为人处事**
    - 1.永不抱怨 多思考如何解决问题 找一些释当的发泄工具
    - 2.不要怕吃亏 能提升自己 多吃点亏
- **心态**
    - 理论就是原理，工程就是问题

    - 不懂原理，解决问题没有思路 技术怎么来的，解决了哪些问题，带来了哪些问题 底层如何工作的
- 多面试 多尝试
    - 不要怀疑自己
    - **技术是其次，自信心是最重要**
    - **说核心的，直击灵魂 干净利索 专业(不是为了装逼 而是切实理解 体会)**
    - **急切 基于证明自己 高手都是淡定**
    - **回答问题 先回答问题，在说自己的理解 重点**
    - **学会分解问题**

0x14-todo

- 未来1/2年学习大数据相关体系

项目

一些建议:一个知识点 分点说成树

基础知识+项目+场景问题

对于面试官抛出的问题。一般从为什么会出现 解决了那些问题 以及遇到的问题

不要知识单纯的回答一个知识点，要将自己的技能树型话。

对于一个知识点，抛出正确答案，然后谈一下自己的理解。就是满分。

面试官

- 连环坑
- 展知识点深度
- 较好的展现自己的水平 以及亮点和加分项

复盘建议

- 笔记重要性 好记性不如烂笔头
- 写技术博客
- 学会复盘面试

0x15-书海遨游

- JVM
    - 周志明 三版
    -
- 并发
- MySQL
-